

UNIVERSITÉ DE NICE CÔTE D'AZUR,
ÉCOLE DOCTORALE EN SCIENCES ET TECHNOLOGIES DE
L'INFORMATION ET DE LA COMMUNICATION

A scientific journey in the realm of signal and data processing: Compressed sensing, sparse
approximation, and applications

HDR

submitted as requirements for the degree of

HABILITATION À DIRIGER DES RECHERCHES

in Computer Engineering, Control and Signal Processing (CNU Section 61)

by

Jean-Luc Bouchot

Presented publicly on December 19th, 2025, after positive reviews from

Professor Richard Küng
Professor Uwe Naumann
Dr. Clarice Poon (Reader)

Johannes Kepler University
RWTH Aachen
University of Warwick

In front of the HDR Committee:

Associate Professor Roza Aceska
Professor Richard Küng
Professor Frédéric Morain-Nicolier
Professor Uwe Naumann
Dr. Clarice Poon (Reader)
Professor Alexander Powell
Professor Jun Yu

Ball State University, USA
Johannes Kepler University, Austria
Université de Reims Champagne Ardennes, France
RWTH Aachen, Germany
University of Warwick, UK
Vanderbilt University, USA
University of Umeå, Sweden

TABLE OF CONTENTS

	Page
LIST OF FIGURES	iv
LIST OF TABLES	vii
LIST OF ALGORITHMS	viii
ACKNOWLEDGMENTS	ix
ABSTRACT OF THE HDR	xi
RÉSUMÉ HDR	xiii
1 An overview of my research	1
2 Structure-based image processing: discrepancy norm and distance trans- forms	20
2.1 General background	20
2.2 Some algorithmic concerns for industrial automatic quality control	24
2.3 Some axiomatic results against classical measures	30
2.4 Some constructive results	32
2.5 Further research topics in structures in signals	35
3 Compressed sensing and sparse approximation	37
3.1 An introduction to compressed sensing and sparse approximation	37
3.1.1 A primer on compressed sensing	37
3.1.2 Sensing in a distributed environment	43
3.2 Iterative algorithms for the recovery of sparse vectors	47
3.3 Sparse numerical approximation to parametric PDEs	55
3.3.1 General background	56
3.3.2 Basics of weighted sparsity	58
3.3.3 Single level approximation	62
3.3.4 Multi-level approximation	67
3.3.5 Computational work	76
3.3.6 Numerical validation	79
3.3.7 Numerical results	82

4	Algorithmic differentiation	87
4.1	AD and adjoint AD	87
4.2	Profiling strategy	90
4.3	What is coming in my AD world	99
5	Outlook	102
	Bibliography	103

LIST OF FIGURES

	Page
1.1 The discrepancy norm unit ball in 2 dimensions. The odd-shaped hexagon implies an interesting behavior in the computations of distances in which the norm of a vector, by contrast to classical Minkowski distances, depends on the order of its elements thereby considering structures.	6
1.2 Aligning two signals is a non trivial task when the metric used is not monotonic with respect to shifts. Left, a signal of interest, right the autocorrelation function with various metrics. The classical correlation (based on the ℓ_2 norm) is not monotonic with respect to the displacement.	7
1.3 The minimum ℓ_1 norm solution (red diamond) is more likely to touch the linear constraints (orange line) along the canonical axis (hence forcing many components to be 0) than the ℓ_2 ball is (blue circle).	10
1.4 In the proposed fusion frames approach, information is measured locally, in each subspace, and then the local pieces are fused together by the centralized processing unit. This allows for larger complexities of the signals, or lower complexity of the sensor design. A represents the sensing matrix and the P_i 's are projection onto subspaces.	12
1.5 In the proposed multi-level approach, most of the calculations are spent on a cheap, coarse grid ($l = 1$ in the Figure). As we refine the model (l growing), we reduce exponentially the number of function evaluations, yet preserving the overall uniform approximation.	15
1.6 Illustration of checkpointing. The upstream section (U) and downstream (D) are not checkpointed and the double array symbolizes the need to save necessary variable in order to compute the backward sweep (single array to the left). C is a checkpointed chunk. Its forward sweep is computed twice, albeit without further stacking <i>local</i> variables. The checkpoint (Ckp) saves all useful variables stemming from U required to recompute C . These variables are popped (<i>CkpPop</i>) from the stack prior to the second forward sweep.	16
2.1 Examples of OptLDMap runs on images of our customers a to f and test images of the TILDA database [105]. The first column is always the input image, the second the defectiveness result of OptLDMap before thresholding and the last column an overlay between input image and the thresholded result. The window sizes are 30×20 for the airbag hose, 25×25 for the woven fabrics and 40×40 for the others. Threshold is fixed to 0.9.	26

2.2	Examples of the applicability test on textile defect images. For each example the test image (T) as well as the output (O) of the algorithm is shown. Reference images are not shown. The algorithm output is for illustration purposes not thresholded.	27
3.1	The use of ℓ_p norms forces the non zero elements to concentrate along the canonical axis, thereby approaching true sparsity (defined through the ℓ_0 norm) more and more as $p \rightarrow 0$	39
3.2	Weights in the minimization allow to drive the solution of the ℓ_1 minimization problem towards a more plausible solution: the ℓ_2 solution and the constraints have not changed, yet minimization weighted ℓ_1 problems changes the solution.	60
3.3	Expectation in the L^2 norm of the MC estimation (blue) and CSPG (green) methods.	66
3.4	Convergence pointwise for various sparsity levels.	67
3.5	Evolution of computing time and approximation accuracy as the number of levels changes.	83
3.6	Evolution of computing time and approximation accuracy as the first approximation level changes.	84
3.7	Evolution of computing time and approximation accuracy as the dimensionality increases.	86
4.1	Illustration of the process of storing (pushing) states at run time during the forward sweep in order to recover (pop) them in the backward sweep.	89
4.2	An example of code with backward sweep included. Data-flow reversal is all about being able to recover the red V's even though their values have been overwritten in memory.	90
4.3	(top)A piece of code C is being checkpointed, i.e., all its useful variables are stored for efficient recomputations while the following code segment is being executed with a store-all strategy. (bottom)A nested code in which some sections are checkpointed for recomputations, while the gray boxed sections are directly executed in store-all mode.	91
4.4	Sketch of execution for the adjoint of a code arbitrarily split as a sequence of three parts U, C, and D. Thick arrows to the right stand for forward sweeps, running the primal code together with storing intermediate values on the stack. Thick arrows to the left stand for backward sweeps, that retrieve values from the stack and use them while propagating gradients backwards. Time goes top-down, and inside each line time follows the arrows' directions. . . .	92
4.5	Sketch of execution for the adjoint of the same code than in Fig. 4.4, having checkpointed the section C. Time goes top-down, and inside each line time follows the arrows' directions. The thin arrow to the right stands for the extra execution of the primal C, and the black and white bullets stand respectively for the writing and reading of the <i>snapshot</i> , i.e. a set of variables sufficient for identical duplicate execution of C.	92

- 4.6 Tuning the adjoint of **streamice** by profiling. Each checkpointing configuration is shown as a dot whose coordinates are peak stack size and run time of the adjoint. The red and blue dotted lines show the evolution of performance of the adjoint by repeatedly following the suggestions made by our profiling tool. The red line follows a “run-time gain first” strategy, the blue line a “careful on memory” strategy. Time measurements are averaged on 5 runs. Smaller gray dots show the performance of 250 random checkpointing configurations. Green dots show combination with binomial checkpointing, an algorithm used for optimal checkpointing of time-stepping loops. 97

LIST OF TABLES

	Page
2.1 Examples of kernels and distance measures that follow the construction principles of Theorem 2.3 or Corollary 2.2 with summation as aggregation function	31
3.1 Dependency of the number of samples on the sparsity. These figures are valid only for the set of parameters described in the text.	66
3.2 Dimensionality of the ansatz space and compute time required for generating it as well as the sensing matrix with changing dimensionality of the parameter space. $J = 2, L = 4$	85
3.3 Dimensionality of the ansatz space and compute time required for generating it as well as the sensing matrix with changing number of levels, $d = 20, L = 5$.	85

LIST OF ALGORITHMS

	Page
1 Basic OptLDMap	25
2 Fused distributed sensing recovery framework	46
3 Iterative Hard Thresholding	48
4 Orthogonal Matching Pursuit	49
5 Hard Thresholding Pursuit	51
6 f -Hard Thresholding Pursuit	52
7 Pseudo-code for the Compressed-Sensing-Petrov-Galerkin Algorithm	64

ACKNOWLEDGMENTS

Thanksgiving marks this singular time where readers and the writer meet. It marks the beginning of the reading journey for the former and the end of the writing part for the latter. With these words the tale of my scientific journey reaches a crossroad, dare I say a non-differentiable point, allowing me to officially supervise future generations of scientists. I am especially honored to have such a diverse and international jury to witness this day.

It is with great pleasure that I thank the reviewers of this HDR manuscript. Foremost, thanks are directed to Richard Küeng, whom I met in 2013 during one of my first conferences on the American side of things (the so cleverly called FFT's) and with whom we've had regular interactions throughout my career. It so happens that I had been at JKU too and spent some time in Hagenberg, a place dear to him. Interestingly enough, after many years abroad he is now back home, and so am I! It is a real joy to have you on board, and as president of this jury no less!, for my next journey. Clarice Poon has always been an inspiration through my career. I always found her papers inspired and she is a model in what I would like to be as a researcher. Recognized for a deep understanding of mathematics applied to signal and image processing as well as machine learning, I am grateful to her for agreeing to review my manuscript. Uwe Naumann, a man who needs no introduction in the world of algorithmic differentiation, has kindly agreed to review this manuscript too. His expertise in both scientific computing and AD (and computer science in general) is an invaluable asset to the review of the HDR. With him too we share work locations (history), both at Inria Sophia and Aachen.

I would also like to express my deepest thanks to the jury members who have agreed to postpone ever so slightly their winter vacations. On top of the reviewers mentioned above I am grateful to Jun Yu for his interest in my work and with whom exchanging ideas is both easy and fruitful. I'm looking forward to finally getting results out and not just ideas! Alex Powell and Roza Aceska were both at Vanderbilt when we first met and I am happy to have kept in touch with them. Teaching at SEAMS or Academia Sinica remains one of my fondest memories as a teacher and an experience I have greatly enjoyed. This is, of course, not even scratching the surface of the interesting research that they both deliver to the signal processing community! Last but not least in this list is Frédéric Morain-Nicolier. The common denominator of the manuscripts for my Ph.D. thesis and this HDR, always listening and always jolly, it is an enormous honor to have you part of this jury!

Of course, achieving an HDR is not a single man job. I have benefitted from the contributions of former students (in particular Benjamin Bykowski, Marcel Leenings). Moreover this would not have been possible had I not met many people on the way who have shared ideas and interests with me. The list would be too long here, but I would like to thank all my co-authors and people I have met on my scientific journey as well as the institutions (Johannes Kepler University, Drexel University, RWTH Aachen, Beijing Institute of Technology, Inria Sophia) and funding agencies (FWF, NSF, ERC, Clay Institute) for allowing me to do my own research. In particular I am forever in debt to those who have taken me under their guidance. Simon Foucart (then at Drexel, now at Texas A & M) and Holger Rauhut (then

at RWTH Aachen, now at LMU) have been great mentors in the world of numerical analysis and compressed sensing. More recently, I have had the honor to work under the guidance of a (if not *the*) pope of source transform AD, Laurent Hascoët, who has shown me not only how to be a good computer scientist, but has also managed to re-calibrate me for rigorous research after some years in the industry. Thank you! On a similar note, I want to thank Jean-Baptiste Caillau for his eternal guidance, both scientifically and humanly, and this since my early days as an undergraduate.

Finally, I want to thank all my friends and family for being around all the time, and Marie for her constant support.

ABSTRACT OF THE HDR

A scientific journey in the realm of signal and data processing: Compressed sensing, sparse approximation, and applications

By

Jean-Luc Bouchot

Habilitation à Diriger des Recherches in Computer Engineering, Control and Signal Processing (CNU Section 61)

Université de Nice Côte d’Azur, Centre Inria, Sophia Antipolis, 2025

This *Habilitation à Diriger des Recherches* reviews my academic work since defending my Ph.D. in October 2012. It presents my journey in the scientific world with an emphasis on algorithmics and image and signal processing. In particular a strong interplay between theoretical findings and real-world applications is highlighted.

The first part focuses on structure-based image similarities and processing. Starting with the (re-)introduction of Hermann Weyl’s discrepancy norm, it naturally evolves into the study of similarities which behave monotonically with respect to shifts. The highlights of this chapter lie in the negative results that most of classical measure cannot satisfy the monotonicity of autocorrelation and in the introduction of a novel similarity measure satisfying the aforementioned property when facing shifts in the image. From a computational point of view, the issue with the non-differentiable parts is circumvented by using ℓ_p norm approximations, also allowing, after some manipulations, fast computations.

In a second part, the link from such problem to model-based approximation is made, going from the notion of structure as object in the image to structure in the signal model. It is also the opportunity to review my work in the area of *compressed sensing*. In particular, after setting the context and reviewing the important ideas, I introduce my main contributions: some novel algorithms for sparse recovery (and de facto for feature selection) and their analysis in terms of number of iterations, as well as the introduction of a distributed sensing framework based on fusion frames. Some applications of compressed sensing to the field of scientific computing and more specifically to the problem of approximating parametric PDEs in very high dimensional parametric spaces is presented. Overall, the algorithm introduced in this section offers a solution to the parametric problem (akin to a *training* phase in machine learning) which has a computational cost comparable to the evaluation of a single solution.

The next section is dedicated to my most recent research in Algorithmic Differentiation, a fundamental yet often overlooked aspect of scientific computing, optimization, and machine learning. In particular it introduces a novel algorithm for profiling checkpointing strategies

in scientific computing applications via Source-Transform AD. This fundamental aspect of optimization is the art of knowing when to recompute chunks of code versus when to store useful variables. This may have an important impact in the world of learning and very large scale scientific computing.

This HDR concludes with some questions of interest in the near future, mixing my previous research in signal processing, scientific computing, computer science, and computer vision. Moreover, throughout this journey, many open questions are highlighted which will drive my future research and supervisions.

RÉSUMÉ HDR

A scientific journey in the realm of signal and data processing: Compressed sensing, sparse approximation, and applications

Par

Jean-Luc Bouchot

Habilitation à Diriger des Recherches en Génie informatique, automatique, et traitement du signal (CNU Section 61)

Université de Nice Côte d'Azur, Centre Inria, Sophia Antipolis, 2025

Cette Habilitation à Diriger des Recherches (HDR) présente mon travail académique depuis la soutenance de ma thèse en octobre 2012. Dans cette HDR, je reviens sur mon parcours scientifique, en mettant l'accent sur l'algorithmique et le traitement d'images et de signaux. L'interaction étroite entre les résultats théoriques et les applications concrètes est particulièrement mise en évidence.

La première partie se concentre sur les similarités pour le traitement d'images basées sur la structure. Après la (ré)introduction de la norme de divergence d'Hermann Weyl, elle évolue naturellement vers l'étude des similarités qui se comportent de manière monotone face aux translations. Les points forts de ce chapitre résident dans les résultats négatifs selon lesquels la plupart des mesures classiques ne peuvent pas satisfaire la monotonie de l'autocorrélation, et dans l'introduction d'une nouvelle mesure de similarité satisfaisant la propriété susmentionnée face aux décalages de l'image. D'un point de vue computationnel, le problème des éléments non différentiables est évité grâce à l'utilisation d'approximations par une norme ℓ_p , permettant également, après quelques manipulations, des calculs rapides.

Dans une deuxième partie, le lien entre ce problème et l'approximation par modèle est établi, passant de la notion de structure comme objet dans l'image à la structure dans le modèle de signal. C'est également l'occasion de passer en revue mes travaux dans le domaine de l'échantillonnage compressé. En particulier, après avoir rappelé le contexte et passé en revue les idées importantes, je présente mes principales contributions : de nouveaux algorithmes pour l'approximation parcimonieuse (et de facto pour la sélection de caractéristiques) et leur analyse en termes de nombre d'itérations, ainsi que l'introduction d'un modèle de traitement distribué basé sur des trames de fusion. Des applications du *compressed sensing* au domaine du calcul scientifique, et plus particulièrement au problème de l'approximation des EDP paramétriques dans les espaces de très grande dimension (problème notoirement connu pour souffrir de la malédiction de la dimension), sont présentées. Globalement, l'algorithme présenté dans cette section offre une solution au problème paramétrique (semblable à une

phase d'apprentissage en apprentissage automatique), dont le coût de calcul est comparable à celui de l'évaluation d'une solution unique.

La section suivante est consacrée à mes recherches les plus récentes en différenciation algorithmique, un aspect fondamental mais souvent négligé du calcul scientifique, de l'optimisation et de l'apprentissage automatique. Je présente notamment un nouvel algorithme de profilage des stratégies de points de contrôle (*checkpointing*) dans les applications de calcul scientifique via une approche de différenciation par transformation de sources. Cet aspect fondamental de l'optimisation est l'art de savoir quand recalculer des fragments de code versus quand stocker des variables utiles. Cela pourrait avoir un impact important dans le monde de l'apprentissage et du calcul scientifique à très grande échelle.

Cette HDR se conclut par des sujets intéressants pour un avenir proche, combinant mes recherches antérieures en traitement du signal, calcul scientifique, informatique et vision par ordinateur. De plus, tout au long de ce parcours, de nombreuses questions ouvertes sont rappelées, qui orienteront mes directions de recherches et supervisions futures.

Chapter 1

An overview of my research

This manuscript reviews my journey into the world of mathematical signal and data processing with a strong emphasis on applications. Throughout, *signal* is to be understood as an abstract concept covering typical 1D signals, 2D images, or any sort of volumetric data. Similarly, data is just a word for any (potentially very) high dimensional vector.

Passionate about human-computer interactions in a broad sense, my research path resembles my personal interests: a mixture of mathematics, computer science, and engineering¹. I tend to always tie a problem arising from applied sciences or engineering, together with its underlying mathematical concepts. I describe the problem in mathematical terms and work on it in order to finally implement its solution or simulation on a computer, thereby advancing the computer science community as well. Understanding at a deep theoretical level the sciences behind applied problems stemming from engineering and the applications is at the core of my motivation. My journey in the scientific world started when I discovered computer vision while working at T-Systems International GmbH (within the R & D department), implementing algorithms for visual recognition on mobile devices. This was in the stone age of smart phones² and even the most advanced mobile phones did not have an autofocus on their (low-quality) cameras, let alone infra-red or depth sensors. I was facing my first challenging scientific problem: trying to make sense of a blurry image, from a low quality camera, on a device with low memory and compute performances. This was my first encounter of image processing, and already entailed a fair part of alignment and knowledge-based mathematical systems, two components which have been the common and main threads running through my career, both as a researcher and a (industrial) scientist.

From there, understanding anything related to images and decision making (knowledge-based mathematical systems) was an obvious choice for my Ph.D., and the path awaiting me. Indeed, involved in an FWF³ project in Austria within the FLLL⁴/Department of

¹, and some oxford commas.

²I had the opportunity back then to work on the first Android phone in Germany.

³Fonds zur Förderung der wissenschaftlichen Forschung

⁴Fuzzy Logic Lab Linz, headed by Prof. E.P. Klement, one of the popes of fuzzy logic.

Knowledge-based Mathematical Systems at the Johannes Kepler University of Linz, I have investigated the use of the discrepancy norm in image processing. A major part of interest here was the fact that the discrepancy norm is monotonic with respect to shifts, a concept I detail below. The link with my prior experience was clear: being able to quantify the misalignment is key to aligning or finding objects in images!

This first research experience has only made me eager to understand things at a deeper level. For instance, is this monotonicity property characteristic to the discrepancy norm? If yes, why is this the case? If no, how can we create other measures with a similar behavior. This was only the tip of the iceberg, and motivated me to strengthen my mathematical understanding of such phenomenon and, from a more general point of view, model-based image processing. This opened the door for me to study compressed sensing and its related topics both in data processing (metagenomics studies, greedy and sparse approximation) and in theoretical problems (sparse optimization, numerical analysis). Compressed sensing can be roughly seen as the art of finding low-dimensional structures in high dimensional worlds – this is a neat application of Ockham’s razor. At Drexel University, I held a double position as a researcher and instructor. As an adjunct professor – an extra activity – I taught various mathematics classes with a strong emphasis on numerics and implementations. As a postdoc – my main activity – working with Simon Foucart, I worked on an NSF⁵ project studying the use of sparse methods and representations in metagenomics data survey. My curiosity stroke again, and I decided to dig deeper in the theory of bioinformatics in general and computational genomics more specifically – this reached its pinnacle in a graduate-level class I created from scratch and taught at RWTH Aachen 2 years later, entitled *Mathematical and Computational Genomics*. Within the two years spent at Drexel University, I have taught many maths classes with a strong practical aspect (for instance, teaching *Differential equations for engineers* which had a major application side to it) and had the chance to spread my recent results on greedy sparse approximation and algorithms at numerous (invited) conferences and seminars.

Following this, I decided to continue pursuing my mathematical studies and mixed it with more computer science and numerical analysis. Arriving at RWTH Aachen to work with Holger Rauhut in Germany, I started working on scientific computing and applied mathematics. My contributions involved applying sparse methods to polynomial approximation schemes in order to numerically compute approximate solutions to some parametric elliptic PDEs. This was a great opportunity to mix all of my previous experiences: some data analysis tools combined with computer science and the mathematics of approximation theory. The peak of my contribution is the development of a piece of software ⁶ and some brand new results on multi-level approximation in order to break what is known as *the curse of dimensionality*. Some parts of this work are still ongoing today, but the beginnings of them allowed me to supervise a master’s thesis in mathematics and publish our results. I did not leave my first (scientific) love aside and eventually worked on image processing again, but with a pinch of applied mathematics in it, in order to develop a method for some distributed sensing and recovery of 2D images, specifically looking at the problem of denoising MR Images, from

⁵National Science Fund

⁶<https://github.com/jlbouchot/CSPDEs/> , albeit far from being industrial grade software

locally unavailable data – this is not unrelated to the problem of *federated learning* today.

After four fruitful years at RWTH Aachen as a postdoc⁷ where I have created two new graduate level courses (*Mathematical foundations of machine learning* on top of the genomics class mentioned earlier), published a handful of papers (which will be developed upon in the upcoming sections), and presented at many conferences, I have joined Beijing Institute of Technology as a faculty member in the School of Mathematics and Statistics. This choice was motivated both for the curious layman that I am to discover a new culture and for the potential to develop new scientific collaborations. This was met with success on both levels. Indeed I was given the opportunity to visit new research centers, namely the Center for Data Science within the School of Mathematics at Zhejiang University, and attend conferences in Japan, which would be more difficult to handle from a Western-based institution. On the teaching side of things, I got invited to teach at the Academia Sinica in Taipei, a leading institution for mathematics in Taiwan, and as a lecturer for a SEAMS⁸ School on signal processing in the Philippines. Both opportunities would have been difficult to satisfy had I not been in an Asian institution. Research-wise I have continued working on the parametric PDE side of my research and started looking first at the approximation of Hilbert-valued solutions to parametric PDEs (a piece of work which is almost done) and second at the approximation of Banach-space valued solutions (a work that is still at its infancy today).

Personal family reasons have made me leave Asia and therefore academia abruptly, but I kept a keen interest on applied research. I have therefore joined Continental UK developing novel algorithms first in the field of computer vision (for instance the calibration of multicamera systems at high speeds) and then on multimodal sensor fusion (combining radar information with camera images to generate an understanding of the surroundings). Many of the algorithms – albeit proprietary – I have designed are now found in millions of cars. The implementation, first as a proof of concept and then in production, was done with a team of engineers whom I supervised on a technical level and with respect to tight schedules given by customers. Once again, politics (Brexit and Covid alike) have made me reconsider a career switch and finally after many years abroad I landed in France and joined Artelys as an expert engineer. There I have been involved as a project manager in the Fortissimo 4 project, an initiative within the H2020 projects from the European commission. The goal was to develop methods for computing (approximate) solutions to large optimization problems via HPC Clusters. Artelys was concerned with problems arising from the energy sector and modeled as tough Mixed Integer Non Linear Programs (MINLP) in very large dimensions (many assets which can be controlled, each of them through many parameters) and under uncertainty (many stochastic parameters describing the weather and/or potential outages). Eventually, we designed and developed a framework for high performance computing which would deliver solutions within days instead of months if not years.⁹ Artelys was also a great

⁷It is worth mentioning that the German system works in a peculiar way, when seen from a French prism. For instance, a postdoc lasts much longer and there are virtually no such thing as a *Maître de Conférence* – This is less and less true, with the development of tenure track like positions. – and only true faculty members are worthy of official supervision of students, even though postdoc's do teach and supervise, in practice...

⁸South-East Asian Mathematical Society

⁹An important study has been published using our algorithms developed within this project [39]. I will

opportunity for me to contribute to various open source software, pushing ever so slightly the boundary of open sciences.¹⁰

Finally, after almost 5 years in the industry developing novel algorithms, I have rejoined academia with a broader perspective on the whole mathematical and applied computing landscape. These few years working in the industry, supervising teams of junior and senior engineers and leading the development of highly technical algorithms, I have learned both the ways of passing on knowledge and setting reachable targets albeit exploring parts of the unknown worlds in the sciences and engineering. I am now tackling problems in automatic differentiation a major pillar in solving scientific computing problems from mesh optimization to the now infamous *backpropagation*. Optimization has been a corner stone of all my academic and industrial research so far, and being able to understand the subject matter of applications will drive my motivation to advance the world of automated derivatives even further!

The tale you are about to read recounts my experience through this scientific journey for which a somewhat chronological order has been chosen, without specific reason.

Structural (similarity) measures in image processing

This *habilitation à diriger des recherches* (HDR) manuscript starts with an insight in the mathematics behind some structural similarities. The term *structural similarities* does not bear any official mathematical meaning and is simply a term I first discovered through the *Structural Similarity Image Measure* (SSIM) paper [120]. Since then, I have decided to continue using this word to describe anything that carries some sort of information about the structures (edges / corners / ...) within an image. This very broad description includes a very diverse panel of tools such as (but not limited to) the Sobel operators, JPEG's cosine or wavelet transforms, and more intricate objects, such as distance transforms (used in particular for binary images and analysis of very old documents) and the *measures*¹¹ that I am about to describe.

not expand any further on this topic in this manuscript as it is orthogonal to my usual line of research. But it is worth mentioning that this work attracted some public attention as I got to talk at various conferences (Teratec, Applied AI Conference) and was part of an invited panel of experts at the HPC Industry Summit in Berlin in October 2023.

¹⁰I contributed to software developed by RTE's R & D department PowSyBl and Dynawo.

¹¹*measure* is used here as a metric, or an index, or some quantification of something, and not necessarily, unless specified otherwise, a measure in the mathematical sense. As a consequence, one may talk about similarity or dissimilarity measures even though the axioms of a normed or a metric space are not satisfied.

Throughout my doctoral research I investigated the use of Hermann Weyl's discrepancy concept [122] in the context of image and signal processing. My research based itself upon the work of Moser [91], in which the old definition is given a more contemporary view specifically used for the analysis of volumetric data. For the case of a sampled 1-D signal f of size N , the discrepancy norm is expressed as

$$\|f\|_D = \max_{1 \leq i \leq j \leq N} \left| \sum_{k=i}^j f_k \right|. \quad (1.1)$$

Its continuous counterpart for one-dimensional μ measurable functions is given by

$$\|f\|_D = \max_{a < b} \left| \int_a^b f(x) d\mu(x) \right|$$

It was shown in [91] that the discrepancy norm (even in higher dimensions) satisfies a certain monotonicity property of its autocorrelation function.

Definition 1.1: Monotonicity of misalignments

Let $d : L(\mathbb{R}, \mu) \times L(\mathbb{R}, \mu) \rightarrow \mathbb{R}^+$ denote any sort of discrepancy measure defined for pairs of μ measurable functions. Let T_t define the translation operator with parameter t . d is said to be *monotonic with respect to shifts* if, for any f , $\lambda \geq 1$, $t \in \mathbb{R}$, then

$$d(f, T_t(f)) \leq d(f, T_{\lambda t}f). \quad (1.2)$$

Informally speaking, restricted to the set of positive signals, the discrepancy norm is monotonic in the sense of Definition 1.1. This has been the root for my research during my Ph.D. studies and has been documented in various publications. More importantly, this has piqued my curiosity: is this monotonicity property characteristic to the discrepancy norm? What makes this behavior so remarkable? In particular towards the end of my Ph.D. and in the years following my defense, while employed at Drexel University, I investigated two important questions:

1. How can the discrepancy norm be approximated by differentiable functions for solving alignment problems?
2. Is this non-monotonicity property in the presence of shifts characteristic to the discrepancy norm?

The so-called discrepancy norm, in the form revisited by Moser [91] in the context of data processing, is computed by looking at regions of maximum divergences of statistical values, as seen from its expression in Eq. (1.1). As such, it is inherently non differentiable (as illustrated by the kinks in its unit ball seen in Fig. 1.1) and has a computing time exponential with the dimensions. Some (over?) simplifications through integral images allow for faster computing

(see Eq.(1.3)) but the problem of non differentiability remains. This issue left aside, one of the main advantages of the discrepancy norm is its monotonicity with respect to shifts, which makes it an invaluable candidate for assessing the quality of alignment, thereby useful for any registration problem. This is particularly useful for instance for detecting unusual patterns or defects in repeating patterns, see our work, as part of my Ph.D., on woven textiles in airbag hoses [32].

As one can see from the use of the max operator, the discrepancy norm searches for the segment of largest divergence, and as such it deserves the title of *structural similarity* – the atomic unit of structure being, in the 1D case, the segment. It is also worth noting that this structural aspect becomes obvious when looking at the fact that the norm is dependent on the position of the entries within a vector, which is not the case for usual Minkowski-based metrics. For instance, $\|[1, -1, 1]\|_D = 1$ whereas $\|[1, 1, -1]\|_D = 2$.

Looking back to Definition 1.1, we notice that the computational cost of the discrepancy norm is of order $\mathcal{O}(N^2)$. However an equivalent, faster, definition is given with the use of integral images (again, expressed for one dimensional arrays):

$$\|f\|_D = \max_{1 \leq j \leq N} \left\{ 0, \sum_{k=1}^j f_k \right\} - \min_{1 \leq i \leq N} \left\{ 0, \sum_{k=1}^i f_k \right\}. \quad (1.3)$$

Note that the number of integral images to be computed increases exponentially with the dimension of the array as we will see in Section 2 and it has only limited use for high dimensional arrays.

Interestingly, after my Ph.D. in [25] we introduced an approximation of the discrepancy norm and controlled its error through the use of L^p norms for some p wisely chosen. It also has the advantage of allowing an autocorrelation function to be computed faster through the use of convolution operators, see Thm. 2.2. It is shown on some synthetic signals that this process allows for using numerical optimization schemes for solving the problem of registering two signals. Unlike previous methods which used some divided differences to approximate some sort of gradient of the discrepancy norm, our L^p approximation no longer suffers from long plateaus. These plateaus render moot any local optimization technique, unless one uses tedious inertia-based methods. This is nicely illustrated on Fig. 1.2, where we see large pieces of constant values on the dashed curve representing the discrepancy norm-based autocorrelation. L^p norm approximations allow to suppress these plateaus, even if ever so slightly, as seen from the green dotted curve on Fig. 1.2. Here, the approximation was done with a very small p values (namely $p = 1.5$), and it was shown in [25] that the p norm-based

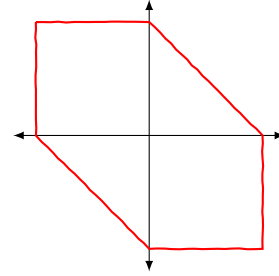


Figure 1.1: The discrepancy norm unit ball in 2 dimensions. The odd-shaped hexagon implies an interesting behavior in the computations of distances in which the norm of a vector, by contrast to classical Minkowski distances, depends on the order of its elements thereby considering structures.

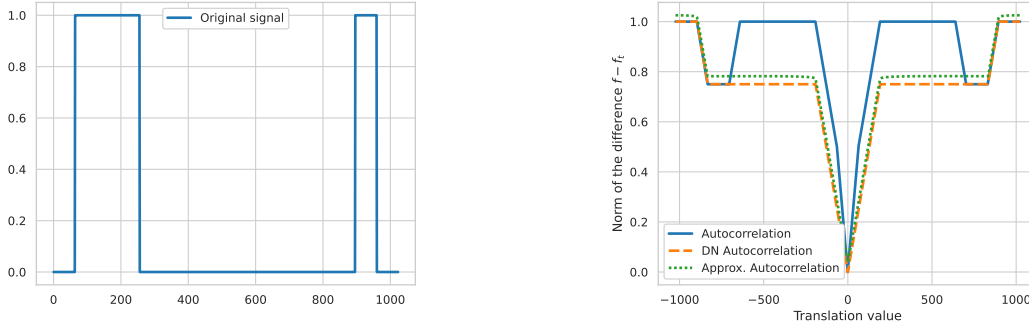


Figure 1.2: Aligning two signals is a non trivial task when the metric used is not monotonic with respect to shifts. Left, a signal of interest, right the autocorrelation function with various metrics. The classical correlation (based on the ℓ_2 norm) is not monotonic with respect to the displacement.

approximation is an estimation of the discrepancy norm from above with an error bounded by $2\ln(N+1)/p$ where N is the length of the signal (the number of sampling points). Note that the classical autocorrelation (i.e. based on the differences between the original signal and its translated measured using the ℓ_2 norm) is represented with a solid line. In our toy example, although it has a strong peak towards the perfect alignment (around 0), it has many local minimum as well as plateaus, making it a less favorable candidate for solving the alignment problem. A generalization of the discrepancy norm for all functions in a measurable space, with a finite measure, was also proposed in [25].

It is also worth mentioning that this approach has more potential than the original definition of the discrepancy norm might suggest, in particular when one is interested in registering images in scale, rotation, and translation domains. This scheme has yet to be evaluated in details and is still the matter of some relatively dormant yet ongoing work.

So far, everything was about the discrepancy norm and its behavior when facing shifts in the patterns, as expressed in Eq. (1.2), whence answering question 1 above. We now look at question 2: is this monotonicity behavior characteristic to the discrepancy norm? To answer this question, we extend the idea to the more general concept of similarity / dissimilarity measures on images and signals. Again, *measures* are not meant, here, as the mathematical object, but rather as the formula used to quantify how visually close (or apart) two objects are. It is worth mentioning that one may define highly complex measures [120]. In fact, research in psychology tends to favor measures which do not even respect the triangle inequality [117, 116]. Such approaches have regained interest in the (deep) learning community recently [101, 52].

With this in mind, we have worked on defining a novel similarity measure taking the inherent structures of an image into account which (provably) has this monotonicity behavior in the presence of misalignments [30]. We coined the term *scaled distance transforms* to describe a whole family of metrics. The definition of this family of metrics is based on measuring

distances (in the Hausdorff sense) between sets of edges at different scales. It is intrinsically relying on the notion of structures through the use of scale spaces and an edge detector. This was motivated, among others, by work in the computer vision community, in which features are found at a certain scale within the image. This is, for instance, related to the SIFT (Scale Invariant Feature Transform) keypoint detection process [85]. We proved in our original publication [30], that any member of this family of metrics satisfies the monotonicity property with respect to shifts, thereby answering question 2 above. But this result only raises more questions related to the set of measures which might satisfy the monotonicity property. This is still unclear but some negative results are detailed in the corresponding Chapter 2 below. With this novel scaled distance transform based approach, we face a similar problem as with the discrepancy norm: the computation involves a max operator rendering the measure non differentiable. In a similar fashion, approximating this with a p -norms allows for noise robustness and efficient computation of optimization algorithms.

Sparse modeling in high dimensional data

Following my first experience in image and signal processing, I have chosen to delve deeper into more theoretical concepts around data analysis. Going back to the notion of structures in images, it is important to bear in mind the notion of representations. As already mentioned, JPEG or wavelet transforms are just other ways of representing data, after an analysis operator. By their construction, they tend to highly concentrate structures on subsets of the basis vectors in the transform domain. In particular, it is often good to differentiate a *data fidelity* term from a *model fidelity* term and problems arising in image processing can often be stated as

$$\begin{cases} \min F(\mathbf{x}, \mathbf{y}) \\ \text{subject to } M(\mathbf{x}) \leq \varepsilon \end{cases} \quad (1.4)$$

Here $\mathbf{x}$ represents the sought-after image or signal, F represents the data fidelity term, $\mathbf{y}$ represents the data or measurements, and M is a function representing the model fitting term. A basic image denoising problem is a prototypical instance of such problems in which $\mathbf{y} = \mathbf{x} + \mathbf{e}$, for some noise $\mathbf{e}$, is the noisy image, $F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2^2$ is a measure of divergence between the original (noisy) image and the (hopefully) noiseless, recovered image, and M might be the total variation operator, aiming to preserve edges in the image. As a consequence, this is yet another look at structures (via the M operator) and similarities (via the F operator) in signals. Edges, or any salient features, being relatively rare (when compared to the number of pixels) in a natural image, the notion of sparsity comes into play. I started looking at sparse modeling concepts, with their applications in the sciences.

Specifically, I got interested in the topic by looking at metagenomics studies. Metagenomics is the study of genomes in a given environment. Of interest to many people is the human microbiome project which studies the (DNA of the) microbiomes that a subject (in this case, a human) carry with them. Typical samples are taken from swabs in their mouth, guts, hands, etc... It is believed that the microbiome is an important marker for potential diseases and/or cure to many health problems such as the Inflammatory Bowel Disease. It may also be used in tracking certain strains of viruses for instance by looking at the microbiome in the subway [89]. Metagenomics typically handles very large data, for which fast and reliable algorithms need to be defined. This was my first steps in both fields of greedy/iterative algorithms and exploitation of large metagenomics data survey.

Within this topic I have worked on answering the following questions:

1. Can we analyze the number of iterations required for convergence in some iterative and/or greedy algorithms?
2. Can greedy algorithms be used in understanding metagenomic studies?
3. How can such algorithms be adapted to large data and/or distributed settings?

Metagenomic studies are interested in making sense of samples of environmental or human microbiomes by sequencing all bacteria at a given sample. The problem is two-fold: how can we reconstruct the bacterial population in the sample from k -mers¹² sequence counts and then what information does the knowledge of the population of bacteria tell us on the sample. For the latter, a typical question is to link the bacterial population carried by an host to certain outside factors or diseases. For both perspectives, it is highly likely that the resulting vector be sparse:

- one may only find a very small subset of bacteria out of the whole spectrum of existing bacteria,
- only a small portion of the bacteria found, which itself is only a small fraction of the bacterial world, may be related to certain diseases.

In any case, having efficient methods for sparse approximation is important, and this has kept me busy a fair part of my post doctoral life. In particular, I have made significant progress in the world of computational compressed sensing.

¹²a k -mer is similar to an n -gram in natural language processing: it is a word of length k on the $\{A, C, G, T\}$ alphabet describing the nucleotides in a genome sequence.

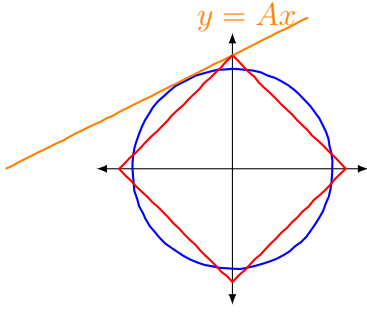


Figure 1.3: The minimum ℓ_1 norm solution (red diamond) is more likely to touch the linear constraints (orange line) along the canonical axis (hence forcing many components to be 0) than the ℓ_2 ball is (blue circle).

I have contributed both on the theoretical aspects and on the practical aspects. From a theoretical point of view, I have started analyzing convergences of the Hard Thresholding Pursuit (HTP) algorithm[55]. Before I detail the HTP family of algorithms (this is the purpose of Sec. 3.2), let me recall some basic notions. I denote by ℓ_0 the counting norm (which is not a norm in a mathematical sense) defined as the number of non zero elements in a vector: $\|\mathbf{x}\|_0 = |\{i : x_i \neq 0\}|$. An N dimensional vector $\mathbf{x}$ is said to be s sparse, for some $s \in \mathbb{N}$, if $\|\mathbf{x}\|_0 \leq s$. Basic compressed sensing can be modeled in a similar fashion to Eq. (1.4) in which the data fidelity term is the ℓ_2 norm of the measurements, $F(\mathbf{x}, \mathbf{y}) = \|\mathbf{Ax} - \mathbf{y}\|_2^2$, for some sensing

matrix A and some known measurements $\mathbf{y} \in \mathbb{R}^m$, and the model fitting term is expressed as $\|\mathbf{x}\|_0 \leq s$. Here the model is the set of s sparse vectors. The modeling assumes that we have some knowledge of the sparsity level of the sought-after signal which might not be possible in practice. After some manipulations, we may rewrite the problem as

$$\begin{cases} \min \|\mathbf{x}\|_0 \\ \text{s.t.} \quad \mathbf{Ax} = \mathbf{y} \quad \text{or,} \\ \|\mathbf{Ax} - \mathbf{y}\|_2 \leq \varepsilon \text{ in the presence of noise} \end{cases} \quad (1.5)$$

which allows us to actually try to minimize the number of non zero elements, enforcing a sparse solution satisfying the constraints, as illustrated in Fig. 1.3.

In particular, we are interested in the regime where we have much fewer observations compared to the number of features, i.e. $A \in \mathbb{R}^{m \times N}$ or $\mathbb{C}^{m \times N}$ with m (the number of observations) much smaller than the number of features N . Basic linear algebra suggests that the kernel of the operator A is non empty, and finding a unique solution to $\mathbf{Ax} = \mathbf{y}$ is out of reach. However, adding the knowledge that we want to find the sparsest solution makes it a solvable problem.

The original effort of the community was concentrated on relaxing the ℓ_0 norm in order to get a tractable algorithm. This is known as the LASSO¹³ and tries to minimize the ℓ_1 norm instead of the counting norm. A geometric motivation is illustrated in Fig. 1.3. We see that by looking at the minimum ℓ_1 norm solution, we find solutions satisfying the constraints (along the orange line) which are aligned with the canonical axis, i.e. for which some of the other coordinates are 0.

The HTP algorithm chooses a completely different approach. It is an iterative algorithm which tries to find the best features at each iteration and then projects the observations

¹³Least Absolute Shrinkage and Selection Operator, stemming from the statistics community

onto the current set of best features. It is formally described through iterating the following two steps, starting from an initial s -sparse guess $\mathbf{x}^{(0)}$

$$S^{(n+1)} = \{\text{indices of } s \text{ largest entries of } |\mathbf{x}^{(n)} + A^*(\mathbf{y} - A\mathbf{x}^{(n)})|\}\quad (1.6)$$

$$\mathbf{x}^{(n+1)} = \operatorname{argmin} \|A\mathbf{x} - \mathbf{y}\|_2 \quad \text{s.t.} \quad \operatorname{supp}(\mathbf{x}) \subseteq S^{(n+1)}\quad (1.7)$$

The intuition behind this strategy stems from the fact that $\mathbf{x}^{(n)} + A^*(\mathbf{y} - A\mathbf{x}^{(n)})$ acts as a proxy for the true underlying s -sparse vector $\mathbf{x}^\#$, since, assuming a noiseless scenario for the time being, $\mathbf{y} = A\mathbf{x}^\#$. In this case, if we assume some form of isometry on A , we have, at least restricted to the set of sparse vectors, $A^*A \approx I$ and therefore $\mathbf{x}^{(n)} + A^*(\mathbf{y} - A\mathbf{x}^{(n)}) \approx \mathbf{x}^\#$.

In [27] we have extended the convergence analysis from Foucart [55] to estimate the number of iterations required for convergence. We could prove that within cs iterations, where $c \leq 3$ is a constant dependent on a Restricted Isometry Constant δ (see Def. 3.3) and s is the number of non-zero elements in the sought-after vector $\mathbf{x}^\#$. This was a very positive result as this was the first uniform recovery result for greedy methods given a linear number of measurements. It was indeed expected to require at least $s^{3/2}$ samples for uniform recovery, as indicated in [102]. Our analysis also concluded with some nonuniform recovery results, in particular when the measurements process is done from subsampling Fourier matrices. We showed that the recovery of an s sparse vector $\mathbf{x}^\#$ from measurements $\mathbf{y} = A\mathbf{x}^\# + \mathbf{e}$ where A is a sensing matrix obtained by randomly subsampling a Fourier matrix and $\mathbf{e}$ is some noise vector can be done in exactly s iterations. This nonuniform recovery comes at a cost of adding some restrictions such as an exponentially fast decay of the non zero elements of the sought-after vector. This is obviously a strong assumption, but empirical results suggest that it should be possible to lift the shape restriction and obtain similar results for general shaped vectors but the proof of this hypothesis still evades us to this day.

This original iterative approach HTP was extended to make it greedy and speed up the computation in [27, 23]. The adaptation is done by simply replacing the s largest entries with the n largest entries (or $f(n)$ largest entries for any non decreasing function f of n which eventually reaches the target sparsity s) in Eq. (1.6). Note that the term *greedy* is a bit misleading as it implies that the set of active features only grows iteration after iteration, whereas the novel Graded HTP (coined GHTP or even f HTP for the general index selection approach) not only increases the size at each iteration but also reshuffles the index set completely. This reshuffling of the selected indices at each iteration is key for its fast (in terms of the number of iterations) convergence. This graded version of the algorithm enjoys similar convergence properties as the original, fixed support size, algorithm. However, the choice of the index selection function is important since an active set growing too large too fast means that the Restricted Isometry Conditions, useful for ensuring the convergence of the algorithm, is hardly satisfied. Finding an adaptive set size function is still an open question.

Finally, this work on greedy approaches was brought back, albeit with a little twist of information theory, to the realm of metagenomics. A first review article of general sparse optimization and machine learning approaches to metagenomic data survey was written [33].

Besides, a forward greedy selection method for feature selection was developed in the context of metagenomic data [50]. The proposed algorithm had the advantage of being careful when adding a feature to limit redundancy. This redundancy in features is something of interest in a broader aspect.

Some signals are simply too big to be tractable by classical algorithms. Or some signals might just not be sparse enough for compressed sensing to work with a given design matrix. Similarly, some setups are such that only partial information are available at a given location (or a given time) as is the case in SAR images¹⁴, and the whole signal can only be reconstructed when all the pieces are eventually available and *glued*¹⁵ together. Such signals are often represented with a fusion frame.

Sparse approximation can be done in this mathematical framework via compressed sensing when a vector lies in a fusion frame structure [1, 2]. Here we are interested in splitting an input vector into (potentially overlapping)

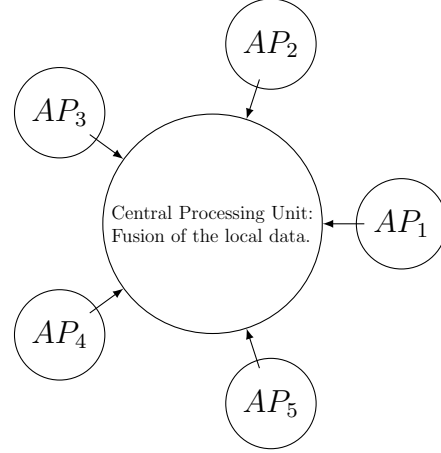


Figure 1.4: In the proposed fusion frames approach, information is measured locally, in each subspace, and then the local pieces are fused together by the centralized processing unit. This allows for larger complexities of the signals, or lower complexity of the sensor design. A represents the sensing matrix and the P_i 's are projection onto subspaces.

smaller subspaces, each one described with its own frame. It allows the treatment of very large vector sizes and the recovery of much more complex signals $\mathbf{x}$, which would be intractable by traditional compressed sampling or other recovery strategies. The challenge here is to handle redundancies both within a frame and between frame subspaces.

The fusion frames structure can be defined as an overcomplete set of subspaces, each containing a redundant basis. More formally, given an index set I , let $\mathcal{W} := \{W_i | i \in I\}$ be a family of closed subspaces in a Hilbert space $\mathcal{H}$ (typically $\mathbb{C}^n$ or $\ell_2(\mathbb{N})$). We denote the orthogonal projections onto W_i by P_i . Then $\mathcal{W}$ is a fusion frame, if there exist two constants $C, D > 0$ such that

$$C\|f\|^2 \leq \sum_{i \in I} \|P_i(f)\|^2 \leq D\|f\|^2 \text{ for all } f \in \mathcal{H}.$$

In our papers [1, 2] we showed that recovery of a dense signal is possible using q sensing

¹⁴Synthetic Aperture Radar

¹⁵One might want to use the term *fused* instead

devices, each having a number of measurements m decreasing linearly with q : $m \asymp 1/q$. This allows to, potentially, drastically reduce the hardware costs of sensors at the price of having more sensors. This could be imposed by the physical devices or an economic choice.

Sparse approximation and parametric PDEs

Sparsity in signals and data is just one key aspect of my research. Understanding phenomenon where sparsity plays a role is another major part of the work. While I have discussed the case of metagenomics studies above, in this Section, I discuss problems arising in scientific computing. A main problem that caught my interest is the study of efficient numerical schemes for high dimensional stochastic and (high-dimensional) parametric PDEs via generalized polynomial chaos expansions [26, 31].

The problem is formalized in its most general form as trying to approximate an unknown mapping $u : \mathbf{y} \mapsto u(\cdot; \mathbf{y}) \in \mathcal{X}$. Here $u(\cdot; \mathbf{y}) \in \mathcal{X}$ is defined in a separable Banach space $\mathcal{X}$, and for a spatial variable $\mathbf{x}$ in a domain Ω . For a given parameter vector $\mathbf{y} \in U$, u is the solution to an operator equation of the form of

$$A(\mathbf{x}; \mathbf{y})u(\mathbf{x}; \mathbf{y}) = f(\mathbf{x}).$$

To be precise, one would need to define boundary conditions, a domain, a function space, ... precisely. This is the purpose of the detailed Chapter 3, specifically Section 3.3. In particular, we are interested in the case where the operator is affine with respect to its parameters

$$A(\mathbf{x}; \mathbf{y}) = A_0(\mathbf{x}) + \sum_{j \geq 1} y_j A_j(\mathbf{x}). \quad (1.8)$$

Such models typically arise in stochastic PDEs, where the operator A may represent, for instance, any stochastic elliptic or parabolic equation. In this case, a Karhunen-Loève decomposition yields an infinite but countable space U and an expansion of the form (1.8). For the sake of simplicity I limit the exposition to this affine operator case but other more general models, such as a holomorphic dependence of the solution map on the parameters, may work – See for instance the Master’s thesis I supervised [36]. From a functional point of view, I assume that the operator $A : \mathcal{L}(\mathcal{X}, \mathcal{Y}')$ (for $\mathcal{X}$ and $\mathcal{Y}$ two separable metric spaces and where $\mathcal{Y}'$ defines the topological dual of $\mathcal{Y}$).

In Eq. (1.8), A_0 usually describes a *mean field* while the following operators $A_j, j \geq 1$, represent *fluctuations*. As a first step, I describe the problem of recovering a *Quantity of*

Interest of the full solution $u(\cdot; \mathbf{y}) \in \mathcal{X}$, $F(\mathbf{y}) := \mathcal{G}(u(\cdot; \mathbf{y}))$, $\mathcal{G} \in \mathcal{X}'$ being a given bounded linear functional. One can think for instance of the average flux through a membrane or an average temperature in a body.

To approximate the solution map, the use of *generalized polynomial chaos (GPC)* on a tensorized Chebyshev basis helps

$$F(\mathbf{y}) = \sum_{\nu \in \mathcal{F}} F_\nu T_\nu(\mathbf{y}),$$

where $\mathcal{F} := \{\nu \in \mathbb{N}_0^N : |\text{supp}(\nu)| < \infty\}$ represents the set of multi-indices with finite support.

Under some strong ellipticity assumptions, one can show that the sequence of Chebyshev coefficients is compressible, i.e. for some $0 < p \leq 1$, $\|(F_\nu)_{\nu \in \mathcal{F}}\|_p < \infty$. Note that this p summability for a $p \leq 1$ describes the compressibility of a signal or vector akin to what was described in the previous section regarding the ℓ_1 norm. Indeed the unit ball of ℓ_p norms for $p \leq 1$ (again, they are not norms unless $p = 1$) are curvy stars with kinks along the canonical axis. The closer the p goes to 0 the closer it gets to the counting ℓ_0 norm – a graphical representation of this phenomenon is given in Fig. 3.1. One way to quantify what *compressibility* means, is to look at Stechkin type approximation results, which we detail in Chapter 3.

In this context, I am interested in numerical approximations of the solution map $\mathbf{y} \mapsto F(\mathbf{y})$ using a polynomial approximation. For this, given m snapshots of solutions $b_i = F(\mathbf{y}^{(i)})$ for $1 \leq i \leq m$ and $(\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(m)})$, m wisely chosen sampling points in U , the goal is to find a sparse solution to the following linear system, reminiscent of compressed sensing

$$b = \Phi F, \quad b = (F(\mathbf{y}^{(i)}))_{1 \leq i \leq m}, \quad F = (F_\nu)_\nu, \quad \Phi = (T_\nu(\mathbf{y}^{(i)}))_{1 \leq i \leq m, \nu \in \mathcal{F}}.$$

Here noise cannot be avoided. At the very least it should be considered in order to account for numerical computations, floating point arithmetic, and certain discretizations. This will be made clear in the detailed chapter below.

Using the framework of weighted compressed sensing [104], it can be shown that the s -sparse approximation $\tilde{F}$ is close to the true solution F in the (Bochner) L^2 and L^∞ norms [103, 26]:

$$\|F - \tilde{F}\|_{L^2} \leq C \left(\frac{\log^2(m) \log(N)}{m} \right)^{1/p-1/2}, \quad \|F - \tilde{F}\|_{L^\infty} \leq C \left(\frac{\log^2(m) \log(N)}{m} \right)^{1/p-1}, \quad (1.9)$$

where the number of snapshots scales as $m \asymp s \log^2(s) \log(N)$ and are computed to a *good enough* accuracy. This number of samples is almost optimal [9].

Each sample involving the accurate numerical estimation of a (potentially) complicated PDE, we have introduced an accelerated framework based on a multi-level decomposition. The intuition of the multi-level approach is to first compute a coarse solution, using many cheap

solutions to the differential equations. The solution is then improved by adding finer details, but estimated from fewer and fewer (increasingly computationally expensive) samples. For a goal accuracy of h_L^{t+r} , where t and r are regularity parameters for the data and the functional $\mathcal{G}$, respectively, and h_L is a discretization parameter, we consider a sequence of meshes with discretization parameters $\{h_l\}_{l=0}^L$. The solution $F(\mathbf{y})$ is then approximated by a telescopic sum

$$F(\mathbf{y}) \approx F^L(\mathbf{y}) := \sum_{l=1}^L F^l(\mathbf{y}) - F^{l-1}(\mathbf{y}),$$

where $F^l(\mathbf{y})$ is the numerical solution associated to the mesh with parameter h_l and we define $F^0 \equiv 0$.

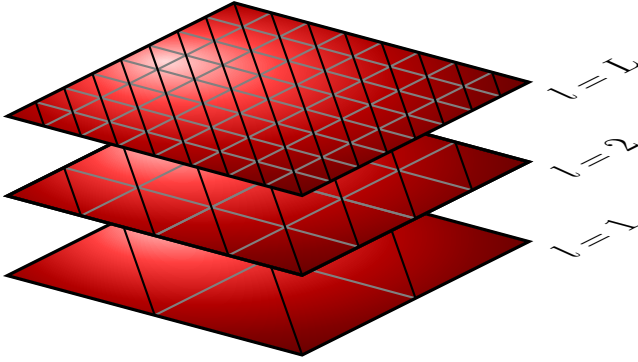


Figure 1.5: In the proposed multi-level approach, most of the calculations are spent on a cheap, coarse grid ($l = 1$ in the Figure). As we refine the model (l growing), we reduce exponentially the number of function evaluations, yet preserving the overall uniform approximation.

$2^{(L-l)(t+t')}$ samples per level, we deliver a solution that is accurate to the level of the finest grid in a total work comparable to the computational complexity of a **single evaluation** at the finest level.

Our novel *Multi-level Compressed Sensing Petrov-Galerkin (MLC-SPG)* [31] method¹⁶ works by approximating every difference between levels (so-called *details* of the solution) by means of weighted ℓ_1 -minimization. We proved that, given L consecutive levels of approximation, it is possible to compute a numerical approximation of the parametric map $\mathbf{y} \mapsto u(\cdot; \mathbf{y})$ with a computational complexity scaling as $\mathcal{O}(2^{nL})$, where n is the dimension of the spatial coordinate $\mathbf{y} \in \mathcal{X}$ of the phenomenon under scrutiny (typically, in the computational sciences, $n \in \{1, 2, 3\}$). As a consequence, computing $m_l \asymp$

Algorithmic differentiation

¹⁶This work comes together with a python implementation: <https://github.com/jlbouchot/CSPDEs>. In general, all algorithms are available for use from my homepage, some in Matlab, some in Python.

Since the beginning of 2024, as an advanced researcher at the Inria Centre of Université Côte d’Azur, I have investigated open problems in algorithmic differentiation (AD) and contributed to the development and the support of the open source software **Tapenade**. Differentiating functions is ubiquitous to all areas of research I have carried out in the past, shall it be in the context of signal alignment, or sparse approximation, or even in the industry, dealing with real time and/or massive data. From a larger point of view, differentiating (via AD or by other means) is a prerequisite for any type of optimization.

Since joining Inria I have worked on two main problems related to algorithmic and automatic differentiation

1. What are good strategies for checkpointing call-graphs in adjoint differentiation?
2. How can source transformation adjoint code be generated for the Julia language?

Regarding the first problem, optimally checkpointing calls on a code is a highly non trivial task, which, in fact, is known to be NP-Complete [97]. But first let me review some basics in AD. The idea behind **adjoint** algorithmic differentiation is to have two passes on a piece of code: a forward sweep, computing the output values from the input, and a backward sweep, somewhat akin to what the machine learning community now calls backpropagation, responsible for propagating / accumulating derivative information. In particular, in order to propagate the variations from the outputs back to the inputs, one needs to make sure that computed variables are available and kept with the appropriate values during the backward sweep, i.e.

the values they had when they were used in the forward sweep. One strategy is to save on a stack, during the forward sweep, all values which are needed but overwritten at a later time in the code. These values are then popped whenever they are needed for the backward sweep. This usually entails a prohibitive cost in memory and one may choose to only save the variables useful to recompute a whole chunk of code. This process is called *checkpointing* and in the graph on Fig. 1.6, a sample piece of code C , surrounded by an upstream chunk U and a downstream chunk D is being checkpointed. Both U and D are considered not checkpointed in this representation. In this scenario, C is being computed twice. Before its first computation, a snapshot of the current state of the system (containing various useful variables) is taken and stored in memory, as illustrated by the black dot on Fig. 1.6. During the first sweep of

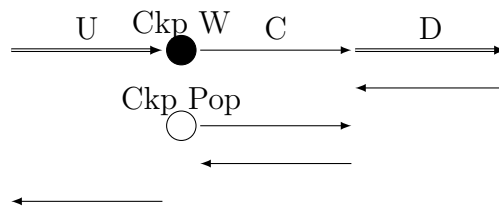


Figure 1.6: Illustration of checkpointing. The upstream section (U) and downstream (D) are not checkpointed and the double array symbolizes the need to save necessary variable in order to compute the backward sweep (single array to the left). C is a checkpointed chunk. Its forward sweep is computed twice, albeit without further stacking *local* variables. The checkpoint (**Ckp**) saves all useful variables stemming from U required to recompute C . These variables are popped (*CkpPop*) from the stack prior to the second forward sweep.

$\mathcal{C}$, variables are computed in a memoryless fashion, where only the variables which are both computed in $\mathcal{C}$ and useful to the downstream $\mathcal{D}$ are computed. Before $\mathcal{C}$ is run a second time, the previous state of the system is loaded (popped from the stack), which is illustrated with the white dot. During the second run, both forward and backward sweeps of $\mathcal{C}$ are done.

Usually stacking variables on the fly is more memory intensive than recomputing useful variables on the fly. Recomputation obviously increases the computing time and balancing what is worth recomputing against what should be saved is crucial to generating efficient adjoint codes, especially in large codes used in scientific computing.

In [67] we presented a strategy for empirically and heuristically suggesting adequate checkpointing locations. By analyzing the call graph at runtime, we are able to estimate the memory cost and time gained if certain calls were to be checkpointed. It was verified empirically that choosing a strategy such as *memory efficiency* first or *compute efficiency* first roughly approaches what may be seen as a Pareto front where compute time and memory usage are used as criteria. It is important to notice that the numerical experiments have been launched on a piece of code with over 80 candidate calls for checkpointing, all of which may have a True/False state, hence yielding over 2^{80} possibilities for checkpointing strategies. Our heuristics is fairly easy to use, yet deliver great results, achieving almost $3\times$ speed-up of the adjoint computation with very limited memory overhead when compared to the default *checkpoint all* strategy¹⁷. This is illustrated on Fig. 4.6 on page 97. For the sake of comparison, random samples from the set of potential checkpointing strategies have been added in light gray. It is worth mentioning that most of the checkpointing suggested by our profiling tool are better (closer to the lower left corner) than random checkpointing strategies.

Regarding the second aspect of my current activities in the world of algorithmic differentiation, I am interested in differentiating Julia code. This is a recent direction of my research and results are still preliminary. Julia however exposes some interesting aspects into the theoretical foundations of AD. For instance, it is an untyped language making it challenging for source transformation tools to decide what is differentiable (e.g. applied to floats) from what is not (e.g. applied to integers). In an operator overloading differentiation process, this can be decided at run time, but source transformation does not, a priori, allow this. This leads to certain challenges in the way data are represented within the code analysis. The versatility of the data structures makes it also an interesting language combining many interesting features of modern languages such as meta programming and garbage collection, a very active topic in source transformation AD [66]. Garbage collection is precisely an active area of research and resolving this (together with object oriented programming) would surely go a long way in applying source transformation for adjoint computation on many more contemporary languages.

¹⁷The *checkpoint-all* strategy is the baseline as it is the one which has the least memory usage, making sure to avoid any out-of-memory issues.

Outline

The rest of this manuscript discusses the previous sections in deeper (and more technical!) details. Chapter 2 is dedicated to the aftermath¹⁸ of my Ph.D. thesis and deals with structural similarities and the important role that the discrepancy norm can play. It details some work which were initiated as I was still a student and only matured enough for publication some months / years after my defense. It concludes with some opportunity to continue some research with the new insight expertise gained from later experiences. Chapter 3 reviews my work in the area of compressed sensing and its applications, mainly in the approximation of parametric PDEs and distributed sensing. Chapter 4 introduces my new playground: the realm of algorithmic differentiation.

Given the multidisciplinary of my research and the combination of academic and industrial experiences I had, I had to make some choices regarding what to present in this *Habilitation à diriger des recherches*. In particular, albeit it was a motivation for studying sparse representation, I do not consider an expert in metagenomics data survey analysis and have chosen not to spend much time on this topic here. I would like however to cite the two papers in this area [33, 50]. An interesting aspect of this subject is now the impact of metagenomics data analysis in machine learning and finding relevant features for certain traits. Moreover, studies in this field often lack data which are missing not at random in the best case and one would benefit from recent results on the subject. In the worst case, the data have been falsely entered and finding such false entries is also a question of interest.

In the next chapter, I describe the analysis of similarity measures from structures in images and data. It is worth pointing out that as I was a Ph.D. student at the Fuzzy Logic Lab. Linz, I was involved in working with Edwin Lughofer on evolving systems [86] and how to quantify redundancies in clusters. This had implicitly a measure of structural closeness involved.

Another structure based mathematical image processing tool which I do not detail in this thesis is the concept of monogenic signals. Roughly speaking, this comes from analyzing local instantaneous frequencies in images by means of Cauchy principal values [17]. This is important for quantifying the local structures in an image. The motivation for this work started as we were analyzing structures in polymer materials by means of local dissimilarity maps [110, 81]. It then drifted a bit.

In the world of data approximation, radial basis functions (RBFs) play a major role. Similarly, they tend to highlight local structures by allowing to tailor the size of the kernel used in the approximation, thereby allowing to discriminate structures locally. Together with Keaton Hamm, we have analyzed the importance of noise and jitter in RBF interpolation schemes [28].

¹⁸This is a lucky name in a mathsy manuscript which the English language allows!

However much interest and scientific curiosity I had the opportunity to develop during my industrial appointments at Artelys and Continental, I prefer not to discuss these topics here. I would simply like to add two main points on this industrial experience here. The first point is about the scientific developments done at both companies. As an employee at Continental I have developed algorithms for sensor fusions and camera calibration which are both optimization problems which could benefit from my recent work on AD (see for instance this paper, which deals with the subject [109] – this was however not a novel idea! [100]). At Artelys, I was part of a European wide H2020 project, heavy on research. In particular, as part of the project, we were given the opportunity to contribute to some policy making analysis, using our novel tools [39]. The second important point of my industrial experience is the amount of supervision and project management. Although these are not academic supervisions in a strict sense, I had the responsibility of the technical supervision of teams of R & D engineers and doctors degrees in optimization or signal processing. This was done in parallel with project deadlines imposed by customers or stakeholders, not unlike what is expected from the supervision of doctoral candidates! This was done

Finally Chapter 5 glues all the pieces together, circling back to some problems in computer vision and details some research ideas for the future.

Throughout the manuscript, some ideas for future research are given. They are highlighted with a gray box. Some of them are simple constructions from previous research and may be given to early stage researchers. Some others however may keep a whole team busy for some time. Either way, it is by far what will keep me active in the near future!

In the remainder of this manuscript, bold face characters such as $\mathbf{x}$ and $\mathbf{t}$ will denote vectors in $\mathbb{R}^N$ or $\mathbb{C}^N$. $\mathbb{K}$ will denote $\mathbb{R}$ or $\mathbb{C}$. Script characters will denote either spatial variables of dimension n , such as $\mathbf{x}$, or parameters, such as $\mathbf{y}$. Matrices and linear operators will be denoted by capital letters A , unless stated otherwise. m will typically denote a number of measurements, and I often talk about $A \in \mathbb{K}^{m \times N}$ as the design or sensing or measurement matrix. L will denote the set of Lebesgue-measurable mappings while $\mathcal{L}$ usually denotes the set of linear operators. Superscripted parenthesis will be used to denote either sequences of variables in an iterative algorithm, for instance in $\mathbf{x}^{(k)}$, or a certain random sample, such as $\mathbf{y}^{(k)}$. Superscripted $\#$ denotes the ground truth while superscripted $*$ denotes an approximation. As a consequence, I will often quantify the approximations by looking at $\|\mathbf{x}^\# - \mathbf{x}^*\|$ for some norm.

Chapter 2

Structure-based image processing: discrepancy norm and distance transforms

2.1 General background

In the context of image processing and throughout this Section, I consider the problem of image registration, which consists of aligning images as best as we can. To set the background, let me recall the concept of *autocorrelation* functions, introduced implicitly in Def. 1.1 in the introduction. Throughout this section, μ denotes a finite positive measure.

Definition 2.1: Autocorrelation functions

Let $d : L(\mathbb{R}^n, \mu) \times L(\mathbb{R}^n, \mu) \rightarrow \mathbb{R}^+$ denote any sort of discrepancy measure defined for pairs of μ measurable functions. Let $T_{\mathbf{t}}$ define the translation operator with parameter $\mathbf{t}$ defined as $T_{\mathbf{t}}(f)(\mathbf{x}) = f(\mathbf{x} - \mathbf{t})$. The autocorrelation operator $a_d[f]$ is defined as

$$a_d[f](\mathbf{t}) = d(f, T_{\mathbf{t}}(f)), \forall \mathbf{t} \in \mathbb{R}^n \quad (2.1)$$

Within the definition, I assume that d , a way to measure (dis-)similarity between signals or functions, is given. Typical choices are among classical Minkowski distances or f -divergences. Later I will discuss why the discrepancy or scaled-distance transforms might be better suited

than these examples in case of misalignments. The current exposition is formulated for signals defined on the real line or on Cartesian products of $\mathbb{R}$ in case of images and volumetric data. It is important to be careful when dealing with repetitive, periodic, patterns which would therefore be defined on a torus. In particular the definition of monotonicity (given as Def. 1.1 on page 5) in its current form cannot apply to periodic functions and some care should be taken to work with such problems. I will not develop any further on this specific subject here.

In general, the image or signal one wants to align is not quite the original image f but a tempered version $g = \mathcal{A}(f)$ for some unknown operator $\mathcal{A}$. $\mathcal{A}$ might represent any operator such as a shift, a shearing, a scaling, a rotation, a combination of them or anything else which returns yet another *signal*. In the case where $\mathcal{A}$ is a translation operator, we are no longer interested in the autocorrelation function but solely the d -based correlation:

Definition 2.2: Correlation functions

Let $d : L(\mathbb{R}^n, \mu) \times L(\mathbb{R}^n, \mu) \rightarrow \mathbb{R}^+$ denote any sort of discrepancy measure defined for pairs of n -dimensional μ measurable functions. Let $f \in L(\mathbb{R}^n, \mu)$ be a reference pattern and let $T_{\mathbf{t}}$ define the translation operator with parameter $\mathbf{t} \in \mathbb{R}^n$. The correlation operator $C_d[f]$ is such that, for any function $g \in L(\mathbb{R}^n, \mu)$,

$$C_d[f](g) : \begin{cases} \mathbb{R}^n & \rightarrow \mathbb{R} \\ \mathbf{t} & \mapsto d(f, T_{\mathbf{t}}(g)) \end{cases} \quad (2.2)$$

This correlation function may be extended to general operator $\mathcal{A}$ defined for a certain parameter vector θ . This is not the purpose of the current exposition.

The challenge of aligning two signals (also often called image registration), is to find an appropriate $\mathbf{t}^*$ such that the correlation function with a reference signal f is minimal, or as close to the minimal as can be. I.e. given a certain measurement $g = \mathcal{A}(f)$, one wishes to solve

$$\text{Find } \mathbf{t}^* \in \mathbb{R}^n \quad \text{such that } \mathbf{t}^* \in \operatorname{argmin} C_d[f](g)(\mathbf{t}). \quad (2.3)$$

In case the perturbation operator $\mathcal{A}$ is a translation operator itself with parameter $\mathbf{t}^\#$, then the correlation function is the translation with parameter $\mathbf{t}^\#$ of the autocorrelation function, i.e.

$$C_d[f](T_{\mathbf{t}^\#}(f)) = T_{\mathbf{t}^\#}(a_d(f)). \quad (2.4)$$

Moreover, the monotonicity of the autocorrelation function plays a major role in developing algorithms in this case, as it allows descent algorithms to find the unique optimal translation parameter $\mathbf{t}^* = \mathbf{t}^\#$ (or approximates it in case of noisy measurements).

Many approaches have been designed to solve the alignment problem. They may be categorized as follows:

- Feature based: one find interesting or salient features in a query image and tries to

match them within a target image. One may use any sort of optimization algorithms and feature detection and description.

- Pixel-based: this is a little bit outdated and lacks robustness to many image distortions, occlusions, view point changes, ...
- Structure-based: one tries to compare structures inherent to a signal and match these structures. The discrepancy norm introduced later falls in this category.

It is worth mentioning that nowadays deep learning is assumed to be the leader in anything related to image processing, but I would argue that this is yet another feature-based approach. Indeed, repeated convolutions layers simply give access to increasingly complex features. These features are then used in a decision process, typically modeled with a set of fully connected layers, but any other decision making algorithms would work too.

My Ph.D. started with the observation that Hermann Weyl's discrepancy concept may be applied to volumetric data[91, 122]. Before digging in the details, let me recall some background on Weyl's discrepancy. Originally, in [122], Weyl introduced the discrepancy to quantify the randomness of sequences when compared to a uniform distribution.

Definition 2.3: Weyl's discrepancy

Let $S_N = \alpha_1, \dots, \alpha_N$ be a sequence of N random numbers in the interval $[0, 1]$. Weyl's discrepancy D_N is defined as

$$D_N = \sup_{0 \leq a \leq b \leq 1} \left| \frac{|S_N \cap [a, b]|}{N} - (b - a) \right| \quad (2.5)$$

The interesting point of this definition is the following

$$\lim_{N \rightarrow \infty} D_N = 0 \Leftrightarrow S_N \text{ is uniformly distributed.}$$

Almost a century later, Moser [91], based on previous work in image processing [13], carried a thorough analysis of the novel discrepancy concept, which he coined the discrepancy norm and defined it on integral images as

Definition 2.4: Modern discrepancy for volumetric data

Let $n \in \mathbb{N}$ and $f \in L(\mathbb{R}^n, \mu)$ be a μ measurable function.

$$\|f\|_D = \max_{\iota \in \{-1,1\}^d} D^{(\iota)}(f) \quad (2.6)$$

where

$$D^{(\iota)}(f) = \left(\sup_{\mathbf{s} \in \mathbb{R}^d} I_f^\iota(\mathbf{s}) - \inf_{\mathbf{s} \in \mathbb{R}^d} I_f^\iota(\mathbf{s}) \right) \text{ and} \quad (2.7)$$

$$I_f^\iota(\mathbf{s}) = \int_{T_{\mathbf{s}}(\mathcal{Q}_\iota)} f d\mu. \quad (2.8)$$

with

$$\mathcal{Q}_\iota = \{(\lambda_1 \iota_1, \lambda_2 \iota_2, \dots, \lambda_n \iota_n)^T, \lambda_i \geq 0, 1 \leq i \leq n\}$$

and

$$T_{\mathbf{s}}(A) = \{\mathbf{x} - \mathbf{s}, \mathbf{x} \in A\}.$$

In particular, for $n = 2$, the four $\mathcal{Q}$'s define the four integral images in the two-dimensional plane, facilitating the implementation of the discrepancy norm. This is not the only choice for extension of the discrepancy norm to high-dimensional data, and I encourage the reader to read the paper by Moser [91] to learn more. He also proved the following monotonicity property with respect to shifts.

Proposition 2.1: Monotonicity of the discrepancy norm's misalignment

Let $n \in \mathbb{N}$ and $f \in L(\mathbb{R}^n, \mu)$, $f \geq 0$, a positive μ measurable function. Then

$$\int f d\mu > 0 \Rightarrow a_{\|\cdot\|_D}[f](\mathbf{t}) = 0 \Leftrightarrow \mathbf{t} = 0, \text{ and} \quad (2.9)$$

$$\forall \mathbf{t} \in \mathbb{R}^n, \forall 0 \leq \lambda_1 \leq \lambda_2, a_{\|\cdot\|_D}[f](\lambda_1 \mathbf{t}) \leq a_{\|\cdot\|_D}[f](\lambda_2 \mathbf{t}). \quad (2.10)$$

Consequently, within my Ph.D. I studied both some theoretical problems and some practical ones around this norm, and continued working on this subject for a short time after my Ph.D. defense. I briefly review these aspects of my past research, with some ideas on how to revive the subject on my side.

Open question 2.1: Space of low-discrepancy functions

From a functional analytic point of view, spaces of functions with low discrepancy are fairly unknown. If we compare to Besov or Sobolev spaces which are known to include curvature information, all that is known about the space of low-discrepancy function is that it is related to Alexiewicz spaces [94]. Understanding the specifics of such a topology would go a long way in adapting algorithms based on the discrepancy norm in the future.

Similarly, can we relax the link between Alexiewicz’s topology and the discrepancy norm to describe a family of spaces based on the set of monotonicity-satisfying metrics? (such as those described in the later sections)

2.2 Some algorithmic concerns for industrial automatic quality control

With all the background in mind, it is just a small gap from the basic idea of the discrepancy concept to the problem of finding defect in regularly textured surfaces.

Here, f denotes a pseudo-periodic input image, and $A_{i,j}$ relates to the patch of size $m \times n$ centered at pixel (i, j) of the query image f . g denotes a defect-free *golden template* used as reference pattern which is assumed to contain more than a single period of the (pseudo-)periodic pattern. The defectiveness at a pixel (i, j) is determined by finding the best potential location for $A_{i,j}$ within the golden template. It is done by solving the following optimization problem

$$\alpha(i, j) = \min_{\tilde{g} \in \mathcal{G}} \|A_{i,j} - \tilde{g}\|_D, \quad (2.11)$$

where $\mathcal{G}$ represents all possible $m \times n$ sub-windows of the reference pattern g . If the match is *good enough*, i.e. $\alpha(i, j)$ is small enough, we consider the local patch to be defect-free. Of course *good enough* always depends on the applications and we do not dare giving explicit guidelines on this matter.

The process was first introduced as OptLDMap in [32], see Algorithm 1. The optimization was done using a BFGS method with some pre-optimization steps done with a fixed number of DIRECT iterations [78, 107]. A speed improvement was suggested using a coarse-to-fine analysis in which larger areas are first registered and only in the case where the dissimilarity is too big (i.e. larger than a prescribed threshold) would the algorithm divide the search sizes

Algorithm 1 Basic OptLDMap

```
1: function OPTLDMap(referenceImage, defectImage)
2:   input reference image
3:   for  $i = 0, n, 2n, \dots$  do
4:     for  $j = 0, m, 2m, \dots$  do
5:       Extract  $m \times n$  patch  $A_{i,j}$  starting at  $(x, y)$  of defectImage
6:       Solve optimization problem (2.11)
7:       Save resulting dissimilarity for current patch into resultImage
8:     end for
9:   end for
10:  binarize resultImage
11:  return resultImage
12: end function.
```

by two. This allowed to drastically reduce the computation times. The coarse-to-fine design rejection method is nicely illustrated in the second column of Fig. 2.1. Indeed, one sees large blocks where the decision was made to reject an area as defective while other smaller blocks have been reduced in size until labeling them, very locally, as faulty.

Although the coarse-to-fine approach helped in reducing the compute time, it was still far from being usable in industrial processes. In [111], we introduced some improvements related to

1. the sliding window size: we used the approach of [83]
2. the compute time, which we adapted via statistical discrimination: if we can find starting points of the optimization which are close enough to being well aligned, then there is no need for further investigations. This allowed to reduce the compute time from minutes to fractions of seconds.

Some final results are further illustrated in Fig. 2.2.

On the track of working out optimization problems based on the discrepancy norm, I have also investigated differentiable approximation schemes. This is the purpose of the paper [25] for the general case of continuous multivariate functions. It is based on some exploratory work [29] which evaluated such schemes on times series. This approximation is based upon generalized means.

Definition 2.5: Generalized means

Let $\phi : \mathbb{R} \rightarrow \mathbb{R}$ be continuous and injective, its generalized mean over f is defined as

$$M_\phi(f) = \phi^{-1} \left(\frac{\int_{\mathbb{R}^n} \phi(f(\mathbf{x})) d\mu(\mathbf{x})}{\int_{\mathbb{R}^n} d\mu} \right)$$

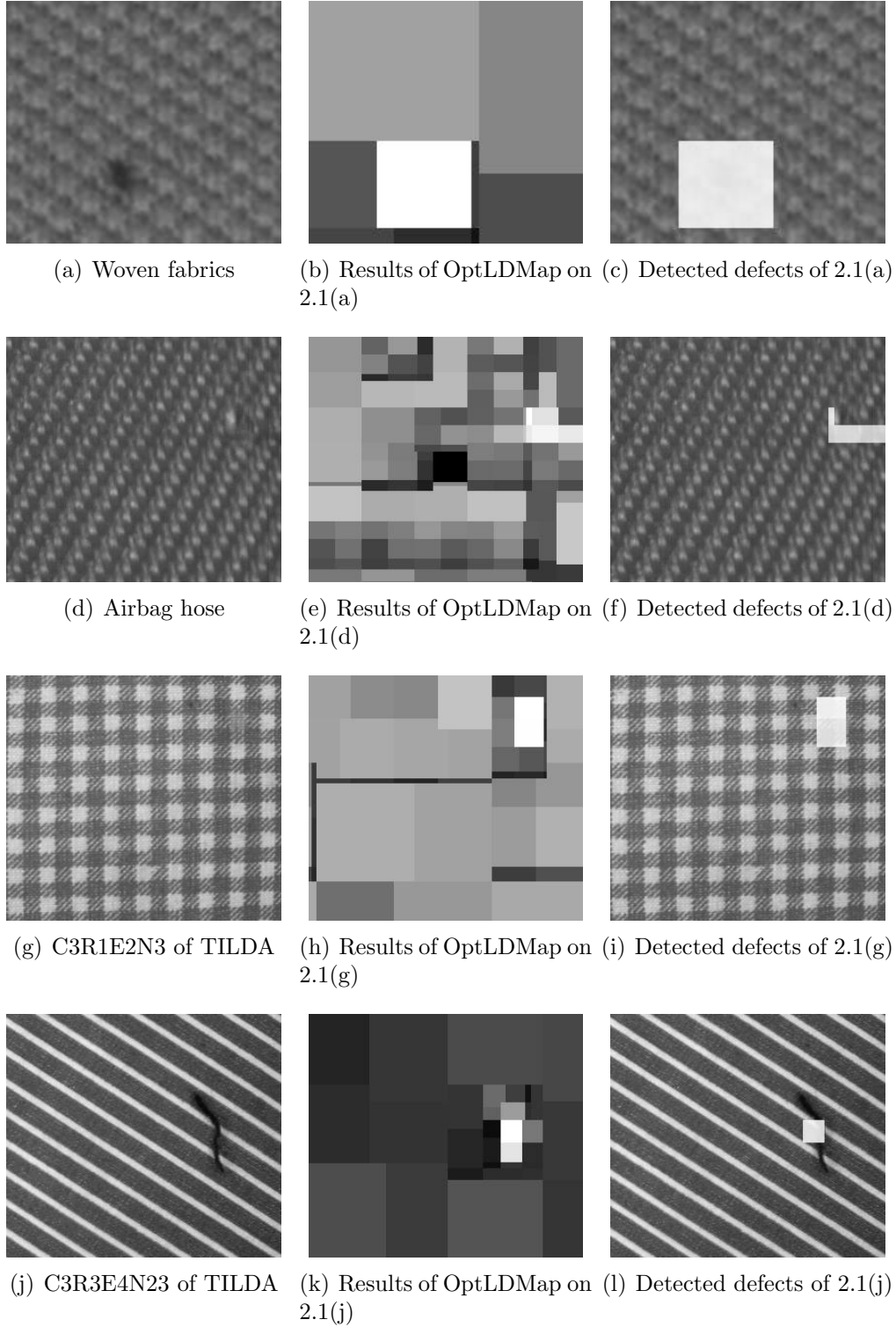


Figure 2.1: Examples of OptLDMap runs on images of our customers a to f and test images of the TILDA database [105]. The first column is always the input image, the second the defectiveness result of OptLDMap before thresholding and the last column an overlay between input image and the thresholded result. The window sizes are 30×20 for the airbag hose, 25×25 for the woven fabrics and 40×40 for the others. Threshold is fixed to 0.9.

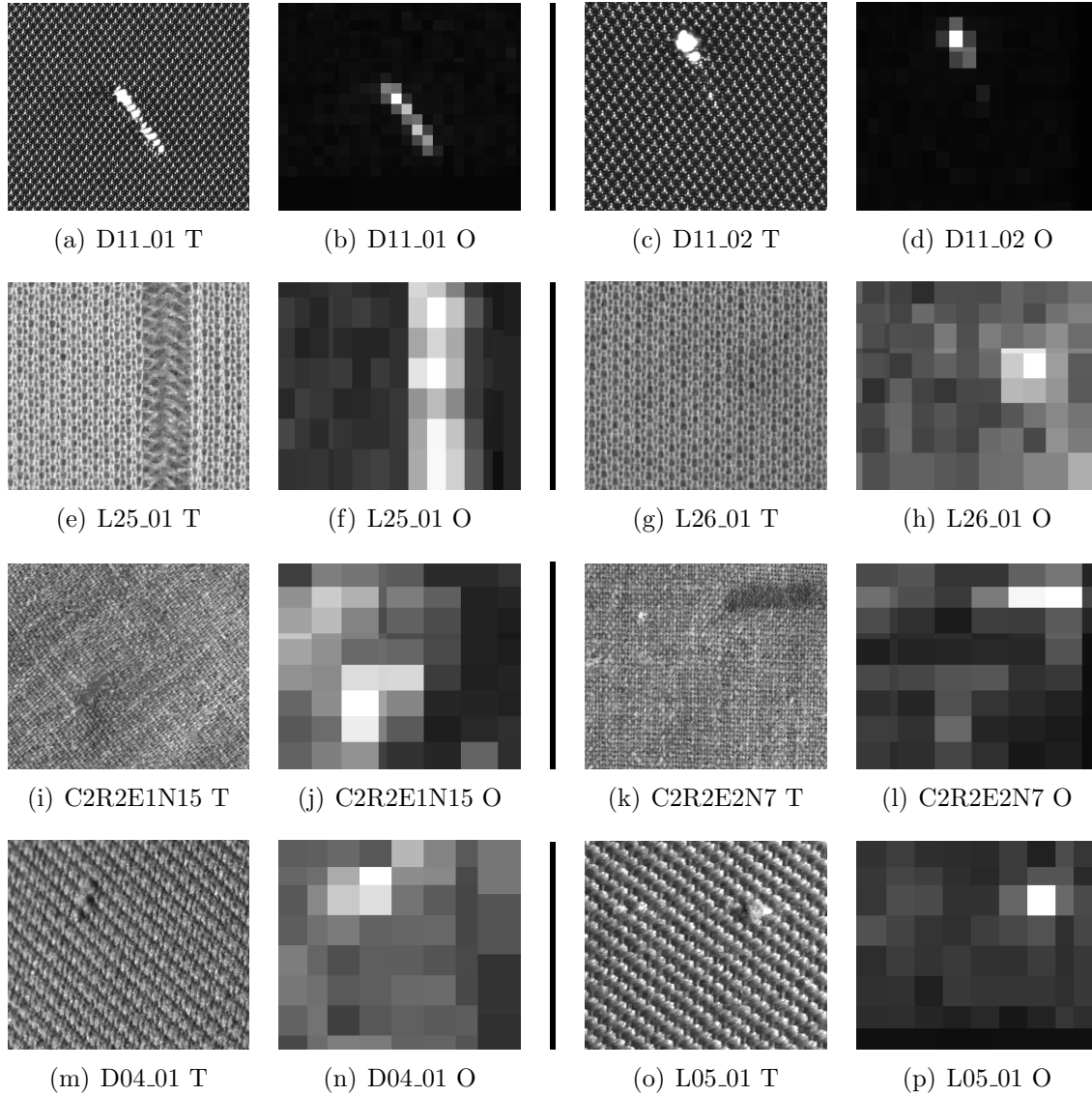


Figure 2.2: Examples of the applicability test on textile defect images. For each example the test image (T) as well as the output (O) of the algorithm is shown. Reference images are not shown. The algorithm output is for illustration purposes not thresholded.

The following bounding property holds $\inf_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \leq M_\phi(f) \leq \sup_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x})$ (where $\inf$ and $\sup$ are understood as their essential definitions: $\text{ess. sup}(f) = \inf \{C \in \mathbb{R} : \mu\{\mathbf{x} : f(\mathbf{x}) > C\} = 0\}$). For any $p \in \mathbb{R}$, p -means M_p are defined using $\phi(x) = x^p$.

Before I introduce the proposed approximation in the general case, I start with the univariate case. Let me first define χ as the exponential of the cumulative function, i.e.:

$$\chi : \begin{cases} L(\mathbb{R}, \mu) & \rightarrow L(\mathbb{R}, \mu) \\ f & \mapsto \chi(f) : \chi(f)(\mathbf{x}) = e^{\int_{-\infty}^x f(t) d\mu(t)} \end{cases}$$

The approximation then reads

$$\Gamma_p : \begin{cases} L(\mathbb{R}, \mu) & \rightarrow \mathbb{R}^+ \\ f & \mapsto \Gamma_p(f) := \ln \left(\frac{M_p(\chi(f))}{M_{-p}(\chi(f))} \right) \end{cases}$$

Luckily, we could prove [25] that this approximation can be made as good as one needs.

Theorem 2.1: Approximation of the discrepancy norm for 1D continuous functions

Γ_p defines an approximation of the discrepancy norm in the sense that

- i) $\forall f \in L(\mathbb{R}, \mu), \Gamma_p(f) \xrightarrow{p \rightarrow \infty} \|f\|_D$
- ii) $\Gamma_p(f) \leq \|f\|_D$
- iii) $\forall \varepsilon > 0, \exists p^* : \forall p \in \mathbb{R}, p \geq p^* \Rightarrow |\Gamma_p(f) - \|f\|_D| \leq \varepsilon$

In terms of optimization, it comes with a closed form formula for its derivative. But more importantly, such a formulation allows for faster processing times when dealing with finding the optimal translation parameters. Indeed, it can be expressed as a convolution operator.

Theorem 2.2: Misalignment with convolutions

Let f and g be two measurable functions in the finite σ -algebra, the correlation function h can be computed by means of two convolution operators (up to a normalization factor):

$$h = \frac{1}{p} \ln (\chi_p(f) * \chi_p(-g)) + \frac{1}{p} \ln (\chi_p(g) * \chi_p(-f))$$

where $\chi_p(\mathbf{x}) = e^{p \int_{-\infty}^x f(y) d\mu(y)}$

The generalization of the approximation scheme to multivariate functions works via a two-level p -norm approximation. First, the directional discrepancies (2.7) are approximated through p -norms, and then the max over the directions is approximated via yet another q -norm.

Definition 2.6: Approximation of the multivariate discrepancy norm

Let $p, q > 0$ and for all $\iota \in \{-1, +1\}^n$ let

$$\Gamma_p^{(\iota)}(f) := \ln \left(M_p(\chi^{(\iota)}(f)) \cdot M_p(\chi^{(\iota)}(-f)) \right) = \ln \left(\frac{\|\chi^{(\iota)}(f)\|_p}{\mu(\mathbb{R}^d)^{1/p}} \cdot \frac{\|\chi^{(\iota)}(-f)\|_p}{\mu(\mathbb{R}^d)^{1/p}} \right)$$

with

$$\chi^{(\iota)}(f)(\mathbf{x}) = e^{\iota \cdot \int_{[\mathbf{x}]} f(s) d\mu s},$$

being an approximation to the directional discrepancy (2.7) and define the vector of directional approximations $\vec{\Gamma}_p = \left[\Gamma_p^{(\iota(1))}, \dots, \Gamma_p^{(\iota(2^d))} \right]^T$. The approximation to the multivariate discrepancy norm is given as

$$\Gamma_{p,q}^{(d)}(f) := \left(\frac{\sum_{\iota \in \{-1,1\}^d} \Gamma_p^{(\iota)^q}(f)}{2^d} \right)^{1/q} = M_q(\vec{\Gamma}_p)$$

We could prove the following result

Corollary 2.1: Convergence of $\Gamma_{p,q}$

Given a function $f \in L(\mathbb{R}^d, \mathbb{R}, \mu)$, the following holds:

$$\forall \varepsilon > 0, \exists \mathbf{p}^{(d)} := (p^{(d)}, q^{(d)}) : \forall \mathbf{p} \geq \mathbf{p}^{(d)}, |\Gamma_{p,q}^{(d)}(f) - \|f\|_D| \leq \varepsilon$$

Open question 2.2: Scale-rotation-translation alignment

The importance and usefulness of the discrepancy norm as a tool for solving alignment problems in which only translations are involved is now clear. In many areas however, one needs to figure out the rotation and scaling parameters too. This subject could be of interest to an early stage researcher willing to understand more about Fourier transforms in images. Moreover, after this first step in research they could work on fast scheme for implementing full alignment (rotation + scale + translation).

One interesting question would be the robustness to noise. Given the plus and minuses involved in the definition of the discrepancy norm, it is expected that the discrepancy norm would be highly robust against Gaussian noise or salt and pepper noise. Overall, additive noises with 0 mean should be nicely suited to be used with the discrepancy norm.

2.3 Some axiomatic results against classical measures

Motivated by the monotonicity property and the fact that the classic L^2 correlation functions fails at quantifying the amount of displacement (see Fig. 1.2), we have described some basic criteria which are sufficient to ensure that the monotonicity fails. Interestingly enough, these criteria, albeit fairly general and intuitive, are sufficient to disqualify most known measures of (dis-)similarity.

In [90], we have proven the following constructive and negative result.

Theorem 2.3: Non-monotonicity of pixel based measures

The construction

$$\Delta_{[\mathcal{A}, \mathcal{S}, \mathcal{C}]}(f, g) := \mathcal{S}(\mathcal{A}_x(\mathcal{C}(f(x), g(x)))) \quad (2.12)$$

induces a function

$$\Delta_{[\mathcal{A}, \mathcal{S}, \mathcal{C}]} : \Psi \times \Psi \rightarrow \mathbb{R}$$

that does not satisfy the monotonicity condition (MC) under the assumption that Ψ is an admissible space of functions $f : \mathbb{Z} \subseteq \mathbb{R} \rightarrow \mathbb{R}$ that contains at least the set of pairwise differences of indicator functions of finite subsets of $\mathbb{Z}$, $\mathcal{C} : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ is a coordinate function that satisfies

(C1) $\mathcal{C}$ is commutative,

(C2) $\mathcal{C}(1, 0) \neq \mathcal{C}(1, 1)$,

(C3) $\mathcal{C}(0, 0) = \min\{\mathcal{C}(1, 0), \mathcal{C}(1, 1)\}$,

the aggregation function $\mathcal{A} : \mathbb{R}^n \rightarrow \mathbb{R}$ is

(A1) commutative and

(A2) strictly monotonically increasing or decreasing in each component, respectively,

and the scaling function $\mathcal{S} : \mathbb{R} \rightarrow \mathbb{R}$ is strictly monotonically increasing or decreasing in each component, respectively.

This is based on the following counter example:

$$h(\cdot) := 1_{\{0\}}(\cdot) + 1_{\{2\}}(\cdot). \quad (2.13)$$

The result is fairly general in the sense that it does not impose many conditions on the function $\mathcal{C}$ acting at the coordinate level. The other two functions are simply aggregating the results of every pixel comparison without handling any sort of structure in the data.

Note that one can combine two axiomatically constructed dissimilarity metrics via a binary operation $\odot : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ which either preserves or reverses the ordering in both arguments.

Corollary 2.2: More pixel-based counter examples

Let

$$\Delta_1[\mathcal{A}_1, \mathcal{S}_1, \mathcal{C}_1](f, g) := \mathcal{S}_1(\mathcal{A}_{1x}(\mathcal{C}_1(f(x), g(x))))$$

$$\Delta_2[\mathcal{A}_2, \mathcal{S}_2, \mathcal{C}_2](f, g) := \mathcal{S}_2(\mathcal{A}_{2x}(\mathcal{C}_2(f(x), g(x))))$$

be functions following the construction principle (2.12) Assume that $\odot : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ is an operation that is strictly monotonic of the same type in each component. Then

$$\Delta(f, g) = \Delta_1[\mathcal{A}_1, \mathcal{S}_1, \mathcal{C}_1](f, g) \odot \Delta_2[\mathcal{A}_2, \mathcal{S}_2, \mathcal{C}_2](f, g)$$

does not satisfy the monotonicity criterion.

Finally, it is worth mentioning that the criteria are general enough to disqualify any pixel based metric, see Table 2.1.

formula	name	remark
$\ f - g\ _p$	Minkowski distance	$\mathcal{C}(a, b) = a - b ^p$, $\mathcal{S}(x) = \sqrt[p]{x}$
$\langle f, g \rangle = \sum_i f_i \cdot g_i$	inner product	$\mathcal{C}(a, b) = a \cdot b$
$e^{-\frac{1}{\sigma} \sum_i (f_i - g_i)^2}$	Gaussian kernel	$\mathcal{S}(x) = \exp(-x/\sigma)$
$-\sqrt{\ f - g\ ^2 + c^2}$	multiquadratic	$\mathcal{S}(x) = -\sqrt{x + c^2}$
$\frac{1}{\sqrt{\ f - g\ ^2 + c^2}}$	inverse multiquadratic	$\mathcal{S}(x) = (\sqrt{x + c^2})^{-1}$
$\ f - g\ ^{2n} \ln(\ f - g\)$	thin plate spline	$\ln, x^n$ as scaling, $\odot(a, b) = a \cdot b$
$\langle f, g \rangle^d, d \in \mathbb{N}$	polynomial kernel	(Cor. 2.2) recursively applied, $\odot(a, b) = a \cdot b$
$(\langle f, g \rangle + c)^d, d \in \mathbb{N}$	inh. polynomial kernel	(Cor. 2.2) recursively applied, $\odot(a, b) = a \cdot b$
$\tanh(\kappa \langle x, y \rangle + \theta)$	sigmoidal kernel	$\mathcal{S}(x) = \tanh(\kappa x + \theta)$

Table 2.1: Examples of kernels and distance measures that follow the construction principles of Theorem 2.3 or Corollary 2.2 with summation as aggregation function

I invite the reader to look at [90] for further constructions which allow to disqualify many more translation invariant kernels, Bessel functions, Dirichlet kernels, splines...

Pixel-based comparisons are not the only family of metrics used for data comparison. As interesting candidates, we want to investigate the use of information theoretic measures and their behavior when facing translations. Such metrics may include for instance mutual information or Kullback Leibler divergence. We can show that such constructions cannot satisfy the monotonicity of their autocorrelation functions, as suggested by the following

Theorem from [90].

Theorem 2.4: Non monotonicity of f -divergences

Let $f : [0, \infty] \rightarrow \mathbb{R} \cup \{+\infty\}$ be a strictly convex and continuous function. For two discrete sequences $A = (a_i)_{i=1}^n \in \mathcal{V}^n$ and $B = (b_i)_{i=1}^n \in \mathcal{V}^n$, $n \in \mathbb{N}$ let

$$D_f(A\|B) = \sum_{v,w \in \mathcal{V}} P_A(v)P_B(w)f\left(\frac{P_{AB}(v,w)}{P_A(v)P_B(w)}\right) \quad (2.14)$$

where $P_{AB}(v,w)$ denotes the joint frequency of occurrence of the pair of values (v,w) , and $P_A(v)$, $P_B(w)$ denote the frequencies of v , w in the corresponding sequences A and B , respectively. Then there are sequences $h : \mathbb{Z} \rightarrow \mathcal{V}$ such that $\chi : \mathbb{N} \rightarrow [0, \infty]$ given by

$$\chi_t = D_f(A_0\|A_t)$$

does not behave monotonically with respect to t , where $A_t(\cdot) = 1_{1,\dots,n}(\cdot) \cdot h(\cdot - t)$.

2.4 Some constructive results

Following these striking results proving that most (dis-)similarity measures are not well-suited for optimization-based registration problems, we asked ourselves whether the discrepancy norm is unique in the sense that its autocorrelation function satisfies the monotonicity condition. This kept me busy towards the end of my Ph.D. and after my defense: is the monotonicity against shifts characteristics to the discrepancy norm? We could eventually answer negatively to this question in [30], when we introduced the *Scaled Distance Transforms*. The idea used here was to fuse the information of gradients at different scales with the quantification of local distances between binary patterns. Our work is based on distance transforms [8, 22], Hausdorff distances [72], and scale spaces [82].

To introduce the concept of scaled distance transforms, I recall some results on binary image comparison. Throughout, we consider two bounded open sets A and B in a metric space $(\mathbb{R}^n, d)$.

Definition 2.7: Distance transform to the foreground

$$\forall \mathbf{x} \in \mathbb{R}^n, DT_d[A](\mathbf{x}) = d(\mathbf{x}, A) = \min_{\mathbf{y} \in A} (d(\mathbf{x}, \mathbf{y})). \quad (2.15)$$

Remark 2.4.1. The output of the distance transform might be infinite for $\mathbf{x} \rightarrow \pm\infty$. We can restrict the domain of definition of the DT to a bounded domain Ω containing A .

Remark 2.4.2. This is generally called the distance to the foreground. A similar definition, with the distance transform to the background (i.e. the complement of A or B) may be defined. The duality between both definition should be clear.

From there, it is easy to define the distance transform of indicator functions. As long as these functions have bounded support we can identify them with their support and consider the distance transform of their support. We may write

$$DT_d[A] = DT_d[\chi_A] \quad (2.16)$$

where χ_A represents the indicator function of the set A . Comparing indicator functions can also be done with the Hausdorff metric [72, 73]. It is defined as follows:

Definition 2.8: Hausdorff distance

Given two bounded sets $A, B \subset (\mathbb{R}^n, d)$, the Hausdorff distance is defined as

$$H(A, B) = \max(\sup_{\mathbf{x} \in A} (d(\mathbf{x}, B)), \sup_{\mathbf{y} \in B} (d(\mathbf{y}, A))) \quad (2.17)$$

This Hausdorff distance can however be reformulated in a simpler manner [8]

$$\begin{aligned} H(A, B) &= \sup_{x \in \Omega} |d(x, A) - d(x, B)| \\ &= \sup_{x \in \Omega} |DT_d[A](x) - DT_d[B](x)| \end{aligned} \quad (2.18)$$

which is in this case implemented by running two distance transform algorithm. The importance of this distance is justified by the following theorem:

Theorem 2.5: Misalignment of the Hausdorff distance

Given a non-empty finite set $A \subset \mathbb{R}^n$ and a vector $\mathbf{t} \in \mathbb{R}^n$, it holds [72]:

$$H(A, T_{\mathbf{t}}A) = \|\mathbf{t}\|$$

where the norm depends on the distance used in the definition above.

As a consequence, the monotonicity along a line of the autocorrelation of indicator functions is trivially verified:

Corollary 2.3: Monotonicity of the Hausdorff metric

Let $\mathbf{t} \in \mathbb{R}^n$ be a vector and $A \subset \mathbb{R}^n$ a non-empty finite set. For any two constants $0 \leq \lambda_1 \leq \lambda_2$, it holds:

$$H(A, T_{\lambda_1 \mathbf{t}}A) \leq H(A, T_{\lambda_2 \mathbf{t}}A)$$

Our contribution [30] was to extend these results to real-valued functions in order to create a novel structure-based dissimilarity enjoying the monotonicity of its autocorrelation. Following ideas coming from scale space analysis and other machine learning tools such as keypoint detection and matching, we decompose an image as edges at different scales. Mathematically speaking, edges are found as points with non vanishing derivatives and vanishing second derivatives. We denote by $E(f; s)$ the set of edges found in a signal f at scale s . Looking first at 1D signals helps easing the presentation. Indeed, in this case, second order directional derivatives carry not ambiguity in terms of their directions. From the wordy description above, edges are expressed as

$$E(f; s) := \{\mathbf{x} \in \mathbb{R} : (\partial_{\mathbf{x}} g_s * f)(\mathbf{x}) \neq 0\} \cap \{\mathbf{x} \in \mathbb{R} : (\partial_{\mathbf{xx}} g_s * f)(\mathbf{x}) = 0\}, \quad (2.19)$$

where g_s denotes a (Gaussian) blurring filter at scale s (corresponding to its blurring factor or its width). This can equivalently be written as $E(f; s) = (\partial_{\mathbf{x}} g_s * f)^{-1}(\mathbb{R} \setminus \{0\}) \cap (\partial_{\mathbf{xx}} g_s * f)^{-1}(\{0\})$, where the inverse should be understood in an algebraic way as

$$\mathbf{x} \in (\partial_{\mathbf{xx}} g_s * f)^{-1}(\{0\}) \Leftrightarrow (\partial_{\mathbf{xx}} g_s * f)(\mathbf{x}) = 0$$

Definition 2.9: Scaled distance transforms

Let $\Omega \subset \mathbb{R}$ be open and bounded and let f be a function defined on Ω with values in $\mathbb{R}$. Its scaled-distance transform is a positive real-valued mapping from $\Omega \times \mathbb{R}^+$:

$$SDT[f] : \begin{cases} \Omega \times \mathbb{R}^+ & \rightarrow \mathbb{R}^+ \\ (\mathbf{x}, s) & \mapsto SDT[f](\mathbf{x}, s) \end{cases} \quad (2.20)$$

$$SDT[f](\mathbf{x}, s) = \min \{d(\mathbf{x}, \mathbf{y}), \mathbf{y} \in E(f; s)\} \quad (2.21)$$

The case of multivariate functions is handled by looking at edges aligned with all the Cartesian coordinates. We say that an edge located at $\mathbf{x}$ is oriented in the i^{th} direction, $1 \leq i \leq n$, if

$$\mathbf{x} \in E_i(f; s) := \{\mathbf{x} \in \mathbb{R} : (\partial_{x_i} g_s * f)(\mathbf{x}) \neq 0\} \cap \{\mathbf{x} \in \mathbb{R} : (\partial_{x_i x_i} g_s * f)(\mathbf{x}) = 0\}. \quad (2.22)$$

The set of all edge locations at scale s is defined as

$$E(f; s) = \bigcup_i E_i(f; s), \quad (2.23)$$

i.e. $\mathbf{x} \in E(f; s)$ means that it belongs to an edge in at least one direction for an object of size (related to) s . Hence we get the following definition:

Definition 2.10: Scaled distance transforms in higher dimensions

Let f be a function defined on $\Omega \subset \mathbb{R}^n$, open and bounded, with values in $\mathbb{R}$. Its scaled-distance transform is a positive real-valued mapping from $\Omega \times \mathbb{R}^+$:

$$SDT[f] : \begin{cases} \Omega \times \mathbb{R}^+ & \rightarrow \mathbb{R}^+ \\ (\mathbf{x}, s) & \mapsto SDT[f](\mathbf{x}, s) \end{cases} \quad (2.24)$$

$$SDT[f](\mathbf{x}, s) = \min \{d(\mathbf{x}, \mathbf{y}), \mathbf{y} \in E(f; s)\} \quad (2.25)$$

Informally, the SDT representation gives an information to the closeness of every point to an object of a given size. It therefore gives a rich description of the structure of the image or the volumetric data under scrutiny.

Now that the scale space representations are clear, we may define a way to compare these objects.

Definition 2.11: Distance between scaled distance transforms

Let δ be a (dis-)similarity measure on $\mathbb{R}^n$ and ν a positive finite measure on $\mathbb{R}^+$. Two scaled edges representations may be compared as

$$\delta_{SDT}(f, g) = \int_{s \geq 0} \delta(SDT[f](\cdot, s), SDT[g](\cdot, s)) d\nu(s). \quad (2.26)$$

We could finally prove the main result of this section: the construction based on scaled distance transforms allows to define a novel metric for comparing volumetric data which enjoys the monotonicity of its autocorrelation.

Theorem 2.6: Monotonicity of the SDT distance

Given a function f in $(\Omega \subset \mathbb{R}^n, d)$. Its autocorrelation function $a_{\delta_{SDT}}[f](\mathbf{t}) := \delta_{SDT}(f, f_{\mathbf{t}})$ where the Hausdorff distance from (2.18) is used as δ in (2.26) is monotonic with respect to the displacement: for a given $\mathbf{t} \in \mathbb{R}^n$ and two constants $0 \leq \lambda_1 \leq \lambda_2$, it holds

$$a_{\delta_{SDT}}[f](\lambda_1 \mathbf{t}) \leq a_{\delta_{SDT}}[f](\lambda_2 \mathbf{t}) \quad (2.27)$$

2.5 Further research topics in structures in signals

Based on what has been done so far, more topics can be the subject of future research. Some of them might be low hanging fruits for young researchers while others seem a bit more demanding.

One important aspect of the discrepancy norm which has yet to be investigated is its use in low-energy consumption image processing and analysis. Indeed, it is known that there is a link between the discrepancy norm as described above and send-on-delta/threshold-based sampling [92, 93]. Such approaches are interesting for low-consumption image processing [121] and could be used together with tailor-made sensors, similar to Silicon Retina cameras. Moreover, some care should be taken, and investigated!, in order to make optimization robust and easy using the specifics of this norm.

Another question arose when dealing with the general set up of signal denoising and defining the associated optimization problem with the discrepancy norm or other monotonicity preserving metrics as a regularizer. One could think for instance at the problem mentioned in Eq. (1.4) in the introduction on page 8. Using the discrepancy norm as a model enforcing term might be interesting but little is known today to the type of signals which may be of use for such a problem. This is reminiscent of the problem mentioned as Open problem 2.1 in which one needs to describe what are the low-discrepancy functions. From an optimization point of view, solving (1.4) as

$$\begin{cases} \min F(\mathbf{x}, \mathbf{y}) \\ \text{s.t. } \|\mathbf{x}\|_D \leq \varepsilon \end{cases}$$

is not trivial. One could use the approximation defined above (see Eq. (2.2)) but the use of the exponential function of the cumulative function could easily lead to overflow, especially for very long signals. Other approaches require to investigate more complex optimization schemes using proximal methods, projected gradients, or subgradients. How these should be implemented and what would be the recovery results for monotonicity preserving metrics is a completely new open realm in signal processing, which could easily keep a Ph.D. student busy!

Chapter 3

Compressed sensing and sparse approximation

3.1 An introduction to compressed sensing and sparse approximation

3.1.1 A primer on compressed sensing

Compressed sensing (CS) is the art of linearly acquiring signals albeit using only a small number of measurements, often far below what is required by the classical Nyquist-Shannon sampling theorem. We consider an unknown signal of interest $\mathbf{x}^\# \in \mathbb{K}^N$. This signal is sampled / measured / sensed as $\mathbf{y} = A\mathbf{x}^\# \in \mathbb{C}^m$ where the number of linear samples $m \ll N$ is much smaller than the size of the ambient space. In case of some corrupted / noisy measurements, we usually have $\mathbf{y} = A\mathbf{x}^\# + \mathbf{e}$ for some unknown noise vector $\mathbf{e} \in \mathbb{K}^m$. A is called the design or measurement or sensing matrix. The challenge of compressed sensing is to recover an approximation of $\mathbf{x}^*$ from the sole knowledge of $\mathbf{y}$ (and A)... and of course a well chosen algorithm! It therefore relates to two problems

1. Find concrete sensing matrices A , and

2. Find reconstruction maps Δ such that

$$\Delta(\mathbf{y}) = \Delta(A\mathbf{x}^\# + \mathbf{e}) \approx \mathbf{x}^\#.$$

It will become clear later how this approximation can be quantified and what are conditions to ensure a good enough approximation.

While both questions are usually not completely independent, I have not worked on the first problem and have concentrated my research on the recovery map and some applications of CS in the sciences.

Basic algebra claims that one might find infinitely many solutions, unless further conditions are specified. The simplest such condition is to think of $\mathbf{x}^\#$ as having only a very small number $s \leq m$ of nonzeros. It is worth mentioning in passing that nonzeros are related to what the machine learning community calls active variables. As such, the subject presented here is linked to regression in statistics. Later, I will also detail some ideas when the $\mathbf{x}^\#$ is not sparse *per se* but sparse in certain structures.

As mentioned in the introduction, the general – noisy – problem may be stated as

$$\begin{cases} \min \|\mathbf{x}\|_0 \\ \text{s.t. } \|A\mathbf{x} - \mathbf{y}\|_2 \leq \varepsilon. \end{cases} \quad (3.1)$$

Here, $\varepsilon > 0$ is an estimation of the noise level in the measurements. An exhaustive exposition on the subject would require discussing noises which are bounded (i.e. whose level is measured in the ℓ_∞ norm). I recommend the interested reader to look at the book by Foucart and Rauhut for a more exhaustive and detailed presentation [56].

Eq. (3.1) being combinatorial in nature, the first challenge was to find relevant approximate models which are algorithmically tractable. The first approach was to replace the ℓ_0 norm with its tightest convex relaxation, the ℓ_1 norm (see Fig. 1.3 in the introduction for a geometrical justification or Fig. 3.1 below for a notion of compressibility).

$$\begin{cases} \min \|\mathbf{x}\|_1 \\ \text{s.t. } \|A\mathbf{x} - \mathbf{y}\|_2 \leq \varepsilon. \end{cases} \quad (3.2)$$

This is the idea behind what is known as the *Basis Pursuit Denoising* or *Quadratically constrained basis pursuit*. It is equivalent to the Lasso [114], known in the statistics community as an algorithm combining variable selection and regression.

Before giving deeper details, I would like to review some important tools, results, and definitions. It has been known for a long time that $2s$ measurements are required for the uniform recovery of s -sparse vectors. Prony's method dating back to the 18th century is one example of successful recovery algorithms. It is also known that more measurements are needed for a stable (i.e. in the presence of noise or non exactly s sparse vectors) recovery. To continue we need a notion of compressibility and stability.

Definition 3.1: Best s term approximation of a vector

Let $s \in \mathbb{N}$ and $0 < p \in \mathbb{R}^+$. The best s -term approximation of a vector $\mathbf{x} \in \mathbb{C}^N$ is defined as

$$\sigma_s(\mathbf{x})_p := \inf \{ \|\mathbf{x} - \mathbf{z}\|_p : \|\mathbf{z}\|_0 \leq s \}. \quad (3.3)$$

Informally speaking, a vector $\mathbf{x}$ is said to be *compressible* if its best s term approximation is small enough or decays fast enough with s , for a small enough p . This decay can be quantified via what some call Stechkin's inequality

Proposition 3.1: Stechkin's inequality

For all $q > p > 0$ and any $\mathbf{x} \in \mathbb{C}^N$, the best s term approximation of $\mathbf{x}$ in the q (quasi) norm is bounded as

$$\sigma_s(\mathbf{x})_q \leq \frac{c_{p,q}}{s^{1/p-1/q}} \|\mathbf{x}\|_p \quad (3.4)$$

where

$$c_{p,q} = \left[\left(\frac{p}{q} \right)^{p/q} \left(1 - \frac{p}{q} \right)^{1-p/q} \right]^{1/p} \leq 1.$$

Two points are worth mentioning here. The Proposition above allows to quantify what signals are compressible (namely those with a small ℓ_p norm for a small p) and how to quantify the decay. Moreover, we will often read Eq. (3.4) with the special values $q = 2, p = 1$, in which case we have

$$\sigma_s(\mathbf{x})_2 \leq \frac{1}{2\sqrt{s}} \|\mathbf{x}\|_1.$$

And we notice that minimizing the ℓ_1 norm of a vector (as described in the basis pursuit denoising Eq. (3.2)), leads to better approximation results, as measured in the residual of the tail.

A question remains when solving this ℓ_1 constrained minimization problem: is the solution sparse? To answer this question, two general conditions have been introduced. Let us start with the null-space property.

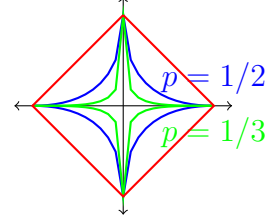


Figure 3.1: The use of ℓ_p norms forces the non zero elements to concentrate along the canonical axis, thereby approaching true sparsity (defined through the ℓ_0 norm) more and more as $p \rightarrow 0$.

Definition 3.2: Null-Space Property (NSP)

A matrix $A \in \mathbb{C}^{m \times N}$ satisfies the robust null space property with constants $0 < \rho < 1$ and $\tau > 0$ relative to a set $S \subset \{1, \dots, N\}$ if

$$\|\mathbf{v}_S\|_1 \leq \rho \|\mathbf{v}_{\bar{S}}\|_1 + \tau \|A\mathbf{v}\|, \forall \mathbf{v} \in \mathbb{C}^N \quad (3.5)$$

It is said to satisfy the robust null space property of order s with constants $0 < \rho < 1$ and $\tau > 0$ if it satisfies the robust null space property relative to any set $S \subset \{1, \dots, N\}$ with $|S| \leq s$.

In the noiseless and exactly s sparse case, one can show that some null-space property is necessary and sufficient for the ℓ_1 minimization with equality constraints to recover the unique s sparse solution. In the more general case, we have the following result.

Theorem 3.1: NSP-based recovery

Let $A \in \mathbb{C}^{m \times N}$ be a matrix satisfying the robust null-space property of order s with constants $0 < \rho < 1$ and $\tau > 0$. Then, for any $\mathbf{x}^\# \in \mathbb{C}^N$, a solution $\mathbf{x}^*$ of Eq. (3.2) with $\mathbf{y} = A\mathbf{x}^\# + \mathbf{e}$ and $\|\mathbf{e}\| \leq \eta$ satisfies the following approximation error

$$\|\mathbf{x}^\# - \mathbf{x}^*\|_1 \leq \frac{2(1+\rho)}{1-\rho} \sigma_s(\mathbf{x}^\#)_1 + \frac{4\tau}{1-\rho} \eta. \quad (3.6)$$

Similar results may be obtained for general ℓ_p estimates, $p \geq 1$ using an adapted version of the null-space property.

A second condition sufficient for recovery is known as the restricted isometry property.

Definition 3.3: Restricted Isometry Property (RIP)

Let $A \in \mathbb{C}^{m \times N}$. A is said to satisfy the s^{th} (or order s -) restricted isometry property with constant $\delta \geq 0$ if for all $\mathbf{x} \in \mathbb{C}^N$ with $\|\mathbf{x}\|_0 \leq s$,

$$(1 - \delta) \|\mathbf{x}\|_2^2 \leq \|A\mathbf{x}\|_2^2 \leq (1 + \delta) \|\mathbf{x}\|_2^2. \quad (3.7)$$

The smallest such δ is called the s^{th} *Restricted Isometry Constant* of A .

Intuitively, the RIP quantifies the amount of distortion a linear operator applies to s sparse vectors. It helps quantifying the results obtained via (3.2):

Theorem 3.2: RIP-based recovery

Let $A \in \mathbb{C}^{m \times N}$ be a matrix such that its RIC of order $2s$ satisfies

$$\delta_{2s} < \frac{4}{41}. \quad (3.8)$$

Then for any $\mathbf{x}^\# \in \mathbb{C}^N$ and $\mathbf{y} \in \mathbb{C}^m$ such that $\|A\mathbf{x}^\# - \mathbf{y}\|_2 \leq \eta$, a solution $\mathbf{x}^*$ to the problem

$$\begin{cases} \min \|\mathbf{x}\|_1 \\ \text{s.t. } \|A\mathbf{x} - \mathbf{y}\|_2 \leq \eta, \end{cases} \quad (3.9)$$

approximates the true solution in the sense that

$$\|\mathbf{x}^\# - \mathbf{x}^*\|_1 \leq C\sigma_s(\mathbf{x}^\#)_1 + D\sqrt{s}\eta, \quad (3.10)$$

$$\|\mathbf{x}^\# - \mathbf{x}^*\|_2 \leq \frac{C}{\sqrt{s}}\sigma_s(\mathbf{x}^\#)_1 + D\eta, \quad (3.11)$$

where C and $D > 0$ are constant depending only on δ_{2s} .

In fact, most of the RIP-based results are proved by reducing to a null-space property.

Theorem 3.3: RIP implies NSP

Let $A \in \mathbb{C}^{m \times N}$ be a matrix with restricted isometry constant δ_{2s} . Then A satisfies the robust null-space property of order s with constants $0 < \rho < 1$ and $\tau > 0$ depending only on δ_{2s} .

As mentioned earlier, this manuscript does not look into the problem of finding adequate sensing matrices. However, we will use two types of matrices throughout our results: fully (subgaussian) random matrices and matrices obtained by random samples of a bounded orthogonal systems or Riesz basis. The following two theorems express the conditions under which these matrices are known to satisfy (with high probability) RIP conditions. Constructing a deterministic matrix with known RIP conditions is still an open problem and even simply verifying the RIP for a given generated matrix is a hard problem [10].

Theorem 3.4: RIP for subgaussian matrices

Let $A \in \mathbb{R}^{m \times N}$ be a matrix whose entries are random variables i.i.d according to a subgaussian distribution with parameters β and κ . Then there exists a constant C depending only on β and κ such that if

$$m \geq 2C\delta^{-2}s \ln(eN/s) \quad (3.12)$$

then the order s restricted isometry constant of $\frac{1}{\sqrt{m}}A$ satisfies $\delta_s < \delta$ with probability at least $1 - 2\exp(-\delta^2 m / (2C))$.

Roughly speaking, a subgaussian matrix enjoys an RIP condition (with high probability) as

soon as the number of measurements scales linearly with the sparsity s and only logarithmically with the ambient dimension N . Similar results exist in the complex case [124, 75]. As other candidates for RIP matrices, subsampled bounded orthogonal systems are known to have nice behaviors in terms of their necessary number of measurements, see [69, 35]

Theorem 3.5: RIP for Bounded orthogonal systems

Let $(\psi_j)_{1 \leq j \leq N}$ be a Riesz system with constants $0 < a < b < \infty$ and such that $K := \max_{1 \leq j \leq N} \|\psi_j\|_\infty$. Let $\delta \in (1 - \frac{a}{b}, 1)$. Then with probability at least $1 - 2 \exp(-D \min(1, b^2) \eta^2 m / K^2 / s)$ the matrix $A \in \mathbb{C}^{m \times N}$ whose entries are defined as

$$A_{i,j} = \frac{1}{\sqrt{mb}} (\psi_j(\omega_i))$$

where ω_i are drawn i.i.d. at random, satisfies the order s restricted isometry property with restricted isometry constant δ provided

$$m \geq C \max(b^{-2}, 1) K^2 \eta^{-2} s \log^2(s K^2 \max(b^{-2}, 1) \eta^{-2}) \log(eN). \quad (3.13)$$

In particular, Eq. (3.13) implies that the number of samples scales quadratically with the bound K , a remark which will justify the introduction of weights for parametric PDEs in Sec. 3.3. Combining results from Theorem 3.2 together with Thms. 3.4 and 3.5 shows that recovering sparse solutions from underdetermined systems is indeed possible via convex optimization, assuming one agrees to some randomness in the design matrix.

Sometimes, signals are not sparse in the original basis but only sparse in a transformed representation. This falls under the realm of what is known as model-based compressed sensing, in which more information is known about the sought after signal $\mathbf{x}^\#$.

Some common adaptations of the sparse approximation problem are done in order to convey a *sparse in basis* information. As a first step into model-based compressed sensing, one finds mainly two kinds of approaches: ℓ_1 analysis and ℓ_1 synthesis. In both cases, we are given a matrix $D \in \mathbb{C}^{N \times M}$, usually called a dictionary or sparsifying basis, and no longer assume that $\mathbf{x}$ is sparse, but instead that $\mathbf{x} = D\mathbf{u}$ and $\mathbf{u}$ is sparse (i.e. $\|\mathbf{u}\|_0$ is small). Said differently, only a small number of coefficients are sufficient to synthesize $\mathbf{x}$. The optimization problems are adapted into

$$\begin{cases} \min \|D^* \mathbf{x}\|_0 \\ \text{s.t. } \|A\mathbf{x} - \mathbf{y}\|_2 \leq \varepsilon. \end{cases} \quad \text{for the } \ell_1 \text{ analysis approach} \quad (3.14)$$

$$\begin{cases} \min \|\mathbf{u}\|_0 \\ \text{s.t. } \|AD\mathbf{u} - \mathbf{y}\|_2 \leq \varepsilon. \end{cases} \quad \text{for the } \ell_1 \text{ synthesis approach.} \quad (3.15)$$

Optimization problems have been re-analyzed for the sake of this new viewpoint, but since I have not worked on this, I choose not to explicitly detail this here.

3.1.2 Sensing in a distributed environment

We have seen in Thm. 3.4 and 3.5 that all recovery results so far require a number of measurements which is larger than something that depends linearly on s and only logarithmically on the ambient dimension. Unfortunately in some cases, it still might not be possible to handle such a large number of measurements. For instance, we might have legal limitations or production costs which could get prohibitive when generating larger sensors. When thinking about this, we suggested another approach within the realm of model-based compressed sensing. The underlying idea is to split the information into (overlapping or not) subsets of information, as suggested in the Figure 1.4 in the introduction on page 12. The approach we chose is to distribute the information in a fusion frame structure. As usual, before detailing the novelties of the suggested solution, I want to review some basics on fusion frames, going back to the definition of frames.

Definition 3.4: Frame systems

A sequence $\mathcal{F} = \{f_i\}_{i \in I}$ in a Hilbert space $\mathcal{H}$ is a frame [40] for $\mathcal{H}$ if there exist two constants $0 < A \leq B < \infty$ (called the lower and upper frame bounds respectively) such that

$$A\|f\|^2 \leq \sum_{i \in I} |\langle f, f_i \rangle|^2 \leq B\|f\|^2 \text{ for all } f \in \mathcal{H}. \quad (3.16)$$

Roughly speaking, a frame can be seen as an overcomplete basis. In signal processing, this is often the framework used to denote a dictionary. The frame analysis and synthesis operators are respectively given by

$$T(f) = \{\langle f, f_i \rangle\}_{i \in I} \text{ and } T^*(\{c_i\}_{i \in I}) = \sum_{i \in I} c_i f_i,$$

for all $f \in \mathcal{H}$ and $\{c_i\}_{i \in I} \in \ell_2(I)$. It shows that the frame operator $S := T^*T$ is a positive, self-adjoint and invertible operator; this means that recovery of any $f \in \mathcal{H}$ is possible, if Sf is known. Although S^{-1} might be computationally challenging, we may use the concept of dual frames. Indeed, an important aspect about frames is that they come with at least one so-called dual frame $\mathcal{G} = \{g_i\}_{i \in I}$, which satisfies

$$f = \sum_{i \in I} \langle f, f_i \rangle g_i = \sum_{i \in I} \langle f, g_i \rangle f_i \text{ for all } f \in \mathcal{H}. \quad (3.17)$$

As a consequence, recovering an original function f is easily done by mixing frames and their duals.

Frames work by adding redundancy in the representation. Fusion frames add a layer of distribution on top of this in the sense that redundancy is added in terms of subspaces. We are interested in spanning overlapping subspaces with their respective projections.

Definition 3.5: Fusion frames

Given an index set I , let $\mathcal{W} := \{W_i \mid i \in I\}$ be a family of closed subspaces in $\mathcal{H}$. We denote the orthogonal projections^a onto W_i by P_i . Then $\mathcal{W}$ is a fusion frame, if there exist two constants $0 < C \leq D < \infty$ such that

$$C\|f\|^2 \leq \sum_{i \in I} \|P_i(f)\|^2 \leq D\|f\|^2 \text{ for all } f \in \mathcal{H}. \quad (3.18)$$

^aan operator P is an orthogonal projection if $P^2 = P$ and $P^* = P$

It is important to notice that every subspace might have its own frame basis, as per Def. 3.4. These local frames may be concatenated in order to form yet another frame for the global (ambient) space. If $\mathcal{F}_i := \{f_{ij} \mid j \in J_i\}$ is a frame for W_i , $i \in I$. Then $\{(W_i, \mathcal{F}_i) \mid i \in I\}$ is a fusion frames system for $\mathcal{H}$. In fact fusion frames and frames are not unrelated. It is known [37] that the following statements are equivalent.

- $\cup_{i \in I} \{f_{ij} \mid j \in J_i\}$ is a frame for $\mathcal{H}$.
- $\{W_i\}_{i \in I}$ is a fusion frame for $\mathcal{H}$.

Similar to classical frame theory, the analysis operator T is defined by

$$T(f) := \{P_i(f)\}_{i \in I} \text{ for all } f \in \mathcal{H},$$

while its adjoint operator is the synthesis operator $T^* : (\sum_{i \in I} \oplus W_i)_{\ell_2} \rightarrow \mathcal{H}$, defined by

$$T^*(f) = \sum_i f_i, \text{ where } f = \{f_i\}_{i \in I} \in \left(\sum_{i \in I} \oplus W_i \right)_{\ell_2}.$$

The fusion frame operator $S = T^*T$ is given by

$$S(f) = \sum_{i \in I} P_i(f).$$

S is again a positive and invertible operator on $\mathcal{H}$. In particular, it holds $CI \leq S \leq DI$, with I denoting the identity.

If the dual frames $\mathcal{G}_i$, $i \in I$, for each local frames are known then the fusion frame operator can be expressed in terms of the local (dual) frames [38]:

$$S = \sum_{i \in I} T_{\mathcal{G}_i}^* T_{\mathcal{F}_i} = \sum_{i \in I} T_{\mathcal{F}_i}^* T_{\mathcal{G}_i}.$$

In [1, 2], we have extended the notion of sparsity to describe local sparsities, i.e. some restrictions of the number of nonzeros in each subspace. From there, we extended the

notion of null-space and restricted isometry properties for the distributed sparsity setting and extended the results mentioned earlier to this signal model.

Definition 3.6: Distributed sparsity

A signal $\mathbf{x} \in \mathbb{C}^N$ is said to be $\mathbf{s} = (s_1, \dots, s_n)$ -distributed sparse with respect to a fusion frame $\mathcal{W} = (W_i, P_i)_{1 \leq i \leq n}$, if $\|P_i(\mathbf{x})\|_0 \leq s_i$, for every $1 \leq i \leq n$. $\mathbf{s}$ is called the *sparsity pattern* of $\mathbf{x}$ with respect to $\mathcal{W}$.

In the case that the sparsity of the signal is uniformly distributed among the subspaces ($s_i = s/n$), the usual compressed sensing recovery guarantees ensure us that

$$m \asymp s_i \log(N/s_i)$$

observations per subspace are required for a stable and robust recovery of the signal. This accounts for a total number of measurements scaling as $m_T \asymp ns_i \log(N/s_i) = s \log(N/s_i)$. In other words, we are able to recover similar sparsities as in the classical CS framework, while using only *cheap* sensors at a small extra computation cost. But in opposition to the classical theory most of the computations can be easily carried in a distributed or embedded setting, where only pieces of the information are available. Only the fusion process requires all the local pieces of information to compute the final estimation of a signal.

Definition 3.7: Distributed best approximation

Let $\mathcal{W} = (W_i, P_i)_{i=1}^n$ be a fusion frame, and let $\mathbf{x} \in \mathbb{K}^N$. For $p > 0$ and a sparsity pattern $\mathbf{s} = (s_1, \dots, s_n)$ with $s_i \in \mathbb{N}$ for all $1 \leq i \leq n$, the ℓ^p errors of best $\mathbf{s}$ -term approximations are defined as the vector

$$\sigma_{\mathbf{s}}^{\mathcal{W}}(\mathbf{x})_p := (\sigma_{s_1}(P_1\mathbf{x}), \sigma_{s_2}(P_2\mathbf{x}), \dots, \sigma_{s_n}(P_n\mathbf{x}))^T$$

We may now adapt the null-space property to this signal model.

Definition 3.8: Robust and stable partial null space property (RP-NSP)

Let n be an integer and $\mathcal{W} = (W_i, P_i)_{i=1}^n$ be a fusion frame for $\mathbb{K}^N$. Let $\mathbf{s} = (s_1, \dots, s_n)$ be a sequence of non negative numbers representing the distributed sparsity pattern with respect to $\mathcal{W}$. For a number $q \geq 1$, a sensing matrix $\mathbf{A} \in \mathbb{K}^{m \times N}$ is said to satisfy the ℓ_q -RP-NSP with pattern $\mathbf{s}$ with respect to $\mathcal{W}$ and with constants $\rho_1, \dots, \rho_n \in (0, 1)$ and $\tau_1, \dots, \tau_n > 0$ if

$$\|(P_i\mathbf{v})_{S_i}\|_q \leq \frac{\rho_i}{s^{1-1/q}} \|(P_i\mathbf{v})_{\overline{S_i}}\|_1 + \tau_i \|\mathbf{A}\mathbf{v}\|_2, \quad \text{for all } \mathbf{v} \in \mathbb{K}^N, 1 \leq i \leq n, S_i \subset W_i, \text{ and } |S_i| \leq s_i.$$

This definition is reminiscent of the work on sparse recovery with partially known support [11]. The difference here being that there is no need to enforce a condition on the vector $\mathbf{v}$ to lie in the range of the other subspaces. In a sense, this is taken care of by the fusion process and the fact that we have multiple measurement vectors. Similarly, we may

extend the concept of RIP to signals in a distributed setting.

Definition 3.9: Partial-RIP (P-RIP)

Let $\mathcal{W} = (W_i, P_i)_{i=1}^n$ be a fusion frame, and let $\mathbf{A} \in \mathbb{K}^{m \times N}$. Assume that $\mathbf{A}$ satisfies the $RIP(s_i, \delta_i)$ on W_i , with $\delta_i \in (0, 1)$, $i \in I = \{1, \dots, n\}$. Then, we say that $\mathbf{A} \in \mathbb{K}^{m \times N}$ satisfies the partial RIP with respect to $\mathcal{W}$, with bounds $\delta_1, \dots, \delta_n$ and sparsity pattern $\mathbf{s} = (s_1, \dots, s_n)$.

We may exploit the link between global and local frames in order to obtain an expression for the P-RIP which is similar to the traditional RIP.

Proposition 3.2: Distributed RIP implies RIP

Let $\mathcal{W} = (W_i)_{i \in I}$ be a fusion frame (with frame bounds $0 < C \leq D < \infty$). Let $A \in \mathbb{K}^{m \times N}$ satisfy the P-RIP with respect to $\mathcal{W}$, with bounds $\delta_1, \dots, \delta_n$ and sparsity pattern $\mathbf{s} = (s_1, \dots, s_n)$, and let $C_o = C \min_i \{1 - \delta_i\}$, $D_o = D \max_i \{1 + \delta_i\}$. Then, for any $\mathbf{v} \in \mathbb{K}^N$,

$$C_o \|\mathbf{v}\|_2^2 \leq \sum_i \|A\mathbf{v}_i\|_2^2 \leq D_o \|\mathbf{v}\|_2^2.$$

Finally, we describe our approach in Algo. 2. The idea is the following For a given cheap sensor modeled with a sensing matrix $A \in \mathbb{R}^{m \times N}$, we sample projected signals in the various subspaces as $\mathbf{y}^{(k)} = AP_k \mathbf{x}^\# + \mathbf{e}^{(k)}$ and recover local pieces of information $\mathbf{x}^{(k)*}$ via a basis pursuit algorithm or some sparse recovery algorithm (such as those presented in the next section). Once all pieces of information have been estimated, the inverse fusion frame operator allows to fuse all the pieces of information into the original signal.

Algorithm 2 Fused distributed sensing recovery framework

```

1: function FCS( $(P_i)_{1 \leq i \leq n}$ ,  $A$ ,  $\eta$ )
2:    $\mathbf{x}^0 = 0$ 
3:   for  $k = 0, \dots, n$  do
4:     Measure  $k^{\text{th}}$  vector  $\mathbf{y}^{(k)} \leftarrow AP_k \mathbf{x} + \mathbf{e}^{(k)}$ 
5:     Recover local information  $\mathbf{x}^{(k)*}$  as the solution to  $\text{argmin} \|\mathbf{z}\|_1$  s.t.  $\|AP_k \mathbf{z} - \mathbf{y}^{(k)}\|_2 \leq \eta$ 
6:     Add local information  $\hat{\mathbf{x}} \leftarrow \hat{\mathbf{x}} + \mathbf{x}^{(k)*}$ 
7:      $k \leftarrow k + 1$ 
8:   end for
9:   return  $\mathbf{x}^* = S^{-1} \hat{\mathbf{x}}$ 
10: end function.

```

Altogether we can adapt the proof techniques from the traditional compressed sensing setup in order to obtain P-RIP-based recovery guarantees.

Theorem 3.6: Distributed RIP Recovery

Let $\mathcal{W} = (W_i, P_i)_{i=1}^n$ be a fusion frame for $\mathbb{K}^N$ with fusion frame operator S and fusion frame bounds $0 < C \leq D < \infty$. Let $A \in \mathbb{K}^{m \times N}$ be a matrix satisfying the P-RIP(2s, δ) where $\mathbf{s} = (s_1, \dots, s_n)$ is a sparsity pattern with respect to $\mathcal{W}$ and $\delta = (\delta_1, \dots, \delta_n)$ with $\delta_i < 4/\sqrt{41}$, for all $1 \leq i \leq n$. Then any $\mathbf{s}$ distributed-sparse vector $\mathbf{x}^\#$ in $\mathcal{W}$ can be recovered by solving n sparse recovery problems such as (3.2) from the measurements $\mathbf{y}^{(k)}, 1 \leq k \leq n$.

Moreover, assuming the noise vectors $\mathbf{e}^{(k)}, 1 \leq k \leq n$ are controlled by $\|\mathbf{e}^{(i)}\|_2 \leq \eta_i, 1 \leq i \leq n$. Let $\mathbf{x}^* = S^{-1} \sum \mathbf{x}^{(i)*}$. Then

$$\|\mathbf{x}^* - \mathbf{x}^\#\|_2 \leq \frac{1}{C} \sum_{i=1}^n \alpha_i \frac{\sigma_{\mathbf{s}}^{\mathcal{W}}(\mathbf{x}^\#)_{1,i}}{\sqrt{s_i}} + \beta_i \eta_i$$

where α_i and β_i depend only on the RIP constants δ_i .

To be tractable, the algorithm requires an efficient computation of the inverse frame operator. This is usually not a trivial thing. Luckily, it is known that it can be approximated via what is known as the (fusion) frame algorithm [38].

Proposition 3.3: Fusion frame algorithm

Let $(W_i)_{i \in I}$ be a fusion frame in $\mathbb{K}^N$, with fusion frame operator $S = S_W$, and fusion frame bounds C, D . Further, let $\mathbf{x} \in \mathbb{K}^N$, and define the sequence $(\mathbf{x}_k)$ by $\mathbf{x}^{(0)} = 0$ and $\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + \frac{2}{C+D} S(\mathbf{x}^\# - \mathbf{x}_{k-1}), k \geq 1$. Then we have $\mathbf{x}^\# = \lim_{k \rightarrow \infty} \mathbf{x}_k$, with the error estimate $\|\mathbf{x} - \mathbf{x}_k\| \leq \left(\frac{D-C}{D+C}\right)^k \|\mathbf{x}^\#\|$.

3.2 Iterative algorithms for the recovery of sparse vectors

So far we have reviewed the literature on sparse recovery via ℓ_1 minimization techniques and introduced a novel model-based strategy for distributed sensing. In this section we discussed iterative and greedy approaches which approximate the solution to the original problem without resorting to the convex relaxation of the original problem (3.1). In particular, based on the work of Foucart on Hard Thresholding Pursuit (HTP) [55] we asked ourselves how one can estimate the number of iterations for convergence of HTP [27]. This led to generalizations of HTP in which earlier iterations are even cheaper to compute [23]. To provide some

context on iterative methods for sparse recovery, I start by reviewing three of the main algorithms – Iterative Hard Thresholding (IHT), Orthogonal Matching Pursuit (OMP), and Hard Thresholding Pursuit (HTP) – and introduce their recovery guarantee results before introducing our new research: the estimation of the number of iterations for HTP, and the variants GHTP and f HTP. Let me introduce the ambiguous *hard thresholding operator* H_s which will be defined either as the extraction of the s largest entries, in magnitude, of a vector or as the vector whose s largest entries are unchanged while the remaining ones are set to 0. The intention of one or the other definition should be clear in context. One may also write H_S for a subset of indices $S \subset \{1, \dots, N\}$ to denote the operator restricting the input to its entries supported on S . In a similar fashion, for an index set $S \subset \{1, \dots, N\}$ of size s , the notation $\mathbf{x}_S$ will denote either the s -dimensional vector whose entries are precisely those from $\mathbf{x}$ supported on S , or the N dimensional vector such that $\mathbf{x}_i = 0$ for $i \notin S$. We start with the most intuitive iterative and thresholding-based algorithm: IHT, whose detailed algorithm is given as Algo. 3.

Algorithm 3 Iterative Hard Thresholding

```

1: function IHT( $A \in \mathbb{C}^{m \times N}$ ,  $s$ ,  $\mathbf{y} \in \mathbb{C}^m$ ,  $\mathbf{x}^{(0)} \in \mathbb{C}^N$ )
2:   Inputs: a design matrix  $A$ , an initial guess  $\mathbf{x}^{(0)}$ , measurements  $\mathbf{y}$ , a target sparsity  $s$ 
3:   repeat
4:      $\mathbf{u}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + A^*(\mathbf{y} - A\mathbf{x}^{(k)})$ 
5:      $\mathbf{x}^{(k+1)} \leftarrow H_s(\mathbf{u}^{(k+1)})$ 
6:      $k \leftarrow k + 1$ 
7:   until a stopping criterion is met
8:   return  $\mathbf{x}^{(k)}$ 
9: end function.

```

They are two key elements in the IHT algorithm. The first thing to look at is the computation of $\mathbf{u}^{(n)}$ which is nothing else than a gradient step of the ℓ_2 error $\|A\mathbf{x}^{(n)} - \mathbf{y}\|_2^2$. The second step enforces an s -sparse solution by thresholding all *irrelevant* entries. As a consequence, IHT alternates the minimization of the data-fidelity term (the gradient of the ℓ_2 norm) with a model-fidelity enforcement (the fact that the sought after signal belongs to the set of s -sparse vectors). IHT robustly approximates the original $\mathbf{x}^\#$ from $\mathbf{y} = A\mathbf{x}^\# + \mathbf{e}$ [20, 56]. Note that the recovery results come in many flavors in terms of RIP and I give one such result here and another one later, which will combine results for HTP (see Thm. 3.9).

Theorem 3.7: Recovery by IHT

Let $\mathbf{x}^\# \in \mathbb{C}^N$ be an arbitrary vector, $\mathbf{y} = A\mathbf{x}^\# + \mathbf{e}$ where $A \in \mathbb{C}^{m \times N}$ is a sensing matrix. Let $s > 0$ and define S has the index set of the s largest entries of $\mathbf{x}^\#$. If A satisfies an RIP of order $3s$ with constant $\delta_{3s} < 1/8$, then the k^{th} iterate of IHT approximates $\mathbf{x}^\#$ in the sense that

$$\|\mathbf{x}^\# - \mathbf{x}^{(k)}\|_2 \leq \frac{1}{2^k} \|\mathbf{x}_S^\#\|_2 + 5\varepsilon, \quad (3.19)$$

where

$$\varepsilon = \sigma_s(\mathbf{x}^\#)_2 + \frac{1}{2} \sigma_s(\mathbf{x}^\#)_1 + \|\mathbf{e}\|_2$$

In particular, after

$$k^* = \left\lceil \log_2 \left(\frac{\|\mathbf{x}_S^\#\|_2}{\varepsilon} \right) \right\rceil \quad (3.20)$$

iterations, the approximation via IHT satisfies

$$\|\mathbf{x}^\# - \mathbf{x}^{(k^*)}\|_2 \leq 6\varepsilon.$$

This result, although not optimal in terms of RIC, allows for an easy stopping criteria in terms of number of iterations. The overall complexity is simply $\mathcal{O}(k^*Nm)$, or less in case of structured operators A (as it may be the case for Fourier matrices for instance). Various conditions and applications in optimal control, machine learning, matrix completions, ... have been found in the literature since this first result [19, 18, 113, 74, 99, 125, 7].

As a second iterative approach, we look at the archetypal greedy algorithm Orthogonal Matching Pursuit (OMP). It is detailed in Algorithm 4

Algorithm 4 Orthogonal Matching Pursuit

```

1: function OMP( $A \in \mathbb{C}^{m \times N}$ ,  $s$ ,  $\mathbf{y} \in \mathbb{C}^m$ ,  $\mathbf{x}^{(0)}$ )
2:   Inputs: a design matrix  $A$ , measurements  $\mathbf{y}$ 
3:   Set  $k \leftarrow 0$ ,  $\Lambda^{(0)} \leftarrow \text{supp}(\mathbf{x}^{(0)})$ 
4:   repeat
5:      $j_{k+1} \leftarrow \text{argmax} \{ |(A^*(\mathbf{y} - A\mathbf{x}^{(k)}))_j| \}$ 
6:      $\Lambda^{(k+1)} \leftarrow \Lambda^{(k)} \cup \{j_{k+1}\}$ 
7:      $\mathbf{x}^{(k+1)} \leftarrow \text{argmin}_{\mathbf{z} \in \mathbb{C}^N} \{ \|\mathbf{y} - A\mathbf{z}\|_2, \text{supp} \subset \Lambda^{(k+1)} \}$ 
8:      $k \leftarrow k + 1$ 
9:   until a stopping criterion is met
10:  return  $\mathbf{x}^{(k)}$ 
11: end function.
```

A trivial starting point is such that $\mathbf{x}^{(0)} = \mathbf{0}$ and consequently the set of active indices, at the beginning of the algorithm is void, $\Lambda^{(0)} = \emptyset$. Having a nonzero starting point could help in adding some partial information into the solution but may also get the iterations in the

wrong direction, in case an unwanted index was part of the $\Lambda^{(0)}$. It is not a trivial task to figure out what a good stopping criterion might be. It could be expressed as a threshold on the norm on the residual, a threshold on the evolution of the norm of the residual, a number of iterations, ... OMP is a **greedy** algorithm as it slowly builds up the active feature space Λ until it is sufficiently large to accurately approximate the given data $\mathbf{y}$. The idea in OMP is slightly different than IHT in that the index selection is based on the highest correlation between the residual and the available features. Note that once an index has been selected and added to the active set, it can never be selected again nor be rejected at a later iteration. Although the solution of the n^{th} , for $n > s$, iterate of OMP is not s sparse (but n sparse) as one would wish, the algorithm is guaranteed to converge to a good enough approximation of the solution in a linear (in terms of important variables in the sought-after signal) number of iterations.

Theorem 3.8: Recovery via OMP

Assume $A \in \mathbb{C}^{m \times N}$ satisfies a RIP of order $13s$ with constant

$$\delta_{13s} < \frac{1}{6}.$$

Then there exists a constant $C > 0$ depending only on δ_{13s} such that, for all $\mathbf{x}^\# \in \mathbb{C}^N$ and noise vector $\mathbf{e} \in \mathbb{C}^m$ with the vector of measurements $\mathbf{y} = A\mathbf{x}^\# + \mathbf{e}$ the sequence of iterations of OMP leads to

$$\|\mathbf{y} - A\mathbf{x}^{(12s)}\|_2 \leq C\|A\mathbf{x}_S^\# + \mathbf{e}\|_2 \quad (3.21)$$

for any $S \subset \{1, \dots, N\}$ such that $|S| = s$. Moreover if its RIC of order $26s$ is such that

$$\delta_{26s} < \frac{1}{6}$$

then there exist constants $C, D > 0$ such that for any $1 \leq p \leq 2$, the final iterate of OMP satisfy

$$\|\mathbf{x}^\# - \mathbf{x}^{(24s)}\|_p \leq \frac{C}{s^{1-1/p}} \sigma_s(\mathbf{x}^\#)_1 + Ds^{1/p-1/2} \|\mathbf{e}\|_2. \quad (3.22)$$

Overall, we have an approximation which can be made as close as possible to the original vector $\mathbf{x}^\#$. Although without the limitation on the sparsity of the output vector, the risk of overfitting the noise is real. The number of iterations is suboptimal here and the resulting approximation is much less sparse than expected. However, every iteration is very cheap to compute thanks to the Sherman–Morrison formula since only rank one updates are required for the projection step.

Of particular interest to me is the Hard Thresholding Pursuit (HTP) introduced by Foucart [55], yet another algorithm for sparse recovery. It was the basis for our paper on the number of iterations required for convergence [27] and further extensions [23]. Its definition is summarized as Algorithm 5

Algorithm 5 Hard Thresholding Pursuit

```

1: function HTP( $A \in \mathbb{C}^{m \times N}$ ,  $s$ ,  $\mathbf{y} \in \mathbb{C}^m$ ,  $\mathbf{x}^{(0)}$ )
2:   Inputs: a design matrix  $A$ , an initial guess  $\mathbf{x}^0$ , measurements  $\mathbf{y}$ , a target sparsity  $s$ 
3:   repeat
4:      $\mathbf{u}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + A^*(\mathbf{y} - A\mathbf{x}^{(k)})$ 
5:      $\Lambda^{(k+1)} \leftarrow$  set of  $s$  largest entries in magnitude of  $\mathbf{u}^{(k+1)}$ 
6:      $\mathbf{x}^{(k+1)} \leftarrow \operatorname{argmin}_{\mathbf{z} \in \mathbb{C}^N} \{ \|\mathbf{y} - A\mathbf{z}\|_2, \operatorname{supp} \subset \Lambda^{(k+1)} \}$ 
7:      $k \leftarrow k + 1$ 
8:   until a stopping criterion is met
9:   return  $\mathbf{x}^{(k)}$ 
10: end function.

```

HTP works by combining a bit of both greedy and thresholding algorithms by selecting the best feature set at a given iteration and then finding the best approximation (in the ℓ_2 sense) on this set of features. In contrast to the previous algorithms, an intuitive stopping criterion is the detection of cycles in the set of active indices after a certain number of iterations k^* , i.e. finding a integer c such that $\Lambda^{(k+c)} = \Lambda^{(k)}$ for all $k \geq k^*$. As mentioned earlier, one may prove results for the recovery of HTP similar to those for IHT, taken from [56].

Theorem 3.9: Recovery via HTP or IHT

Let $A \in \mathbb{C}^{m \times N}$ be a matrix whose $3s$ RIC satisfies

$$\delta_{3s} < \frac{1}{\sqrt{3}}.$$

Then for all $\mathbf{x}^\# \in \mathbb{C}^N$, $\mathbf{e} \in \mathbb{C}^m$, $S \subset \{1, \dots, N\}$ with $|S| = s$, and $\mathbf{y} = A\mathbf{x}^\# + \mathbf{e}$, the sequence of iterates generated by IHT or HTP satisfies

$$\|\mathbf{x}^{(k)} - \mathbf{x}_S^\#\|_2 \leq \rho^k \|\mathbf{x}^{(0)} - \mathbf{x}_S^\#\|_2 + \tau \|A\mathbf{x}_S^\# + \mathbf{e}\|_2 \quad (3.23)$$

with

$$\rho = \sqrt{3}\delta_{3s}$$

for IHT and

$$\rho = \sqrt{2\delta_{3s}^2 / (1 - \delta_{2s}^2)}$$

for HTP.

Simply said, this Theorem ensures a geometric decay of the error with respect to the number of iterations. If we compare the results from IHT and HTP in Thm. 3.9, one might think that both algorithms (IHT and HTP) converge equally well, even though HTP's iterations are much more computationally expensive. It is important to look in details the value of the ρ in each cases. Indeed, assuming $\delta_{3s} < 1/\sqrt{3}$, we have $\sqrt{2\delta_{3s}^2 / (1 - \delta_{2s}^2)} < \sqrt{3}\delta_{3s}$ and therefore the actual speed of convergence may be much faster for HTP (in terms of number of iterations).

Assessing the number of iterations is one of the results for our paper [27]. Moreover, we have extended HTP rendering it *greedier* by letting the size of the active set of indices increase with the number of iterations. I generalized this even further in [23] by allowing more involved index set selection functions and analyzing their impacts on the recovery conditions. The most general HTP (coined *f*HTP) is given as Algo. 6. The only constraints on the function $f : \mathbb{N} \rightarrow \mathbb{N}$ is that it be non decreasing and that there exists a constant k_0 such that $f(k) \geq s$ for all $k \geq k_0$; in other words, we make sure to reach an active index size at least as large as the sparsity of the sought-after signal. I give more detail on this function f in the next paragraphs.

Algorithm 6 *f*-Hard Thresholding Pursuit

```

1: function f-HTP( $A \in \mathbb{C}^{m \times N}$ ,  $s$ ,  $\mathbf{y} \in \mathbb{C}^m$ ,  $\mathbf{x}^{(0)}$ )
2:   Inputs: a design matrix  $A$ , an initial guess  $\mathbf{x}^{(0)}$ , measurements  $\mathbf{y}$ , a target sparsity  $s$ 
3:   repeat
4:      $\mathbf{u}^{(k+1)} = \mathbf{x}^{(k)} + A^*(\mathbf{y} - A\mathbf{x}^{(k)})$ 
5:      $\Lambda^{(k+1)} = \text{set of } f(k+1) \text{ largest entries in magnitude of } \mathbf{u}^{(k+1)}$ 
6:      $\mathbf{x}^{(k+1)} = \operatorname{argmin}_{\mathbf{z} \in \mathbb{C}^N} \{ \|\mathbf{y} - A\mathbf{z}\|_2, \operatorname{supp} \subset \Lambda^{(k+1)} \}$ 
7:      $k = k + 1$ 
8:   until a stopping criterion is met
9:   return  $\mathbf{x}^{(k)}$ 
10: end function.

```

The only difference with the original HTP lies in the index selection. We select $f(k)$ indices at iteration k instead of the imposed size s for HTP. In particular, the constant case $f(k) = s$ ($k_0 = 0$) recovers Foucart's HTP algorithm [55] while the case $f(k) = k$ ($k_0 = s$) corresponds to the GHTP algorithm introduced in [27]. In [27] we proved the following recovery result.

Theorem 3.10: Uniform recovery via Graded HTP

If the restricted isometry constant of the matrix $A \in \mathbb{C}^{m \times N}$ obeys

$$\delta_{3s} \leq \frac{1}{3} \text{ for (HTP), and } \delta_{9s} \leq \frac{1}{3} \text{ for (GHTP),}$$

then the sequences $(\mathbf{x}^{(k)})_k$ produced by (HTP) or (GHTP) with $\mathbf{y} = A\mathbf{x}^\# + \mathbf{e} \in \mathbb{C}^m$ for some s -sparse $\mathbf{x}^\# \in \mathbb{C}^N$ and some $\mathbf{e} \in \mathbb{C}^m$ with $\|\mathbf{e}\|_2 \leq \gamma \widetilde{\mathbf{x}}^\#_s$ where $\widetilde{\mathbf{x}}$ corresponds to the vector $\mathbf{x}$ whose entries are sorted in decreasing order according to their magnitudes, satisfy

$$\|\mathbf{x}^\# - \mathbf{x}^{(k^*)}\|_2 \leq d\|\mathbf{e}\|_2, \quad k^* \leq cs.$$

The constants $c \leq 3$ for (HTP) and $c \leq 4$ for (GHTP), $d \leq 2.45$, and $\gamma \geq 0.079$ depend only on δ_{3s} or δ_{9s} .

The generalization of this theorem for the generalized family of algorithms *f*HTP reads

Theorem 3.11: Recovery via f HTP

If the restricted isometry constant of the matrix $A \in \mathbb{C}^{m \times N}$ obeys

$$\delta_{2f(k^*)+k_0} \leq \frac{1}{3},$$

then the sequence $(\mathbf{x}^{(k)})$ produced by (f -HTP) with $\mathbf{y} = A\mathbf{x}^\# + \mathbf{e} \in \mathbb{C}^m$ for some s -sparse $\mathbf{x}^\# \in \mathbb{C}^N$ and some $\mathbf{e} \in \mathbb{C}^m$ with $\|\mathbf{e}\|_2 \leq \gamma \widetilde{\mathbf{x}^\#}_s$ satisfies

$$\|\mathbf{x} - \mathbf{x}^{k^*}\|_2 \leq d\|\mathbf{e}\|_2, \quad k^* \leq cs.$$

The constants $c \leq 4$, $d \leq 2.45$, and $\gamma \geq 0.079$ depend only on $\delta_{s+2f(k^*)}$.

The choice of the function f as a mechanism to choose the number of indices selected at a given iteration is an interesting one and should be carefully analyzed based on the needs. It is suggested to have a logarithmic shape for such a function. Indeed, this entails picking many indices in the earlier stages of the algorithms, where indices can still be rejected, and the slowly reducing the rate at which the number of indices increases. Theoretically speaking, it allows to keep the restrictions on the RIP manageable and comparable to other state of the art algorithms.

Open question 3.1: Index selection function

One may select $f(k) = k$ or $f(k) = 2k$ or $f(k) = C \log(k)$ but it is probably better to have an adaptive way of selecting indices, i.e. defining $f(k+1) = f(k) + g(k, A, \mathbf{x}^{(0)}, \mathbf{y})$ where g would depend on the norm of the residual, of the norm of the decay. This relatively low-hanging fruit would be a good topic for a Master student interested in either numerical analysis of algorithms or signal processing. It may also be continued as a Ph.D. subject by working out some proof in terms of non uniform recovery, see Open Problem 3.2.

Another important aspect to analyze is the number of indices which are found in both $\Lambda^{(k+1)}$ and $\Lambda^{(k)}$. The first numerical experiments suggest that the intersection is far from void, but this needs to be quantified properly. In particular, understanding this aspect would allow to think about potential faster updates of the projection through low-rank updates.

One of the main advantages of the f HTP / GHTP methods over OMP is that a wrongly¹ selected index at an early iteration may be later rejected with greedy HTP while it is kept forever in OMP. This justifies that one may take bigger risks towards the beginning of the algorithm with f HTP. It is also the reason why nonuniform recovery in exactly s iterations is possible via GHTP which is not guaranteed with OMP. This is the subject of the next theorems.

¹wrongly means here that it is not necessarily among the top s components in the final solution

Theorem 3.12: Non uniform recovery via GHTP

Let $\lambda \geq 1$ and let $\mathbf{x}^\# \in \mathbb{C}^N$ be an s -sparse vector. Define $\widetilde{\mathbf{x}^\#}_1$ be the largest entry (in magnitude) of $\mathbf{x}^\#$ and let $\widetilde{\mathbf{x}^\#}_s$ be its s^{th} largest entry. Assume that $\widetilde{\mathbf{x}^\#}_1 \leq \lambda \widetilde{\mathbf{x}^\#}_s$. If $A \in \mathbb{R}^{m \times N}$ is a matrix whose entries are drawn iid at random according to a subgaussian distribution with mean 0 and variance $1/m$. Assuming

$$m \geq Cs \ln(N),$$

then it occurs with probability at least $1 - 2N^{-c}$ that, for all $\mathbf{e} \in \mathbb{C}^m$ with $\|\mathbf{e}\|_2 \leq \gamma \widetilde{\mathbf{x}^\#}_s$, the sequences $(\Lambda^{(k)})$ and $(\mathbf{x}^{(k)})$ produced by (GHTP) with $\mathbf{y} = A\mathbf{x}^\# + \mathbf{e}$ satisfy, at iteration s ,

$$\Lambda^{(s)} = \text{supp}(\mathbf{x}^\#) \quad \text{and} \quad \|\mathbf{x}^\# - \mathbf{x}^{(s)}\|_2 \leq d \|\mathbf{e}\|_2.$$

The constants γ and d depend only on λ , while the constant C depends on γ and c .

This theorem shows that for vectors that are relatively flat, i.e. their slow decay is controlled by the λ parameter, exactly s iterations are required to recover the s largest entries of the original signal $\mathbf{x}^\#$.

The next theorem handles the case of a vector with geometric decay, a strong assumption, but allows

1. to use Fourier matrices and matrices from bounded orthonormal systems
2. to go below the usually required number of measurements for (sub)Gaussian matrices

Theorem 3.13: Nonuniform recovery via GHTP with Fourier matrices

Let $\mu \in (0, 1)$ and let $\mathbf{x}^\# \in \mathbb{C}^N$ be an s -sparse vector such that $\widetilde{\mathbf{x}^\#}_{j+1} \leq \mu \widetilde{\mathbf{x}^\#}_j$ for all $j \in \{1, \dots, s-1\}$, with $\widetilde{\mathbf{x}^\#}_j$ denoting the j^{th} largest entry in magnitude of $\mathbf{x}^\#$. If $A \in \mathbb{R}^{m \times N}$ is a (normalized) subgaussian random matrix with

$$m \geq \max\{Cs, C' \ln(N)\},$$

or if $A \in \mathbb{C}^{m \times N}$ is a (normalized) matrix whose entries are random samples of a bounded orthonormal system with

$$m \geq C'' s \ln(N),$$

then it occurs with probability at least $1 - 2N^{-c}$ that, for all $\mathbf{e} \in \mathbb{C}^m$ with $\|\mathbf{e}\|_2 \leq \gamma \widetilde{\mathbf{x}^\#}_s$, the sequences $(\Lambda^{(k)})$ and $(\mathbf{x}^{(k)})$ produced by (GHTP) with $\mathbf{y} = A\mathbf{x}^\# + \mathbf{e}$ satisfy, at iteration s ,

$$\Lambda^{(s)} = \text{supp}(\mathbf{x}^\#) \quad \text{and} \quad \|\mathbf{x}^\# - \mathbf{x}^{(s)}\|_2 \leq d \|\mathbf{e}\|_2.$$

The constants C , γ , and d depend only on μ , while the constants C' and C'' depend on μ and c .

Note that these non uniform results can also be generalized to the case of the more general f -HTP algorithms, but are much harder to read. Note also that the results are valid for all error vectors $\mathbf{e}$ uniformly.

Open question 3.2: Recovery of general shaped vectors

Numerically, it was verified that the claim in the two previous theorems remain valid for any shape of vectors. The proof still eludes us today. This is an interesting research topic for an advanced Master student / early stage Ph.D. student in order to gain better understanding of algorithms for signal processing. Moreover, understanding this would also help developing novel proofs for model-based signal recovery, such as non negative vectors (in which the absolute values in the index selection within f -HTP is omitted) or even quantized version of the f -HTP family of algorithms.

3.3 Sparse numerical approximation to parametric PDEs

3.3.1 General background

Interested in the applications of compressed sensing, I later started working on the approximation of numerical schemes for parametric and stochastic PDEs. The final aspect of our research [31], detailed in the next Section 3.3.4, shows that with a reasonable compute costs – in fact close to the evaluation cost of a single solution – one can approximate uniformly for all parameters, in very high dimensions, the parametric solution to an operator equation. I detail here mainly the compressed sensing aspect of the methods, keeping the numerical PDE aspects to a minimum. I am interested in the numerical approximation of a generic linear operator of the form

$$A(\cdot; \mathbf{y})u(\cdot; \mathbf{y}) = f(\cdot). \quad (3.24)$$

$\mathbf{y} \in U$ corresponds to a vector of parameters, which is assumed to be high-dimensional, rendering the uniform approximation of the map $\mathbf{y} \mapsto u(\cdot; \mathbf{y}) \in \mathcal{X}$ difficult due to the *curse of dimensionality* (see [16, 43, 42] for a precise description of this phenomenon). $A(\cdot; \mathbf{y}) : \mathcal{X} \rightarrow \mathcal{Y}'$ is a linear operator from a reflexive Banach space $\mathcal{X}$ to the dual of another reflexive Banach space $\mathcal{Y}$. In particular I am interested in the case of affine linear equations of the form

$$A(\cdot; \mathbf{y}) = A_0(\cdot) + \sum_{j=1}^d y_j A_j(\cdot) \quad (3.25)$$

with $A_0, A_1, \dots$ being bounded, linear operators. This is motivated by problems stemming from the stochastic PDE community, once a Karhunen-Loève decomposition has been applied to the stochastic fields. However the approaches detailed here are applicable to other cases, even non-linear operators, or shape optimization (see the Master thesis I supervised [36]), as long as certain holomorphic dependencies apply, see [42]. For simplicity we look at the situation that $U = [-1, 1]^d$ or $U = [-1, 1]^{\mathbb{N}}$ (corresponding to $d = \infty$) and approximate a functional of the solution, i.e. we aim to approximate

$$G : U \rightarrow \mathbb{R}, \quad \mathbf{y} \mapsto G(\mathbf{y}) = \mathcal{G}(u(\cdot; \mathbf{y})).$$

$\mathcal{G}$ is a bounded linear map denoting a *quantity of interest*.

Open question 3.3: Recovery of the full solution

The work presented here concentrates on the recovery of functionals of solutions to operator equations. Adapting the proposed approach to the recovery of the full solution is still relatively open. An ongoing work aims at providing similar approximation results for the case of the recovery of the whole function, within a Hilbert space, see the draft [24], but some work needs to be done to make it work in a multi-level setting and in a general Banach space.

Our *compressed sensing approximation* [26, 31], which is based on preliminary works of

Rauhut and Schwab [103] and Rauhut and Ward [104], relies on the expansion of the solution map on a generalized polynomial chaos

$$u(\mathbf{x}; \mathbf{y}) = \sum_{\nu \in \mathcal{F}} u_\nu(\mathbf{x}) \psi_\nu(\mathbf{y}) \quad (3.26)$$

$$G(\mathbf{y}) = \sum_{\nu \in \mathcal{F}} g_\nu \psi_\nu(\mathbf{y}), \quad g_\nu = \mathcal{G}(u_\nu), \quad (3.27)$$

where $(\psi_\nu)_{\nu \in \mathcal{F}}$ denotes an orthonormal system of $L^2(U, \eta)$ for a probability measure η on U . Here, the index set

$$\mathcal{F} := \{\nu \in \mathbb{N}_0^\mathbb{N} : |\text{supp}(\nu)| < \infty\} \quad (3.28)$$

denotes the (countable) set of multi-indices with finite support with, for $\nu \in \mathcal{F}$, $\text{supp}(\nu) := \{j \in \mathbb{N} : \nu_j \neq 0\}$. An approximation of G can therefore be obtained via computing an approximation $(g_\nu^*)_{\nu \in \mathcal{F}}$ to the sequence $(g_\nu)_{\nu \in \mathcal{F}}$.

The idea of the approach is to (approximately) recover the sequence of coefficients $(g_\nu)_{\nu \in \mathcal{F}}$ from the knowledge – usually a numerical approximation – of a small number m of solutions

$$b_i = G(\mathbf{y}^{(i)}) = \mathcal{G}(u(\cdot; \mathbf{y}^{(i)})), \quad i = 1, \dots, m, \quad (3.29)$$

where each sample $\mathbf{y}^{(i)}$ is drawn i.i.d. at random according to η .

The validation of the approximation method detailed below requires 4 steps:

1. Verify the sparsity of the expansion
2. Truncate the set of multi-indices to a manageable, finite dimensional subset
3. Estimate the numerical approximation error of a single PDE solve
4. Combine such estimates with the approximation results from compressed sensing such as Thm. 3.2

In Section 3.3.4, we reduce the computation costs via a multi level scheme and some aspects of this list (in particular the first point) will need to be reviewed.

The suggested approach may work only under the condition that the expansion be sparse, which I detail now. Most of the single level approach detailed here is analyzed in the original work of Rauhut and Schwab [103] with the numerical details found in [26]. In particular, we consider tensorized Chebyshev polynomials defined as

$$T_\nu(\mathbf{y}) = \prod_{j=1}^{\infty} T_{\nu_j}(y_j) = \prod_{j \in \text{supp}(\nu)} T_{\nu_j}(y_j), \quad \mathbf{y} \in U, \quad \nu \in \mathcal{F}, \quad (3.30)$$

with the univariate Chebyshev polynomials defined by

$$T_0(t) \equiv 1, \quad T_j(t) = \sqrt{2} \cos(j \arccos(t)), \quad j = 1, 2, 3, \dots \quad (3.31)$$

The univariate Chebyshev polynomials T_j defined in (3.31) form an orthonormal system in $L^2([-1, 1]; \sigma)$, i.e. $\int_{-1}^1 T_k(t) T_l(t) d\sigma(t) = \delta_{k,l}$ for all $k, l \in \mathbb{N}_0$, where the probability measure σ is defined on $[-1, 1]$ as $d\sigma(t) := \frac{dt}{\pi\sqrt{1-t^2}}$. Similarly, with the product probability (Chebyshev) measure

$$d\eta(\mathbf{y}) := \bigotimes_{j \geq 1} d\sigma(y_j) = \bigotimes_{j \geq 1} \frac{dy_j}{\pi\sqrt{1-y_j^2}}, \quad (3.32)$$

the tensorized Chebyshev polynomials (3.30) are orthonormal with respect to η :

$$\int_{\mathbf{y} \in U} T_\mu(\mathbf{y}) T_\nu(\mathbf{y}) d\eta(\mathbf{y}) = \delta_{\mu,\nu}, \quad \mu, \nu \in \mathcal{F}.$$

A Neumann series argument [103, 49, 106] implies that if A_0 is invertible and there exists a $0 < \delta < 1$ such that

$$\sum_j \|A_0^{-1} A_j\|_{\mathcal{L}(\mathcal{X}, \mathcal{X})} \leq 1 - \delta < 1 \quad (3.33)$$

then the operator $A(\cdot; \mathbf{y})$ is uniformly (with respect to $\mathbf{y}$) invertible and

$$\|u(\cdot; \mathbf{y})\|_{\mathcal{X}} \leq \frac{\|f\|_{\mathcal{Y}'}}{1 - \delta}. \quad (3.34)$$

3.3.2 Basics of weighted sparsity

Noting from Eq. (3.13) that the number of samples scales quadratically with the bound K of the basis system and that $\|T_\nu\|_\infty = 2^{\|\nu\|/2}$ for all $\nu \in \mathcal{F}$, it is clear that one would require a number of samples scaling as (omitting some elements) $m \geq Cs \log^2(s) 2^d$, i.e. an exponential number of samples! To break this *curse of dimensionality*, one introduces [104] the concept of weighted sparsity. We recall only the most important results here and refer the reader to [104] for more details.

Definition 3.10: Weighted sparsity

Let Λ denote some index set of size $N = |\Lambda|$. For a given weight vector $\omega \in \mathbb{R}^N$ with $\omega_\nu \geq 1$ for all $\nu \in \Lambda$, a vector $\mathbf{x} \in \mathbb{R}^N$ is called *weighted s -sparse* for some $s \geq 1$ if

$$\|\mathbf{x}\|_{0,\omega} := \sum_{\nu \in \Lambda: x_\nu \neq 0} \omega_\nu^2 \leq s.$$

We also define weighted ℓ_p (quasi-)norms as

$$\|\mathbf{x}\|_{\omega,p}^p := \sum_{\nu \in \Lambda} \omega_\nu^{2-p} |x_\nu|^p, \quad 0 < p \leq 2. \quad (3.35)$$

In particular, it holds $\|\mathbf{x}\|_{\omega,2} = \|\mathbf{x}\|_2$ for any weight sequence ω and $\|\mathbf{x}\|_{\omega,1} = \|\mathbf{x} \odot \omega\|_1$, where $\odot$ signifies pointwise multiplication. The *error of best weighted s -term approximation* in $\ell_{\omega,p}$ is defined as

$$\sigma_s(\mathbf{x})_{\omega,p} := \inf_{\mathbf{z}: \|\mathbf{z}\|_{\omega,0} \leq s} \|\mathbf{x} - \mathbf{z}\|_{\omega,p}.$$

and a variant of Stechkin's estimate reads

$$\sigma_s(\mathbf{x})_{\omega,q} \leq (s - \|\omega\|_\infty^2)^{1/q-1/p} \|\mathbf{x}\|_{\omega,p}. \quad (3.36)$$

As it was the case for the classical sparsity, a small weighted ℓ_p norm for $p < 1$ is a suitable marker for good enough weighted sparse approximations. Similar to Eqs. (3.1) and (3.2), we can redefine weighted ℓ_1 minimization programs

$$\begin{cases} \min \|\mathbf{x}\|_{\omega,1} \\ \text{s.t. } \|\Phi \mathbf{x} - \mathbf{y}\|_2 \leq \varepsilon, \end{cases} \quad (3.37)$$

where Φ is a matrix stemming from a bounded orthogonal system, see below. Adding weights allow to tailor the solution towards more likely scenarios, as illustrated in Fig. 3.2. I like to see weights from two perspectives. The first one comes from a data sciences point of view. Similar to eigenvalues values of the correlation matrix in PCA, the weights describe the amount of variability tolerated along a certain axis. Therefore, larger weights mean that the deviations along the associated axis are more tolerated than along axis with smaller weights. The other point of view is to see the weights as a way to quantify the importance of certain parameters: larger weights mean larger impact on the solution. This is in fact my favorite way to think about weighted sparse recovery in the current context.

One may also redefine the thresholding algorithms where the index selection is replaced by the selection of the weighted s largest entries (in magnitude). This is however an NP hard problem and only approximation of this operator can be easily implemented. One strategy is to replace the IHT with the following updates:

$$\mathbf{x}^{(n+1)} = \tilde{H}_{\omega,3s}(\mathbf{x}^{(n)} + A^*(\mathbf{y} - A\mathbf{x}^{(n)})) \quad (3.38)$$

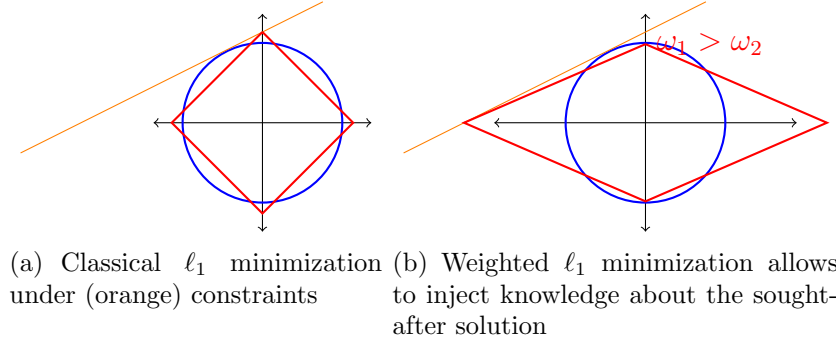


Figure 3.2: Weights in the minimization allow to drive the solution of the ℓ_1 minimization problem towards a more plausible solution: the ℓ_2 solution and the constraints have not changed, yet minimization weighted ℓ_1 problems changes the solution.

where $\tilde{H}_{\omega,s}$ is an approximation to the hard thresholding operator. One such example is given by the following principle. Let $\pi : \{1, \dots, N\} \rightarrow \{1, \dots, N\}$ be a permutation such that

$$\frac{|x_{\pi(1)}|}{\omega_{\pi(1)}} \geq \frac{|x_{\pi(2)}|}{\omega_{\pi(2)}} \geq \dots \geq \frac{|x_{\pi(N)}|}{\omega_{\pi(N)}}$$

and k be the largest integer such that $\sum_{j=1}^k \omega_{\pi(j)}^2 \leq s$ and define $S = \{\pi(j), 1 \leq j \leq k\}$. Then we may use $\tilde{H}_{\omega,s}(\mathbf{x})_i = \mathbf{x}_i$ if $i \in S$ and 0 elsewhere.

Open question 3.4: Iterative and greedy weighted sparse recovery

Finding efficient ways to solve the Hard Thresholding operator in the weighted problem is no trivial task. Although, ℓ_1 minimization (or convex / conic programming in general) is a theoretically sound way to go, it might not be computationally tractable in large instances. Greedy and iterative approaches are usually better suited, but their generalization(s) using weights is not clear. Some results are already out [76, 36, 53], but finer analysis are still required. It is also worth investigating the use of knapsack optimization here to find a better solution. This would correspond to a great opportunity for young researchers to enter the world of optimization, algorithmics, and signal processing.

We have two recovery guarantees for weighted sparse approximation. Note that the current theorems are written using BOS and not random Gaussian matrices. This is motivated by the fact that having i.i.d. entries in a matrix would render moot any prior knowledge embedded in the weights.

Theorem 3.14: Weighted ℓ_1 recovery with Bounded Orthonormal Systems

Let Λ be an index set and $(\varphi_j)_{j \in \Lambda}$ be a bounded orthonormal system with probability measure η . Assume that A is a matrix obtained from sampling this orthogonal system, i.e. $\Phi_{i,j} = \varphi_j(\mathbf{y}^{(i)})$, $j \in \Lambda$ and $1 \leq i \leq m$ for some samples $\mathbf{y}^{(i)}$ drawn i.i.d. at random from η . Let $\omega = (\omega_1, \dots, \omega_N)$ with $\omega_i \geq 1$ be a weight vector. Φ satisfies $\delta_{\omega, 3s} < 1/3$ implies that the solution $\mathbf{x}^*$ of (3.37) with $\mathbf{y} = A\mathbf{x}^\# + \mathbf{e}$ and $\|\mathbf{e}\| \leq \varepsilon$ satisfies

$$\|\mathbf{x}^* - \mathbf{x}^\#\|_{\omega,1} \leq c_1 \sigma_s(\mathbf{x}^\#)_{\omega,1} + d_1 \sqrt{s} \varepsilon, \quad (3.39)$$

$$\|\mathbf{x}^* - \mathbf{x}^\#\|_2 \leq c_2 \frac{\sigma_s(\mathbf{x}^\#)_{\omega,1}}{\sqrt{s}} + d_2 \varepsilon, \quad (3.40)$$

for constants c_1, c_2, d_1, d_2 only depending on $\delta_{\omega, 3s}$.

Theorem 3.15: Weighted sparse recovery via IHT for BOS

Let $(\varphi_j)_{j \in \Lambda}$ be a bounded orthonormal system with probability measure η . Assume that A is a matrix obtained from sampling this orthogonal system, i.e. $\Phi_{i,j} = \varphi_j(\mathbf{y}^{(i)})$, $j \in \Lambda$ and $1 \leq i \leq m$ for some samples $\mathbf{y}^{(i)}$ drawn i.i.d. at random from η . Let $\omega = (\omega_1, \dots, \omega_{|\Lambda|})$ with $\omega_i \geq 1$ be a weight vector. If A satisfies $\delta_{\omega, 7s} < 1/\sqrt{3}$ for some $s \geq 2\|\omega\|_\infty^2$ and assume that weighted IHT is run with parameter s and $\mathbf{y} = A\mathbf{x}^\# + \mathbf{e}$. Then the iterates $\mathbf{x}^{(j)}$ satisfy

$$\|\mathbf{x}^\# - \mathbf{x}^{(j)}\|_{\omega,1} \leq c_1 \sigma_{s/2}(\mathbf{x}^\#)_{\omega,1} + d_1 \sqrt{s} \|\mathbf{e}\|_2 + \rho^j \sqrt{s/2} \|\mathbf{x}\|_2, \quad (3.41)$$

$$\|\mathbf{x}^\# - \mathbf{x}^{(j)}\|_2 \leq c_2 \frac{\sigma_{s/2}(\mathbf{x}^\#)_{\omega,1}}{\sqrt{s}} + d_2 \|\mathbf{e}\|_2 + \rho^j \|\mathbf{x}\|_2, \quad (3.42)$$

where c_1, c_2, d_1, d_2 and $0 < \rho < 1$ depend on $\delta_{\omega, 7s}$.

In the previous two theorems, it is assumed that the index set is finite. In practical cases, it will be infinite and some truncation needs to be taken into account. This is the purposes of the following results. The proof is done by assuming that the components that have been truncated out can be estimated using tail estimates from high-dimensional probabilities. The following theorem reverts to a generic infinite $\mathcal{F}$ to distinguish from the finite Λ in the previous theorems in defining the index sets. Moreover, the following theorem will play a crucial role in the section on multi-level approximation.

Theorem 3.16: wIHT for function approximation

Let $G : U \rightarrow \mathbb{R}$ be such that a decomposition $G = \sum_{\nu \in \mathcal{F}} g_\nu \phi_j$ exists. Assume that the sequence of coefficients $\mathbf{g} = (g_\nu)_{\nu \in \mathcal{F}} \in \ell_{\omega,p}(\mathcal{F})$ for some $0 < p \leq 1$ and weights ω , for some infinite $\mathcal{F}$.

Let $s \geq 4\|\omega\|_\infty^2$ and define $\Gamma^s = \{\nu \in \mathcal{F} : \omega_\nu^2 \leq s/2\}$. For $\rho \in (0, 1)$ let

$$m \geq C_\rho s \log^2(s) \log(|\Gamma^s|), \quad (3.43)$$

where C_ρ is a constant only depending on ρ .

Assume that $\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(m)}$ are chosen independently at random according to the orthogonalizing measure and let

$$b_i = G(\mathbf{y}^{(i)}) + e_i, \quad i = 1, \dots, m,$$

be perturbed random samples of G . Let $\Phi \in \mathbb{R}^{m \times |\Gamma^s|}$ be the sampling matrix defined in the previous theorem and let $\mathbf{g}^{(k)} \in \mathbb{R}^{\Gamma^s}$ be the k^{th} iterate of weighted IHT applied to the vector $\mathbf{b}$.

Then there exist constants $c, c_0 > 0$ such that, with probability at least $1 - e^{-cm/s} \geq 1 - (|\Gamma^s|)^{-c_0}$, the function

$$G^{(k)}(\mathbf{y}) = \sum_{\nu \in \Gamma^s} g_\nu^{(k)} T_\nu(\mathbf{y})$$

satisfies

$$\|G - G^{(k)}\|_\infty \leq C\|\mathbf{g}\|_{\omega,p} s^{1-1/p} + D\sqrt{\frac{s}{m}}\|\mathbf{e}\|_2 + \rho^k \sqrt{s}\|\mathbf{g}\|_2, \quad (3.44)$$

$$\|G - G^{(k)}\|_{L^2(U,\eta)} \leq C\|\mathbf{g}\|_{\omega,p} s^{1/2-1/p} + D\frac{\|\mathbf{e}\|_2}{\sqrt{m}} + \rho^k \|\mathbf{g}\|_2, \quad (3.45)$$

where $C, D > 0$ are absolute constants.

3.3.3 Single level approximation

With the background on weighted compressed sensing in mind, I now give details about the *Single Level Compressed Sensing Petrov Galerkin* method [103, 26]. I review the various elements as stated above, such as the compressibility of the solution map on Chebyshev polynomials. Let $d \in \mathbb{N} \cup \{\infty\}$ be the dimension of the parameter vector, for a suitable weight sequence $\mathbf{v} = (v_j)_{j=1}^d \in [1, \infty)^{d^2}$ and a compressibility potential $0 < p \leq 1$, we

²We want to make the distinction clear between two weights sequences involved in this section: the weights associated to certain operators in Eq. (3.25), which we denote by $\mathbf{v}$, and the weights associated to

assume that there exists a positive constant $\kappa_{v,p} < 1$ such that

$$\sum_{j=1}^d \|A_0^{-1} A_j\|_{\mathcal{L}(\mathcal{X}, \mathcal{X})} v_j^{(2-p)/p} \leq \kappa_{v,p} \quad \text{and} \quad \sum_{j=1}^d \|A_0^{-1} A_j\|_{\mathcal{L}(\mathcal{X}, \mathcal{X})}^p v_j^{2-p} < \infty. \quad (3.46)$$

For $\nu \in \mathcal{F}$, set

$$\omega_\nu = 2^{\|\nu\|_0/2} \mathbf{v}^\nu = \prod_{j \in \text{supp } \nu} (\sqrt{2} v_j^{\nu_j}). \quad (3.47)$$

Then the sequence of norms $(\|u_\nu\|_{\mathcal{X}})$ is contained in $\ell_{\omega,p}(\mathcal{F})$, i.e.,

$$\sum_{\nu \in \mathcal{F}} \omega_\nu^{2-p} \|u_\nu\|_{\mathcal{X}}^p < \infty. \quad (3.48)$$

Applying the continuous linear functional $\mathcal{G} : \mathcal{X} \rightarrow \mathbb{R}$ to $u(\cdot; \mathbf{y})$ gives an expansion

$$G(\mathbf{y}) = \mathcal{G}(u(\cdot; \mathbf{y})) = \sum_{\nu \in \mathcal{F}} g_\nu T_\nu(\mathbf{y}) \quad (3.49)$$

with real-valued coefficients $g_\nu = \mathcal{G}(u_\nu)$ satisfying

$$\sum_{\nu \in \mathcal{F}} \omega_\nu^{2-p} |g_\nu|^p < \infty. \quad (3.50)$$

So far, all computations are only theoretical and assume potentially infinite dimensional spaces: the space of parameters $\mathbf{y}$, associated to the linear operators $A_1, A_2, \dots$, and the space of tensorized Chebyshev polynomials. In order to have a working implementation of the method, one needs to truncate both expansions:

1. Polynomial set truncation
2. Operator expansion truncation

Regarding the polynomial set, for a target weighted sparsity parameter s it was suggested in [104, 103] to use the initial set of indices

$$\Gamma = \Gamma^s = \{\nu \in \mathcal{F} : \omega_\nu^2 \leq s/2\}. \quad (3.51)$$

Note that in the case of finite d , the set Γ is finite if $v_j > 1$ for all j and in the case of infinite d , Γ is finite if additionally $v_j \rightarrow \infty$ as $j \rightarrow \infty$. Once this set of multi-indices is selected, one may truncate the expansion itself to only useful elements. We replace the sampling points $\mathbf{y}^{(i)} = (y_1^{(i)}, y_2^{(i)}, \dots)$ with the truncated ones $\tilde{\mathbf{y}}^{(i)} = (y_1^{(i)}, \dots, y_k^{(i)}, 0, 0, \dots)$, where k is such that $T_\nu(y_1, y_2, \dots) = T_\nu(y_1, \dots, y_k, 0, 0, \dots)$ for all $\nu \in \Gamma^s$, i.e., $k \geq \max\{j : \nu_j \geq 1, \nu \in \Gamma^s\}$.

the tensorized Chebyshev coefficients, which are the ones which will be used for the sparse approximation problem and which we usually denote with ω .

We introduce $\mathcal{F}_{(k)} = \{\nu \in \mathcal{F} : \nu_j = 0 \text{ for all } j > k\}$ and write

$$\bar{G} = \sum_{\nu \in \mathcal{F}_k} g_\nu T_\nu.$$

Then we have, for all $i = 1, \dots, m$,

$$G(\tilde{\mathbf{y}}^{(i)}) = \bar{G}(\tilde{\mathbf{y}}^{(i)}) = \bar{G}(\mathbf{y}^{(i)})$$

The samples $G(\tilde{\mathbf{y}}^{(i)}) = \bar{G}(\mathbf{y}^{(i)})$ can be seen as perturbed samples of the sought-after signal and the analysis for the recovery results carry over (see for instance [31]).

The overall single level approximation, as detailed in [103, 26] is detailed in Algorithm 7 and the recovery results are summarized as Theorem 3.17. Notice that the algorithm is written with a general index set $\mathcal{J}_0^s$. Indeed, one could have another strategy in selecting the potentially relevant indices. The Theorem will use explicitly $\mathcal{J}_0^s = \Gamma^s$.

Algorithm 7 Pseudo-code for the Compressed-Sensing-Petrov-Galerkin Algorithm

- 1: **function** CSPG(Weights $(v_j)_{j \geq 1}$, target accuracy ε and sparsity s , a multiindex set $\mathcal{J}_0^s$ with $N = |\mathcal{J}_0^s| < \infty$, a compressibility parameter $0 < p < 1$, and a number of samples m)
- 2: input reference image
- 3: **for** $l = 1, \dots, m$ **do**
- 4: Draw $\mathbf{y}^{(l)}$ at random
- 5: Numerically estimate $b^{(l)}$
- 6: **end for**.

$$\begin{aligned} (\Phi)_{l,\nu} &\leftarrow T_\nu(\mathbf{y}^{(l)}), \quad \nu \in \mathcal{J}_0^s, 1 \leq l \leq m \\ \omega_\nu &\leftarrow 2^{\|\nu\|_0/2} \prod_{j:\nu_j \neq 0} v_j^{\nu_j}, \quad \nu \in \mathcal{J}_0^s \end{aligned} \tag{3.52}$$

- 7: Compute $\mathbf{g}^*$ as the solution of

$$\min \|\mathbf{g}\|_{\omega,1}, \quad \text{s.t. } \|\Phi \mathbf{g} - \mathbf{b}\|_2 \leq 2\sqrt{m}\varepsilon \tag{3.53}$$

- 8: **return** $\mathbf{g}^*$
 - 9: **end function**.
-

Theorem 3.17: Recovery results via Single-level CSPG [103, 26]

Let $u(\cdot; \mathbf{y})$ be the solution to Eq. (3.24) such that A_0 is boundedly invertible and $\sum_{j \geq 1} \beta_{0,j} \leq \kappa \in (0, 1)$ where $\beta_{0,j} := \|A_0^{-1} A_j\|_{\mathcal{L}(\mathcal{X}, \mathcal{X})}$, and the scales of smoothness spaces have a parametric regularity property (see [103, Assumption 2.3]). If the sequence $(v_j)_{j \geq 1}$ is such that $\sum_j \beta_{0,j} v_j^{(2-p)/p} \leq \kappa_{v,p} \in (0, 1)$ and $(\beta_{0,j})_j \in \ell_{v,p}$ then $\mathbf{g} \in \ell_{\omega,p}$ with ω as in Eq. (3.52). Choose an accuracy $\varepsilon > 0$ and sparsity s such that

$$\sqrt{5} \, 4^{1-1/p} s^{1/2-1/p} \|\mathbf{g}\|_{\omega,p} \leq \varepsilon \leq C_2 s^{1/2-1/p} \|g\|_{\omega,p}, \quad (3.54)$$

with $C_2 > \sqrt{5} 4^{1-1/p}$ independent of s and define $\mathcal{J}_0^s := \{\nu \in \mathcal{F} : 2^{\|\nu\|_0} \prod_{j: \nu_j \neq 0} v_j^{2\nu_j} \leq s/2\}$ with $N := |\mathcal{J}_0^s|$. Pick $m \asymp Cs \log^2(s) \log(N)$ (C a universal constant) samples i.i.d at random w.r.t. the Chebyshev distribution and define $F^*(\mathbf{y}) := \sum_{\nu \in \mathcal{J}_0^s} \mathbf{g}_\nu^*(\mathbf{y})$ with $\mathbf{g}^*$ solution of Eq. (3.53). Then, there exists a (universal) constant C' such that, with probability at least $1 - 2N^{-\log^2(s)}$, it holds:

$$\|F - F^*\|_2 \leq C' \|\mathbf{g}\|_{\omega,p} s^{1/2-1/p} \leq C_g'' \left(\frac{\log^2(m) \log(N)}{m} \right)^{1/p-1/2}, \quad (3.55)$$

$$\|F - F^*\|_\infty \leq C' \|\mathbf{g}\|_{\omega,p} s^{1-1/p} \leq C_g'' \left(\frac{\log^2(m) \log(N)}{m} \right)^{1/p-1}, \quad (3.56)$$

where C_g'' depends on C' and $\|\mathbf{g}\|_{\omega,p}$.

A piece of Python software was developed in order to numerically validate the theoretical results.³ These results were presented at a conference based on the work with my Master's student [26] and are recalled below together within the Figures 3.3 3.4.

For the purpose of the numerical illustrations, we considered approximations of problems in 1-dimensional physical domains. We are mostly interested in the high-dimensionality of the parameter space U and analyze, here, only the convergence property of the method, not its computational cost (this is the subject of the next Section together with some speed improvements). Specifically, we model a diffusion equation

$$-\nabla \cdot (a \nabla u) = f, \quad (3.57)$$

where a admits the affine-parametric expansion

$$a(\mathbf{x}; \mathbf{y}) = a_0(\mathbf{x}) + \sum_{j \geq 1} y_j a_j(\mathbf{x}). \quad (3.58)$$

As already introduced, we consider $\mathbf{y} = (y_j)_{j \geq 1} \in U$ and $\mathbf{x} \in D = [0, 1]$. For the practical

³All numerical results can be reproduced, albeit with some tuning required, from my personal github account: <https://github.com/jlbouchot/CSPDEs/>.

Table 3.1: Dependency of the number of samples on the sparsity. These figures are valid only for the set of parameters described in the text.

s	30	100	200	300	400	500
N	1554	5712	10006	12690	15601	17043
m	441	1731	3685	5670	7725	9744

numerical results, we use trigonometric polynomials as basis functions a_j in (3.58):

$$a_{2j-1}(x) = \frac{\cos(j\pi x)}{j^2}, \quad a_{2j}(x) = \frac{\sin(j\pi x)}{j^2}. \quad (3.59)$$

Our paper [26] gives further details about the various parameters involved. We have used only $m = 2s \log(N)$ with $\mathcal{J}_0^s$ denoting the initial set of important coefficients of the GPC, and $N := |\mathcal{J}_0^s|$, as suggested by results on nonuniform recovery from random matrices in compressed sensing - see [56, Ch.9.2]. For the purpose of the experiments, we let s vary from 30 to 500, see Table 3.1 for examples of values for m and N as functions of s .

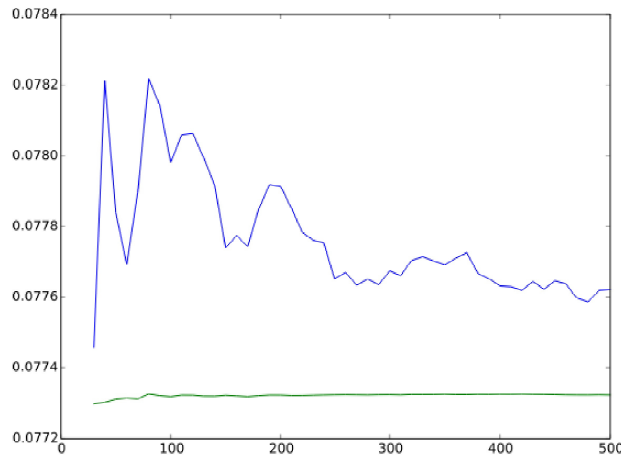


Figure 3.3: Expectation in the L^2 norm of the MC estimation (blue) and CSPG (green) methods.

The use case depicted in Fig. 3.3 uses the functional $\mathcal{G} : \mathcal{X} \rightarrow \mathbb{R}$ defined as $\mathcal{G}(u(\cdot; \mathbf{y})) := \int_{\mathbf{x} \in D} u(\mathbf{x}; \mathbf{y}) d\mathbf{x}$, and $D = [0, 1]$. Fig. 3.3 illustrates the convergence of both the Monte-Carlo and the CSPG method. Note that usual Monte Carlo methods can be done only when comparing the first coefficient g_0 of the expansion (3.49), i.e. the expectation of the function. Indeed, for $F(\mathbf{y}) := \sum_{\nu \in \mathcal{F}} g_\nu T_\nu(y)$, for T_ν the tensorized Chebyshev polynomials, it holds $\mathbb{E}[F] = g_0$. Monte-Carlo estimation is done as the empirical mean $E_m[F] := \frac{1}{m} \sum_{l=1}^m F(\mathbf{y}^{(l)})$. It is known (see [12, Lemma 4.1]) that for a number of samples m drawn at random it holds

$$\|E_m[F] - \mathbb{E}[F]\|_{L^2(U, \eta)} \leq m^{-1/2} \|F\|_{L^2(U, \eta)}. \quad (3.60)$$

The x -axis of Fig. 3.3 corresponds to the sparsity parameter s , or, seen differently, to the number of samples used (see Table 3.1 for the correspondence). The CSPG method shows a

much better convergence of the expectation and a much more stable behavior – a questions that has not been studied so far.

We also look at the pointwise convergence of the approximation in U with respect to the sparsity (or number of samples). Fig. 3.4 shows the approximations obtained for various sparsity values. The upper left corner shows the computed solution of $t \mapsto \tilde{F}(t, 0, 0, \dots)$ for $-1 \leq t \leq 1$. The remaining three quadrants correspond to the (pointwise) error for $t \mapsto F^*(t, 0, 0, 0, \dots)$ (upper right graph), $t \mapsto F^*(0, t, 0, 0, \dots)$ (lower left graph), $t \mapsto F^*(0, 0, t, 0, \dots)$ (upper right graph), for $-1 \leq t \leq 1$. The CSPG approximated solutions converge, as s increases, to the true solution.

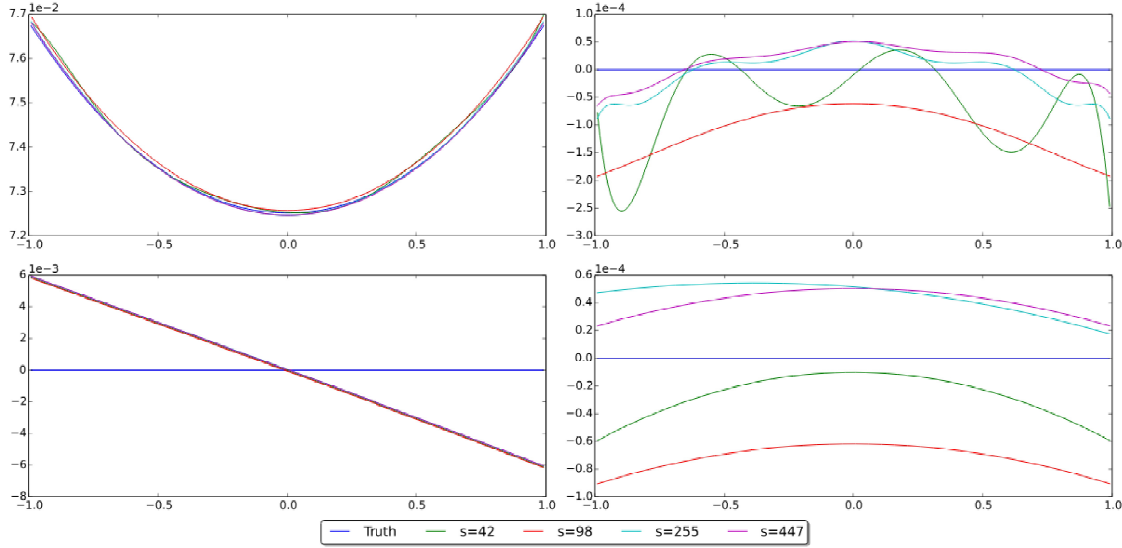


Figure 3.4: Convergence pointwise for various sparsity levels.

3.3.4 Multi-level approximation

In the introduction, I already mentioned the idea behind the multi-level extension of the compressed-sensing based method. In particular, in the previous section, I detailed the uniform approximation, with respect to the parameter vector $\mathbf{y}$, of the solution map $\mathbf{y} \mapsto u(\cdot; \mathbf{y})$ (or a functional thereof to simplify). I showed that, in order to achieve a target accuracy up to a certain $\varepsilon > 0$ one needs to compute a relatively small number of samples at very high accuracy and apply compressed sensing / sparse recovery techniques from the knowledge of said samples. However, the samples are required to be accurate to that precise ε . This can be quite difficult to achieve, especially in higher spatial dimensions, which is typically the case in fluid dynamics. In order to reduce the overall computational costs, we introduced [31] (and are still analyzing it!) a novel method combining ideas from the previous single level compressed sensing Petrov-Galerkin approximation, with some multi-level schemes. We refer to it as the *Multi-Level Compressed Sensing Petrov-Galerkin* (MLCSPG for short) approximation.

Multi-level schemes are similar to coarse-to-fine approaches used in image processing, in which, say, a rough synthesized image is generated at a very cheap cost, and details are then later added, increasing the complexity of the added information at each level. A visual presentation of the idea in the context of numerical PDEs is found in the introduction, in Fig. 1.5 (page 15). Here, the discretization spaces double in sizes (or, said differently, the finite elements are halved at each level) and a very coarse solution to the parametric map is computed at first, finer details being added only later in the refinements. The goal is to spend most of the samples (potentially much more than in the single level approach) at a very coarse level and exponentially decrease the number of samples with the current level of details. The presentation here focuses on the compressibility and recovery aspects of the methods, while the PDE aspects are kept to a minimum for brevity.

In contrast to the single level approach, which was discretized only for a given accuracy, we need to compute approximate solutions at various discretization accuracy. To this end, we introduce two dense, one-parameter families of discretization spaces $\{\mathcal{X}^h\}_{h>0} \subset \mathcal{X}$ and $\{\mathcal{Y}^h\}_{h>0} \subset \mathcal{Y}$ of respectively equal finite dimensions $N^h := \dim(\mathcal{X}^h) = \dim(\mathcal{Y}^h)$ and assume that the parametric operator $A(\cdot; \mathbf{y})$ fulfills the discrete and uniform inf – sup conditions: there exists a $\mu > 0$ such that for any $h > 0$ and $\mathbf{y} \in U$

$$\begin{cases} \inf_{0 \neq v^h \in \mathcal{X}^h} \sup_{0 \neq w^h \in \mathcal{Y}^h} \frac{\langle A(\cdot; \mathbf{y})v^h, w^h \rangle}{\|v^h\|_{\mathcal{X}} \|w^h\|_{\mathcal{Y}}} \geq \mu > 0 \\ \inf_{0 \neq w^h \in \mathcal{Y}^h} \sup_{0 \neq v^h \in \mathcal{X}^h} \frac{\langle A(\cdot; \mathbf{y})v^h, w^h \rangle}{\|v^h\|_{\mathcal{X}} \|w^h\|_{\mathcal{Y}}} \geq \mu > 0 . \end{cases} \quad (3.61)$$

The Petrov-Galerkin (PG for short) projections G^h are defined as the solutions to the weak variational problems

$$\text{find } u^h(\mathbf{y}) := G^h(\mathbf{y})(u(\cdot; \mathbf{y})) \in \mathcal{X}^h \text{ such that } \langle A(\mathbf{y})u^h(\cdot; \mathbf{y}), v^h \rangle = \langle f, v^h \rangle \quad \text{for all } v^h \in \mathcal{Y}^h . \quad (3.62)$$

The following classical result (see for example [21, Chapter 6]) is fundamental in assessing the quality of the numerical solutions

Proposition 3.4: Uniform boundedness of PG projections

Let $\mathcal{X}^h$ and $\mathcal{Y}^h$ be discretization spaces for the PG method, such that the uniform discrete inf – sup conditions (3.61) are fulfilled and assume that the bilinear operator $\mathcal{X} \times \mathcal{Y} \ni (u, w) \mapsto \langle A(\cdot; \mathbf{y})u, w \rangle$ is continuous, uniformly with respect to $\mathbf{y} \in U$. Then the PG projections $G^h(\mathbf{y}) : \mathcal{X} \rightarrow \mathcal{X}^h$ are well-defined linear operators, whose norms are uniformly bounded with respect to the parameters $\mathbf{y}$ and h , i.e.,

$$\sup_{\mathbf{y} \in U} \sup_{h > 0} \|u^h(\cdot; \mathbf{y})\|_{\mathcal{X}} \leq \frac{1}{\mu} \|f\|_{\mathcal{Y}'}, \quad (3.63)$$

$$\sup_{\mathbf{y} \in U} \sup_{h > 0} \|G^h(\mathbf{y})\|_{\mathcal{L}(\mathcal{X})} \leq \frac{C}{\mu} \quad (3.64)$$

The Galerkin projections are uniformly quasi-optimal: for every $\mathbf{y} \in U$ we have the a-priori error bound

$$\|u(\cdot; \mathbf{y}) - u^h(\cdot; \mathbf{y})\|_{\mathcal{X}} \leq \left(1 + \frac{C}{\mu}\right) \inf_{v^h \in \mathcal{X}^h} \|u(\cdot; \mathbf{y}) - v^h\|_{\mathcal{X}}. \quad (3.65)$$

It is classical to make use of some knowledge on the smoothness of the solution together with the discretization spaces. In particular, we introduce, for a parameter $0 \leq t \leq \bar{t}$ representing the regularity of the data, smoothness scales $\{\mathcal{X}_t\}_{t \geq 0}$, resp. $\{\mathcal{Y}_t\}_{t \geq 0}$ and assume that the parametric family $A(\cdot; \mathbf{y}) \in \mathcal{L}(\mathcal{X}, \mathcal{Y}')$ is regular within this smoothness scale:

$$\mathcal{X}_t \subset \mathcal{X}_0 = \mathcal{X}, \quad \mathcal{Y}_t \subset \mathcal{Y}_0 = \mathcal{Y}, \quad 0 \leq t \leq \bar{t}. \quad (3.66)$$

t may be seen as a differentiation order within Sobolev or Besov spaces for instance, which may be interpolated in case of non-integer parameters. Similarly, to encode the smoothness of the functional, we introduce a scale of smoothness on the dual side with $\mathcal{X}'_r := (\mathcal{X}')_r$, and $\mathcal{Y}'_r := (\mathcal{Y}')_r$,

$$\mathcal{X}'_r \subset \mathcal{X}'_0 = \mathcal{X}', \quad \mathcal{Y}'_r \subset \mathcal{Y}'_0 = \mathcal{Y}' \quad 0 \leq r \leq \bar{t}. \quad (3.67)$$

The discretization spaces are typically parametrized by the meshwidth $h > 0$ in finite element discretizations of fixed polynomial degree, on a quasiuniform triangulation of the physical bounded, polyhedral domain D . We assume that these spaces admit *linear approximation properties* in the smoothness scales (3.66), (3.67),

$$\begin{aligned} \inf_{w^h \in \mathcal{X}^h} \|w - w^h\|_{\mathcal{X}} &\leq C_t h^t \|w\|_{\mathcal{X}_t}, & \text{for all } w \in \mathcal{X}_t, \\ \inf_{v^h \in \mathcal{Y}^h} \|v - v^h\|_{\mathcal{Y}} &\leq C_r h^r \|v\|_{\mathcal{Y}_r}, & \text{for all } v \in \mathcal{Y}_r. \end{aligned} \quad (3.68)$$

Open question 3.5: MLCSPG via Higher order FEM

The current presentation looks at the linear finite element methods. It would be possible to include the case of higher order FEM by propagating a PG discretization error of order $k > 1$ through all remaining parts of our analysis. The consequences of this accuracy improvement would need to be worked out precisely, in particular in terms of compute costs, optimal number of levels, starting level of approximation, etc... This would keep a doctoral student busy throughout their postgraduate studies.

Together with the bounded invertibility of the family of operators $A(\mathbf{y})$, it holds

$$\|u(\cdot; \mathbf{y}) - u^h(\cdot; \mathbf{y})\|_{\mathcal{X}} \stackrel{\text{Eq. (3.65)}}{\leq} C \inf_{v^h \in \mathcal{X}^h} \|u(\cdot; \mathbf{y}) - v^h\|_{\mathcal{X}} \stackrel{\text{Eq. (3.68)}}{\leq} c_t h^t \|u(\cdot; \mathbf{y})\|_{\mathcal{X}_t} \leq C_t h^t \|f\|_{\mathcal{Y}_t'.} \quad (3.69)$$

The last inequality is a consequence of some bounded uniform invertibility of the operator $A(\cdot; \mathbf{y})$ in the smoothness scale. This dependency on the smoothness parameter is reflected in the constant C_t which depends on $\sup_{\mathbf{y} \in U} \|A(\mathbf{y})^{-1}\|_{\mathcal{L}(\mathcal{Y}_t', \mathcal{X}_t)}$ and therefore on the smoothness t . However it does not depend on the discretization parameter h .

Looking at functionals of solution only, we can assume adjoint regularity for the map $\mathcal{G}$. Formally, there exists $r \geq 0$, such that $\mathcal{G} \in \mathcal{X}_r'$, and such that the solution $w_{\mathcal{G}}(\mathbf{y}) \in \mathcal{Y}'$ of the *parametric, adjoint problem*

$$A(\cdot; \mathbf{y})^* w_{\mathcal{G}}(\mathbf{y}) = \mathcal{G} \quad (3.70)$$

satisfies $w_{\mathcal{G}}(\mathbf{y}) \in \mathcal{Y}'_r$ for some $0 \leq r \leq \bar{t}$ uniformly with respect to $\mathbf{y}$, i.e.

$$\sup_{\mathbf{y} \in U} \|w_{\mathcal{G}}(\mathbf{y})\|_{\mathcal{Y}'} \leq C \|\mathcal{G}\|_{\mathcal{X}_r'}. \quad (3.71)$$

Combining all assumptions and results (in particular Eqs. (3.71), (3.61), and (3.68)), we can apply an Aubin-Nitsche duality argument similar to [80]. It follows what is known as *superconvergence*, i.e. the discretized functional $G^h(\mathbf{y}) := \mathcal{G}(u^h(\cdot; \mathbf{y}))$, defined for $\mathbf{y} \in U$, which is the numerical application of the functional to the PG approximation (3.62) of the solution $u(\cdot; \mathbf{y}) \in \mathcal{X}$ defined in the discretization space $\mathcal{X}^h$ of parameter h . This superconvergence reads

$$|G(\mathbf{y}) - G^h(\mathbf{y})| \leq C_{t+r} h^{t+r} \|f\|_{\mathcal{Y}_t} \|\mathcal{G}\|_{\mathcal{X}_r'}. \quad (3.72)$$

Before detailing the proposed decomposition, I need to introduce two important levels of discretization J and L with $0 \leq J \leq L$. $L \in \mathbb{N}$ is related to the target discretization. In particular, assuming a target accuracy of ε and dyadic mesh refinements, we are looking to have $2^{-L} \approx \varepsilon$. Consequently, I refer to L as the discretization fineness targeted throughout the MLCSPG approach. To initiate the process it is also required to define a first, cheap, numerical approximation. This is the role of the $J \in \mathbb{N}$. It corresponds to the coarsest discretization level computed. For a parameter $\mathbf{y} \in U$ we may write the PG projection

$u^L(\cdot; \mathbf{y}) \in \mathcal{X}^{h_L}$ of $u(\cdot; \mathbf{y})$ at the targeted discretization level L as a telescoping sum

$$u^L(\cdot; \mathbf{y}) = u^J(\cdot; \mathbf{y}) + \sum_{l=J+1}^L (u^l(\cdot; \mathbf{y}) - u^{l-1}(\cdot; \mathbf{y})) = u^J(\cdot; \mathbf{y}) + \sum_{l=J+1}^L \Delta u^l(\cdot; \mathbf{y}), \quad (3.73)$$

where $\Delta u^l(\cdot; \mathbf{y}) = u^l(\cdot; \mathbf{y}) - u^{l-1}(\cdot; \mathbf{y})$. The last sum will be empty in the single level (SLC-SPG) case $J = L$. Applying the functional $\mathcal{G}$ gives the corresponding telescoping sum

$$G_L(\mathbf{y}) = G_J(\mathbf{y}) + \sum_{l=J+1}^L \Delta G_l(\mathbf{y}). \quad (3.74)$$

To facilitate the reading the parametrization of the approximation and spaces will be equivalently annotated with l or h_l .

Our multilevel approach aims at computing approximations for G_J and ΔG_l , $l = J+1, \dots, L$ via compressive sensing in order obtain an approximation to G_L . Similar to what has been done before, we look at Chebyshev expansions of the coarsest level (studied in the previous section) as well as the details $J < l \leq L$,

$$\Delta G_l(\mathbf{y}) = \sum_{\nu \in \mathcal{F}} (\Delta g_l)_\nu T_\nu(\mathbf{y}).$$

A multilevel approach becomes effective once $\Delta G^l(\mathbf{y})$ and $\Delta u^l(\mathbf{y})$, decay quickly with increasing l with respect to a suitable norm. Since we will exploit sparsity of the Chebyshev expansion of $\Delta u^l(\mathbf{y})$ as well, we study the weighted $\ell_{\omega,p}$ norm of the sequence $((\Delta g_l)_\nu)_{\nu \in \mathcal{F}}$ and its scaling with respect to h_l . Inequality (3.69) motivates then to study $u(\cdot; \mathbf{y})$ within the smoothness spaces $\mathcal{X}_t$. This requires to make certain smoothness assumptions on the operator equation that we combine in the following definition.

Definition 3.11: Assumption $S(t, p, \mathbf{v}, \kappa)$

A sequence $(A_0, A_1, A_2, \dots) \subset \mathcal{L}(\mathcal{X}, \mathcal{Y}')$ satisfies assumption $S(t, p, \mathbf{v}, \kappa)$ for a weight sequence $\mathbf{v} = (v_j)_{j \geq 1} \subset [1, \infty)^\mathbb{N}$, a constant $0 < \kappa < 1$, $0 \leq t \leq \bar{t}$ and $0 < p \leq 1$ if

1. the operators A_j are contained in $\mathcal{L}(\mathcal{X}_t, \mathcal{Y}'_t)$ for all $j \geq 0$, and the nominal operator $A_0 \in \mathcal{L}(\mathcal{X}_t, \mathcal{Y}'_t)$ is boundedly invertible, and
2. the sequence $\mathbf{a}_t = (a_{t,j})_{j \geq 1}$ with entries

$$a_{t,j} := \|A_0^{-1} A_j\|_{\mathcal{L}(\mathcal{X}_t)} = \|(A_0^{-1} A_j)^*\|_{\mathcal{L}(\mathcal{X}'_t)} \quad (3.75)$$

satisfies

$$\sum_{j \geq 1} a_{t,j} v_j^{(2-p)/p} \leq \kappa, \text{ and} \quad (3.76)$$

$$\sum_{j \geq 1} a_{t,j}^p v_j^{2-p} < \infty. \quad (3.77)$$

Eq. (3.76) ensures the invertibility of the expansion via a Neumann series, whereas Eq. (3.77) ensures the weighted compressibility of the expansion. Our next result represents an extension of previous (weighted) ℓ_p -estimates [103, 65], see also [44, 43], of Chebyshev coefficients of parametric solutions to smoothness spaces. Here, $\mathbf{v} = \mathbf{1}$ denotes the constant sequence $v_j = 1$, $j \geq 1$.

Theorem 3.18: Weighted ℓ_p estimates for details

Assume a sequence $(A_0, A_1, A_2, \dots) \subset \mathcal{L}(\mathcal{X}, \mathcal{Y}')$ satisfies conditions $S(0, 1, \mathbf{1}, \kappa_1)$ and $S(t, p, \mathbf{v}, \kappa_{\mathbf{v}, p})$ for some $0 < p \leq 1$ and $0 \leq t \leq \bar{t}$, with $\bar{t}$ as in (3.66) - (3.71), with certain constants $0 < \kappa_1, \kappa_{\mathbf{v}, p} < 1$ and a weight sequence $\mathbf{v} \subset [1, \infty)^{\mathbb{N}}$. Assume in addition that the approximation property (3.68) holds for the discretization spaces $\mathcal{X}^{h_l}$, $l = 0, 1, 2, 3, \dots$. Let the weights ω_ν be defined as in (3.47), i.e., $\omega_\nu = 2^{\|\nu\|_0/2} \mathbf{v}^\nu$. Then the following holds:

1. The parametric operators $A(\cdot; \mathbf{y}) = A_0(\cdot) + \sum_{j \geq 1} y_j A_j(\cdot)$ are uniformly boundedly invertible in $\mathcal{L}(\mathcal{X}, \mathcal{Y}')$ and in $\mathcal{L}(\mathcal{X}_t, \mathcal{Y}'_t)$, for $0 < t \leq \bar{t}$.
2. The parametric solution $u(\cdot; \mathbf{y})$ to $A(\cdot; \mathbf{y})u = f$, $\mathbf{y} \in U$, exists and is uniformly bounded in $\mathcal{X}$ and in $\mathcal{X}_t$.
3. The Chebyshev coefficients $(\Delta u^l)_\nu$ satisfy

$$\|((\Delta u^l)_\nu)_{\nu \in \mathcal{F}}\|_{\omega, p} \leq C h_l^t \|f\|_{\mathcal{Y}'_t}.$$

4. If in addition $\mathcal{G} \in \mathcal{X}'_r$ for some $0 \leq r \leq \bar{t}$, then the Chebyshev coefficients $(\Delta g_l)_\nu$ of $\Delta G_l(\mathbf{y})$ satisfy

$$\|((\Delta g_l)_\nu)_{\nu \in \mathcal{F}}\|_{\omega, p} \leq D h_l^{t+r} \|\mathcal{G}\|_{\mathcal{X}'_r} \|f\|_{\mathcal{Y}'_t}.$$

Here, the constants $C, D > 0$ only depend on $\kappa_0, \kappa_{\mathbf{v}, p}$ and on the sequences $\mathbf{a}_0$ and $\mathbf{a}_t$ in Definition (3.11).

With the compressibility of the expansion verified, our suggestion to approximate $\Delta u^l(\mathbf{y}) := u^l(\mathbf{y}) - u^{l-1}(\mathbf{y})$ or $\Delta G_l = G_l - G_{l-1}$ via compressed sensing methods is justified. Looking back at the telescopic sum in (3.74), we can approximate every term via sparse approximation similar to what was presented in the previous section on SLCSPPG, i.e. via convex optimization or iterative methods. To this end, for each refinement level $l = J, \dots, L$, we choose an appropriate number m_l of sampled parameter vectors $\mathbf{y}_l^{(1)}, \dots, \mathbf{y}_l^{(m_l)}$, compute the PG approximations $u^J(\cdot; \mathbf{y}_J^{(i)})$ as well as $u^l(\cdot; \mathbf{y}_l^{(i)})$ and $u^{l-1}(\cdot; \mathbf{y}_l^{(i)})$, $l = J+1, \dots, L$, by solving (possibly in parallel) the corresponding finite dimensional linear systems (3.62) corresponding to the PG discretization and apply the functional $\mathcal{G}$ to obtain the samples $G_l(\mathbf{y}_l^{(i)}) = \mathcal{G}(u^l(\cdot; \mathbf{y}_l^{(i)}))$ and $G_{l-1}(\mathbf{y}_l^{(i)}) = \mathcal{G}(u^{l-1}(\cdot; \mathbf{y}_l^{(i)}))$. The samples used for the detailed approximation at level l , $J < l \leq L$ are then obtained through

$$\Delta G_l(\mathbf{y}_l^{(i)}) = G_l(\mathbf{y}_l^{(i)}) - G_{l-1}(\mathbf{y}_l^{(i)}), \quad i = 1, \dots, m_l.$$

From these computations, one approximates the coefficients in the tensorized Chebyshev expansion of the details ΔG_l via a weighted sparse approximation method detailed in the previous Sections. This yields approximations $\widehat{\Delta G}_l(\mathbf{y})$, $l = J+1, \dots, L^4$. In parallel, an

⁴Notice the switch from the * as an indicator of solutions to approximation problems to a hat. * would just add ambiguity to the notation and require more parenthesis.

approximation to G_J is computed via a SLCS PG approach from m_J samples and the final estimator is eventually computed as

$$G_L^{\text{MLCS}}(\mathbf{y}) := \widehat{G}_J(\mathbf{y}) + \sum_{l=J+1}^L \widehat{\Delta G}_l(\mathbf{y}) \quad (3.78)$$

which provides an approximation of the parametric solution $G = G(\mathbf{y})$.

Estimating the error bounds is done by abusing the triangle inequality for the MLCS approximation $G_L^{\text{MLCS}}(\mathbf{y})$ in (3.78). Specifically

$$|G(\mathbf{y}) - G_L^{\text{MLCS}}(\mathbf{y})| \leq |G(\mathbf{y}) - G_L(\mathbf{y})| + |G_J(\mathbf{y}) - \widehat{G}_J(\mathbf{y})| + \sum_{l=J+1}^L |\Delta G_l(\mathbf{y}) - \widehat{\Delta G}_l(\mathbf{y})|. \quad (3.79)$$

The first term on the right hand side of Eq. (3.79) can be bounded, using the uniform parametric regularity, the uniform parametric inf-sup condition, and the approximation property by

$$|G(\mathbf{y}) - G_L(\mathbf{y})| \leq C_{t+r} h_L^{t+r} \|f\|_{\mathcal{Y}_t'} \|\mathcal{G}\|_{\mathcal{X}_r'}. \quad (3.80)$$

The second term and the terms in the sum in (3.79) can be estimated via Theorem 3.17 under the assumption that the Chebyshev coefficients $(g_J)_\nu$ of G_J are in ℓ_{ω, p_0} for some $p_0 \in (0, 1)$ (as detailed in the previous Section) and the Chebyshev coefficients $(\Delta g_l)_\nu$ of ΔG_l , for $l = J + 1, \dots, L$ belong to weighted ℓ_p spaces for some $p \in (0, 1)$, i.e.,

$$\|(g_J)_\nu\|_{\omega, p_0} < \infty \quad \text{and} \quad \|(\Delta g_l)_\nu\|_{\omega, p} \leq \infty \text{ for } l = J + 1, \dots, L,$$

which is verified through Thm. 3.18 Here, the weight sequence ω is defined as in (3.47) for a sequence $\mathbf{v}$ with $v_j > 1$ and $v_j \rightarrow \infty$ as $j \rightarrow \infty$.

Assuming for simplicity here that the computation of the functional is exact, given level dependent sparsities s_l , we define

$$\Gamma_l = \Gamma^{s_l} = \{\nu \in \mathcal{F} : \omega_\nu^2 \leq s_l/2\}$$

we assume that the number m_l of samples at each level satisfies

$$m_l \geq C s_l \log^2(s_l) \log(|\Gamma_l|) \quad (3.81)$$

for an appropriate constant $C > 0$. Assuming the sparse recovery method is *good enough*⁵

⁵we choose not to dig into details here on the specifics of the recovery method. Any sparse recovery method will work fine, as long as it is guaranteed to be stable and robust, as was the case for the algorithms presented in the previous Sections. The remainder of this manuscript deals solely with IHT, but similar analysis can be done for other recovery algorithms.

Theorem 3.17 gives the error bounds

$$\left| G_J(\mathbf{y}) - \widehat{G}_J(\mathbf{y}) \right| \leq C \| (g_J)_{\nu \in \mathcal{F}} \|_{\omega, p} s_J^{1-1/p}, \quad (3.82)$$

$$\left| \Delta G_l(\mathbf{y}) - \widehat{\Delta G}_l(\mathbf{y}) \right| \leq C \| (\Delta g_l)_{\nu \in \mathcal{F}} \|_{\omega, p} s_l^{1-1/p}, \quad l = J+1, \dots, L. \quad (3.83)$$

which hold with high probability. (It also provides estimates in $L^2(U, \eta)$ for which, however, I will not outline the details here.)

If the operator equation satisfies $S(0, p_0, \mathbf{v}, \kappa_0)$ (see Def. 3.11) with some $\kappa_0 \in (0, 1)$, then this implies that

$$\left| G_J(\mathbf{y}) - \widehat{G}_J(\mathbf{y}) \right| \leq C' \| \mathcal{G} \|_{\mathcal{X}'} \| f \|_{\mathcal{Y}'} s_J^{1-1/p_0} \quad \text{for all } \mathbf{y} \in U. \quad (3.84)$$

see also (3.50).

If additionally the operator equation satisfies $S(t, p, \mathbf{v}, \kappa_t)$ (see Def. 3.11) then Theorem 3.18 implies

$$\forall \mathbf{y} \in U : \quad \left| \Delta G_l(\mathbf{y}) - \widehat{\Delta G}_l(\mathbf{y}) \right| \leq C'' \| \mathcal{G} \|_{\mathcal{X}_t'} \| f \|_{\mathcal{Y}_t'} h_l^{t+r} s_l^{1-1/p}, \quad l = J+1, \dots, L. \quad (3.85)$$

The constants C', C'' only depend on the operators $A_0, A_1, \dots$

In order to achieve a given overall target accuracy of the order h_L^{t+r} , see (3.80), with small computational effort we can adapt the weighted sparsities s_l to the hierarchy of discretization levels $J \leq l \leq L$. For simplicity, we propose to balance the above error bounds among discretization levels which leads to the choices $s_J^{1-1/p_0} \asymp h_L^{t+r}$ and $h_l^{t+r} s_l^{1-1/p} \asymp h_L^{t+r}$. Since we assume $h_l \asymp 2^{-l}$, this motivates the choices

$$s_J \asymp 2^{L \frac{p_0(t+r)}{1-p_0}}, \quad s_l \asymp 2^{(L-l) \frac{p(t+r)}{1-p}}, \quad l = J+1, \dots, L. \quad (3.86)$$

As announced at the beginning of the Section, we see that the number of samples decreases exponentially with the current level. Moreover, the complexity for computing the solutions at finer levels is much higher than that at coarser levels. Overall, the method achieves a bound on the accuracy as described in the next theorem.

Theorem 3.19: Error bound for PDE approximation via MLCSPG

Under the assumptions described above, with probability at least

$$1 - \sum_{l=J}^L \exp(-cm_l/s_l) \geq 1 - \sum_{l=J}^L s_l^{-c' \log(s_l)}$$

with the constant c' independent of J and L , the multilevel compressive sensing method satisfies the error bound

$$\sup_{\mathbf{y} \in U} |G(\mathbf{y}) - G_L^{\text{MLCS}}(\mathbf{y})| \leq C(1 + K_1 + K_2) \|f\|_{\mathcal{Y}_t} \|\mathcal{G}\|_{\mathcal{X}_r} 2^{-L(t+r)} \quad (3.87)$$

where K_1, K_2 are constants implied in (3.86).

3.3.5 Computational work

The method presented so far is motivated by the opportunity to reduce the computational costs of the numerical approximation scheme for the operator equation (3.24) and, in our context, the case of affine linear dependencies (3.25). We analyze the computational costs for the case of the approximation using polynomial weights. The costs are split into two major parts: the computational solves for the PDEs on the one hand and the ℓ_0 sparse recovery on the other. The costs for inference are then fairly simple, since they involve only a matrix vector product – and are anyway not involved in the *learning* part of the approximation.

To set the background, we assume that our PG projections happen thanks to a linear solver such as multigrid methods (as described for instance in [64, 123]). Such solvers compute approximate solutions of the PG projections at a cost scaling only linearly with the number of unknown in the mesh. In our particular case of dyadic refinements, the number of unknowns is exponential with the discretization level which leads to a computational cost for a single solve at level l

$$w_l \asymp h_l^{-n} \asymp 2^{nl},$$

The work of computing m_l snapshot solutions at discretization level $l \geq J$, therefore scales as

$$W_l = m_l w_l \asymp m_l 2^{nl}. \quad (3.88)$$

Whence the total cost for the PDE part at a given level $l, J \leq l \leq L$ is This gives rise to the following complexity estimates, where we focus on polynomially growing weights.

Proposition 3.5: Total cost for PDE solves within MLCSPG

Assume the telescopic estimator described in Eq. (3.78) and that the sparsity per level is given by (3.86). Assume moreover that the set of multiindices at level l is chosen as

$$\Gamma_l = \{\nu \in \mathcal{F} : \omega_\nu^2 \leq s_l/2\} \quad (3.89)$$

and that the number of samples per level is chosen according to Eq. (3.81), i.e.

$$m_l \asymp s_l \log^2(s_l) \log(|\Gamma_l|), \quad l = J, \dots, L. \quad (3.90)$$

Suppose that the weight sequence $\mathbf{v} \subset [1, \infty)^\mathbb{N}$ grows polynomially, i.e., $v_j \geq \beta j^\alpha$, $j \in \mathbb{N}$, for some $\beta > 1$ and $\alpha > 0$. We set

$$\sigma_p = \frac{(t+r)p}{1-p}. \quad (3.91)$$

Further suppose given a linear complexity solver for the PG discretizations. Then the total work $W = W_J + W_\Delta$ with W_J the total cost for the first numerical approximation and W_Δ the total cost for all the samples for all the details, can be estimated as

$$\begin{aligned} W_J &\leq C \gamma_{\alpha,\beta} B^{\frac{p_0}{1-p_0}} (\sigma_{p_0} L)^4 2^{L\sigma_{p_0} + nJ}, \\ W_\Delta &\leq C_{\sigma_p, n} \gamma_{\alpha,\beta} \times \begin{cases} (L-J)^{\frac{p}{1-p}} \log^4(L-J) 2^{nL} & \text{if } \sigma_p < n, \\ (L-J)^{5+\frac{p}{1-p}} 2^{nL} & \text{if } \sigma_p = n, \\ (L-J)^{4+\frac{p}{1-p}} 2^{\sigma_p L - J(\sigma_p - n)} & \text{if } \sigma_p > n. \end{cases} \end{aligned} \quad (3.92)$$

Here, $B = \frac{\|f\|_{\mathcal{Y}} \|\mathcal{G}\|_{\mathcal{X}'} D_{p_0}}{\|f\|_{\mathcal{Y}'} \|\mathcal{G}\|_{\mathcal{X}'} K_2}$. The constant $C_{\sigma_p, n}$ only depends on σ_p and n and D_{p_0} on the constants involved in the choices of s_l and m_l .

As a consequence, we may wish to balance both terms. Targeting an accuracy

$$\varepsilon = 2^{-L(t+r)},$$

at level L and looking at the case of strong compressibility of the expansion, which is materialized by $p \ll 1$, we can assume $\sigma_p \leq n$ (this is even more true when dealing with larger spatial dimensions, as is the case in computational fluid for instance). This leads to the optimal choice

$$J = \left\lceil \max \left\{ 1 - \frac{(t+r)p_0}{n(1-p_0)}, 0 \right\} \frac{\log(\varepsilon^{-1})}{t+r} \right\rceil \leq L$$

and gives the final overall estimate for the PDE part of the MLCSPG approximation

$$W \leq C' \left(\frac{\log_2(\varepsilon^{-1})}{t+r} \right)^{\frac{p}{1-p}} \log^4 \left(\frac{\log_2(\varepsilon^{-1})}{t+r} \right) \varepsilon^{-\frac{n}{t+r}}. \quad (3.93)$$

It is worth noticing that this is, up to logarithmic factors and multiplicative constants,

precisely the complexity $\varepsilon^{-\frac{n}{t+r}}$ for one solve of the operator equation.

In order to analyze the costs for the sparse recovery parts of the algorithm, we focus on the case where we used the weighted version of IHT together with the proposed approximation of the hard thresholding operator. Looking at Eqs. (3.45) and (3.44) we see that we may balance the two error terms in order to get a number of iterations per level as

$$k_l \asymp \log(s_l)$$

with sparsity parameter $2s_l$. The complexity of the sparse recovery part at level l , V_l , is therefore given by the computation of $k = k_l$ iterations of weighted IHT. This is nothing else than some matrix multiplications of size $m_l \times N_l$ followed by sorting and thresholding, which has complexity $N_l \log(N_l)$ with $N_l = |\Gamma^{s_l}|$. With the choice (3.90) of m_l we conclude that

$$V_l \asymp k_l \cdot m_l \cdot N_l \asymp \log(s_l) \cdot s_l \log^2(s_l) \log(N_l) \cdot N_l \asymp N_l s_l \log^3(s_l) \log(N_l),$$

and the total complexity being

$$V = \sum_{l=J}^L V_l \asymp \sum_{l=J}^L N_l s_l \log^3(s_l) \log(N_l).$$

For the specifics of polynomially growing weights we can derive the next theorem.

Theorem 3.20: Computational work bound for the sparse recovery within MLCSPG

Assume the scenario described in the previous theorem, especially regarding the choices of s_l , m_l , and Γ^l . Suppose that the weight sequence $\mathbf{v} \subset [1, \infty)^\mathbb{N}$ grows polynomially, i.e., $v_j \geq \beta j^\alpha$, $j \in \mathbb{N}$, for some $\beta > 1$ and $\alpha > 0$. Set $\sigma_p = (t+r)p/(1-p)$.

One can approximately balance the work for the first level approximation and the work for all the sparse approximations of the details with the choice $\rho(L-J)^2 \approx (\log(2)\sigma_{p_0}^2 + \log(\kappa)\sigma_{p_0})L^2$ i.e.,

$$J = \left\lceil \max \left\{ 1 - \sqrt{\frac{(\log(2)\sigma_{p_0}^2 + \log(\kappa)\sigma_{p_0})}{\rho}}, 0 \right\} L \right\rceil, \quad (3.94)$$

resulting in

$$V \lesssim L^5 2^{\tau L^2}$$

with $\tau = \max\{\rho, \log(2)\sigma_{p_0}^2 + \log(\kappa)\sigma_{p_0}\}$ and κ a constant depending on the data f and $\mathcal{G}$ and the constants involved in the asymptotics in (3.86).

One may re-express this in terms of the target accuracy $\varepsilon = 2^{-L(t+r)}$ as

$$V \leq C'' \left(\frac{\log(1/\varepsilon)}{t+r} \right)^5 \varepsilon^{-\frac{\tau}{(t+r)^2} \log_2(1/\varepsilon)}. \quad (3.95)$$

Ignoring constants and logarithmic terms (besides the one in the exponent), if

$$\tau < \frac{n(t+r)}{\log(\varepsilon^{-1})},$$

then the estimate (3.95) of the computational work for the compressive reconstruction is smaller than the estimate (3.93) for the computational work for solving the samples of the solution of the operator equation.

Note that the total PDE computational costs assumes a single core architecture. It may be better suited to use more high quality samples when a massively parallel and efficient HPC cluster is used. The discussion above and, in particular, the balance between PDE solve times and sparse recovery method is presented on a sequential architecture and using IHT as sparse solver. This should be adapted to everyone's needs and algorithmic and compute architecture's choices since both computational costs are competitive in the selection of the optimal J .

3.3.6 Numerical validation

We illustrate the theoretical findings with numerical experiments. We reconsider the problem involved in the numerical verifications of the SLCSPG method.

$$A(\mathbf{x}, \mathbf{y})u(\mathbf{x}, \mathbf{y}) = f(\mathbf{x}), \quad \mathbf{x} \in D = [0, 1]^n, \mathbf{y} \in U = [-1, 1]^d.$$

Theoretically, depending on the situation, n will be taking values in $\{1, 2, 3\}$. We will keep 2 dimensional problems in this validation. d will be specified in due time. The diffusion coefficient will always be expressed as an affine expansion, similar to Eq. (3.58)

$$\begin{aligned} A(\mathbf{x}, \mathbf{y})u(\mathbf{x}, \mathbf{y}) &= -\nabla (a(\mathbf{x}, \mathbf{y})\nabla u(\mathbf{x}, \mathbf{y})), \\ a(\mathbf{x}, \mathbf{y}) &= \bar{a} + \varsigma \sum_{j=1}^d y_j a_j(\mathbf{x}) \end{aligned} \tag{3.96}$$

with $\bar{a}$ being a constant function. Note the addition of the parameter ς when compared to Eq.(3.58) which allows us to control the importance of the fluctuations with respect to the mean field thereby ensuring the summability conditions involved in the novel Assumptions 3.11, required for the multi-level case (specifically for the compressibility of the details).

For the numerical experiments we consider the problem of a diffusion coefficient expanded on a set of cosine functions

$$a_j(\mathbf{x}) = \frac{\cos(\pi j \|\mathbf{x}\|)}{(1 + \Upsilon_j)^\mu} \tag{3.97}$$

where Υ is a constant. The larger Υ is, the faster the fluctuations decay. As smoothness parameters of the data we consider $t = r = 1$. All the examples below have been implemented

in Python 3.5 and above. For the PDE solves we use the tools developed via the FEniCS project [6, 84].⁶

Checking the hypothesis

As I mentioned in the theoretical section, in order to break the curse of dimensionality, it is necessary to embed some prior information on the expansion of the diffusion coefficient by means of the weights v_j which are set as

$$v_j = cj^\alpha \quad (3.98)$$

for some constants $c, \alpha \geq 0$. We are interested in the case of polynomially growing weights, i.e., $\alpha > 0$, thereby ensuring that $v_j \rightarrow \infty$ as $j \rightarrow \infty$. These weights imply the weights on the Chebyshev expansion defined as

$$\omega_\nu = \sqrt{2}^{\|\nu\|_0} \mathbf{v}^\nu, \quad \nu \in \mathcal{F}.$$

The weighted summability of the Chebyshev expansion of the solution map is obtained from the (weighted) summability of the coefficient expansion. Here, we have

$$\|a_j\|_\infty \asymp j^{-\mu}.$$

The sequence $(\|a_j\|_\infty)_j \in \ell_{\mathbf{v}, p_0}$ if

$$p_0 > \frac{2\alpha + 1}{\alpha + \mu}. \quad (3.99)$$

Once the constants α and μ are chosen, we can estimate p_0 .

Given the smoothness $t = 1$, we can look at the summability in the smoothness scale, which corresponds, in this case, to the summability of the norms of the gradients. Similarly one obtains

$$\|\nabla a_j\|_\infty \asymp j^{-\mu+1}$$

from which it directly follows that we need

$$p > \frac{2\alpha + 1}{\alpha + \mu - 1}. \quad (3.100)$$

⁶Note that all the code for reproducible research and further use is available from one of the authors' github page: <https://github.com/jlbouchot/CSPDEs>. Installation can be done (relatively) efficiently via a .yaml file.

The interesting case in Eq. (3.86) is when σ_p (see Eq. (3.91)) is such that $\sigma_p < 1$ which implies a highly compressible expansion. Typically

$$p < \frac{1}{3},$$

so that, together with (3.100), we have the constraint on α and μ that

$$\mu > 5\alpha + 4. \quad (3.101)$$

We enforce the uniform ellipticity assumption by imposing the stronger conditions (assuming a constant mean field $\bar{a}$)

$$\varsigma \sum_{j \geq 1} \|\psi_j\|_{\infty} v_j^{(2-p_0)/p_0} < \min_{x \in D} \bar{a}(x) = \bar{a}, \quad (3.102)$$

$$\varsigma \sum_{j \geq 1} \|\nabla \psi_j\|_{\infty} v_j^{(2-p)/p} < \min_{x \in D} \bar{a}(x) = \bar{a}. \quad (3.103)$$

We control the summability indices p, p_0 by tuning the parameter ς describing the importance of the fluctuations with respect to the mean-field. Following the computations for the weighted summability assumptions above, it holds

$$\begin{aligned} \sum_{j \geq 1} \|\psi_j\|_{\infty} v_j^{\frac{2-p_0}{p_0}} &\leq \sum_{j \geq 1} (1 + \Upsilon j)^{-\mu} c^{(2-p_0)/p_0} j^{\alpha(2-p_0)/p_0} \\ &\leq c^{(2-p_0)/p_0} \sum_{j \geq 1} (1 + \Upsilon j)^{-\mu} j^{\alpha(2-p_0)/p_0}. \end{aligned} \quad (3.104)$$

Let us set constants simplifying the exposition. For the rest of the experiments we limit ourselves to the case where $p_0 = p = 0.33 < 1/3$ and $\alpha = 1/6$ and $\mu = 5$. These values indeed satisfy the conditions above and (3.104) reads

$$\sum_{j \geq 1} \|\psi_j\|_{\infty} v_j^{\frac{2-p_0}{p_0}} \leq c^4 \sum_{j \geq 1} (1 + \Upsilon j)^{-5} j^{2/3}.$$

We note in passing that this series is convergent for any value of $\mu > 5/3$.

Notice that $(1 + \Upsilon j) \geq \Upsilon j$ and so $(1 + \Upsilon j)^{-\mu} \leq \Upsilon^{-\mu} j^{-\mu}$ and the upper bound finally becomes

$$\sum_{j \geq 1} \|\psi_j\|_{\infty} v_j^{\frac{2-p_0}{p_0}} \leq c^4 \Upsilon^{-5} \sum_{j \geq 1} j^{-13/3}.$$

The function $t \mapsto j^{-13/3}$ decaying for $j > 0$, a comparison with an integral yields

$$\sum_{j \geq 1} \|\psi_j\|_{\infty} v_j^{\frac{2-p_0}{p_0}} \leq \frac{c^4}{\Upsilon^5} \frac{13}{10}$$

and Eq. (3.102) is satisfied as soon as

$$\frac{\varsigma}{\Upsilon^5} < \frac{\bar{a}}{\frac{13}{10}c^4}.$$

A similar analysis for Eq. (3.103) implies the condition

$$\frac{\varsigma}{\Upsilon^5} < \frac{\bar{a}}{\frac{10\pi}{7}c^4}.$$

In the numerical experiments, we set $c = 1.005$, $\bar{a} = 10$, $\Upsilon = 1$, and $\varsigma = 1$ which ensures that all summability assumptions are satisfied.

3.3.7 Numerical results

For the remainder of the experiments, we set the level 0 discretization size to $h_0 = 1/20$ and consider $d = 20$ cosine functions in the expansion and set the physical dimension to $n = 2$. Moreover, whenever an error is computed it is done by comparing the numerical solution obtained from our Single- or Multi-level approach with a numerical solution obtained on a grid with a mesh size 4 times smaller than the finer grid used in the experiments. Since we are testing up to $L = 6$, the ground truth solution is computed on a grid which would be appearing at level 8, i.e. for $h_{\text{ground truth}} = 1/5120$.

We first aim at verifying that the accuracy of the solution indeed increases twofold for each new level added. Fig. 3.5 shows the evolution of both the compute times for the sparse recovery, PDE solves, and total time, as well as the accuracy of the solutions. We point out that the time is the actual total CPU core time, assuming every single PDE solve is done on a unique CPU. The experiments always start with J such that $L - J = 2$ approximation levels are used and we let L grow from 2 to 6. The accuracy decays as expected. As suggested by the analysis in the previous sections a trade-off between number of levels, speed, and accuracy needs to be found, as the computation costs explode as we reach $L = 6$. This is due to the needs in memory to build the sensing matrix. Indeed, with increasing L it follows that s_l increases which has an impact on both the number of rows (the number of samples) which increases with the sparsity and the number of columns (the size of the potential active

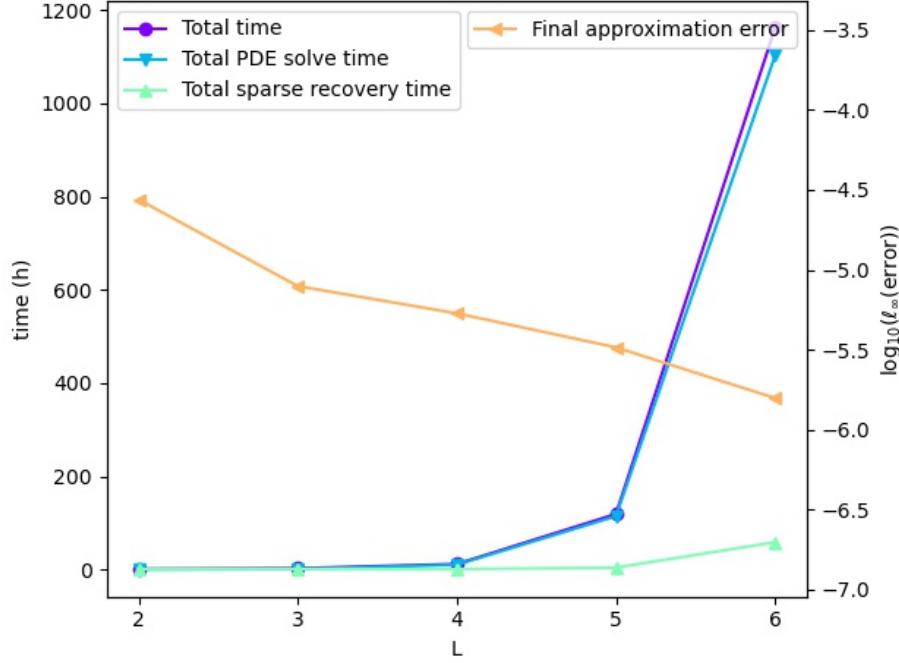


Figure 3.5: Evolution of computing time and approximation accuracy as the number of levels changes.

set Γ_l) of the sensing matrix.

Second we empirically verify that the optimal choice of J in terms of computing time might not be found for lower values of J . For this set of tests, we run the experiments with a fixed $L = 5$ and let J vary from $J = 1$ to $J = 5$, the latter corresponding to the single level case. From Fig. 3.6 we first notice that the accuracy is stable, varying by a factor of 2 only, and remaining in the $10^{-5.5}$ range, for all experiments, thereby validating our theory. Moreover, the case $J = 2$, seems to offer the best performances in terms of its computing time, both for the sparse recovery part and for the PDE solves. In particular, the multilevel method outperforms the single level method. This also justifies the analysis carried out in the analysis of the computational costs, where it was shown that J must be chosen strictly between 0 and L . We want to emphasize here that the optimal value for J is very much dependent on many factors including the scenario / PDE involved, the machine(s) on which the computations are done, the implementation of the method itself, etc... The set of experiments presented here are simply illustrating this phenomenon and might not be easily transferable to other problems.

I would also like to point out one last phenomenon which I have not introduced yet. It is worth keeping in mind that, albeit much smaller than the sparse recovery or PDE computations, the time for actually estimating the ansatz spaces Γ_l and generating the matrix of Chebyshev coefficients is not free, especially for coarser levels (small l 's). Constructing the ansatz space might have a prohibitive cost for large d and large s . See Tables 3.3 and 3.2 for some figures.

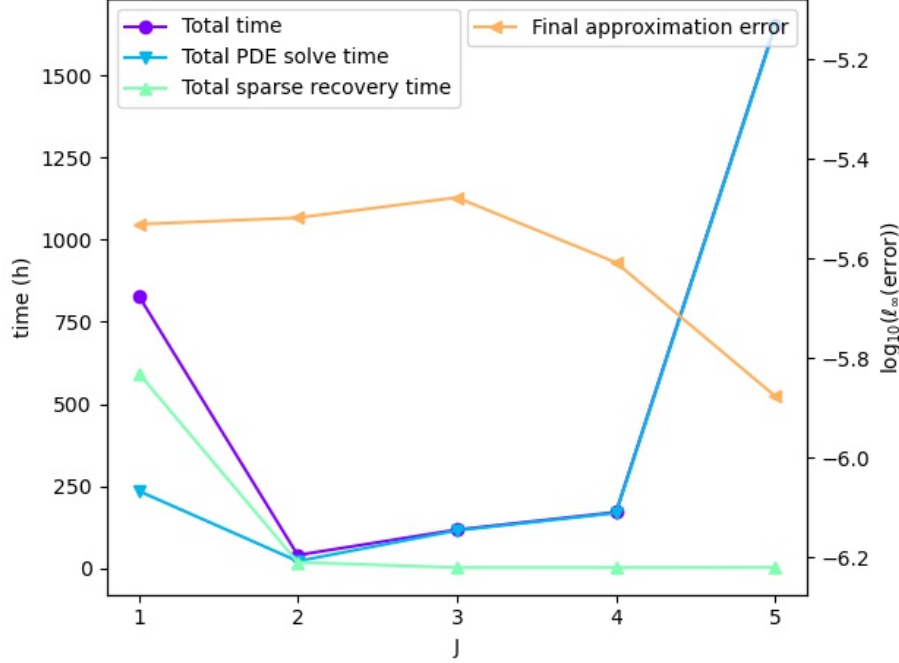


Figure 3.6: Evolution of computing time and approximation accuracy as the first approximation level changes.

Open question 3.6: Handling large sensing matrices

In some case, it might be useful to extend the ranges of levels used. The sensing matrices becoming too large for larger ranges, it is important to develop fast matrix vector multiplication. In our case, this would be about developing fast Chebyshev transforms similar to the FFT [54, 51, 77]. Some work has been done recently on the topic of sparse Radon transform, and this could be a good source of ideas [5]. Combining such results with efficient greedy algorithms for the weighted case (see Problem ??).

Finally, we test the robustness of our approach against varying dimensions. Setting $J = 2$ and $L = 4$, we let d , the number of cosine basis function in the expansion, vary from 16 to 50.

Fig. 3.7 shows that the compute time is only mildly impacted by the increase in dimensionality while the accuracy remains stable. The increase in compute time is a consequence of the increase of both the size of the ansatz space and the number of samples. The sparse recovery computing time is impacted two folds by the increase of the dimension of the parameter space.

Table 3.2: Dimensionality of the ansatz space and compute time required for generating it as well as the sensing matrix with changing dimensionality of the parameter space. $J = 2$, $L = 4$.

d	ℓ	$ \Gamma_\ell $	m	t_A (s)	t_{Γ_ℓ}
16	3	69225	17575	111.08	0.29
	4	10811	5128	24.02	0.19
	5	1538	1256	3.88	0.02
20	3	73919	17678	113.48	0.37
	4	11171	5146	22.97	0.24
	5	1542	1256	3.68	0.02
26	3	78740	17778	114.29	0.52
	4	11603	5167	24.31	0.50
	5	1548	1257	4.17	0.04
32	3	81926	17840	121.77	0.79
	4	11947	5183	28.10	0.61
	5	1554	1257	4.60	0.08
40	3	84778	17894	130.10	1.90
	4	12267	5197	27.17	1.85
	5	1562	1258	4.38	0.16
50	3	87144	17938	141.07	4.15
	4	12525	5209	22.71	4.24
	5	1572	1259	4.88	0.27

Table 3.3: Dimensionality of the ansatz space and compute time required for generating it as well as the sensing matrix with changing number of levels, $d = 20$, $L = 5$.

J	ℓ	$ \Gamma_\ell $	m	t_A (s)	t_{Γ_ℓ}
1	1	3374260	263397	1724.24	7.84
	2	764117	92625	456.86	1.84
	3	167340	31035	120.46	0.88
	4	26597	9421	27.86	0.49
	5	4661	2541	5.08	0.02
2	2	507225	69713	307.98	1.18
	3	107765	22734	81.43	0.87
	4	15862	6714	17.44	0.55
	5	2669	1736	3.07	0.01
3	3	150745	28871	124.15	0.51
	4	11171	5146	14.76	0.46
	5	1542	1256	2.30	0.01
4	4	150745	28871	106.01	0.48
	5	242	477	0.53	0.14
5	5	150745	28871	119.93	0.48

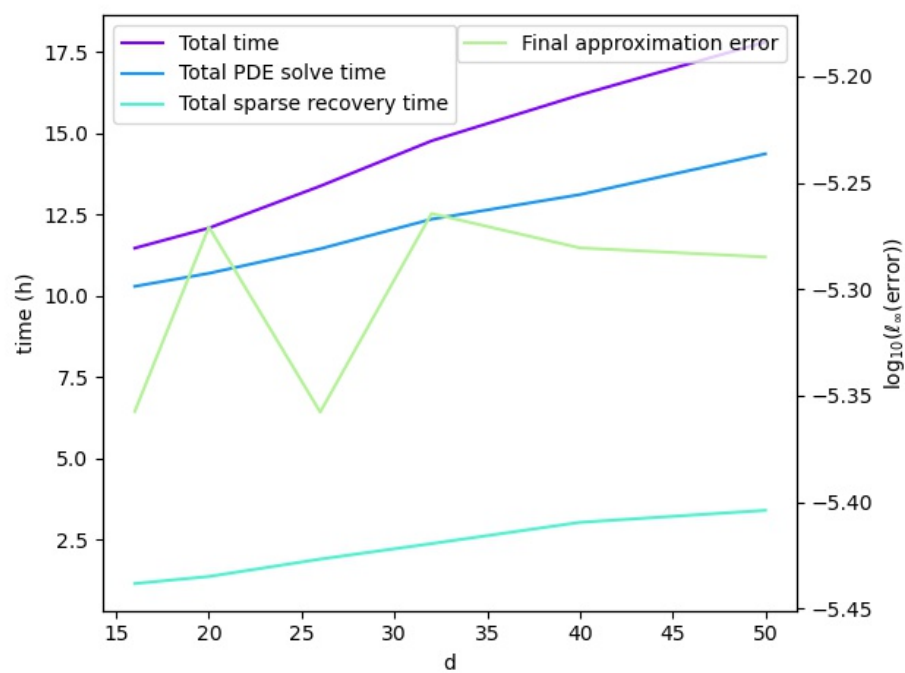


Figure 3.7: Evolution of computing time and approximation accuracy as the dimensionality increases.

Chapter 4

Algorithmic differentiation

4.1 AD and adjoint AD

In the context of approximating parametric PDEs, one might be interested to optimize certain *things* over some parameter space. A particularly important aspect is the possibility of the compressed sensing method presented in the previous to be used in shape optimization. Indeed, while I only briefly mentioned it earlier, the uniform ellipticity assumptions and the compressibility of the solutions are generally valid for parametrized shapes (this is done by lifting the real variable and Chebyshev expansion to a complex variable and Laurent series, see for instance [46, 115]). Similarly, going back to the problem of aligning two images, one needs optimization and, in this process, some knowledge of gradient and derivatives. Or, using a denoising like problem, such as the one motivated by the constrained optimization (1.4), one needs the knowledge of the derivatives. Motivated by these facts, I recently got interested in the possibility to use *algorithmic differentiation*¹ to create a complete pipeline for all the problems introduced in the previous chapters. Typically, scientific computing for linear operators and machine learning has already attracted a few AD people [45]. AD has been my major topic of research since joining Inria in January 2025.

¹Some people prefer the term *automatic differentiation*. Either way, let's call it *AD*!

Algorithmic Differentiation (AD) is *the art of differentiating computer programs* [98]. The field is usually split into two main branches [63]: operator overloading (OO) and source transform (ST). While OO is very interesting and broadly used in certain fields, I am particularly interested in the ST-AD concept. Here, we consider a piece of programming code implemented in a *fairly* high-level² language and generate another piece of code, implementing the derivatives, in that same high-level language. Source transformation happens as a static process, while operator overloading computes the derivative at run time.

Another dichotomy exists between what is called the *tangent* (or forward or direct) mode and the *adjoint* (or backward or reverse) derivatives. Both will eventually compute derivatives, but the way the various values are propagated changes drastically. One way to understand the difference between both approaches is by looking at examples. Consider the following composite function, and assume only univariate functions for simplicity,

$$x_v = u(x_u), \quad x_w = v(x_v), \quad y = w(x_w) \quad (4.1)$$

$$y = f(x) = w(v(u(x))). \quad (4.2)$$

Then, the derivative of f is computed using the chain rule as

$$\begin{aligned} f'(x) &= w'(x_w)v'(x_v)u'(x), \text{ where} \\ x_v &= u(x) \text{ and} \\ x_w &= v(x_v). \end{aligned}$$

The difference between forward and backward mode lies in the priorities affected to the chain of operations. In particular, the choice $f'(x) = w'(x_w)(v'(x_v)u'(x))$ implies a forward mode AD in which the derivative are evaluated in the same sequence as the primal code. However, the choice $f'(x) = (w'(x_w)v'(x_v))u'(x)$ evaluates backwards from the primal code, this is known as the adjoint mode AD. In fact, when one has many more compositions, the bracketing choice is important, especially when the function involved aren't univariate. This is an active area of research, which I choose not to discuss further here. The interested reader may check [96] or references therein.

Looking at the forward mode, we always have to provide a seed somehow (the rightmost element) which is also called the direction of differentiation. Typically, one could consider the $\dot{x} = dx/dx = 1$ derivative. Then, a chain of (Jacobian) matrix-vector products is computed. This is called the Jacobian-Vector-Product (JVP) variant. In reverse mode, the seed is given for the leftmost element as $\bar{y}$. Again one could consider the value $\bar{y} = dy/dy = 1$ and then propagate left to right, computing Vector-Jacobian-Products (VJP). Of course, if the computations are embedded in a larger group of computations, these seeds would be the outputs of the values surrounding the current piece of code.

This mathematical formality being clear, let us look into differentiating computer programs.

²High level means here that it has not been compiled into some intermediate representation. Even though C is often perceived as a low-level language, it is still considered high-level in this exposition.

A program computes **out** from **in**, via intermediate variables, which we usually call **v**:

$$\mathbf{in} \rightarrow \mathbf{v}_1 \rightarrow \mathbf{v}_2 \rightarrow \cdots \rightarrow \mathbf{v}_9 \rightarrow \mathbf{out}$$

In forward mode $\frac{d\mathbf{v}}{d\mathbf{in}}$, we compute JVPs:

$$1.0 = \frac{d\mathbf{in}}{d\mathbf{in}} \rightarrow \frac{d\mathbf{v}_1}{d\mathbf{in}} \rightarrow \frac{d\mathbf{v}_2}{d\mathbf{in}} \rightarrow \cdots \rightarrow \frac{d\mathbf{v}_9}{d\mathbf{in}} \rightarrow \frac{d\mathbf{out}}{d\mathbf{in}}$$

In comparison, the reverse mode propagates backwards via adjoint VJPs

$$\frac{d\mathbf{out}}{d\mathbf{in}} \leftarrow \frac{d\mathbf{out}}{d\mathbf{v}_1} \leftarrow \frac{d\mathbf{out}}{d\mathbf{v}_2} \leftarrow \cdots \leftarrow \frac{d\mathbf{out}}{d\mathbf{v}_9} \leftarrow \frac{d\mathbf{out}}{d\mathbf{out}} = 1.0$$

Whilst analyzing a whole program, we often refer to the set of all variables which exist at one point during the program as **V** and index this set with the current instruction I_p . Therefore $\mathbf{V}_p$ refer to the state of all variables at instruction I_p . With this notation in mind, the nickname *adjoint* becomes clear when writing out the matrix vector products.

$$\overline{\mathbf{in}} = f'^T(\mathbf{in}).\overline{\mathbf{out}} = f_1'^T(\mathbf{v}_0) \cdots f_{p-1}'^T(\mathbf{v}_{p-2}).f_p'^T(\mathbf{v}_{p-1}).\overline{\mathbf{out}}$$

The challenge in the adjoint AD is related to what is known as *data-flow reversal*. In general, computing an adjoint implies first computing the *primal* code, i.e. computing the output of the function, before accumulating the derivative information via a backward sweep. This is illustrated in Fig. 4.1. The problem involved in the data-flow reversal issue is that one needs

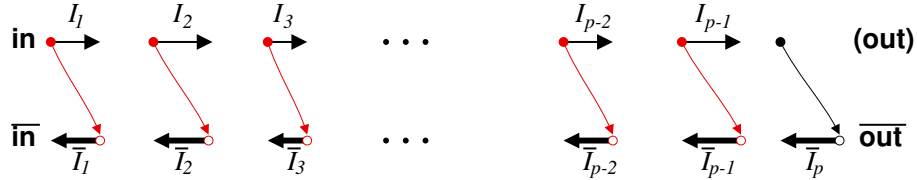


Figure 4.1: Illustration of the process of storing (pushing) states at run time during the forward sweep in order to recover (pop) them in the backward sweep.

to make sure that the variables used in evaluating the derivatives in the backward sweep have the same values as when they were used when evaluating the primal code during the forward sweep. In Fig. 4.1 red filled dots highlight the fact that the state of the variables need to be preserved during the forward sweep. Similarly the empty red dots illustrate the need to recover the state as it was during the execution of the primal code, during the backward pass. Throughout, an arrow pointing to the right denotes a forward sweep, while a thick arrow pointing left denotes a backward sweep. Full dots represent the process of storing in memory while empty circles are their dual, i.e., reading or loading from memory. The states of the program is stored during the forward sweep, after each instructions. It is then read during the backward sweep (highlighted with the bars).

```

I1 ;
...
Ip-1 ;
 $\bar{V} = \text{out} ;$ 
 $\bar{V} = f_p^{tt}(V_{p-1}) * \bar{V} ;$ 
 $\bar{V} = f_{p-1}^{tt}(V_{p-2}) * \bar{V} ;$ 
...
 $\bar{V} = f_1^{tt}(V_0) * \bar{V} ;$ 
in =  $\bar{V}$  ;

```

Figure 4.2: An example of code with backward sweep included. Data-flow reversal is all about being able to recover the red V 's even though their values have been overwritten in memory.

$V_0, V_1, \dots, V_{p-1}$ are stored successively into the same V , see Fig. 4.2 for an illustration of the adjoint code. In most programs the state V_{p-2} is lost, overwritten by I_{p-1} .

Here again, there are two main strategies: *recompute-all* or *store-all*. In the recompute-all strategy, the variables which are needed at a given adjoint instruction are recomputed from the beginning. Of course, given the static nature of ST-AD, one can benefit from code analysis and various optimizations and remove any dead or useless code for a specific variable, leading to reasonable compute times in the end. In the store-all approach, a variable is added to a stack in memory every time it is *used* and then it is read (popped from the stack) whenever it is needed in the backward sweep. In fact storing when use is probably not the best options, and one might prefer store-on-kill strategies in which variables are stacked if and when they are overwritten and only in the case that they are needed for the adjoint computation. They are then popped at the appropriate time during the backward sweep.

4.2 Profiling strategy

Neither *store-all* nor *recompute-all* is satisfactory in practice. The recompute-all strategy creates a CPU overhead while the store-all strategy very often leads to memory overflow. Consequently, the solution lies in a trade-off between both approaches: recompute what is cheap to compute and demanding in memory and store what is cheap in memory and compute intensive. Unfortunately, the world is not easily split between these two options and what is expensive or cheap might depend on the needs and architectures. As it turns out, the idea of storing some variables versus recomputing some other is not trivial to set up. This is where *checkpointing* the code is important. *Checkpointing* is the art of saving the state of the system at certain locations in order to recover it and recompute a chunk of

the code for the needs of the derivative. On selected checkpointed sections of code of the program, which might be nested, we don't store intermediate values on the stack. Instead, we re-execute this chunk of code at a later time, when we are ready for the backward sweep. This is illustrated in Fig. 4.3. It is expected that efficient checkpointing strategies allow both

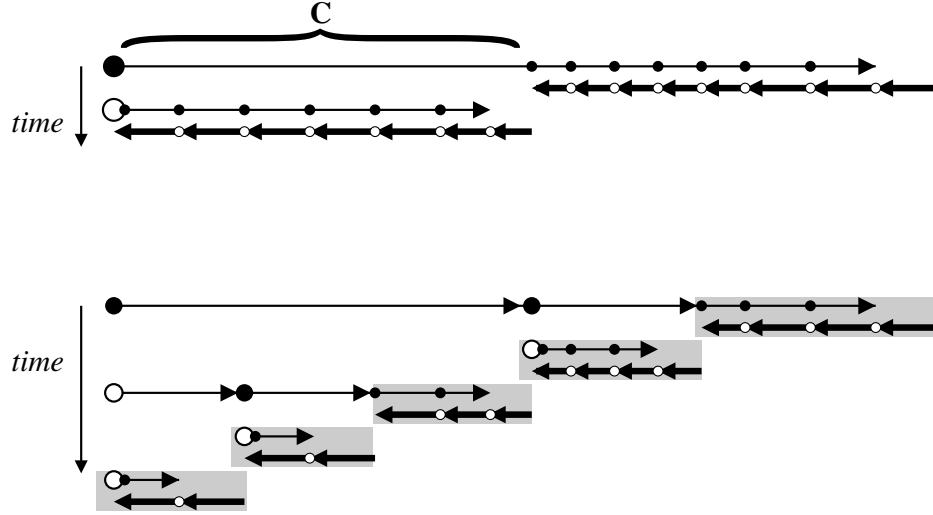


Figure 4.3: (top) A piece of code C is being checkpointed, i.e., all its useful variables are stored for efficient recomputations while the following code segment is being executed with a store-all strategy. (bottom) A nested code in which some sections are checkpointed for recomputations, while the gray boxed sections are directly executed in store-all mode.

the memory and CPU usages to grow logarithmically with the size of the primal code.

One may ask a question: how should one decide which sections of code should be checkpointed versus which sections of code should be kept in store-all. In [67] we designed and evaluated a tool for profiling call-graphs in terms of memory and compute time in order to justify certain checkpointing locations in the primal code. We are interested only in what is called *call graph checkpointing*. This is a setting in which checkpoints happen only at certain locations where `CALL`'s appear (or re-entrant sections which could be transformed into a `CALL`). It is known to be an NP-Complete problem [95] and solutions exist only in very specific cases [61, 62, 119, 112].

We denote by $\mathcal{S}$ the set of all possible checkpoint locations. A checkpointing configuration is a binay map $c : \mathcal{S} \rightarrow \{0, 1\}$ describing whether a checkpoint $X \in \mathcal{S}$ at a certain location of the primal code is activated ($c(X) = 1$) or inhibited ($c(X) = 0$). Keeping in mind that a program is seen as a sequence of instructions $P = I_1; I_2; \dots; I_n$ we analyze a given run of P by decomposing the code in three sections: an upstream section U , a downstream section D , and a piece of code of interest C , considered as a potential candidate for checkpointing. Figure 4.4 illustrates the execution of the associated adjoint code, in the base case where no checkpointing is active.

On this figure, at the turn point between the forward and backward sweeps, the stack on which all intermediate useful values are stored reaches its maximum size Tn . By comparison,

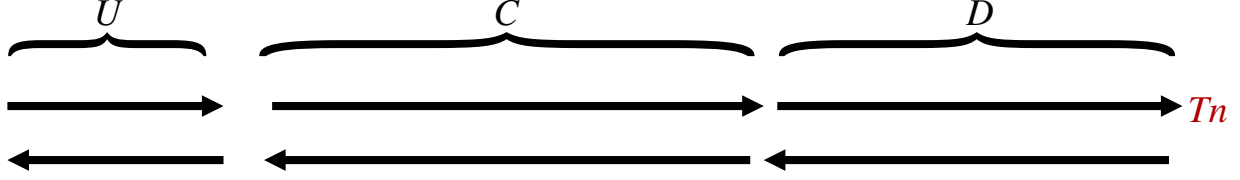


Figure 4.4: Sketch of execution for the adjoint of a code arbitrarily split as a sequence of three parts U , C , and D . Thick arrows to the right stand for forward sweeps, running the primal code together with storing intermediate values on the stack. Thick arrows to the left stand for backward sweeps, that retrieve values from the stack and use them while propagating gradients backwards. Time goes top-down, and inside each line time follows the arrows' directions.

we look at the data-flow for the adjoint of the same piece of primal code $U; C; D$, but where we chose to checkpoint C , as depicted in Fig. 4.5. In the case that $c(C) = 1$ and $c(D) = 0$, the

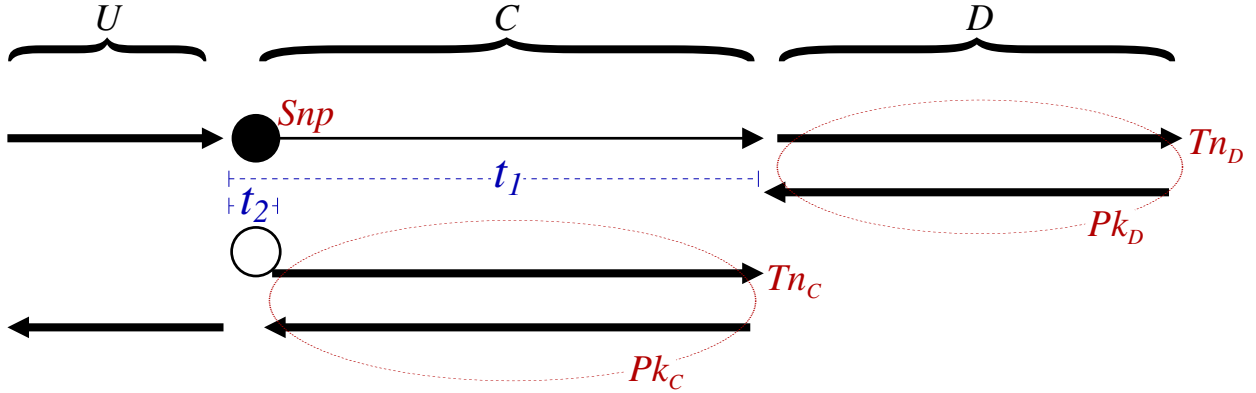


Figure 4.5: Sketch of execution for the adjoint of the same code than in Fig. 4.4, having checkpointed the section C . Time goes top-down, and inside each line time follows the arrows' directions. The thin arrow to the right stands for the extra execution of the primal C , and the black and white bullets stand respectively for the writing and reading of the *snapshot*, i.e. a set of variables sufficient for identical duplicate execution of C .

code fragment C is executed once, without storing any values in the stack, and computing only values which are useful for the downstream. Then D is executed in a store-all mode, storing all intermediate values on the stack. Eventually, at the turn point, the stack reaches a local maximum Tn_D . Similarly, another local maximum stack size Tn_C is reached at the turn point of C . In this case, the primal (forward) execution is duplicated³ adding an extra time t_1 in the compute cost. The hope is that both of the local peak stack sizes in the checkpointed case be lower than that of the default, no checkpoint, case. Note that the sizes of the arrows in the figures do not bear any information and are simply following the ordering of the primal code. As such Tn_D might very well be smaller than Tn_C . Although memory usage is usually

³*Duplicated* is not exactly accurate. Indeed, some variables might not be used in the downstream section and therefore, a code analysis would prevent its execution. Similarly, instructions from C which are used only in computing certain values in D might disappear from the second primal execution of C . We do not go any deeper into these analysis here, but they represent a fundamental part of ST-AD.

linear with the runtime, applying checkpointing recursively on nested code can reduce this behavior to a logarithmic growth both in terms of peak stack size and compute time.

Before we continue the details, we want to emphasize the distinction between static and dynamic call-graph locations. Profiling the various calls requires running the code and analyzing the results for a given set of data. There are two main consequences to this:

- Results are dependent on the current run, with the current parameters. Other runs might not yield exactly the same results.
- The call sites profiled are dynamic ones.

A dynamic call is defined as one when the execution actually reaches a static call site. A static call site is a location within the original primal code where a `CALL` to a procedure is found. Working with ST-AD, the strategy chosen for implementing the checkpointing is one where all `CALL`'s to a given procedure are checkpointed or not and we do not specify precisely a call location. This remark together with the fact that the checkpointing may be applied nested structures calls for attention. Indeed, nothing prevents code sections `D` or `U` to call the fragment `C`. Even `C` itself might have a recursive call. This implies that all call of a given procedure in the primal code will be checkpointed, whether they are reached at a single dynamic site or multiple.

As a starting configuration for the profiling, we assume that all calls are checkpointed, $c(X) = 1, \forall X \in \mathcal{S}$. This is motivated by the fact that it is the safest option, in terms of memory usage, and reduces the risk of memory saturation. It implies however a potentially much longer run time. The goal is to find a configuration c such that the memory and runtime are manageable for a given experiment. For some current configuration c , for every potential checkpoint X which is currently activated ($c(X) = 1$), we expect our profiling tool to give us the following key figures about inhibiting X :

- the associated run-time benefit $\Delta t(X)$, i.e. the run-time difference when X is switched from activated to inhibited. It should always be negative (run-time decreases) since we are trading storage for recomputation.
- the associated memory cost $\Delta Pk(X)$, i.e. the peak stack size difference when X is switched from activated to inhibited. It is in general positive except for some extreme cases – mainly due to either some static code optimization or to side effects of variables being stored in a later code fragment.

Looking back at Fig. 4.5 and assuming that checkpoints may be nested in the adjoint code, any of `U`, `C`, and `D` might have any number of checkpointed segment X . Some of these checkpoints might even be the candidate `C` itself. We want to analyze the memory cost and time benefits from inhibiting a certain checkpoint. To this end let us introduce some notations and vocabulary. A sequence of a forward pass followed by a backward pass is called

a *round trip* and is highlighted with a bar over a code section. For instance, $\bar{D}$ corresponds to the round trip of D on the right hand side of Fig. 4.5. In this case, the round trip $\bar{D}$ is precisely the concatenation of a forward sweep $\vec{D}$ and its associated backward sweep $\overleftarrow{D}$. Round trips may be understood on a sequence of code segments and depend on the checkpointing configuration. For instance, the round trip for the case $c(U) = c(C) = c(D) = 0$ is illustrated in Fig. 4.4 and is in fact the sequence

$$\overline{U; C; D} = \vec{U}; \vec{C}; \bar{D}; \overleftarrow{C}; \overleftarrow{U}.$$

Fig. 4.5 shows an examples of round trip in the case $c(C) = 1$. It reads

$$\overline{U; C; D} = \vec{U}; C; \bar{D}; \overleftarrow{C}; \overleftarrow{U}.$$

This figure also highlights two dark dots: the filled one corresponds to saving a snapshot of the various useful variables. They are the variables that one needs to restore in order to be used in the next pass of C . The empty dot is precisely this restoring operation.

As illustrated in Figs. 4.5 and 4.4 and taking into account that there may be nested checkpoints inside D , we distinguish the peak stack sizes Tn_D and Pk_D : Tn_D is the (local) peak reached at the turn point at the end of $\vec{D}$, whereas Pk_D is the (global) maximum peak stack size attained at one point, anywhere, during the round trip on D , i.e. at the turn point after $\vec{D}$ but also at the turn point of any checkpoint N nested in D . Nothing prevents $\vec{N}$ from storing more intermediate values than D and therefore $Pk_D \geq Tn_D$ in general. In the sequel we will need to keep track of both Tn_D and Pk_D . The same remark applies to C or any other candidate checkpoint in $\mathcal{S}$.

Profiling is performed on the adjoint code corresponding to the current checkpointing configuration c . We want the profiling tool to return, for each currently active static checkpoint $X \in \mathcal{S}, c(X) = 1$, the cost in memory $\Delta Pk(X)$ versus the benefits in time $\Delta t(X)$ for inhibiting X . The values $\Delta Tn(X)$ are irrelevant for the final profiling but are useful for our reasoning. To keep notation lightweight, we drop the X since all costs and benefits can be evaluated independently from the actual X .

As the code is being executed for a given checkpointing configuration c , the various values for memory costs and time benefits are evaluated for all segments X with $c(X) = 1$. Then, these values are propagated bottom-up and right-to-left in the dynamic call graph. As an illustration, consider the situation in Fig. 4.5 ($c(U) = c(D) = 0, c(C) = 1$). Peak memory and compute time are computed on the fly for the round trip on the sequence $C; D$:

$$\overline{C; D} = C; \bar{D}; \overleftarrow{C}.$$

As the run progresses, we will, first compute the Δ values on $\bar{D}$ and $\overleftarrow{C}$. The computation of the Δ 's from the inhibition of any checkpoint C or D is done by propagating and accumulating the values already calculated from the single round trips $\bar{D}$ and $\overleftarrow{C}$. To ease the notation again, we indicate as a subscript the sequence of code on which the values are computed. Therefore our goal is, given $\Delta t_C, \Delta Tn_C, \Delta Pk_C$ about the round trip on C and likewise $\Delta t_D, \Delta Tn_D,$

ΔPk_D about the round trip on D, to compute Δt_{CD} , ΔTn_{CD} , and ΔPk_{CD} about the round trip on C;D. This computation should be done for all active checkpoint X , irrelevant of where they appear (within C or D or both) and how many times they are met during the current run.

To start the propagation, we look at the case of a piece of code containing no activated checkpoint. The profiling can return the basis times and stack sizes and in this case $Pk = Tn$. Δ values are all zero for all X . Now, to propagate the costs/benefits estimations, we consider an activated checkpoint $X \in \mathcal{S}$ which may or may not be nested within C or D. We analyze the interactions between nested checkpoints. Looking at Fig. 4.5 and thinking about the run time gives

$$t_{CD} = t_1 + t_D + t_2 + t_C \quad (4.3)$$

where t_1 is the extra time to take the snapshot and run the primal C, and t_2 is the time to read the snapshot. Looking at the stack sizes we obtain

$$Tn_{CD} = Tn_D \quad (4.4)$$

$$Pk_{CD} = \max(Pk_D, Pk_C) \quad (4.5)$$

We want to quantify the changes in value of t_{CD} , Tn_{CD} , and Pk_{CD} compared to the base case (Eqs. (4.3), (4.4), (4.5)) when X switches from activated to inhibited. We need to distinguish the cases $X = C$ from $X \neq C$. If $X \neq C$, C is unaffected by the switch and we remain in the situation depicted in Fig. 4.5. It is straightforward to see that accumulating values estimated independently on each segments C and D into the section C;D can be written as

$$\Delta t_{CD} = \Delta t_D + \Delta t_C \quad (4.6)$$

for the time benefits and

$$\Delta Tn_{CD} = \Delta Tn_D \quad (4.7)$$

and

$$\Delta Pk_{CD} = \max \begin{pmatrix} Pk_D + \Delta Pk_D \\ Pk_C + \Delta Pk_C \end{pmatrix} - Pk_{CD} \quad (4.8)$$

for the memory usage.

The updates are more involved in the case $X = C$ as we switch from the situation depicted in Fig 4.5 to the one in Fig. 4.4. The time benefits is now larger (smaller negative value) as we spare the time needed for the recomputation of C and the storing and reading of the snapshot:

$$\Delta t_{CD} = \Delta t_D + \Delta t_C - t_1 - t_2 \quad (4.9)$$

The stack space used by $\vec{C}$ now accumulates with that used by the round trip $\bar{D}$. Stack size after $\vec{C}$ in Fig. 4.4 is indeed $Tn_C + \Delta Tn_C$. On top of it is accumulated the stack space specifically used by $\vec{D}$ and $\bar{D}$ which are respectively $Tn_D + \Delta Tn_D - Snp$ and $Pk_D + \Delta Pk_D - Snp$, where Snp is the stack size measured just after taking the snapshot.

$$\Delta Tn_{CD} = Tn_C + \Delta Tn_C + \Delta Tn_D - Snp \quad (4.10)$$

$$\Delta Pk_{CD} = \max \left(\begin{array}{l} Tn_C + \Delta Tn_C + Pk_D + \Delta Pk_D - Snp \\ Pk_C + \Delta Pk_C \\ - Pk_{CD} \end{array} \right) \quad (4.11)$$

It is worth mentioning that in some extreme cases, the memory cost may in fact be negative (i.e. a benefit). This happens when the snapshot stored when checkpointing a segment is larger than the cumulation of the intermediate values stored in the stack. One should never checkpoint such a segment. In theory, such cases should not happen, but other optimizations in the code generated perturb this analysis. Understanding and quantifying these side effects is an interesting question on its own.

We keep propagating these values throughout the run of the code and end up with a report summarizing the costs and benefits of inhibiting certain checkpoints. It is the responsibility of the end user to then decide which one to inhibit next. In theory, results are only valid for a given configuration c . Therefore one only knows which is the next best site whose checkpoint should be deactivated.

Open question 4.1: Inhibiting many checkpoints from a single profile

Balancing between checkpointing and not checkpointing function calls is a major issue for large scientific applications. Finding a good enough configuration requires then to reiterate the profiling with the new updated configuration which might be extremely time consuming. This is arguably a strong limitation. Two questions are of interests here:

- a) can a report on the profiling be used to infer information of profiling at a deeper level?
- b) if yes, how should we gain this information?

Indeed, the current way of profiling a computation is to checkpoint everything and profile a run of the application. The report suggest the most *important* (importance is a mixed notion of memory consumption versus compute cost) call to checkpoint and then iteratively run the profiler again until a satisfying run-time / memory consumption state is obtained.

May be defining an optimization problem could help and one could expect to extrapolate a sub-optimal checkpointing configuration from much fewer runs. This is the strategy chosen in [34] where an Integer Linear Programming problem is solved to facilitate the choice of checkpointing configuration.

The proposed profiling was tested on an example from the Massachusetts Institute of Technology General Circulation Model (MITgcm) [88, 3]. It is an academic code famous in the climate science community for its ice stream and ice shelf simulations, its synchronous ice-ocean coupling capabilities, and its adjoint [79, 59]. Differentiating by hand such a code being close to impossible, it was always designed to be compatible with AD tools, traditionally TAF [58] and recently using Tapenade [57]. It is thus a key component of the Estimating the Circulation and Climate of the Ocean (ECCO) state estimate: a data as-

simulation product widely employed by the physical oceanography research community. The `halfpipe_streamice` test case uses the package `streamice` to simulate the flow of land ice in a valley. The simulation runs for 80 timesteps on a 40×20 horizontal grid. The individual time steps are more complex than a typical MITgcm simulation since they contain fixed point iteration based solvers to solve the non-linear stress balance. Their adjoint is built according to Christianson’s two-phases algorithm [41, 60]. Typically, run-time performance of an adjoint code is measured by the ratio with the run-time of the primal. The adjoint code by Tapenade using the default *checkpoint all* strategy yielded a 7.0 ratio, which is fair but not as good as what OpenAD or TAF provide. This diagnostic was the starting point and motivation for the profiling tool presented above. The hope is to reduce this ratio whilst keeping a reasonable memory footprint. We have used our profiling tool iteratively to find better checkpointing configurations and reported this search in Fig. 4.6. The initial configuration with all checkpoints activated (indicated with the black *no binomial* arrow) has an average run-time of 155.4 seconds and a peak stack size of 119.1 megabytes. The number of static checkpoint locations is 85, yielding an enormous 2^{85} number of possible configurations to explore.

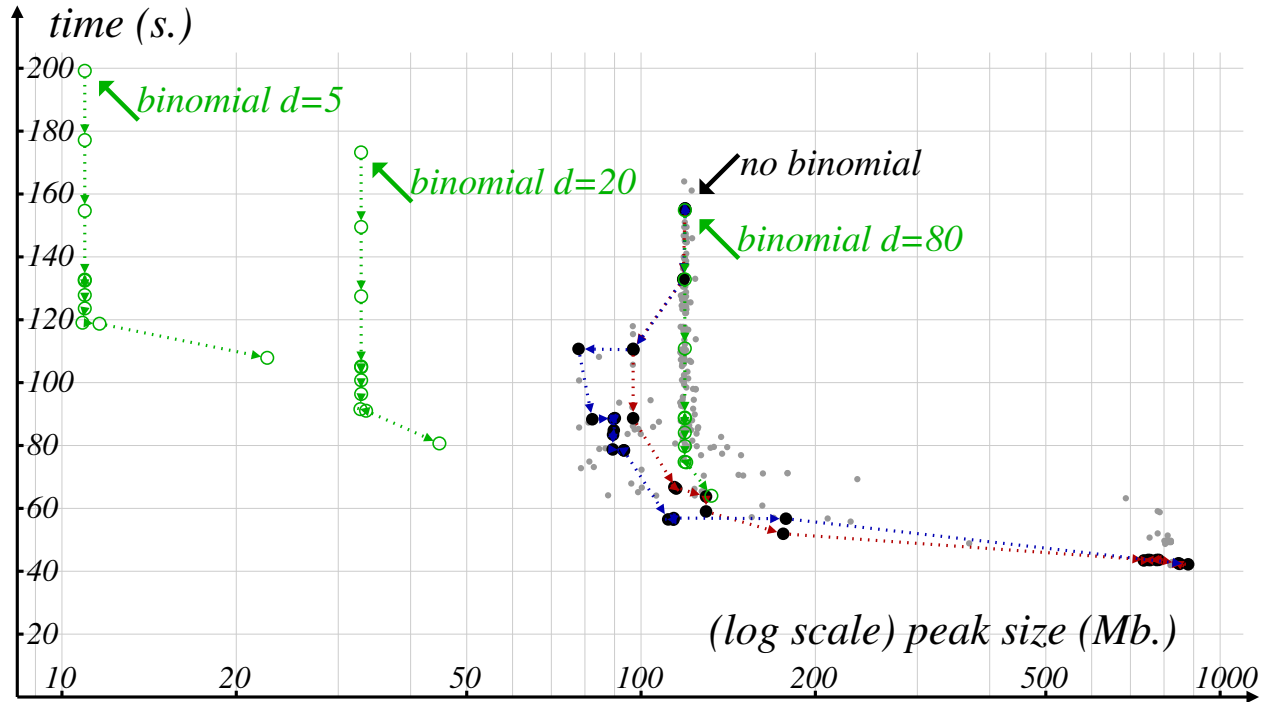


Figure 4.6: Tuning the adjoint of `streamice` by profiling. Each checkpointing configuration is shown as a dot whose coordinates are peak stack size and run time of the adjoint. The red and blue dotted lines show the evolution of performance of the adjoint by repeatedly following the suggestions made by our profiling tool. The red line follows a “run-time gain first” strategy, the blue line a “careful on memory” strategy. Time measurements are averaged on 5 runs. Smaller gray dots show the performance of 250 random checkpointing configurations. Green dots show combination with binomial checkpointing, an algorithm used for optimal checkpointing of time-stepping loops.

The suggestions returned by the profiled adjoint code are sorted in 3 categories:

- Static checkpoints whose inhibition give a run-time gain (as always), and a stack peak gain. Those are rare cases where the snapshot is larger than the amount of intermediate values stored in the forward sweep. This is generally due to overestimation of the AD data-flow analysis on arrays, where activity for a single cell of an array implies activity of the whole array. This sort of profiling result means that the checkpoint should be inhibited without hesitation.
- Static checkpoints whose inhibition give no change of the stack peak. This happens when the effect of inhibiting the checkpoint indeed modifies the size of one local peak of the stack, but this peak is smaller than the global peak (refer to Eq. (4.11)) and therefore the change is hidden. The checkpoint is a good candidate for inhibition, as this will gain some run time without changing the global peak memory size.
- Static checkpoints whose inhibition will increase the stack peak size. Decision to inhibit it results from a trade off decision, comparing with the run-time expected benefit and possibly the available budget in memory space.

As these suggestions allow for several alternative decisions, the end-user may apply different strategies. Two examples are presented in Fig. 4.6: one (red line) follows the lowest run-time path. The other one, the blue curve, follows a low-memory cost path. The process was the same for each experiments: select the or a few of the best suggestions, then perform a new differentiation of the code inhibiting the selected checkpoints and then run again the profiling. Fig. 4.6 shows with black dots the time and memory performances of the successive adjoint codes after each step. Gray dots show the performance of 250 randomly chosen checkpointing configurations. We observe that they are mostly situated above and right of the red and blue lines, indicating that the profiling suggestions, combined with these strategies, take performance close to the front of optimal configurations.

Remembering our goal to reduce the ratio of run time of the adjoint code with respect to the duration of the primal code, we extract some specific runs found on the red and blue paths. Some tested configurations illustrated in Fig. 4.6, significant time gains for small memory cost, if not even small memory gains. For instance one dot on the red line exhibits a stack peak of 176.0 Mb, roughly 50% above the initial 119.1 Mb, but cutting run time to a third of its original value, i.e. from 155.4 s to 51.9s. This means a slowdown ratio for the adjoint code of 2.4, a drastic reduction from the initial 7.0. 50% memory increase from the default *checkpoint all* strategy might still mean a lot in very large scientific computing applications. One could limit themselves on another dot on the blue line for which a peak stack size of 111.41 Mb, even less than the initial 119.1 Mb, was reached, but reducing the run time to 56.49 s., i.e. achieving a very competitive adjoint-to-primal ratio of 2.6.

In passing, we have also compared the application of the various checkpointing strategies with the binomial checkpointing, which is known to be optimal in case of time-stepping loops. The binomial checkpointing is very different in the sense that it does not look for

specific call sites to check points and consequently, it is a good candidate for collaborating with the suggested strategies.

4.3 What is coming in my AD world

A year in a half in the AD world has made me wonder about the next steps. There are many topics of interest in this very important, yet a little bit abandoned, subject. I detail here some ideas that are of interest to the community and on which I am actively (or soon to be) working. They appear somewhat independent from one another but are all mandatory bricks in the AD world to keep ST-AD up to pace for today's standard.

One important aspect is the evolution of programming languages. ST-AD has traditionally been applied to imperative languages such as **C** and **Fortran**.⁴ These strongly typed languages allow for *easy* type checking and other static code analysis. These analysis are much harder to handle when looking at object oriented languages, in particular if they allow templates. On the other hand even timeless languages such as **C** and **Fortran** evolve and allow more and more flexible constructs. For instance, function handlers and objects have appeared in recent **Fortran** standards and one needs to make ST-AD tools evolve with them.

Open question 4.2: Object oriented programming

Allowing pieces of code to handle Objects in the **Tapenade** tool is of prime importance and may require to adapt bits of the internal data representation. This is already ongoing with some recent adaptations for function handlers, but still require thorough tests and thinking.

One important aspect in modern languages is also the ability to offer inheritance, polymorphism, and method overloading. This is an important aspect for certain modern languages, like **Julia**, or some object oriented languages.

⁴This authoritarian statement must be tempered. Some work has been done with **C++** via **Clang** and gave birth to **Clad** [118]. Similarly, **Java** has had its moment for AD with the **ADiJaC** tool [108]

Open question 4.3: ST-AD for Julia

Signal and data processing practitioners use mainly modern languages like **Python** and **Julia** for fast prototyping and certain deployments. Both of these languages are dynamically typed – although it is understood in a very liberal sense for **Python**, **Julia** offers some restrictions. Moreover, they use and abuse the concepts of overloading, and **Julia**’s *multiple dispatch* is one of the main advantages highlighted. This renders most of the static code analysis highly complex and is related to the problem of polymorphism in object oriented programming languages. Handling such structures in traditional ST-AD tools would go a long way in being able to apply these tools in modern software architectures.

Some work is already underway on this subject.

Another aspect of modern programming languages is in the line of memory management. It is notorious that **C** is rather tolerant when it comes to memory, but modern languages tend to be more robust. In particular, the concept of *garbage collector* is found in many languages⁵, including **Julia**.

Open question 4.4: AD Memory management

Memory management is of prime importance in programming, and even more so in handling source transformation AD. Indeed, a piece of code which would have **free**’s and which one would like to differentiate, would need to have some sort of revival mechanism in order to generate adjoint code. This relatively trivial task (one could use an external table linking memory values to variables) could keep an early stage researcher busy.

Another important topic is the concept of *garbage collection* which is mandatory to develop, if one hopes to develop ST-AD tools for modern languages in general, and **Julia** in particular. Some theoretical work has already been done on the subject [66], but its precise implementation and development remains to be done.

Finally, putting all the pieces together would allow to have a fairly functional ST-AD tool for the **Julia** language. This would allow to enter the machine learning and data science world which could benefit from ST-AD’s added value.

⁵**Rust** chose another concept, the *ownership* which is a bit different, but still handles memory in better ways than **C** does.

Open question 4.5: Algorithm sparsification

One important topic in the data sciences and neural network theory is to reduce the memory footprint and inference time for very large models. One approach for this (related to the distillation of models) is to reduce the number of parameters. Luckily, by combining AD tools and static code analysis with interval propagation, one can hope to quantify the importance of certain features and the sensitivity of some outputs with respect to said features [4]. ST-AD offers precisely the right framework to analyze, sparsify, and generate efficient codes for reducing the costs of large models.

Chapter 5

Outlook

My scientific journey has followed what appears to be a fairly chaotic path. Driven by many topics of attractions such an unpredictable path is to be expected.¹ This varied interest is reflected in the diversity of problems tackled in this *Habilitation à Diriger des Recherches* and the bouncing back and forth between applications and theory, computer science and mathematics, is what drives my curiosity and allows me to innovate on certain problems. In practice, the common thread of signal and data processing in a very broad sense is always in the background.

Recently, I have been interested in applying my novel knowledge of source-transform AD to problems in computer vision. One question arising in multi-view geometry is how to fuse the images coming from multiple view points or multiple cameras. This problem known as a bundle adjustment requires solving large optimization problems where the Jacobian involved is very sparse. By combining ideas from Algorithmic Sparse Differentiation [71] and source transform strategies, we can generate highly efficient hardware agnostic kernels to compute the sparsity patterns. The implications are many: one can port codes from CPU to GPU without effort and have highly efficient codes for computing the Jacobian sparsity pattern. This in turns allows highly optimized code for solving the bundle adjustment problem. This line of work is currently in progress but already shows promising results. This bundle adjustment example is not the only in demand of multiple and/or heterogeneous architecture portability. One goal with the further development of the ST-AD tool **Tapenade** [68] is to be able to specify pieces of code which should be optimized for GPU-like architectures and SIMD parallelism. The question of finding sparsity patterns in second order derivatives is still very open and interesting to pursue.

Similarly, with the development of the first ST-AD tool for the **Julia** language, I can enter the world of machine learning with all sorts of code optimization and checkpointing strategies, clearly a bottleneck in ML today [15, 87]. Moreover, the world of machine learning is in demand of novel approaches to handle the AD part of learning a model [14]. The interactions

¹The work of Alain Chenciner as well as Liu Cixin's book explain this problem very well!

between the AD and the ML worlds is not limited to this. As mentioned in the Open problem 4.1 one can also hope to improve the speed of the profiler via a learned model. This is highly exploratory but would offer a fair mix of understanding AD, optimization, and ML, surely a topic of interest for my future!

Machine (deep?) learning is also at the forefront of scientific computing recently, and reviewing the way AD is done within it could help. Of particular interest to me is the topic of CNNs for parametric PDEs, thereby combining chapters 3 and 4 together (see [70] for an interesting and recent example). Similarly, some interesting research has appeared in the recent past on structural image similarities [47, 48]. It would be interesting to generalize such approaches to all metrics satisfying the monotonicity as well as include a scale and rotation component to it (as expressed within the Open problem 2.2).

Whilst advancing novel languages in AD, it does not mean that old languages are thrown away. In fact **Fortran** is still evolving on a regular basis, adding new constructs at each iterations (for instance, the use of objects and function pointers in **Fortran** 2003). Such new constructs imply adapting the way static code analysis are done. Function pointers are now better handled within **Tapenade**, thanks to the work on the **Julia** language already carried out, but some work needs to be done for classes. Also, even if **Julia** or **Python** are rapidly gaining interest in the community, they are often only used as interface for calling an engine in a lower level language such as **Fortran** or **C**. With this in mind, adapting the differentiation model to handle multiple languages at once is an important task. Indeed, one could have a piece of code written in **Julia** with a bit of computations done, but which would afterwards call some **C** or **Fortran** engine. This is typically the case of the physics community working with the **QCDNUM** package. This community is also very eager to differentiate parametric integrals. This is, from a theoretical point of view, very easy via *Leibniz formula* but hard to implement in source transform AD. Having a mature enough AD protocol for such a matter would help in many instances where the domain of integration plays a role in a non-differentiable manner.

Last but not least comes the concept of hybrid AD models in which one would try to mix source transformation with operator overloading. Trying to benefit from both world, one could manage to have an AD model which would be robust against many modern constructs such as dynamic polymorphisms in object oriented programming, but gaining the power of static code analysis. The idea here would be to make use of techniques coming from source transformation in order to spare the duplication of information on the tape stemming from operator overloading. This far-fetched goal is also an interesting one, since it would allow to alleviate many limitations left by source transform AD.

Bibliography

- [1] R. Aceska, J.-L. Bouchot, and S. Li. Fusion frames and distributed sparsity. In *Frames and harmonic analysis*, volume 706, pages 47–61. Amer. Math. Soc., 2018.
- [2] R. Aceska, J.-L. Bouchot, and S. Li. Local sparsity and recovery of fusion frame structured signals. *Signal Processing*, 174:107615, 2020.
- [3] A. Adcroft, J.-M. Campin, E. Doddridge, S. Dutkiewicz, C. Evangelinos, D. Ferreira, M. Follows, G. Forget, B. Fox-Kemper, P. Heimbach, C. Hill, E. Hill, H. Hill, O. Jahn, J. Klymak, M. Losch, J. Marshall, G. Maze, M. Mazloff, D. Menemenlis, A. Molod, and J. Scott. *MITgcm User Manual*. Available at <https://mitgcm.readthedocs.io>.
- [4] S. Afghan and U. Naumann. Interval adjoint significance analysis for neural networks. In *International Conference on Computational Science*, pages 365–378. Springer, 2020.
- [5] G. S. Alberti, A. Felisi, M. Santacesaria, and S. I. Trapasso. Compressed sensing for inverse problems and the sample complexity of the sparse Radon transform. *Journal of the European Mathematical Society*, 2025.
- [6] M. S. Alnæs, J. Blechta, J. Hake, A. Johansson, B. Kehlet, A. Logg, C. Richardson, J. Ring, M. E. Rognes, and G. N. Wells. The FEniCS project version 1.5. *Archive of Numerical Software*, 3(100), 2015.
- [7] K. Axiotis and M. Sviridenko. Iterative hard thresholding with adaptive regularization: Sparser solutions without sacrificing runtime. In *International Conference on Machine Learning*, pages 1175–1197. PMLR, 2022.
- [8] A. J. Baddeley. An error metric for binary images. In *Proceedings, International Workshop on Robust Computer Vision*, pages 59–78. Wichmann, 1992.
- [9] B. Bah and R. Ward. The sample complexity of weighted sparse approximation. *IEEE Transactions on Signal Processing*, 64(12):3145–3155, June 2016.
- [10] A. S. Bandeira, E. Dobriban, D. G. Mixon, and W. F. Sawin. Certifying the restricted isometry property is hard. *IEEE transactions on information theory*, 59(6):3448–3450, 2013.
- [11] A. S. Bandeira, K. Scheinberg, and L. N. Vicente. On partial sparse recovery, 2013.

- [12] A. Barth, C. Schwab, and N. Zollinger. Multi-level Monte Carlo finite element method for elliptic PDEs with stochastic coefficients. *Numerische Mathematik*, 119(1):123–161, 2011.
- [13] P. Bauer, U. Bodenhofer, and E. P. Klement. A fuzzy algorithm for pixel classification based on the discrepancy norm. *Neurocomputing*, page 128190, 2024.
- [14] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind. Automatic differentiation in machine learning: a survey. *Journal of machine learning research*, 18(153):1–43, 2018.
- [15] O. Beaumont, J. Herrmann, G. Pallez, and A. Shilova. Optimal memory-aware back-propagation of deep join networks. *Philosophical Transactions of the Royal Society A*, 378(2166):20190049, 2020.
- [16] R. Bellman. *Dynamic programming*. Princeton Landmarks in Mathematics. Princeton University Press, Princeton, NJ, 2010. Reprint of the 1957 edition, With a new introduction by Stuart Dreyfus.
- [17] S. Bernstein, J.-L. Bouchot, M. Reinhardt, and B. Heise. Generalized analytic signals in image processing: comparison, theory and applications. In *Quaternion and Clifford Fourier Transforms and Wavelets*, pages 221–246. Springer, 2013.
- [18] T. Blumensath. Accelerated iterative hard thresholding. *Signal Processing*, 92(3):752–756, 2012.
- [19] T. Blumensath and M. E. Davies. How to use the iterative hard thresholding algorithm. In *SPARS’09-Signal Processing with Adaptive Sparse Structured Representations*, 2009.
- [20] T. Blumensath and M. E. Davies. Iterative hard thresholding for compressed sensing. *Applied and computational harmonic analysis*, 27(3):265–274, 2009.
- [21] D. Boffi, F. Brezzi, and M. Fortin. *Mixed finite element methods and applications*, volume 44 of *Springer Series in Computational Mathematics*. Springer, Heidelberg, 2013.
- [22] G. Borgefors. Distance transformations in digital images. *Computer vision, graphics, and image processing*, 34(3):344–371, 1986.
- [23] J.-L. Bouchot. A generalized class of hard thresholding algorithms for sparse signal recovery. In *Approximation Theory XIV: San Antonio 2013*, pages 45–63. Springer, 2014.
- [24] J.-L. Bouchot. Weighted compressed sensing for function-valued mapping approximation and applications. In preparation, October 2025.
- [25] J.-L. Bouchot and F. Bauer. Discrepancy norm: Approximation and variations. *Journal of Computational and Applied Mathematics*, 272:162–179, 2014.

- [26] J.-L. Bouchot, B. Bykowski, H. Rauhut, and C. Schwab. Compressed sensing Petrov-Galerkin approximations for parametric PDEs. In *Proc. 2015 International Conference on Sampling Theory and Applications (SampTA)*, pages 528–532. IEEE, 2015.
- [27] J.-L. Bouchot, S. Foucart, and P. Hitczenko. Hard thresholding pursuit algorithms: number of iterations. *Applied and Computational Harmonic Analysis*, 41(2):412–435, 2016.
- [28] J.-L. Bouchot and K. Hamm. Stability and robustness of RBF interpolation. *Sampling Theory in Signal and Image Processing*, 16(1):37–53, 2017.
- [29] J.-L. Bouchot, J. Himmelbauer, and M. Bernhard. On autocorrelation based on Hermann Weyl’s discrepancy norm for time series analysis. In *The 2010 International Joint Conference on Neural Networks (IJCNN)*, pages 1–7. IEEE, 2010.
- [30] J.-L. Bouchot and F. Morain-Nicolier. Scaled distance transforms and monotonicity of autocorrelations. *IEEE Signal Processing Letters*, 31(10):1235–1239, 2014.
- [31] J.-L. Bouchot, H. Rauhut, and C. Schwab. Multi-level compressed sensing Petrov-Galerkin methods for numerical solutions of high-dimensional parametric PDEs. In preparation, September 2025.
- [32] J.-L. Bouchot, G. Stübl, and B. A. Moser. A template matching approach based on the discrepancy norm for defect detection on regularly textured surfaces. In J.-C. Pinoli, J. Debayle, Y. Gavet, F. Gruy, and C. Lambert, editors, *Tenth International Conference on Quality Control by Artificial Vision*, volume 8000, page 80000K. International Society for Optics and Photonics, SPIE, 2011.
- [33] J.-L. Bouchot, W. L. Trimble, G. Ditzler, Y. Lan, S. Essinger, and G. Rosen. *Advances in Machine Learning for Processing and Comparison of Metagenomic Data*, pages 295–329. Elsevier, 2014.
- [34] A. Boudaoud, A. Calotoiu, M. Copik, and T. Hoefler. DaCe AD: Unifying high-performance automatic differentiation for machine learning and scientific computing, 2025.
- [35] S. Brugiapaglia, S. Dirksen, H. C. Jung, and H. Rauhut. Sparse recovery in bounded Riesz systems with applications to numerical methods for PDEs. *Applied and Computational Harmonic Analysis*, 53:231–269, 2021.
- [36] B. Bykowski. Weighted ℓ_1 minimization methods for numerical approximations of parametric PDEs under uncertainty quantification. Master’s thesis, Chair for Mathematics C (Analysis), RWTH Aachen, June 2015.
- [37] P. G. Casazza and G. Kutyniok. Frames of subspaces. *Contemporary Mathematics*, 345:87–113, 2004.
- [38] P. G. Casazza, G. Kutyniok, and S. Li. Fusion frames and distributed processing. *Applied and computational harmonic analysis*, 25(1):114–132, 2008.

- [39] M. Chammas, I. Arduin, L. Cornaggia, P. Briere, L. Lacombe, J. Lecarpentier, T. Rousset, J.-L. Bouchot, S. Saing, G. Verger, et al. PEPS5 study on the interest of energy storage and of power-to-X-final report+ appendix to the PEPS5 study: report on the second life of batteries+ technological sheets: Technical-economic information related to technologies of electric, thermal, hydrogen, power-to-heat and power-to-gas storage, 2023.
- [40] O. Christensen et al. *An introduction to frames and Riesz bases*, volume 7. Springer, 2003.
- [41] B. Christianson. Reverse accumulation and attractive fixed points. *Optimization methods and software*, pages 311–326, 1994.
- [42] A. Cohen, A. Chkifa, and C. Schwab. Breaking the curse of dimensionality in sparse polynomial approximation of parametric PDEs. *Journ. Math. Pures et Appliquees*, 103(2):400–428, 2015.
- [43] A. Cohen and R. DeVore. Approximation of high-dimensional parametric PDEs. *Acta Numerica*, 2015.
- [44] A. Cohen, R. Devore, and C. Schwab. Analytic regularity and polynomial approximation of parametric and stochastic elliptic PDE’s. *Analysis and Applications*, 9(01):11–47, 2011.
- [45] F. Dangel, T. Siebert, M. Zeinhofer, and A. Walther. Collapsing Taylor mode automatic differentiation, 2025.
- [46] P. J. Davis. *Interpolation and approximation*. Courier Corporation, 1975.
- [47] M. Diaw, A. Delahaies, J. Landré, F. Retraint, and F. Morain-Nicolier. Modeling a local dissimilarity map with weibull distribution—application to 2-class and multi-class image classification. *IEEE Access*, 10:35750–35767, 2022.
- [48] M. Diaw, F. Retraint, F. Morain-Nicolier, A. Delahaies, and J. Landré. New statistic detector for structural image similarity. *IEEE Transactions on Signal Processing*, 2025.
- [49] J. Dick, F. Y. Kuo, Q. T. LeGia, and C. Schwab. Multi-level higher order QMC Galerkin discretization for affine parametric operator equations. *SIAM J. Numer. Anal.*, 54(4):2541–2568, 2016.
- [50] G. Ditzler, Y. Lan, J.-L. Bouchot, and G. Rosen. *Variable Selection to Improve Classification of Metagenomes*, pages 1–9. Springer New York, New York, NY, 2013.
- [51] M. Döhler, S. Kunis, and D. Potts. Nonequispaced hyperbolic cross fast Fourier transform. *SIAM journal on numerical analysis*, 47(6):4415–4428, 2010.
- [52] M. K. B. Doumbouya, D. Jurafsky, and C. D. Manning. Tversky neural networks: Psychologically plausible deep learning with differentiable Tversky similarity, 2025.

- [53] J. Fell. *Weighted ℓ^1 -analysis minimization and stochastic gradient descent for low rank matrix recovery*. Phd thesis, RWTH Aachen University, 2020.
- [54] M. Fenn, S. Kunis, and D. Potts. Fast evaluation of trigonometric polynomials from hyperbolic crosses. *Numerical Algorithms*, 41(4):339–352, 2006.
- [55] S. Foucart. Hard thresholding pursuit: an algorithm for compressive sensing. *SIAM Journal on Numerical Analysis*, 49(6):2543–2563, 2011.
- [56] S. Foucart and H. Rauhut. *A Mathematical Introduction to Compressive Sensing*. Applied and Numerical Harmonic Analysis. Birkhäuser, 2013.
- [57] S. S. Gaikwad, S. H. K. Narayanan, L. Hascoët, J.-M. Campin, H. Pillar, A. Nguyen, J. Hükelheim, P. Hovland, and P. Heimbach. MITgcm-AD v2: Open source tangent linear and adjoint modeling framework for the oceans and atmosphere enabled by the automatic differentiation tool Tapenade. *Future Generation Computer Systems*, 163:107512, 2025.
- [58] R. Giering and T. Kaminski. Recipes for Adjoint Code Construction. *ACM Transactions on Mathematical Software*, 24(4):437–474, 1998.
- [59] D. Goldberg, K. Snow, P. Holland, J. Jordan, J.-M. Campin, P. Heimbach, R. Arthern, and A. Jenkins. Representing grounding line migration in synchronous coupling between a marine ice sheet model and a z-coordinate ocean model. *Ocean Modelling*, 125:45–60, 2018.
- [60] D. N. Goldberg, S. H. K. Narayanan, L. Hascoët, and J. Utke. An optimized treatment for algorithmic differentiation of an important glaciological fixed-point problem. *Geoscientific Model Development*, 9(5):1891–1904, 2016.
- [61] A. Griewank. Achieving logarithmic growth of temporal and spatial complexity in reverse automatic differentiation. *Optimization Methods and software*, 1(1):35–54, 1992.
- [62] A. Griewank and A. Walther. Algorithm 799: revolve: an implementation of checkpointing for the reverse or adjoint mode of computational differentiation. *ACM Transactions on Mathematical Software (TOMS)*, 26(1):19–45, 2000.
- [63] A. Griewank and A. Walther. *Evaluating derivatives: principles and techniques of algorithmic differentiation*. SIAM, 2008.
- [64] W. Hackbusch. *Multigrid methods and applications*, volume 4 of *Springer Series in Computational Mathematics*. Springer-Verlag, Berlin, 1985.
- [65] M. Hansen and C. Schwab. Analytic regularity and nonlinear approximation of a class of parametric semilinear elliptic PDEs. *Mathematische Nachrichten*, 286(8–9):832–860, 2013.
- [66] L. Hascoët. Data-flow reversal and garbage collection. *ACM Transactions on Mathematical Software*, 50(1):1–20, 2024.

- [67] L. Hascoët, J.-L. Bouchot, S. S. Gaikwad, S. H. K. Narayanan, and J. Hückelheim. Profiling checkpointing schedules in adjoint ST-AD. In *AD 2024, 8th International Conference on Algorithmic Differentiation*, 2024.
- [68] L. Hascoët and V. Pascual. The Tapenade Automatic Differentiation tool: Principles, Model, and Specification. *ACM Transactions On Mathematical Software*, 39(3), 2013.
- [69] I. Haviv and O. Regev. The restricted isometry property of subsampled Fourier matrices. In *Geometric Aspects of Functional Analysis: Israel Seminar (GAFA) 2014–2016*, pages 163–179. Springer, 2017.
- [70] C. Heiß, I. Gühring, and M. Eigel. Multilevel CNNs for parametric PDEs. *Journal of Machine Learning Research*, 24(373):1–42, 2023.
- [71] A. Hill and G. Dalle. Sparser, better, faster, stronger: Sparsity detection for efficient automatic differentiation. *Transactions on Machine Learning Research*, 2025.
- [72] D. P. Huttenlocher, G. A. Klanderman, and W. J. Rucklidge. Comparing images using the Hausdorff distance. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, 15(9):850–863, 1993.
- [73] D. P. Huttenlocher and W. J. Rucklidge. A multi-resolution technique for comparing images using the Hausdorff distance. In *Computer Vision and Pattern Recognition, 1993. Proceedings CVPR '93., 1993 IEEE Computer Society Conference on*, pages 705–706, 1993.
- [74] P. Jain, A. Tewari, and P. Kar. On iterative hard thresholding methods for high-dimensional m-estimation. *Advances in neural information processing systems*, 27, 2014.
- [75] O. James. On the RIP of real and complex Gaussian sensing matrices via RIV framework in sparse signal recovery analysis. In *2015 2nd International Conference on Electronics and Communication Systems (ICECS)*, pages 1706–1710, 2015.
- [76] J. Jo. Iterative hard thresholding for weighted sparse approximation. arXiv preprint arXiv:1312.3582, 2018.
- [77] D. Jones, P.-D. Letourneau, M. J. Morse, and M. H. Langston. A sparse fast Chebyshev transform for high-dimensional approximation. *arXiv preprint arXiv:2309.14584*, 2023.
- [78] D. R. Jones, C. D. Perttunen, and B. E. Stuckman. Lipschitzian optimization without the Lipschitz constant. *Journal of Optimization Theory and Applications*, 79(1):157–181, October 1993.
- [79] J. R. Jordan, P. R. Holland, D. Goldberg, K. Snow, R. Arthern, J.-M. Campin, P. Heimbach, and A. Jenkins. Ocean-forced ice-shelf thinning in a synchronously coupled ice-ocean model. *Journal of Geophysical Research: Oceans*, 123(2):864–882, 2018.

- [80] F. Y. Kuo, C. Schwab, and I. H. Sloan. Quasi-Monte Carlo finite element methods for a class of elliptic partial differential equations with random coefficients. *SIAM Journal on Numerical Analysis*, 50(6):3351–3374, 2012.
- [81] E. Leiss-Holzinger, U. Cakmak, B. Heise, J. Bouchot, E. Klement, M. Leitner, D. Stifter, and Z. Major. Evaluation of structural change and local strain distribution in polymers comparatively imaged by FFSA and OCT techniques. *Express Polym. Lett*, 6(3):249–256, 2012.
- [82] T. Lindeberg. Scale-space theory: A basic tool for analyzing structures at different scales. *Journal of Applied Statistics*, 21:225–270, 1994.
- [83] R. A. Lizarraga-Morales, R. E. Sanchez-Yanez, and V. Ayala-Ramirez. Homogeneity cues for texel size estimation of periodic and near-periodic textures. In *Mexican Conference on Pattern Recognition*, pages 220–229. Springer, 2011.
- [84] A. Logg, K.-A. Mardal, G. N. Wells, et al. *Automated Solution of Differential Equations by the Finite Element Method*. Springer, 2012.
- [85] D. G. Lowe. Distinctive image features from scale-invariant keypoints. *International journal of computer vision*, 60:91–110, 2004.
- [86] E. Lughofer, J.-L. Bouchot, and A. Shaker. On-line elimination of local redundancies in evolving fuzzy systems. *Evolving systems*, 2(3):165–187, 2011.
- [87] Q. Luo, M. Li, L. Zhao, and X. Li. StreamBP: Memory-efficient exact backpropagation for long sequence training of LLMs. *arXiv preprint arXiv:2506.03077*, 2025.
- [88] J. Marshall, A. Adcroft, C. Hill, L. Perelman, and C. Heisey. A finite-volume, incompressible Navier Stokes model for studies of the ocean on parallel computers. *J. Geophys. Res. Oceans*, 102:5753–5766, 1997.
- [89] C. Mason, E. Afshinnikoo, S. Ahsannudin, E. Ghedin, T. Read, C. Fraser, J. Dudley, M. Hernandez, C. Bowler, G. Stolovitzky, A. Chernonetz, A. Gray, A. Darling, B. Catherine, P. P. Labaj, A. Graf, H. Noushmehr, S. Moraes, E. Dias-Neto, J. Ugalde, Y. Guo, Y. Zhou, Z. Xie, D. Zheng, H. Zhou, L. Shi, S. Zhu, A. Tang, T. Ivanković, R. Siam, N. Rascovan, H. Richard, I. Lafontaine, C. Baron, N. Nedunuri, B. Prithiviraj, S. Hyat, S. Mehr, K. Banihashemi, N. Segata, H. Suzuki, C. M. Alpuche Aranda, J. Martinez, A. Christopher Dada, O. Osuolale, F. Oguntinyinbo, M. Dybwad, M. Oliveira, A. Fernandes, A. D. Chatziefthimiou, S. Chaker, D. Alexeev, D. Chuvelev, A. Kurilshikov, S. Schuster, G. H. Siwo, S. Jang, S. C. Seo, S. H. Hwang, S. Ossowski, D. Bezdan, K. Udekwu, P. O. Lungjdahl, O. Nikolayeva, U. Sezerman, F. Kelly, S. Mestrury, E. Elhaik, G. Gonnet, L. Schriml, E. Mongodin, C. Huttenhower, J. Gilbert, M. Hernandez, E. Vayndorf, M. Blaser, E. Schadt, J. Eisen, C. Beitel, D. Hirschberg, L. Schriml, and E. Mongodin. The metagenomics and metadesign of the subways and urban biomes (MetaSUB) international consortium inaugural meeting report. *Microbiome*, 4(1):24, 2016.

- [90] B. Moser, G. Stübl, and J.-L. Bouchot. On a non-monotonicity effect of similarity measures. In *Similarity-Based Pattern Recognition*, pages 46–60. Springer, 2011.
- [91] B. A. Moser. A similarity measure for image and volumetric data based on Hermann Weyl’s discrepancy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(11):2321–2329, 2011.
- [92] B. A. Moser. Similarity recovery from threshold-based sampling under general conditions. *IEEE Transactions on Signal Processing*, 65(17):4645–4654, 2017.
- [93] B. A. Moser and M. Lunglmayr. On quasi-isometry of threshold-based sampling. *IEEE Transactions on Signal Processing*, 67(14):3832–3841, 2019.
- [94] B. A. Moser and M. Lunglmayr. Spiking neural networks in the Alexiewicz topology: A new perspective on analysis and error bounds. *Neurocomputing*, page 128190, 2024.
- [95] U. Naumann. Call tree reversal is NP-complete. In *Advances in automatic differentiation*, pages 13–22. Springer, 2008.
- [96] U. Naumann. Optimal jacobian accumulation is NP-complete. *Mathematical Programming*, 112(2):427–441, 2008.
- [97] U. Naumann. DAG reversal is NP-complete. *Journal of Discrete Algorithms*, 7(4):402–410, 2009.
- [98] U. Naumann. *The art of differentiating computer programs: an introduction to algorithmic differentiation*. SIAM, 2011.
- [99] E. Ollila, H.-J. Kim, and V. Koivunen. Robust iterative hard thresholding for compressed sensing. In *2014 6th International Symposium on Communications, Control and Signal Processing (ISCCSP)*, pages 226–229. IEEE, 2014.
- [100] T. Pock, M. Pock, and H. Bischof. Algorithmic differentiation: Application to variational problems in computer vision. *IEEE Trans. Pattern Anal. Mach. Intell.*, 29(7):1180–1193, July 2007.
- [101] J. Rahnema and E. Hüllermeier. Learning Tversky similarity. In *Information Processing and Management of Uncertainty in Knowledge-Based Systems: 18th International Conference, IPMU 2020, Lisbon, Portugal, June 15–19, 2020, Proceedings, Part II 18*, pages 269–280. Springer, 2020.
- [102] H. Rauhut. On the impossibility of uniform sparse reconstruction using greedy methods. *Sampling Theory in Signal and Image Processing*, 7:197–215, 2008.
- [103] H. Rauhut and C. Schwab. Compressive sensing Petrov-Galerkin approximation of high-dimensional parametric operator equations. *Mathematics of Computation*, 86(304):661–700, 2017.
- [104] H. Rauhut and R. Ward. Interpolation via weighted ℓ_1 minimization. *Applied and Computational Harmonic Analysis*, 40(2):321–351, 2016.

- [105] H. Schulz-Mirbach. Ein Referenzdatensatz zur Evaluierung von Sichtprüfungsverfahren für Textiloberflächen. *TU Hamburg: Hamburg, Germany*, 1996.
- [106] C. Schwab. Quasi-Monte Carlo Galerkin discretization of parametric operator equations. In *Monte Carlo and Quasi-Monte Carlo Methods 2012*, pages 613–629. Springer, 2013.
- [107] B. O. Shubert. A sequential method seeking the global maximum of a function. *SIAM Journal on Numerical Analysis*, 9(3):379–388, 1972.
- [108] E. I. Slușanschi and V. Dumitrel. ADiJaC—Automatic differentiation of Java classfiles. *ACM Transactions on Mathematical Software (TOMS)*, 43(2):1–33, 2016.
- [109] F. Srajer, Z. Kukulova, and A. Fitzgibbon. A benchmark of selected algorithmic differentiation tools on some problems in computer vision and machine learning. *Optimization Methods and Software*, 33(4-6):889–906, 2018.
- [110] D. Stifter, E. Leiss-Holzinger, B. Heise, J.-L. Bouchot, Z. Major, M. Pircher, E. Götzinger, B. Baumann, and C. K. Hitzenberger. Spectral domain polarization sensitive optical coherence tomography at 1.55 μm : Novel developments and applications for dynamic studies in materials science. In *Optical Coherence Tomography and Coherence Domain Optical Methods in Biomedicine XV*, volume 7889, pages 143–150. SPIE, 2011.
- [111] G. Stübl, J.-L. Bouchot, P. Haslinger, and B. Moser. Discrepancy norm as fitness function for defect detection on regularly textured surfaces. In A. Pinz, T. Pock, H. Bischof, and F. Leberl, editors, *Pattern Recognition*, pages 428–437, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [112] P. Stumm and A. Walther. Multistage approaches for optimal offline checkpointing. *SIAM Journal on Scientific Computing*, 31(3):1946–1967, 2009.
- [113] J. Tanner and K. Wei. Normalized iterative hard thresholding for matrix completion. *SIAM Journal on Scientific Computing*, 35(5):S104–S125, 2013.
- [114] R. Tibshirani. Regression shrinkage and selection via the LASSO. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 58(1):267–288, 1996.
- [115] L. N. Trefethen. *Approximation theory and approximation practice, extended edition*. SIAM, 2019.
- [116] A. Tversky. Features of similarity. *Psychological review*, 84(4):327, 1977.
- [117] A. Tversky and I. Gati. Similarity, separability, and the triangle inequality. *Psychological review*, 89(2):123, 1982.
- [118] V. Vassilev, M. Vassilev, A. Penev, L. Moneta, and V. Ilieva. Clad – Automatic Differentiation Using Clang and LLVM. *Journal of Physics: Conference Series*, 608(1):012055, may 2015.

- [119] Q. Wang, P. Moin, and G. Iaccarino. Minimal repetition dynamic checkpointing algorithm for unsteady adjoint calculation. *SIAM Journal on Scientific Computing*, 31(4):2549–2567, 2009.
- [120] Z. Wang, A. Bovik, H. Sheikh, and E. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.
- [121] A. Werzi, S. Dorrer, B. A. Moser, and M. Lunglmayr. A hardware architecture for efficient adaptive threshold-based sampling using Weyl’s discrepancy. In *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5. IEEE, 2025.
- [122] H. Weyl. Über die Gleichverteilung von Zahlen mod. Eins. *Mathematische Annalen*, 77:313–352, 1916.
- [123] J. Xu. Iterative methods by space decomposition and subspace correction. *SIAM Rev.*, 34(4):581–613, 1992.
- [124] K. Xu, J. Wang, and B. Shim. The RIP for random matrices with complex Gaussian entries. In *Future Information Technology: FutureTech 2013*, pages 13–19. Springer, 2014.
- [125] X.-T. Yuan, B. Liu, L. Wang, Q. Liu, and D. N. Metaxas. Dual iterative hard thresholding. *Journal of Machine Learning Research*, 21(152):1–50, 2020.

Collected papers

I note in passing that some of the papers I (co-)authored aren't mentioned in this manuscript – besides a brief introduction in the introduction. This chapter aims at listing all my contributions together with a brief description of their content. The remaining part of this manuscript will simply concatenate 6 of my most representative papers published after defending my Ph.D. (they are highlighted in the list below with a little *). The collection below is ordered from most recent to oldest and categorized by types. All contributions have been peer reviewed.

- Journal papers

- J1* Roza Aceska, Jean-Luc Bouchot, and Shidong Li, *Local sparsity and recovery of fusion frames structured signals* Signal Processing, 174:107615, 2020. Description of a distributed sensing and reconstruction approach via fusion frames, found in section 3.1.2.
- J2 Bubacarr Bah and Jean-Luc Bouchot, *Editorial: Recent Developments in Signal Approximation and Reconstruction*, Frontiers in Applied Mathematics and Statistics, 17 March 2020. An editorial contribution for an edited special issue regarding the novel developments in the area of signal approximation and reconstruction. The whole edited volume can be found here: <https://www.frontiersin.org/research-topics/7545/recent-developments-in-signal-approximation-and-reconstruction>
- J3 Jean-Luc Bouchot and Keaton Hamm, *Stability and robustness of RBF Interpolation*, Sampling Theory in Signal and Image Processing, 16,37–53, 2017. Some stability estimations of radial basis function interpolation, with numerical validations.
- J4* Jean-Luc Bouchot, Simon Foucart, and Paweł Hitczenko, *Hard thresholding pursuit algorithms: number of iterations*, Applied and Computational Harmonic Analysis, 41/2, 412–435, 2016. Development and analysis of numerical convergence of a new greedy/iterative algorithm for sparse approximation, this is detailed in section 3.2.
- J5* Jean-Luc Bouchot and Frank Bauer, *Discrepancy norm: Approximation and variations*, Journal of Computational and Applied Mathematics, 272, 162–179,2014. Differentiation and approximation of the discrepancy norm. This is developped in parts of section 2.2.

- J6* Jean-Luc Bouchot and Frédéric Morain-Nicolier, *Scaled-distance-transforms and monotonicity of autocorrelations*, IEEE Signal Processing Letters, 21/10, 1235–1239, 2014. Construction and proof of a novel similarity measure for matrices enjoying the monotonicity property. Check 2.4 for details.
- J7 Elisabeth Leiss-Holzinger, Umut Cakmak, Bettina Heise, Jean-Luc Bouchot, E. Peter Klement, Michael Leitner, David Stifter, and Zoltan Major, *Evaluation of structural change and local strain distribution in polymers comparatively imaged by FFSA and OCT techniques*, Express Polymer Letters, 6/3, 249–256, 2012. Development of image similarity techniques for subsurfacic interferometric images used in polymer material and comparison with physical models.
- J8 Edwin Lughofer, Jean-Luc Bouchot, and Ammar Shaker, *On-Line Elimination of Local Redundancies in Evolving Fuzzy Systems*, Evolving Systems, 2/3, 165–187, 2011. Fusion and fission in evolving systems by means of novel similarity measures.
- Conference proceedings
 - C1* Laurent Hascoët, Jean-Luc Bouchot, Shreyas Sunil Gaikwad, Sri Hari Krishna Narayanan, and Jan Hüchelheim, *Profiling checkpointing schedules in adjoint ST-AD*, XProceedings of the 8th International Conference on Algorithmic Differentiation (AD), 2024. Development of a profiling tool used for helping end user decide on checkpointing strategies in AD based problems, more infos can be found in section 4.
 - C2 Roza Aceska, Jean-Luc Bouchot, and Shidong Li, *Fusion Frames and Distributed Sparsity*, Contemporary Mathematics, 2017. Similar to [J1], with a bit more maths and less application.
 - C3* Jean-Luc Bouchot, Benjamin Bykowski, Holger Rauhut, and Christoph Schwab, *Compressed Sensing Petrov-Galerkin Approximations for Parametric PDEs*, 2015 International Conference on Sampling Theory and Applications (SampTA), Washington, DC, 528–532, 2015. Details the single level CSPG approach with a heavy implementation and numerical testing component (as detailed in section 3.3). The multi level approach detailed afterwards is still under development and does not appear in this list.
 - C4 Jean-Luc Bouchot and Lei Cao, *Numerical solution of underdetermined systems from partial linear circulant measurements*, 2015 International Conference on Sampling Theory and Applications (SampTA), Washington, DC, 264–268, 2015. Gives an approach for solving some sparse approximation problems from partial circulant matrices via completion of the matrix first.
 - C5 Jean-Luc Bouchot, *A generalized class of hard thresholding algorithms for sparse signal recovery*, Approximation Theory XIV: San Antonio 2013, 45–63, 2014. Introduces the f HTP family of algorithms
 - C6 Swanhild Bernstein, Jean-Luc Bouchot, Martin Reinhardt, and Bettina Heise, *Generalized analytic signals in image processing: Comparison, theory and applications*, Quaternion and Clifford Fourier Transforms and Wavelets, 221–246, 2013.

Review article on the use of complex representations (namely monogenic images) for image processing and analysis.

- C7 Gernot Stübl, Jean-Luc Bouchot, Peter Haslinger, and Bernhard Moser, *Discrepancy norm as fitness for defect detection on regularly textured surfaces*, DAGM/OAGM 2012: Pattern Recognition, 428–437, 2012. Numerical improvements on the method for defect detections, as described in section 2.2.
- C8 David Stifter, Elisabeth Leiss-Holzinger, Bettina Heise, Jean-Luc Bouchot, Zoltan Major, Michael Pircher, Erich Götzinger, Bernhard Baumann, Christoph K. Hitzenberger, *Spectral domain polarization sensitive optical coherence tomography at 1.55 μm : novel developments and applications for dynamic studies in materials science*, Optical Coherence Tomography and Coherence Domain Optical Methods in Biomedicine XV, Proceedings of SPIE Vol. 7889 (SPIE, Bellingham, WA 2011) 78890Z, 2011. First introduction of some similarity measure based processing for OCT interferometric images for stress testing polymer materials.
- C9 Jean-Luc Bouchot, Gernot Stübl, and Bernhard Moser, *A template matching approach based on the discrepancy norm for defect detection on regularly textured surfaces*, Proceedings Volume 8000, Tenth International Conference on Quality Control by Artificial Vision; 80000K, 2011. First introduction of the golden-template approach for defect detection, as introduced in section 2.2.
- C10 Bernhard Moser, Gernot Stübl, and Jean-Luc Bouchot, *On a Non-Monotonicity Effect of Similarity Measures*, Similarity-Based Pattern Recognition (SIMBAD 2011), 46-60, Springer, 2011. Counter examples detailed in section 2.3 for similarity measures satisfying the monotonicity of their autocorrelations.
- C11 Jean-Luc Bouchot, Johannes Himmelbauer, and Bernhard Moser, *On autocorrelation based on Hermann Weyl’s discrepancy norm for time series analysis*, The 2010 International Joint Conference on Neural Networks (IJCNN), Barcelona, 1–7, 2010. First introduction of the discrepancy norm’s p norm approximation.
- Book chapters
 - B1 Jean-Luc Bouchot, Will Trimble, Gregory Ditzler, Yemin Lan, Steve Essinger, and Gail Rosen, *Advances in machine learning for processing and comparison of metagenomic data*, Computational Systems Biology (Second Edition), Academic Press, 295–329, 2014. Thorough review of machine learning techniques in the world of metagenomics, from mathematical theories to practical examples via information theory.
 - B2 Gregory Ditzler, Yemin Lan, Jean-Luc Bouchot, and Gail Rosen, *Variable selection to improve classification of metagenomes*, Encyclopedia of metagenomics, 2013. Introduction and numerical evaluation of a forward selection algorithm with reduction of redundancies from information theoretic metrics for metagenomics.
- Misc:
 - M1 Maxime Chammas *et al.*, *PEPS5 study on the interest of energy storage and of power-to-X-Final report+ Appendix to the PEPS5 study: report on the second*

life of batteries+ Technological sheets: Technical-economic information related to technologies of electric, thermal, hydrogen, power-to-heat and power-to-gas storage, Association Technique Energie Environnement - ATEE, 2023. A publication for policy makers using a completely novel software architecture I developed for launching HPC applications on many compute platforms.

As a big believer in open science, I have also released accompanying software for most of my work. Besides the basic proof of concepts available from my github page, I would like to mention bigger projects:

- S1 **MLCSPG**: Started as a proof of concept for the **SLCSPG**, it evolved into a more complex piece of software for the multi level case, including some unit testing and easy (ish) installation processes.
- S2 **Tapenade**: A tool for source transform AD of **C** and **Fortran** (and hopefully soon enough **Julia**). While I am not the instigator of this software, I am now actively maintaining it and adding new features.
- S3+4 As part of Artelys, I have also briefly contributed to some open source software from RTE's R & D department: **dynawo** and **PowSyBl**. Both were interesting projects and taught me a lot on power systems and power networks.

Profiling checkpointing schedules in adjoint ST-AD

Laurent Hascoët *

Jean-Luc Bouchot *

Shreyas Sunil Gaikwad †

Sri Hari Krishna Narayanan ‡

Jan Hückelheim ‡

Abstract

Checkpointing is a cornerstone of data-flow reversal in adjoint algorithmic differentiation. Checkpointing is a storage/recomputation trade-off that can be applied at different levels, one of which being the call tree. We are looking for good placements of checkpoints onto the call tree of a given application, to reduce run time and memory footprint of its adjoint. There is no known optimal solution to this problem other than a combinatorial search on all placements. We propose a heuristics based on run-time profiling of the adjoint code. We describe implementation of this profiling tool in an existing source-transformation AD tool. We demonstrate the interest of this approach on test cases taken from the MITgcm ocean and atmospheric global circulation model. We discuss the limitations of our approach and propose directions to lift them.

1 Introduction

Source-transformation algorithmic differentiation (ST-AD) in its adjoint mode transforms a *primal* code that evaluates some original function into an *adjoint* code that computes its gradient. It is well known [9] that the most efficient implementation of the adjoint code must progress backwards of the original computation, progressively using values originating from the primal execution. The amount of values used grows linearly with the run time of the primal code and, since they are used in the reverse of their production order, their management (*data-flow reversal*) is a key issue that requires a delicate trade-off between storage and recomputation.

This work focuses on one particular setting, where data-flow reversal is primarily done through a stack and the memory cost of this stack is mitigated through a classical storage/recomputation trade-off known as *checkpointing*. Tuning this trade-off is difficult, and optimal tuning exists only in specific cases [8]. In our setting, checkpointing can be applied at every procedure call. When the granularity of procedure calls is not fine enough, checkpointing can also be applied at user-

designated procedure fragments, provided they *could* be turned into procedures. Potential checkpointing locations therefore end up being the nodes of the call graph, possibly extended to procedure fragments, and at run time they are nested as the nodes of the call tree. In this work, we examine the question of finding a subset of the checkpointing locations that will result in good enough execution time given a limited storage budget. Optimality, although desirable, has already been shown to be NP-hard [15]. We will rather explore heuristics, based on profiling results, to decide which checkpoint locations should be activated or inhibited to achieve a better performance.

In section 2, we describe in more detail the principle of checkpointing in the setting of our stack-based data-flow reversal, and illustrate its costs and benefits. Section 3 contrasts with checkpointing in other settings and relates this work with other profiling support in existing AD tools. Section 4 describes the information appropriate to guide the choice of activated checkpoints, and how to gather it from a profiled run. Section 5 discusses implementation of this profiling in an existing source-transformation AD tool, and section 6 applies it to two realistic test-cases taken from the MITgcm code suite. We will show how the developer can achieve a significant performance gain by exploiting the profiling results. In section 7, we come back to some limitations of our proposed approach and discuss how they could be overcome, before concluding in section 8.

2 Our checkpointing model / setting

In our setting, data-flow reversal is achieved by storing intermediate values of the primal execution. Consequently, the adjoint code basically obeys a two-sweeps structure:

1. a first, “*forward*”, sweep runs the primal code, augmented to organize the storage of whatever intermediate values or derivatives will be needed in the second sweep.
2. a second, “*backward*”, sweep propagates the gradients in reverse order, retrieving the intermediate values from the first sweep when needed.

*INRIA, Sophia-Antipolis, France

†Oden Institute for Computational Engineering and Sciences, The University of Texas at Austin, USA

‡Argonne National Laboratory, Lemont, IL, USA

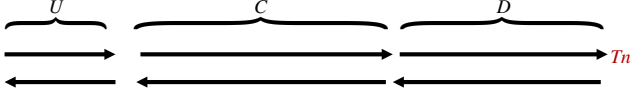


Figure 1: Sketch of execution for the adjoint of a code arbitrarily split as a sequence of three parts U , C , and D . Thick arrows to the right stand for forward sweeps, running the primal code together with storing intermediate values on the stack. Thick arrows to the left stand for backward sweeps, that retrieve values from the stack and use them while propagating gradients backwards. Time goes top-down, and inside each line time follows the arrows direction.

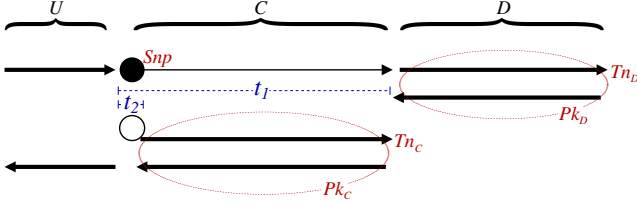


Figure 2: Sketch of execution for the adjoint of the same code than in Fig. 1, this time checkpointing code part C . Time goes top-down, and inside each line time follows the arrows direction. The thin arrow to the right stands for the extra execution of the primal C , and the black and white bullets stand for respectively the writing and reading of the *snapshot*, i.e. a set of variables sufficient for identical duplicate execution of C .

Naturally, the appropriate storage structure for that is a stack. Note that the intermediate values that are not used in the derivative computation, such as those appearing only in linear computations, need not be stored. This can reduce the stack size significantly. AD tools implement several kinds of data-flow analysis to detect many opportunities to reduce the stack usage. Still, stack size generally grows linearly with run-time and the checkpointing technique, which trades recomputation for peak stack size, is unavoidable.

Figure 1 sketches execution of the adjoint code, in the base case where no checkpointing is active. Code fragment C , a candidate for future checkpointing, is singled out from upstream and downstream code U and D , only for future comparison with the case where checkpointing is applied to C . At the turn point between the forward and backward sweeps, the stack reaches its maximum size Tn .

Checkpointing reduces the peak stack size by allowing for one recomputation of a chosen fragment of the primal code, here C as shown in Fig. 2. At the cost of this duplicate execution of primal fragment C ,

which does not write values to the stack, the stack size reaches two local peaks Tn_D and Tn_C , both hopefully smaller than Tn . On the one hand, Tn_D contains intermediate values from U , D , but not from C , plus a so-called “*snapshot*” needed to run C again and which we hope is smaller than the intermediate values in C . On the other hand, Tn_C only contains intermediate values from U and C . Notice that Tn_C does not contain the snapshot, which has been read to prepare for the forward sweep of C and immediately deleted. Symbols Pk_D , Pk_C , and Snp in Fig. 2 denote stack sizes that will be discussed in section 4. It is worth observing on Fig. 2 that the reading and writing of the snapshot blend nicely with the general stack behavior, which allows us to store the snapshot and the other intermediate values onto the same stack. Whatever efficient implementation, or cache level optimization, exists for the stack will be equally beneficial for snapshots. Also notice that *some* values in the snapshot may have been already stored in U , not too deep in the stack. Although some data-flow analysis can detect this and therefore slightly reduce either the snapshot or the stack after U , our experiments have shown that the benefit is very often minor. As discussed in section 7, this sort of extra improvement, implemented in the AD tool we are using here, can be the cause for inaccuracy in the profiling predictions.

Notice that on the figures, the position and length of the arrows do not represent the stack size, but only their correspondence with the primal code. In other words Tn_C may well be larger than Tn_D .

On large codes, checkpointing can and must be applied repeatedly, recursively inside fragments U , C , and D . With a well-balanced choice of the nested checkpoints, i.e. the choice of C and of all other checkpoints nested inside U , C , and D , one can achieve logarithmic growth – with respect to the primal run time – of both the peak stack size and of the number of times a given primal code fragment is repeatedly run. Notice however that the control structure of the primal code restricts the choice of the nested checkpoints. Basically since C will be re-executed, it must have unique entry and exit points. It must also be “*reentrant*”, i.e. contains no adverse side-effect such as an isolated `send` or `receive`, `malloc` or `free`, file I-O, etc. Obvious checkpointing candidates C ’s are all the (reentrant) procedure calls, but one may also define at will additional checkpoints inside procedure bodies, essentially when these additional checkpoints *could* be turned into reentrant procedure calls. In this work, we will assume that one has defined a large enough collection of nested potential checkpoints, and our goal is to use profiling to selectively inhibit or activate these checkpoints to reach

good performance in run time and in peak stack size.

At one extreme, activating all the potential checkpoints (i.e. actually performing checkpointing on them) will severely affect run time due to primal code recomputations. Conversely, inhibiting all potential checkpoints can easily lead to excessive stack memory use. One can look for an optimal trade off for instance: for a given fixed stack memory budget, look for the checkpoints activation that minimizes primal code recomputation time. Previous works [15] have shown that a general answer to this combinatorial problem is NP-hard, possibly resorting to Integer Programming approaches [13]. We will here explore a greedy approach, progressively improving our subset of activated checkpoints. Still, the information returned by profiling could also be fed to optimal search tools.

The combined questions we consider are: what information can we extract from run-time profiling of the adjoint code, and how can we use it to guide selection of potential checkpoints towards a good enough trade off?

3 Related approaches

In the so-called “*recompute-all*” model, primal values are made available by repeatedly running the primal code from a chosen initial point. This approach trades computation time for memory space. The alternative “*store-all*” model stores the needed primal values on a stack, sparing the extra recomputation time at the cost of storage space. On realistic codes that run for long times, both models require a similar notion of checkpointing, and the question of the optimal choice of checkpoints is the same in both models. The tool TAF [5] (“*recompute-all*” model) allows one to fine-tune the checkpointing locations and the variables they must store, through user directives. This manual tuning is (by principle) more powerful than any automated approach, but is labor-intensive. It could as well benefit from some profiling.

In some “*store-all*” models, the values stored on the stack are the intermediate values of variables computed by the primal code, or at least the subset of those that are indeed used in the partial derivatives accumulated during gradient computation. In other models [16] they can be the partial derivatives directly. When storing intermediate values, one can choose to store them upon each primal instruction whose adjoint uses the value. This “*store-on-use*” has the risk of storing the same value several times. The alternative “*store-on-kill*”, which we follow in the sequel, is to store the value only when it is going to be overwritten by the next primal instruction.

The present work considers checkpointing essen-

tially at the level of the call tree. In the restricted setting of a time-stepping loop, several checkpointing schedules have been studied. In particular the well-known *binomial checkpointing* [8] is optimal, in its application case. These checkpointing strategies on loops differ from ours as they amount to choosing the extent of the nested checkpoints over the sequences of time steps, whereas our setting operates on a collection of fixed potential checkpoints on which we search a good subset. Given the nested time-step sequences that binomial checkpointing has found optimal, inhibiting checkpointing on any such sequence can only lead to a less efficient scheme. Conversely we do not consider in our setting the option to enlarge or shrink the code fragments C to which checkpointing may be applied. Binomial checkpointing, or its refinements dedicated to very large snapshots such as H-revolve [11], are independent from the present checkpointing on call trees. We will see in section 6.2 how they may complement each other.

Profiling is one answer to improve performance when no static analysis can guarantee optimality at compile time. Machine learning frameworks such as PyTorch, TensorFlow, and JAX offer automatic differentiation capability as well as profiling. In JAX, a common use of profiling is to understand the GPU or TPU memory usage of a program, for example if trying to debug an out-of-memory problem [1]. A profiler trace can be created by instrumenting the code with the `jax.profiler.start_trace()` and `jax.profiler.stop_trace()` methods. The trace can be viewed later using a graphical interface. As opposed to the approach presented here, the profiling information is not used to select a differentiation strategy.

4 A profiling model for checkpointing

We will distinguish static and dynamic checkpoint locations. We call static a location in the source code, and more precisely in the primal source code. This is where the end-user may conveniently decide to (de-)activate a checkpoint e.g. through a differentiation directive. Dynamic checkpoints are the run-time occurrences when the execution pointer reaches a static checkpoint. There can be several dynamic checkpoints for each static one. It is in principle possible to apply different checkpointing choices to each dynamic occurrence, but this is impractical and leads to a more involved implementation. We therefore choose to attach the profiling information that we seek to each static checkpoint, although it will be collected and accumulated at run-time on each of its dynamic occurrences.

As a starting configuration that we will profile and progressively improve upon, we will activate all potential checkpoints. One reason is that this configuration

gives profiling an easier access to useful information. The other reason is that it is less likely to fail at run-time by lack of memory, although it may take longer. It is therefore a safer starting point when the structure of the primal code is not well known. As the improvement process goes on, the checkpointing configuration is modified according to profiling results, and a new profiling run is in theory needed for each configuration in order to determine the next configuration.

For some current configuration, for every potential checkpoint X which is currently activated, we expect profiling to give us the following key figures about inhibiting X :

- the associated run-time benefit $\Delta t(X)$, i.e. the run-time difference when X is switched from activated to inhibited. It is always negative (run-time decreases).
- the associated memory cost $\Delta Pk(X)$, i.e. the peak stack size difference when X is switched from activated to inhibited. It is in general positive except for some extreme cases.

Let us look back at Fig. 2, now assuming we are in the general case where there can be nested checkpoints in the adjoint codes for C and D (and U). Just view C and D as sections of the primal code, in which there may be any number of checkpointed sections X , one of them coinciding with C . Let us call *round trip* the sequence of a forward sweep and its associated backward sweep. For instance the rightmost part of Fig. 2 is the round trip on D , also written $\overrightarrow{D}$, the sequence of its forward sweep $\overrightarrow{D}$ and backward sweep $\overleftarrow{D}$. The bottom two arrows of the central part of Fig. 2 form the round trip on C . A round trip can include nested checkpoints and for instance the round trip on $U; C; D$ is exactly Fig. 1 (if C is inhibited) or Fig. 2 (if C is activated).

Since we now assume that there may be nested checkpoints inside D , we must distinguish peak stack sizes Tn_D and Pk_D : as defined already, Tn_D is the (local) peak reached at the turn point at the end of $\overrightarrow{D}$, whereas Pk_D is the (global) maximum peak stack size attained during the round trip on D , i.e. at the turn point after $\overrightarrow{D}$ but also at the turn point of any checkpoint N nested in D . Nothing prevents $\overrightarrow{N}$ from storing more intermediate values than D and therefore $Pk_D \geq Tn_D$ in general. In the sequel we will need to keep track of both Tn_D and Pk_D . The same remark applies to C .

Profiling is performed on the adjoint code corresponding to the current checkpointing configuration. We want profiling to return, for each static checkpoint X that is currently active, the cost/benefits $\Delta t(X)$ and

$\Delta Pk(X)$. Only as an intermediate information, profiling will also return $\Delta Tn(X)$. As cost/benefits can be evaluated independently for each X , we will in the sequel drop the (X) to keep notation light.

We will accumulate cost/benefit figures bottom-up and right-to-left on the tree of nested dynamic checkpoints or in other words, bottom-up on the tree of nested round trips. Following Fig. 2, the key recursive operation is therefore to compute these figures for the round trip on the code sequence $C; D$, given these figures for the round trips on C and on D . Notation-wise, we will specify as an index the round trip on which a cost/benefit is computed. Therefore our goal is, given $\Delta t_C, \Delta Tn_C, \Delta Pk_C$ about the round trip on C and likewise $\Delta t_D, \Delta Tn_D, \Delta Pk_D$ about the round trip on D , to compute $\Delta t_{CD}, \Delta Tn_{CD}$, and ΔPk_{CD} about the round trip on $C; D$. We keep in mind that this computation must be made for any active checkpoint X , and therefore in particular for C , but also for any other static checkpoint that may occur, any number of times, in C , in D , or in both. In addition we will also need to keep track of the basis times and stack sizes when none of the currently activated checkpoints is inhibited i.e. regarding notation, compute t_{CD}, Tn_{CD}, Pk_{CD} from $t_C, Tn_C, Pk_C, t_D, Tn_D$, and Pk_D .

For the base case of a piece of code that contains no activated checkpoint, profiling can easily return the basis times and stack sizes. Note that Pk and Tn are then equal. Δ values are all zero for all X .

For the recursive case, it is easy to verify on Fig. 2 that, about run time:

$$(4.1) \quad t_{CD} = t_1 + t_D + t_2 + t_C$$

where t_1 is the extra time to take the snapshot and run the primal C , and t_2 is the time to read the snapshot. Then about stack sizes:

$$(4.2) \quad Tn_{CD} = Tn_D$$

$$(4.3) \quad Pk_{CD} = \max(Pk_D, Pk_C)$$

For each X , cost/benefit results are the changes in the value of t_{CD} , Tn_{CD} , and Pk_{CD} , from the reference case (equations 4.1, 4.2, 4.3) with no additional checkpoint inhibited, to the case where only X is switched to inhibited. These Δ values are computed differently whether X coincides with C or not. If $X \neq C$ then the effect of inhibiting checkpoint X does not affect C and the resulting computation scheme remains as in Fig. 2. Therefore:

$$(4.4) \quad \Delta t_{CD} = \Delta t_D + \Delta t_C$$

$$(4.5) \quad \Delta Tn_{CD} = \Delta Tn_D$$

$$(4.6) \quad \Delta Pk_{CD} = \max \left(\frac{Pk_D + \Delta Pk_D}{Pk_C + \Delta Pk_C} \right) - Pk_{CD}$$

If on the other hand $X = C$, execution when inhibiting X will follow the scheme of Fig. 1. There is no duplicate execution of the primal C so that time benefit becomes:

$$(4.7) \quad \Delta t_{CD} = \Delta t_D + \Delta t_C - t_1 - t_2$$

The stack space used by $\vec{C}$ now accumulates with that used by the round trip $\overleftarrow{D}$. Stack size after $\vec{C}$ in Fig. 1 is indeed $Tn_C + \Delta Tn_C$. On top of it is accumulated the stack space specifically used by $\vec{D}$ and $\overleftarrow{D}$ which are respectively $Tn_D + \Delta Tn_D - Snp$ and $Pk_D + \Delta Pk_D - Snp$, where Snp is the stack size measured just after taking the snapshot.

$$(4.8) \quad \Delta Tn_{CD} = Tn_C + \Delta Tn_C + \Delta Tn_D - Snp$$

$$(4.9) \quad \Delta Pk_{CD} = \max \left(\frac{Tn_C + \Delta Tn_C + Pk_D + \Delta Pk_D - Snp}{Pk_C + \Delta Pk_C} \right) - Pk_{CD}$$

All the times and stack sizes needed can be measured while running with C activated. Note that ΔTn_{CD} and ΔPk_{CD} can be negative in the extreme case where Tn_C is less than SNP , or in other words when the snapshot for checkpointing C is larger than the collection of intermediate values stored during $\vec{C}$. In this extreme case, checkpointing C is never beneficial.

5 Implementation

We implemented our profiling tool inside the source-transformation AD tool Tapenade [10].

5.1 Installing callbacks Section 4 tells us which elementary data must be collected from the profiled run in order to compute the estimated cost/benefits. These are, for each active checkpoint:

- the run times t_1 and t_2 .
- the stack size Snp immediately after taking the snapshot.

and in addition every local peak stack size Tn , measured at each turn point. Collecting this data is implemented

$< \text{code for } \vec{U} >$ <code>CALL adProfileAdj_SNPWrite("C", 42)</code> <code>CALL PUSH(snapshot)</code> <code>CALL adProfileAdj_beginAdvance("C", 42)</code> <code>CALL C()</code> <code>CALL adProfileAdj_endAdvance("C", 42)</code> $< \text{code for } \vec{D} >$
<code>CALL adProfileAdj_turn()</code>
$< \text{code for } \overleftarrow{D} >$ <code>CALL adProfileAdj_SNPRead("C", 42)</code> <code>CALL POP(snapshot)</code> <code>CALL adProfileAdj_beginReverse("C", 42)</code> <code>CALL C()</code> <code>CALL adProfileAdj_endReverse("C", 42)</code> $< \text{code for } \overleftarrow{U} >$

Figure 3: Adjoint code with profiling callbacks inserted, corresponding to the code sketch of Fig 2. Forward sweep on top, turn point in the middle, backward sweep below. Except for `turn`, callbacks take as argument the called function name and the corresponding line number in the primal code, for later reference.

by installing a number of callbacks in the differentiated code. These callbacks are inserted automatically by Tapenade when invoked with the `-profile` command-line option. Figure 3 sketches the instrumented adjoint code produced by Tapenade, corresponding to Fig 2. The added callbacks (highlighted in red) all belong to a new `adProfileAdj` package. Calls occur in the forward sweep before writing a snapshot and before and after calling the duplicate primal C , and symmetrically in the backward sweep before reading the snapshot and before and after calling the round trip $\overleftarrow{C}$ which is $\overleftarrow{C}$; $\overleftarrow{C}$. Last but not least, a callback to `adProfileAdj_turn()` is inserted at each turn point between a forward and a backward sweep.

The profiling mechanism requires minor additional adaptations to blend well with sophisticated structures of the adjoint code such as binomial adjoint (for time-stepping loops) and two-phases adjoint (for fixed-point loops).

5.2 Callback implementation Implementation of the profiling callbacks maintains an internal stack of the activated checkpoints encountered, to retrieve correspondence from forward sweep to backward sweep, between the callbacks on a given dynamic checkpoint. Timing t_1 is deduced from measurements be-

tween `SNPWrite` and `endAdvance`, t_2 between `SNPRead` and `beginReverse`. The final computation of the cost/benefits for the round trip $C;D$ is entirely performed during `endReverse`, when all the needed values t , Tn , Pk , and all Δt , ΔTn , ΔPk are known for C and D . At the end of the profiled run, a readable summary of the cost/benefits is displayed.

Complexity of the profiling process lies in that of callback `endReverse`, that implements all equations 4.1 to 4.9, and that is called once per run-time procedure call (or encounter of a manual checkpoint). In each `endReverse`, equations 4.1, 4.2, and 4.3 need to be evaluated only once, whereas the subsequent equations need to be evaluated once per static checkpoint X . The total complexity is therefore the product of the size of the call tree times the number of static checkpoints. Memory-wise, the data stored by profiling occupies a size at most proportional to the number of static checkpoints times the depth of the call tree.

5.3 Inhibiting / activating checkpoints Bearing in mind that calls are always checkpointed by default, the functionality for inhibition of potential checkpointing locations was already present in the AD tool Tapenade. End-users could play with it but our impression is that very few did so, possibly due to the absence of a profiling tool to guide them. The user interface of Tapenade provides two ways to inhibit checkpointing on calls:

- On the Tapenade command-line, one can use option `-nocheckpoint "foo foo2"`, which will prevent checkpointing on every call to procedures `foo` and `foo2`.
- In the primal source, one can place pragma `$AD NOCHECKPOINT` before definition of a procedure `foo`, which inhibits checkpointing on every call to `foo` (equivalent to the command line option). Alternatively one can place the same pragma before any individual call to `foo`, thus selectively inhibiting this particular call.

5.4 Inhibiting checkpointing on externals Since external routines are not shown to the AD tool, Tapenade provides a way for the end-user to specify information about these externals that are useful for a better differentiation, through a special-purpose specification file. These are typically about types and read/write status of their arguments, or about the differentiable dependency of their outputs on their inputs. The end-user is also required to write the derivative code for these externals. For a non-checkpointed call to an external, the AD tool can produce a better code by knowing which of

the arguments are used in a “*non-linear*” way by the external, or in other words which of the values *returned* by the external must be, when later overwritten, preserved on the stack for future use in the backward sweep. If this information is not provided to the AD tool, the conservative information is built from the read/write status, possibly causing stack storage of more values than necessary. We thus extended the mini-language of the specification file to designate these “non-linear” arguments.

Symmetrically, the differentiated code for a non-checkpointed external `foo` consists of two procedures `foo_fwd` and `foo_bwd` (instead of a single `foo_b` in the classical checkpointed case). The end-user is required to provide their implementation. One consequence of not checkpointing is that `foo_fwd` may be required by the differentiation context to take care of preserving some of its input values, on the stack, and `foo_bwd` must restore these values from the stack at the symmetric location. This extra task for the end-user is easy enough and worth the effort. However, in cases where the user can not do so, the conservative fallback option is to let the AD tool store what is needed immediately before the call to `foo_fwd` and restore just after `foo_bwd`. This will be triggered by another option in the external specification file. Interaction with the end-user thus occurs in two phases:

1. Before differentiation, the end-user can use the specification file to provide the non-linearly used arguments of the external procedures. The user also has the option to accept that the hand-written `foo_fwd` and `foo_bwd` take care of storing and restoring some of their arguments.
2. After differentiation, the AD tool emits messages that tell the user which differentiated external procedures must be provided, specifying which derivatives (of which output with respect to which input) must be computed, and whether they must come in the `foo_b` form or in the `foo_fwd`, `foo_bwd` form. When the user has accepted to do so, messages also specify the input arguments that must be preserved in case they are overwritten in `foo_fwd`.

5.5 Asynchronous stack offloading to files Searching for a good checkpointing scheme is all the more important for large applications that need a huge amount of memory to store the stack. Until recently Tapenade kept this stack in RAM, and this has been noted as an unfortunate limitation by several users. We implemented a new stack mechanism in Tapenade, that triggers offloading of the deeper parts of the stack to

files. It is a variation on virtual memory: when the stack size grows close to a user-defined given limit, the parts of the stack that were pushed early, and therefore that will be popped late, are stored to files and the associated main memory becomes available again. The reciprocal mechanism occurs during the backward sweep. These deep parts of the stack, that are subject to this file write/read mechanism, are in general “dormant” i.e. disjoint from the stack top where all push/pop activity occurs. Therefore Tapenade offers the option of performing the file write/read operations asynchronously, using Posix `pthreads`. When a spare thread is available, this reduces the overhead of the file offloading mechanism.

For instance the application described in section 6.3 makes use of this stack offloading mechanism, with a stack peak that goes over 4 Gb. while never using more main memory than a user-defined limit of 256 Mb.

6 Application

We test our profiling capability on test cases from the Massachusetts Institute of Technology General Circulation Model (MITgcm) [14, 2]. The MITgcm is widely used by the climate science community to simulate planetary atmosphere and ocean circulations, and its dynamical core has been successfully used to also perform ice stream and ice shelf simulations, and thus develop synchronous ice-ocean coupling capabilities as well as its adjoint [12, 7].

MITgcm has always been developed to be compatible with AD tools, specifically TAF [5], enabling the generation of tangent-linear and adjoint models. It is thus a key component of the Estimating the Circulation and Climate of the Ocean (ECCO) state estimate: a data assimilation product widely employed by the physical oceanography research community. The MITgcm has recently been differentiated using Tapenade [4], which is what allows us to perform the profiling experiments discussed next.

6.1 Test streamice: The `halfpipe_streamice` test case uses the package `streamice` to simulate the flow of land ice in a valley. The simulation runs for 80 timesteps on a 40×20 horizontal grid. The individual time steps are more complex than a typical MITgcm simulation since they contain fixed point iteration based solvers to solve the non-linear stress balance. Their adjoint is built according to Christianson’s two-phases algorithm [3, 6].

Adjoint AD of `streamice` was first performed with OpenAD in 2016 [6], and a few years later with Tapenade. Both tools apply the same strategy for the adjoint of the fixed-point loop. In both cases the end-user chose not to apply binomial checkpointing to the outer

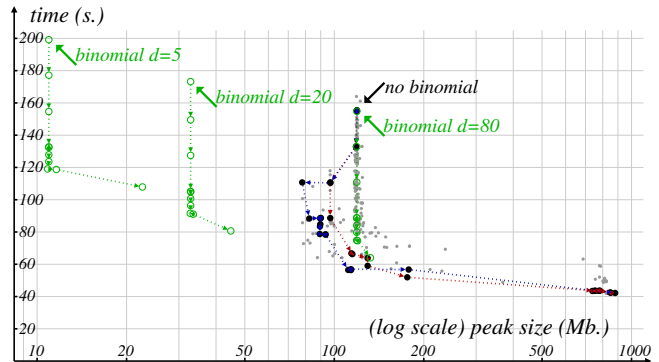


Figure 4: Tuning the adjoint of `streamice` by profiling. Each checkpointing configuration is shown as a dot whose coordinates are peak stack size and run time of the adjoint. The red and blue dotted lines show the evolution of performance of the adjoint by repeatedly following the suggestions made by our profiling tool. The red line follows a “run-time gain first” strategy, the blue line a “careful on memory” strategy. Time measurements are averaged on 5 runs. Smaller gray dots show the performance of 250 random checkpointing configurations. Green dots show combination with binomial checkpointing, discussed in section 6.2

time-stepping loop. The structure of the adjoint codes is therefore similar. Typically, run-time performance of an adjoint code is measured by the ratio with the run-time of the primal. The adjoint code by Tapenade yielded a 7.0 ratio, which is fair but not as good as with OpenAD. Likewise, on most MITgcm test cases, the ratio obtained with Tapenade is not as good as the average ratio obtained with TAF. By default, Tapenade activates checkpointing at each procedure call, whereas OpenAD doesn’t. Checkpointing in TAF is carefully hand-tuned, with methods that have not been published. Certainly, there is room for improvement from the default checkpointing scheme of Tapenade.

Fig. 4 summarizes this search for a better checkpointing configuration. The initial configuration with all checkpoints activated (indicated as “no binomial”) has an average run-time of 155.4 seconds and a peak stack size of 119.1 megabytes. The number of static checkpoint locations is 85, yielding an enormous 2^{85} number of possible configurations to explore.

We use our profiling tool to explore only the most profitable changes. Incidentally, we compare adjoint run-time with and without profiling and find a negligible 1.1 second overhead for profiling. The suggestions returned by the profiled adjoint code are sorted in 3 categories:

- Static checkpoints whose inhibition give a run-time

gain (as always), and a stack peak gain. Those are rare cases where the snapshot is larger than the amount of intermediate values stored in the forward sweep. This is generally due to overestimation of the AD data-flow analysis on arrays, where activity for a single cell of an array implies activity of the whole array. This sort of profiling result means that the checkpoint should be inhibited without hesitation.

- Static checkpoints whose inhibition give no change of the stack peak. This happens when the effect of inhibiting the checkpoint indeed modifies the size of one local peak of the stack, but this peak is smaller than the global peak and therefore the change is hidden. The checkpoint is a good candidate for inhibition, as this will gain some run time without changing the global peak size.
- Static checkpoints whose inhibition will increase the stack peak size. Decision to inhibit it results from a trade off decision, comparing with the run-time expected benefit and possibly the available budget in memory space.

As these suggestions allow for several alternative decisions, the end-user may apply different strategies. Two example strategies are presented in Fig. 4: one (red line) follows the suggestions with the highest run-time gain first, but deferring as much as possible suggestions with a high memory cost. The other (blue line) follows suggestions with the lowest memory cost first, regardless of run-time gain. For both strategies, only one or a few suggestions are selected and applied jointly, then differentiation is performed and the adjoint code is run again, yielding new suggestions. This process is repeated until no suggestion remains, which is when all 85 checkpoints are inhibited. In real life, the end user will probably stop the process before that stage, as the memory cost may go over the available memory budget. Fig. 4 shows with black dots the time and memory performance of the successive adjoint codes after each step. Gray dots show the performance of 250 randomly chosen checkpointing configurations. We observe that they are mostly situated above and right of the red and blue lines, indicating that the profiling suggestions, combined with these strategies, take performance close to the front of optimal configurations.

From the results in Fig. 4, we find a few checkpointing configurations that offer a significant time gain for a relatively small memory cost, or even sometimes a small gain. For instance one dot on the red line exhibits a stack peak of 176.0 Mb, not too much above the initial 119.1 Mb, and a time of 51.9 seconds, a significant gain

from the initial 155.4 s. resulting in a slowdown ratio of 2.4. If memory is really scarce, another dot on the blue line exhibits a stack peak of 111.41 Mb, even less than the initial 119.1 Mb, and a time of 56.49 s. and therefore a still very good ratio of 2.6. Unless memory is very abundant, it is unlikely that the end-user pushes the process to its extreme, costing 880.6 Mb, even if time becomes as short as 42.2 s.

6.2 Binomial checkpointing A natural question is the comparison and interaction between our checkpointing on the call tree and binomial checkpointing on time-stepping loops. Let us introduce binomial checkpointing into the `streamice` test case by adding directive `$AD BINOMIAL-CKP`

on the time-stepping loop. Although the test case features a limited number of time steps (80), our experiments clearly capture the effect of binomial checkpointing and its interaction with our profiling method. Binomial checkpointing is tuned by choosing the maximum number d of “before time-step” snapshots that can be stored together in memory. Figure 4 shows (in green) experiments with an extreme $d = 80$, a more reasonable $d = 20$, and a quite realistic $d = 5$. For each d , dotted arrows show the evolution of performance when following profiling suggestions (highest run-time gain first) until no suggestion remains. The $d = 80$ case is interesting mostly as it makes the link with the experiment with no binomial checkpointing: allowing d to be as high as the total number of time-steps is equivalent to discarding binomial checkpointing altogether and activating checkpointing on the time-stepping loop body, which happens here to be a single procedure call. Therefore all green dots of the $d = 80$ evolution are equivalently checkpointing configurations of the experiment with *no* binomial checkpointing (i.e. black and small gray dots), but they belong to a subset where checkpointing remains activated on the time-stepping loop body. Evolution of the green dots is therefore restricted and cannot reach performance as good as the red and blue evolutions. Notice that the red and blue evolutions almost immediately choose to inhibit checkpointing on the time-stepping loop body.

Binomial checkpointing controls memory usage through the choice of d . A smaller d reduces the memory used to store snapshots, at the cost of a higher number of duplicate execution of time-steps. The green dots for $d = 20$ and $d = 5$ demonstrate that. For all three d , our profiling tool provides profitable suggestions, decreasing run-times by a factor 1.8 to 2.4 at almost no memory cost.

We interpret the almost vertical shape of the green curves by the fact that each configuration systemati-

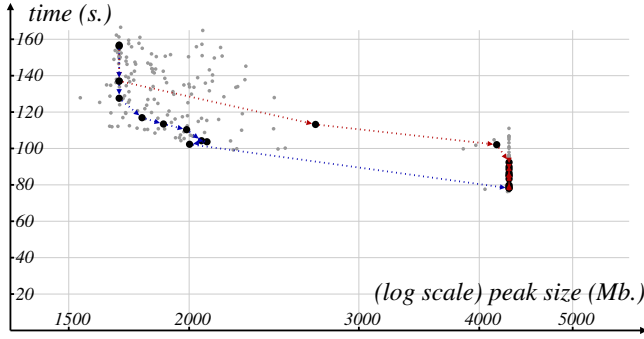


Figure 5: Tuning the adjoint of `tutorial_global_oce_biogeo` by profiling. Each checkpointing configuration is shown as a dot whose coordinates are peak stack size and run time of the adjoint. The red and blue lines show the evolution of performance guided by profiling, respectively with a “run-time gain first” strategy, and a “careful on memory” strategy. Gray dots show the performance of 250 random checkpointing configurations.

cally uses d “time-stepping” snapshots for any activation pattern of the other checkpoints. Observe that with binomial checkpointing, round trips on each time-step always occur separately. Therefore, the memory cost of inhibiting checkpoints inside a time-step is not multiplied by the number of time steps. Profiling suggestions reflect that, by finding lower memory costs (and benefits) to inhibiting checkpoints that are contained in a time-stepping loop with binomial checkpointing. As long as this cost is small compared to that of the d “time-stepping” snapshots, it remains almost invisible. It only becomes visible when almost all checkpointing locations are inhibited.

6.3 Test `tutorial_global_oce_biogeo` : The MIT-gcm test case `tutorial_global_oce_biogeo` is a global simulation on a $128 \times 64 \times 15$ grid. It also simulates the biogeochemistry aspects of the ocean, for example, the carbon uptake and how it is affected by the sea surface temperature anomalies. We simulate 10 steps so that the runtime is similar to the `halfpipe_streamice` test. There is no binomial checkpointing as a default. Figure 5 shows the improvement in performance obtained by applying the suggestions from profiling, with the same two strategies as in Fig 4.

From this experiment it becomes clear that making an informed choice of checkpointing is critical. Indeed, one can easily reduce by 35% the compute time, yet avoiding the massive memory consumption of an extreme “no checkpoint” strategy.

7 Further research

Experience raises a few questions about our approach. One issue is that profiling is run on one given checkpointing configuration, and changing this configuration can lead to another regime with different profiling results, although often only slightly. Therefore it may be safer to re-run differentiation and profiling each time one checkpoint is inhibited. This can prove impractical and we often chose to switch several checkpoints at a time, possibly missing a better configuration. In other words, to which extent can we use one set of profiling suggestions for several checkpoints at a time, or re-use this set even if some more checkpoints are inhibited? This may relate to relative position in the tree of nested round trips.

Alternatively, one can probably extract enough information from the initial profiling run, enabling some separate optimal search tool to directly find the front of “optimal” configurations. This could avoid the need for re-profiling, but would require storage of profiling data proportional to the size of the (dynamic) tree of nested checkpoints, which can be large. In comparison, our approach only stores at a given time data proportional to the depth of the tree of nested checkpoints, times the number of static checkpoints.

Profiling predictions can also be inaccurate due to optimization of the adjoint code, which the AD tool performs using static data-flow analysis. This leads in many cases to inexact predictions, often too optimistic in favor of inhibiting a checkpoint. To understand the issue, recall that an activated checkpoint C causes the forward sweep $\vec{C}$ to be followed immediately by $\overleftarrow{C}$. A good AD tool will take advantage of this to remove some “adjoint-dead” code at the junction of the two sweeps, such as intermediate values pushed then immediately popped from the stack, as well as primal computations that are not needed for the upcoming backward sweep. Taking the snapshot before running the primal C also has the consequence of possibly reducing the set of values that will be stored during $\vec{D}$ and $\vec{C}$ (found by the “TBR” analysis) because the snapshot already stored these values. The benefits of these data-flow analysis is invisible at profiling time. All in all, time and memory performance of the configuration with an activated checkpoint C are not easily modeled, or equivalently inhibiting C often brings improvements that are not exactly those deduced from our profiling measurements. An obvious answer to this disturbing issue would be to de-activate these data-flow analysis in the AD tool, at least during the iterative profiling process. However, one would then optimize a derivative code which is not the same as the final one, with a chosen checkpointing configuration that may not be the most appropriate.

	exact	good	acceptable	inexact
<i>optim-on</i>	24	9	17	28
<i>optim-off</i>	29	17	11	21

Table 1: Exactness of memory cost predictions when activating or deactivating the “*adjoint-dead*” adjoint code optimization. In both settings, we count the number of predictions that are “exact”, not exact but “good” (estimation error less than 0.01Mb), not good but “acceptable” (estimation error less than 0.1Mb).

To validate the discussion above, we have run two series of profiling experiments on the **streamice** test case, that are identical except that one series (*optim-on*) is performed with the standard Tapenade whereas the other (*optim-off*) uses a Tapenade with a deactivated “adjoint-dead” optimization. We could not deactivate the “TBR” analysis, as this changes the adjoint code too radically. In each series of experiments, we have considered the following cost/benefit predictions: on the one hand each prediction corresponding to each suggestion followed along the “careful on memory” itinerary (blue line of Fig. 4), and on the other hand all predictions, followed or not, made by the initial profiling step i.e. on the initial configuration with all potential checkpoints activated. For technical reasons, predictions made about procedures called at several static locations are not considered. For each prediction, we inhibit the corresponding checkpoint, then differentiate and run, thus measuring the actual peak stack change for comparison with the predicted one. Our results shown in Table 1 mostly corroborate our tentative explanation as we observe better predictions on the *optim-off* experiment. Still, this deserves more investigation.

It is visible in figures 4 and 5 that some relatively better checkpointing configurations are not attained by following the two strategies that we experimented with. Visually there are a few gray dots below-left of the dotted lines. We mostly interpret this as a consequence of the few inexact profiling predictions that we just mentioned. It also indicates that, in addition to the red and blue dotted lines on the figures, there may exist better strategies for selecting the next profiling suggestion to follow.

As already mentioned, checkpointing can also be applied at the level of time-stepping loops, for which the well-known binomial checkpointing [8] provides an optimal schedule. On large codes, we need to apply both “kinds” of checkpointing: (optimal) binomial checkpointing on time-stepping loops, and call tree checkpointing (tuned by profiling) on the rest of the code.

Unfortunately we have no unifying framework that encompasses the two. Nevertheless, interactions exist between both: the same stack that we used throughout this work can also accommodate the snapshots of binomial checkpointing. Given a memory budget for this stack, the choice exists to use it to allow for more snapshots for binomial checkpointing, or to inhibit more call-tree checkpoints in the body of the time-steps. Since a cost model exists for both options, the profiling tool could suggest which option is more profitable, thus making one step towards blending the two approaches.

8 Conclusion

The memory/recomputation trade off known as checkpointing is a key ingredient for adjoint (or reverse) differentiation of large applications. Checkpointing can and must be applied at many levels in the target application, in particular at the nodes of the call tree or on iterative loops. The choice of the locations that deserve checkpointing or not has a major impact on the final performance, in run time and memory usage, of the adjoint code. Yet very little guidance is provided to the end-user. We proposed a profiling algorithm that predicts the performance cost/benefit of checkpointing for each node of the call tree. We experimented with greedy strategies that exploit these predictions, progressively improving performance of the adjoint code by adapting its checkpointing schedule. On the two large test cases that we studied, both from the MITgcm test suite, we obtain significant performance gains, up to a two- or three-fold speedup for a minor memory cost. Profiling for better checkpointing appears as very promising.

The profiling algorithm that we proposed is a prototype. It is now available as a component of the AD tool Tapenade. It could be worth exploring more sophisticated strategies to find almost optimal checkpointing schedules, directly instead of step by step, from a similar profiling pass. It also seems profitable to guide the end-user decision better based on profiling results, or to further automate the process. In particular the interplay between call tree checkpointing and binomial checkpointing should be quantified as well, and integrated in the decision strategy.

Acknowledgments

References

- [1] Profiling JAX programs. <https://jax.readthedocs.io/en/latest/profiling.html>, 2024. Online; accessed 13 May 2024.
- [2] Alistair Adcroft, Jean-Michel Campin, Ed Doddridge, Stephanie Dutkiewicz, Constantinos Evangelinos, David Ferreira, Mick Follows, Gael Forget,

- Baylor Fox-Kemper, Patrick Heimbach, Chris Hill, Ed Hill, Helen Hill, Oliver Jahn, Jody Klymak, Martin Losch, John Marshall, Guillaume Maze, Matt Mazloff, Dimitris Menemenlis, Andrea Molod, and Jeff Scott. *MITgcm User Manual*. Available at <https://mitgcm.readthedocs.io>.
- [3] B. Christianson. Reverse accumulation and attractive fixed points. 1994. Original article can be found at: <http://www.informaworld.com/smpp/title-content=t713645924db=all> Copyright Taylor and Francis / Informa. [Originally issued in 1992 as UH Technical Report 258].
 - [4] Shreyas Sunil Gaikwad, Sri Hari Krishna Narayanan, Laurent Hascoët, Jean-Michel Campin, Helen Pillar, An Nguyen, Jan Hückelheim, Paul Hovland, and Patrick Heimbach. Mitgcm-ad v2: Open source tangent linear and adjoint modeling framework for the oceans and atmosphere enabled by the automatic differentiation tool tapenade, 2024.
 - [5] R. Giering and T. Kaminski. Recipes for Adjoint Code Construction. *ACM Transactions on Mathematical Software*, 24(4):437–474.
 - [6] D. N. Goldberg, S. H. K. Narayanan, L. Hascoët, and J. Utke. An optimized treatment for algorithmic differentiation of an important glaciological fixed-point problem. *Geoscientific Model Development*, 9(5):1891–1904, 2016.
 - [7] D.N. Goldberg, K. Snow, P. Holland, J.R. Jordan, J.-M. Campin, P. Heimbach, R. Arthern, and A. Jenkins. Representing grounding line migration in synchronous coupling between a marine ice sheet model and a z-coordinate ocean model. *Ocean Modelling*, 125:45–60, 2018.
 - [8] A. Griewank and A. Walther. Algorithm 799: Revolve: An implementation of checkpoint for the reverse or adjoint mode of computational differentiation. *ACM Transactions on Mathematical Software*, 26(1):19–45, 2000. Also appeared as Technical University of Dresden, Technical Report IOKOMO-04-1997.
 - [9] A. Griewank and A. Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. Number 105 in Other Titles in Applied Mathematics. SIAM, Philadelphia, PA, 2nd edition, 2008.
 - [10] L. Hascoët and V. Pascual. The Tapenade Automatic Differentiation tool: Principles, Model, and Specification. *ACM Transactions On Mathematical Software*, 39(3), 2013.
 - [11] Julien Herrmann and Guillaume Pallez (Aupy). H-revolve: A framework for adjoint computation on synchronous hierarchical platforms. *ACM Trans. Math. Softw.*, 46(2), jun 2020.
 - [12] James R. Jordan, Paul R. Holland, Dan Goldberg, Kate Snow, Robert Arthern, Jean-Michel Campin, Patrick Heimbach, and Adrian Jenkins. Ocean-forced ice-shelf thinning in a synchronously coupled ice-ocean model. *Journal of Geophysical Research: Oceans*, 123(2):864–882, 2018.
 - [13] J. Lotz, U. Naumann, and S. Mitra. *Mixed Integer Programming for Call Tree Reversal*, pages 83–91. 2016.
 - [14] J. Marshall, A. Adcroft, C. Hill, L. Perelman, and C. Heisey. A finite-volume, incompressible Navier Stokes model for studies of the ocean on parallel computers. *J. Geophys. Res. Oceans*, 102:5753–5766, 1997.
 - [15] U. Naumann. DAG reversal is NP-complete. *Journal of Discrete Algorithms*, 7(4):402–410, 2009.
 - [16] J. Utke, U. Naumann, M. Fagan, N. Tallent, M. Strout, P. Heimbach, C. Hill, and C. Wunsch. OpenAD/F: A modular, open-source tool for automatic differentiation of Fortran codes. *ACM Transactions on Mathematical Software*, 34(4):18:1–18:36, 2008.



Local sparsity and recovery of fusion frame structured signals

Roza Aceska^a, Jean-Luc Bouchot^{b,*}, Shidong Li^c

^a Department of Mathematical Sciences, Ball State University, United States

^b School of Mathematics and Statistics, Beijing Institute of Technology, Beijing 102488, China

^c Department of Mathematics, San Francisco State University, United States

ARTICLE INFO

Article history:

Received 1 April 2019

Revised 12 March 2020

Accepted 13 April 2020

Available online 30 April 2020

Keywords:

Compressed sensing

Fusion frame

Sparse signal approximation

ABSTRACT

The problem of recovering signals of high complexity from low quality sensing devices is analyzed via a combination of tools from signal processing and harmonic analysis. By exploiting fusion frames, we introduce a compressed sensing framework in which we split the dense information into subchannels and fuse the local estimations. Each piece of information is measured by linear, potentially low quality sensors, and recovered via compressed sensing. Finally, by a fusion process within the fusion frames, we are able to recover accurately the original signal. We illustrate our findings with numerical experiments, first consider various artificial setups in which we show that splitting a signal via local projections allows for accurate, stable, and robust estimation. We verify that by increasing the size of the fusion frame, a certain robustness to noise is also achieved. While the computational complexity remains relatively low, we achieve stronger recovery performance compared to usual single-device compressed sensing systems. We finally show how our techniques can be applied in various signal processing tasks such as Doppler signal denoising, natural scene scanning and reconstruction, and MR Image reconstruction. In these examples, we empirically verify that visually good reconstructions are obtained, even in highly undersampled and noisy regimes.

© 2020 Elsevier B.V. All rights reserved.

1. Problem statement

In a traditional sampling and reconstruction system, the sensors are designed so that the recovery of the signal(s) of interest is possible. For instance, when considering the sparse recovery problem, one tries to find the sparsest solution $\hat{\mathbf{x}} \in \mathbb{K}^N$ from the noisy measurements $\mathbf{y} = \mathbf{Ax} + \mathbf{e} \in \mathbb{K}^m$ and $m \ll N$. Here $\mathbb{K}$ denotes the field $\mathbb{R}$ or $\mathbb{C}$. This is done by solving the mathematical program

$$\hat{\mathbf{x}} := \operatorname{argmin} \|\mathbf{z}\|_0, \quad \text{subject to } \|\mathbf{Az} - \mathbf{y}\|_2 \leq \eta, \quad \ell^0 - \min$$

This problem is NP-Hard and usually only approximately solved, for instance by solving its convex relaxation, known as the Basis Pursuit denoising

$$\hat{\mathbf{x}} := \operatorname{argmin} \|\mathbf{z}\|_1, \quad \text{subject to } \|\mathbf{Az} - \mathbf{y}\|_2 \leq \eta, \quad \text{BPDN}$$

It is known that for a given complexity (measured by the sparsity in our context) s of the signal $\mathbf{x}$, the number of random subgaussian linear measurements needs to grow as $m \gtrsim s \log(N/s)$ for $\hat{\mathbf{x}}$ to be a good enough approximation to $\mathbf{x}$. Said differently, if the design of a sensor can be made at will, then knowing the complexity of the signal, here characterized by the sparsity, is sufficient

for a stable and robust recovery. This paper looks at the problem of sampling and reconstructing potentially highly non-sparse signals when the quality of the sensors is constrained. We emphasize in passing that throughout this paper, the sought after signal will always be considered of high-complexity. We use the sparsity or density as a measure of complexity, but one could consider other models. We investigate problems where the number of measurements m cannot be chosen based on the complexity of the signals to recover. In the context described above, one would have a limit on the sparsity of the vectors that can be recovered by $s \leq m/\log(N/m)$. These constraints can be due to many reasons such as cost – e.g. using 10 sensors at a coarser resolution is cheaper than one at the finest –, frequency rate – sensors at 2000 THz might not exist for a while –, legal regulation – e.g. in nuclear medicine where one should not expose a patient to too high radiations at once. Problems arise when the signals being sampled are *too dense* for the usual mathematical theories. When one thinks about compressed sensing, the size of the sensor required is driven by a certain measure of complexity of the signals considered. Allowing for the recovery of signals with higher level of complexity entails the use of better sensors. Here we look at the problem differently: first, we assume constraints on the sensor design which are fixed due to some outside reasons. Under these assumptions, we take on the following challenge: split the information carried

* Corresponding author.

E-mail address: jlouchot@gmail.com (J.-L. Bouchot).

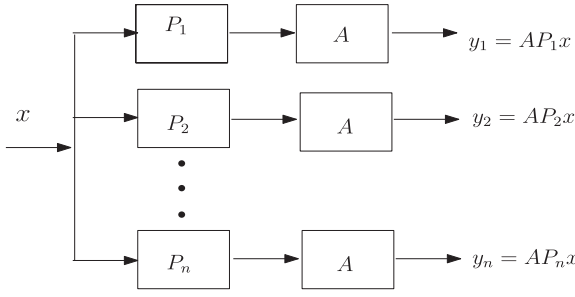


Fig. 1. Sparse linear array geometry.

by the signal in a clever way so that a mathematical recovery is possible.

This paper revisits the theory of fusion frames and applies it to the dense signal recovery problem. We show that by using advanced mathematical techniques stemming from applied harmonic analysis, it is possible to handle very high complexity signals in an efficient and stable manner. Before we dig into the more technical details, we present some real-world scenarios where our framework appears useful, if not essential.

1.1. Examples

1.1.1. Unavailability of high quality observation devices

A typical time-invariant bounded linear operator is always represented by a circulant matrix $\mathbf{A}$. So suppose $\mathbf{A}$ represents a sensing device whose number of rows m is physically limited by the sampling rate (or resolution) of the device. Moreover, consider that the sparsity of the sampled signal $\mathbf{x}$ is substantially larger than what a single observation by $\mathbf{A}$ could handle/recover by various compressed sensing techniques.

In this context, the limitations on the sensing devices combined with the (potentially) high number of non-zeros in the signals makes it impossible for a state-of-the-art algorithm to recover the unknown signal $\mathbf{x}$. As illustrated in Fig. 1, we suggest to apply n such devices in parallel after prefiltering. The fused compressed sensing technique introduced later allows to resolve the problem that otherwise a single device can not! Such scenarios actually exist and show the necessity of the fused compressed sensing technique presented below.

For example, suppose an application requires a sensing device of capacity X , described by a sensing matrix $\mathbf{A}$. In case such a device is either very expensive, or not available, we may choose to combine n parallel projections $\{P_j\}_{j=1}^n$ prior to measuring, and use n low-quality sensing devices of capacity $\frac{1}{n}X$, each described by the sensing matrix $\mathbf{A}$. The (sparse) signal $\mathbf{x}$ is then subsequently recovered by various techniques via each channel and, through the theory of fusion frames [11,13,14], merged into a single vector. As long as $\{P_j\}_{j=1}^n$ are projections - or any filtering operations - with the property that $\mathbf{C}\mathbf{I} \leq \sum_{j=1}^n P_j^* P_j \leq \mathbf{D}\mathbf{I}$ for some $0 < C \leq D < \infty$, such a fusion operation is always possible.

In order for the fusion operator to be bounded away from 0, in our work we assume that the pre-filtering/projections P_i are *complete* in the sense that there won't be "holes" in the signal coverage for a whole class of signals; more precisely, we require that the collection of projections satisfy the fusion frame inequality (2.5).

Evidently, if we design such a fused compressed sensing technique, by enabling the subdivision of sparsity of $\mathbf{x}$ into individual subspaces, the sparse recovery problem becomes a feasible one and can be resolved by multiple sensing devices with low quality or resolution, which are not only widely available but also economical.

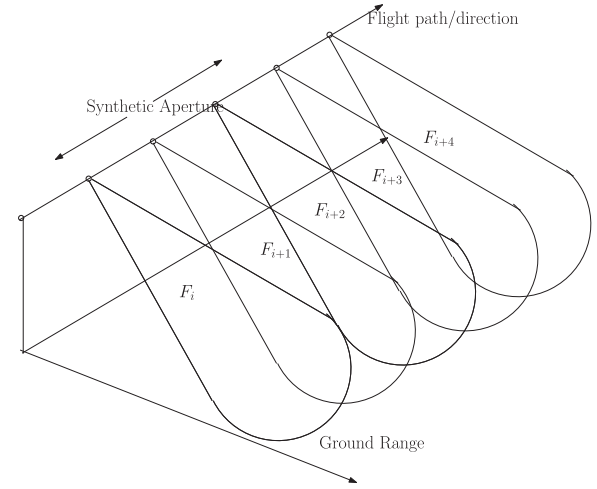


Fig. 2. Synthetic Aperture Imaging beam and data collection mechanism.

The value of fused compressed sensing techniques presented in this paper is clearly reflected in this situation where an otherwise too expensive or impossible problem can now be resolved by using a number of lower resolution/sampling rate devices and by making a *reasonable* number of observations, and processed by the fusion frame theory.

1.1.2. SAR Imaging and spatial filtering

In the Synthetic Aperture Radar (SAR) imaging process, a flying carrier (an airplane or a satellite) emits a sequence of radio waves to the field of observation (and then detects the reflections by the objects in the field). Each radio wave is sent through an antenna with a fixed aperture/size which physically presents an antenna beam (magnetic field) pattern, say $\{F_i\}$.

Consequently, as seen in Fig. 2, during the i^{th} data collection, the antenna beam pattern F_i physically implements a spatial filtering operator. And, naturally, between adjacent i 's (even among several adjacent i 's), the spatial antenna beams F_i 's have intentional overlaps. The actual data observation, for each i , is modeled by $\mathbf{y}^{(i)} = \mathbf{A}F_i\mathbf{x}$, where $\mathbf{x}$ is the field image reflection coefficients that eventually form the SAR image of the field, and $\mathbf{A}$ is a fixed observation matrix determined by the SAR imaging mechanism, which is typically chirping (linearly changing frequencies in time) functions in two different dimensions [19,20,26]. This formulation has a natural "distributed sparsity" due to the spatial filtering operations inherent to the physical beams.

Here, to consider the spatial filtering effect, and to avoid the conventional ad-hoc "alignment" in the flight direction, the fused SAR imaging process needs to be considered [38]. The new fused SAR imaging formulation is not only the should-be rigorous mathematical formulation of the SAR imaging process, but also potentially beneficial to image resolution and to the robustness of the (often) turbulent data collection process.

1.2. Contributions

In this paper we combine mathematical tools from compressed sensing and fusion frame theory to break the limitations on the signal complexity induced by traditional recovery methods.

Our general approach can be described as the two following steps, or summarized in Algorithm 1.

- 1) Estimate the local information $\widehat{\mathbf{x}}^{(i)}$, for $1 \leq i \leq n$ by any sparse recovery method, and

Algorithm 1: Fused distributed sensing recovery framework.

Data: $(P_i)_{1 \leq i \leq n}$, n projections such that $\mathbf{C}\mathbf{I} \leq \sum \|P_i\|_{2 \rightarrow 2}^2 \leq D\mathbf{I}$,
an estimation of the measurement noise η

Result: An estimation $\hat{\mathbf{x}}$ of the vector $\mathbf{x}$

$k \leftarrow 1$;
 $\hat{\mathbf{x}} = \mathbf{0}$;
while $k \leq n$ **do**
 Measure k^{th} vector $\mathbf{y}^{(k)} \leftarrow \mathbf{A}P_k\mathbf{x} + \mathbf{e}^{(k)}$;
 Add local information $\hat{\mathbf{x}} \leftarrow \hat{\mathbf{x}} + \text{argmin} \|\mathbf{z}\|_1$ s.t.
 $\|\mathbf{A}P_k\mathbf{z} - \mathbf{y}^{(k)}\|_2 \leq \eta$;
 $k \leftarrow k + 1$
end
Fusion: $\hat{\mathbf{x}} \leftarrow S^{-1}\hat{\mathbf{x}}$

- 2) Approximate the fused solution $\hat{\mathbf{x}} = S^{-1} \left(\sum_{i=1}^n \widehat{\mathbf{x}}^{(i)} \right)$, either directly or by virtue of the frame algorithm.

Note that the fusion operation either requires the inverse frame operator S^{-1} or can be approximated using the (fusion-) frame algorithm, described at the end of Section 2.2. An important aspect of the framework is that it can either be used sequentially or in a parallel manner. In the sequential approach, we get the local pieces of information one after the other. In this case, as described in Algorithm 1, we have no need to save the local measurements and can update our guess in an online fashion. In the parallel approach, the local measurements are processed independently of each other, and the fusion is done by a central unit, once all the local information have been collected.

In particular, Proposition 3.2 later in the manuscript shows that the robustness is preserved independently from the number of subspaces considered, which can be phrased in simple terms as:

Proposition 1.1. Let $\mathbf{A} \in \mathbb{K}^{m \times N}$, and let $\mathcal{W} = (W_i, P_i)_{i=1}^n$, $n \geq 1$ be a fusion frame. Assume the measurement vectors $\mathbf{y}^{(i)} = \mathbf{A}P_i\mathbf{x} + \mathbf{e}^{(i)}$ are corrupted by some (adversarial) independent noise uniformly bounded $\|\mathbf{e}^{(i)}\|_2 \leq \eta$. Then the solution $\hat{\mathbf{x}}$ generated by Algorithm 1 satisfies

$$\|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 \leq K\eta^2. \quad (1.1)$$

In the more precise statement (Proposition 3.2), we specify precisely the value of the constant $K = 1/C$ where C denotes the lower frame bound. As a special case of Theorem 4.6, we proved the following

Theorem 1.1. Let $\mathcal{W} = (W_i, P_i)_{i=1}^n$ be a fusion frame system for $\mathbb{K}^N$ with frame operator S and frame bounds $0 < C \leq D < \infty$. Let $\mathbf{A} \in \mathbb{K}^{m \times N}$ be a matrix satisfying some Partial RIP condition (see Definition 6). Any s distributed-sparse vector $\mathbf{x}$ whose sparsity is uniformly distributed among the n subspaces (i.e. $s_i = s/n$ is the sparsity of $P_i\mathbf{x}$) can be recovered as

$$\hat{\mathbf{x}} = S^{-1} \left(\sum_{i=1}^n \widehat{\mathbf{x}}^{(i)} \right),$$

where the $\widehat{\mathbf{x}}^{(i)}$ are obtained as solutions to the n local sparse recovery problems

$$\widehat{\mathbf{x}}^{(i)} := \text{argmin} \|\mathbf{z}\|_0, \quad \text{subject to } \|\mathbf{A}P_i\mathbf{z} - \mathbf{y}^{(i)}\|_2 \leq \eta_i.$$

Moreover, assuming the errors to be uniformly bounded by η , and that the RIP constants $\delta_1 = \delta_2 = \dots = \delta_n = \delta < 4/\sqrt{41}$ are the same in all the subspaces, the solution approximate the true vector $\mathbf{x}$ in the following sense:

$$\|\hat{\mathbf{x}} - \mathbf{x}\|_2 \leq \frac{n}{C}\eta$$

where $\eta := \max_i \eta_i$ and $C \leq \frac{960\sqrt{2}}{(16-41\delta^2)^2}$ is a constant depending only on δ .

Note that under the usual compressed sensing assumptions, this allows to have a number of measurements per sampled vector that decay linearly with the number of sensors.

The paper is articulated as follows. We review basics from compressed sensing and fusion frames in Section 2. In Section 3 we explore several theoretical examples to motivate our research, and to emphasize the importance of our findings. Our claims are empirically verified by numerical results. In the flair of traditional compressed sensing, we extend the standard compressed sensing results to the case of recovery where we make explicit use of local redundancy in the fusion frame decomposition in Section 5. Finally, Section 4 derives a mathematical theory allowing to work with fusion frames where sparsity is exploited along a subspace decomposition. These findings extend the traditional compressed sensing problem [25] and show some similarities with recent developments in parallel acquisition [18] and structured sensing [10].

2. General tools and models

2.1. (Traditional) compressed sensing

Compressed sensing (CS) relies (see [23,25] and references therein) on the inherent sparsity of natural signals $\mathbf{x}$ for their recovery from seemingly few measurements $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{e}$ given some linear measurement matrix $\mathbf{A} \in \mathbb{K}^{m \times N}$, with $m \ll N$. Here the vector $\mathbf{e} \in \mathbb{K}^m$ contains the noise and is usually assumed to be bounded, $\|\mathbf{e}\|_2 \leq \eta$. With the sparsity assumption, CS aims at finding (approximate) solutions to the ℓ^0 -min problem. General recovery guarantees ensure that the recovery is stable and robust, that is, the solution satisfies the following approximation bounds

$$\|\hat{\mathbf{x}} - \mathbf{x}\|_2 \leq \frac{C}{\sqrt{s}} \sigma_s(\mathbf{x})_1 + D\eta, \quad (2.1)$$

$$\|\hat{\mathbf{x}} - \mathbf{x}\|_1 \leq C\sigma_s(\mathbf{x})_1 + D\sqrt{s}\eta, \quad (2.2)$$

where $\sigma_s(\mathbf{x})_1 := \inf_{\mathbf{z}: \|\mathbf{z}\|_0 \leq s} \|\mathbf{x} - \mathbf{z}\|_1$ defines the best s -term approximation of $\mathbf{x}$ in the ℓ^1 norm.

Plethora of conditions (some of which we look closer in Section 4) on $\mathbf{A}$ have been derived to ensure that the previous (or similar) estimates hold. These are based on restricted isometry constants ($\delta_{2s} < \sqrt{2} - 1$ [12] or $\delta_{2s} < 4/\sqrt{41}$ [25], $\delta_s < 1/3$), null space properties ($\mathbf{A}$ fulfills the robust and stable NSP(s, ρ, τ) if $\|\mathbf{v}_S\|_1 \leq \rho\|\mathbf{v}_{S^c}\|_1 + \tau\|\mathbf{A}\mathbf{v}\|_2$ for any vector $\mathbf{v} \in \mathbb{K}^N$, and S any index set with $|S| \leq s$), coherence ($\mu_1(s) + \mu_1(s-1) < 1$), quotient property, and so on. Similar conditions and bounds can be found when greedy and thresholding algorithms are used to approximate ℓ^0 -min (see [6–8,24,33,39] for some relatively recent results in this direction).

However, to ensure such recovery bounds, there is still a need for building adequate sensing matrices $\mathbf{A}$. So far, no deterministic matrices $\mathbf{A}$ can be built with reasonable numbers of rows (i.e. with decent, small enough, number of measurements) and we have to rely on randomness to build measurement matrices. It has been shown, for instance, that matrices with independent random subgaussian entries fulfill the RIP(s, δ) provided that the number of measurements scales as $m \asymp \delta^{-2}s \log(N/s)$ [5]. Similar results have been obtained (with different log factors) for structured random matrices or matrices from bounded orthonormal systems, see [35] for instance.

2.2. Frames, fusion frames, and distributed signal processing

By definition, a sequence $\mathcal{F} = \{f_i\}_{i \in I}$ in a Hilbert space $\mathcal{H}$ is a frame [17] for $\mathcal{H}$ if there exist $0 < A \leq B < \infty$ (lower and upper frame bounds) such that

$$A\|f\|^2 \leq \sum_{i \in I} |\langle f, f_i \rangle|^2 \leq B\|f\|^2 \text{ for all } f \in \mathcal{H}. \quad (2.3)$$

The representation space associated with $\mathcal{F}$ is $\ell^2(I)$ and its analysis and synthesis operators are respectively given by

$$T(f) = \{\langle f, f_i \rangle\}_{i \in I} \text{ and } T^*(\{c_i\}_{i \in I}) = \sum_{i \in I} c_i f_i,$$

for all $f \in \mathcal{H}$ and $\{c_i\}_{i \in I} \in \ell^2(I)$. It shows that the frame operator $S := T^*T$ is a positive, self-adjoint and invertible operator; this means that recovery of any $f \in \mathcal{H}$ is possible, if Sf is known; however, computing S^{-1} can be computationally challenging. Luckily, each frame $\mathcal{F}$ is accompanied by at least one so-called dual frame $\mathcal{G} = \{g_i\}_{i \in I}$, which satisfies

$$f = \sum_{i \in I} \langle f, f_i \rangle g_i = \sum_{i \in I} \langle f, g_i \rangle f_i \text{ for all } f \in \mathcal{H}, \quad (2.4)$$

and ensures recovery of the function f . Whenever a frame is A -tight ($A = B$), the problem of function reconstruction is simplified, since the frame operator S in this case is a scalar multiple of the identity operator.

Fusion frames have been initially created [13] to model the setting of a wireless sensor network. Sensor networks are composed of wireless sensors with constraints in their processing power and transmission bandwidth, which reduces the costs but also affects the precision of the system. The sensors are distributed over a significantly large area of interest, to measure, for instance, pollution, temperature, sound, pressure, motion etc. The network is redundant, i.e., there is no orthogonality among sensors, so each sensor can be interpreted as a frame element. In addition, a large sensor network is split into redundant sub-networks; the local measurements within each sub-network are sent to a local sub-station, which submits the gathered information further to a central processing station for final reconstruction.

Every (local) sensor is represented by a single frame vector; that is, each sub-network is related to a frame for a subspace in a Hilbert space. The subspaces have to satisfy a certain overlapping property, which ensures that the overlaps are not too large. The reconstruction in such a system is done in two steps: first, within each subspace the conventional frame reconstruction is employed; then, the local pieces of information serve as the inputs for the fusion frame reconstruction, which reconstructs the initial signal completely.

Definition 1 (Fusion frames). Given an index set I , let $\mathcal{W} := \{W_i \mid i \in I\}$ be a family of closed subspaces in $\mathcal{H}$. We denote the orthogonal projections¹ onto W_i by P_i . Then $\mathcal{W}$ is a fusion frame, if there exist $C, D > 0$ such that

$$C\|f\|^2 \leq \sum_{i \in I} \|P_i(f)\|^2 \leq D\|f\|^2 \text{ for all } f \in \mathcal{H}. \quad (2.5)$$

Remark 2.1. Fusion frames are often accompanied by respective weights. In the weighted case, the frame condition reads $C\|f\|^2 \leq \sum_{i \in I} v_i^2 \|P_i(f)\|^2 \leq D\|f\|^2$ for all $f \in \mathcal{H}$ and some positive weights $(v_i)_{i \in I}$. All (unweighted) results derived below apply mutatis mutandis to the weighted case.

Given a fusion frame $\mathcal{W}$ for a Hilbert space $\mathcal{H}$, let $\mathcal{F}_i := \{f_{ij} \mid j \in J_i\}$ be a frame for W_i , $i \in I$. Then $\{(W_i, \mathcal{F}_i) \mid i \in I\}$ is a fusion frames

system for $\mathcal{H}$. In fact fusion frames and frames are not unrelated. It is known [13] that the following statements are equivalent.

- $\cup_{i \in I} \{f_{ij} \mid j \in J_i\}$ is a frame for $\mathcal{H}$.
- $\{W_i\}_{i \in I}$ is a fusion frame for $\mathcal{H}$.

In particular, if $\{(W_i, \mathcal{F}_i)\}_{i \in I}$ is a fusion frame system for $\mathcal{H}$ with frame bounds C and D , then $\cup_{i \in I} \{f_{ij} \mid j \in J_i\}$ is a frame for $\mathcal{H}$ with frame bounds AC and BD . In the fusion frame theory, an input signal is represented by a collection of vector coefficients that represent the projection onto each subspace. The representation space used in this setting is

$$\left(\sum_{i \in I} \oplus W_i \right)_{\ell^2} = \{ \{f_i\}_{i \in I} \mid f_i \in W_i \}$$

with $\{\|f_i\|\}_{i \in I} \in \ell^2(I)$.

The analysis operator T is then defined by

$$T(f) := \{P_i(f)\}_{i \in I} \text{ for all } f \in \mathcal{H},$$

while its adjoint operator is the synthesis operator $T^* : (\sum_{i \in I} \oplus W_i)_{\ell^2} \rightarrow \mathcal{H}$, defined by

$$T^*(f) = \sum_i f_i, \text{ where } f = \{f_i\}_{i \in I} \in \left(\sum_{i \in I} \oplus W_i \right)_{\ell^2}.$$

The fusion frame operator $S = T^*T$ is given by

$$S(f) = \sum_{i \in I} P_i(f).$$

It is easy to verify that S is a positive and invertible operator on $\mathcal{H}$. In particular, it holds $C\mathbf{I} \leq S \leq D\mathbf{I}$, with $\mathbf{I}$ denoting the identity.

If the dual frames $\mathcal{G}_i$, $i \in I$, for each local frames are known then the fusion frame operator can be expressed in terms of the local (dual) frames [14]:

$$S = \sum_{i \in I} T_{\mathcal{G}_i}^* T_{\mathcal{F}_i} = \sum_{i \in I} T_{\mathcal{F}_i}^* T_{\mathcal{G}_i}.$$

For computational needs, we may only consider the fusion frame operator in finite frame settings, where the fusion frame operator becomes a sum of matrices of each subspace frame operator. The evaluation of the fusion frame operator S and its inverse S^{-1} in finite frame settings are conveniently straightforward. By F_i we denote the frame matrices formed by the frame vectors from $\mathcal{F}_i$, in a column-by-column format. Let G_i be defined in the same way from the dual frame $\{g_{ij}\}_{j \in J_i}$. Then the fusion frame operator is

$$S(f) = \sum_{i \in I} F_i G_i^T f = \sum_{i \in I} G_i F_i^T f.$$

Hence a distributed fusion processing is feasible in an elegant way, since the reconstruction formula for all $f \in \mathcal{H}$ is

$$f = \sum_{i \in I} S^{-1} P_i(f).$$

The standard distributed fusion procedure uses the local projections of each subspace. In this procedure, the local reconstruction takes place first in each subspace W_i , and the inverse fusion frame is applied to each local reconstruction and combined together:

$$f = \sum_{i \in I} S^{-1} P_i(f) = \sum_{i \in I} S^{-1} \left(\sum_{j \in J_i} \langle f, f_{ij} \rangle g_{ij} \right) \text{ for all } f \in \mathcal{H}. \quad (2.6)$$

Alternatively, one may use a reconstruction procedure acting globally, which is possible if the coefficients of signal/function decompositions are available:

$$f = \sum_{i \in I} \sum_{j \in J_i} \langle f, f_{ij} \rangle (S^{-1} g_{ij}) \text{ for all } f \in \mathcal{H}. \quad (2.7)$$

¹ an operator P is an orthogonal projection if $P^2 = P$ and $P^* = P$

The difference in procedure (2.7), compared with a global frame reconstruction lies in the fact that the (global) dual frame $\{S^{-1}g_{ij}\}$ is first calculated at the local level, and then fused into the global dual frame by applying the inverse fusion frame operator. This potentially makes the evaluation of (global) duals much more efficient.

As stated above, computing the inverse (fusion) frame operator is often a challenging task. Instead, one can approximate the solution $\hat{\mathbf{x}}$ by employing the so-called frame algorithm. All we need to start the iterative algorithm is $S\hat{\mathbf{x}}$, which we already have. We recall the relevant result below for completeness.

Proposition 2.1 [14]. *Let $(W_i)_{i \in I}$ be a fusion frame in $\mathbb{K}^N$, with fusion frame operator $S = S_W$, and fusion frame bounds C, D . Further, let $\mathbf{x} \in \mathbb{K}^N$, and define the sequence $(\mathbf{x}_k)$ by $\mathbf{x}_0 = 0$ and $\mathbf{x}_k = \mathbf{x}_{k-1} + \frac{2}{C+D}S(\mathbf{x} - \mathbf{x}_{k-1})$, $k \geq 1$. Then we have $\mathbf{x} = \lim_{k \rightarrow \infty} \mathbf{x}_k$, with the error estimate $\|\mathbf{x} - \mathbf{x}_k\| \leq \left(\frac{D-C}{D+C}\right)^k \|\mathbf{x}\|$.*

Concretely, using the fusion frame reconstruction, the updates read

$$\mathbf{x}_k = \mathbf{x}_{k-1} + \frac{2}{C+D} \sum_{i=1}^n \mathbf{x}^{(i)} - \frac{2}{C+D} \sum_{i=1}^n P_i \mathbf{x}_{k-1}.$$

The middle term is computed once and for all. The following updates then only require some basic matrix-vector multiplications. Note that starting the algorithm with $\hat{\mathbf{x}}_0 = 0$ yields $\hat{\mathbf{x}}_1 = \frac{2}{C+D}S\hat{\mathbf{x}}$.

Similarly, in Section 5, we have $\hat{\mathbf{x}} = S^{-1}\left(\sum_j \mathbf{x}^{(j)}\right)$, and $\hat{f} = D\hat{\mathbf{x}}$. We can find $\hat{\mathbf{x}}$ via the iterative approach of Proposition 2.1, by starting the algorithm with $\hat{\mathbf{x}}_0 = 0$, $\hat{\mathbf{x}}_1 = \frac{2}{C+D}S(\hat{\mathbf{x}}) = \frac{2}{C+D} \sum_j S\mathbf{x}^{(j)}$, and so on. The n -th term approximation of $\hat{f}$ is then estimated with $\hat{f}_n = D\hat{\mathbf{x}}_n$.

2.3. Signal recovery in fusion frames

This work describes an approach for sensing and reconstructing signals in a fusion frame structure. As presented above, given some local information $\mathbf{x}^{(i)} := P_i(\mathbf{x})$, for $1 \leq i \leq n$, a vector can easily be reconstructed by applying the inverse fusion frame operator

$$\mathbf{x} := S^{-1}S(\mathbf{x}) = S^{-1} \sum_{i=1}^n P_i(\mathbf{x}) = S^{-1} \left(\sum_{i=1}^n \mathbf{x}^{(i)} \right),$$

where $\mathbf{x}^{(i)} := P_i(\mathbf{x})$, for $1 \leq i \leq n$. (2.8)

Throughout the work, we assume that the projected vectors are sampled independently from one another, with n devices modeled by the same sensing matrix $\mathbf{A}$. Formally, the problem is as follows *From the measurements $\mathbf{y}^{(i)} = \mathbf{A}\mathbf{x}^{(i)} + \mathbf{e}^{(i)}$, reconstruct an estimation $\hat{\mathbf{x}}^{(i)}$ of the local information to compute the approximation $\hat{\mathbf{x}}$ of the signal $\mathbf{x}$.*

From this point on, there are two ways of thinking about the problem. In a first scenario, the signals are measured in the subspace and the recovery of the $\mathbf{x}^{(i)}$ are done locally. In other words, it accounts for solving n ℓ^0 -min problems (or their approximations via BPDN for instance) in the subspaces, and then transmitting the estimated local signals to a central unit taking care of the fusion via Eq. (2.8). The other approach consists of transmitting the local observations $\mathbf{y}^{(i)} = \mathbf{A}P_i\mathbf{x}$ to a central processing station which takes care of the whole reconstruction process. In this case, the vector $\mathbf{x}$ can be recovered by solving a unique ℓ^0 -min problem directly with, letting $\mathbf{I}_n$ denote the n dimensional identity matrix,

$$\mathbf{y} = \begin{pmatrix} \mathbf{y}^{(1)} \\ \vdots \\ \mathbf{y}^{(n)} \end{pmatrix} = \mathbf{I}_n \otimes \mathbf{A} \begin{bmatrix} P_1 \\ \vdots \\ P_n \end{bmatrix} \mathbf{x} \quad (2.9)$$

While the latter case is interesting on its own (see for instance [18]), we investigate here some results for the first case. Our results can be investigated and generalized further, integrating ideas where the measurements matrix (here, the sensors) vary locally, as is the case in [18] or driven with some structured acquisition (see for instance [10]).

We would like to put our work in context. This paper is not the first one to describe the use of fusion frames in sparse signal recovery. However, it is inherently different from previous works in [3,9]. In [9] the authors provide a framework for recovering sparse fusion frame coefficients. In other words, given a fusion frame system $\{(W_i, P_i)\}_{i=1}^n$ a vector is represented on this fusion frame as a set of n vectors $(\mathbf{x}^{(i)})_{i=1}^n$ where each of the $\mathbf{x}^{(i)}$ corresponds to the coefficient vector in subspace W_i . The main idea of the authors is that the original signal may only lie in few of the n subspaces, implying that most of the $\mathbf{x}^{(i)}$ should be 0. To rephrase the problem, we can say that the vector has to lie in a sparse subset of the original fusion frame. While [9] is concerned with some recovery guarantees under (fusion-) RIP and average case analysis, the paper [3] gives uniform results for subgaussian measurements and derive results on the minimum number of vector measurements required for robust and stable recovery.

We, on the other hand, exploit structures (or sparsity) locally.² We do not ask that only a few of the subspaces be active for a given signal, but that the signal only has a few active components per subspace. This justifies the use of the local properties of fusion frame systems.

3. Examples and applications

This section introduces some theoretical examples where the application of our fusion frame-based recovery shows increased performance over traditional recovery techniques. In particular, we justify in Proposition 3.2 that the fusion frame recovery may provide better robustness against noise.

3.1. Orthogonal canonical projections

Consider the problem of recovering $\mathbf{x} \in \mathbb{K}^N$ from the measurements:

$$\mathbf{y}^{(i)} = \mathbf{A}_i \mathbf{x} + \mathbf{e}^{(i)}, \quad 1 \leq i \leq n,$$

where $\mathbf{A}_i$'s are defined as $\mathbf{A}_i = \mathbf{A}P_i$ and P_i is the orthogonal projection onto Ω_i , with $\Omega_i \subseteq \{1, \dots, N\}$. We assume that the noise vectors $\mathbf{e}^{(i)}$ are uncorrelated and independent, with $\|\mathbf{e}^{(i)}\|_2 \leq \eta_i$. $\mathbf{A}$ corresponds to a matrix of linear measurements in $\mathbb{K}^{m \times N}$. Note that this framework is an example of recovery from Multiple Measurement Vectors (MMV) (see [21,29,36] and references therein) where the signal is the same in every measurements, and the matrices are different for each set of measurements. We consider the problem of recovering local vectors $\mathbf{x}^{(i)}$, for $1 \leq i \leq n$, where the $\mathbf{x}^{(i)}$ are defined as $\mathbf{x}^{(i)} := P_i \mathbf{x}$. In this case, we have that the sparsity of each local signal is at most $N_i := \text{rank}(P_i) = |\Omega_i|$. Once the local pieces of information $\mathbf{x}^{(i)}$ are recovered, the original signal $\mathbf{x}$ can be estimated from a fusion frame like reconstruction:

$$\hat{\mathbf{x}} = S^{-1} \left(\sum_{i=1}^n \hat{\mathbf{x}}^{(i)} \right) = \sum_{i=1}^n S^{-1} \hat{\mathbf{x}}^{(i)}, \quad (3.1)$$

where the fusion frame operator S is defined in the usual way as $S: \mathbb{K}^N \rightarrow \mathbb{K}^N$, $S(\mathbf{x}) = \sum_{i=1}^n P_i(\mathbf{x})$. The problem is to recover the unknown vector $\mathbf{x}$ from the measurements $\{\mathbf{y}^{(i)}\}_{i=1}^n$ by solving local problems. Clearly, a necessary condition for uniform recovery

² This sparsity assumption is needed only when a given frame component has a large dimension. We hope that in most of the cases we can control this dimension to be small enough avoiding the necessity of sparse assumptions. See the following section for more details.

is that we have $\bigcup_{i=1}^n \Omega_i = \{1, \dots, N\}$ in a deterministic setting, or with high probability in a probabilistic setting.

The main idea of our approach is to solve the (very) high-dimensional, high-complexity, and demanding problem $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{e}$ by combining results obtained from n problems that are much easier to solve. We investigate first the case where the orthogonal projections do not overlap, i.e. $\Omega_i \cap \Omega_j = \emptyset$, for any $i, j \in \{1, \dots, n\}$ with $i \neq j$ and $\bigcup_{i=1}^n \Omega_i = \{1, \dots, N\}$. The results developed are reminiscent of some work on partial sparse recovery, when part of the support is already known [4]. In a second step we analyze the use of random and deterministic projections that may overlap for the recovery of the signal $\mathbf{x}$.

Remark 3.1. It is important to pause here and understand the problem above. Knowing the support of the projections, one could work on the local vectors $\mathbf{x}^{(i)}$ of smaller sizes $N_i < N$. Doing this would allow faster computations of the solutions but also has the implication that the sparsity (per subspace) has to remain small. On the other hand, considering large vectors with many zero components allows to keep a sparsity rather large (compared to the subspace dimension). We choose to investigate mainly the latter case as we try to break the limitation on the complexity for a given sensing matrix.

3.1.1. Decomposition via direct sums

It is clear that the dimension N of the ambient space will play an important role. Indeed, if we consider no overlap between the projections, we have two drastically different extreme cases. These scenarios are important as they shed light on the main ideas behind our recovery results.

Casen = N This case is characterized by (up to a permutation of the indices) $\Omega_i = \{i\}$ for all $1 \leq i \leq N$. In other words, each (set of) measurements $\mathbf{y}^{(i)} = \mathbf{A}_i \mathbf{x} + \mathbf{e}^{(i)}$ gives information about a single entry x_i of the input vector $\mathbf{x}$ via an overdetermined linear system. In this case, assuming $\mathbf{A}$ has full rank, we can compute an estimate $\hat{x}_i$ of the entry x_i as the solution to the ℓ^2 minimization problem:

$$\hat{x}_i := \operatorname{argmin}_{x \in \mathbb{R}} \|\mathbf{y}^{(i)} - x \mathbf{a}_i\|_2^2 = \frac{\mathbf{a}_i^T \mathbf{y}^{(i)}}{\mathbf{a}_i^T \mathbf{a}_i} \quad (3.2)$$

where $\mathbf{a}_i$ denotes the i^{th} column of the matrix $\mathbf{A}$. The N independent ℓ^2 minimizations ensure that the final solution $\hat{\mathbf{x}}$ satisfies the bound

$$\|\mathbf{x} - \hat{\mathbf{x}}\|_2 = \sqrt{\sum_{i=1}^N \frac{|\langle \mathbf{a}_i, \mathbf{e}^{(i)} \rangle|^2}{\|\mathbf{a}_i\|_2^4}} \leq \sqrt{\sum_{i=1}^N \frac{\|\mathbf{e}^{(i)}\|_2^2}{\|\mathbf{a}_i\|_2^2}} \leq \sum_{i=1}^N \frac{\|\mathbf{e}^{(i)}\|_2}{\|\mathbf{a}_i\|_2}. \quad (3.3)$$

In particular, in the noiseless scenario ($\mathbf{e}^{(i)} = 0$, for all i), the recovery is exact. In terms of (normalized) matrices from Gaussian measurements, which ensures that $\mathbf{A}$ has full rank – even numerically – with high-probability [16,37], it holds $\mathbb{E}\|\mathbf{a}_i\|_2 = 1$ and it follows $\mathbb{E}\|\mathbf{x} - \hat{\mathbf{x}}\|_2 \leq \sum_{i=1}^N \mathbb{E}\|\mathbf{e}^{(i)}\|_2$.

Notice that even though we have only one fixed matrix $\mathbf{A} \in \mathbb{K}^{m \times N}$, (obviously) we are no longer limited to the recovery of sparse vectors, which is the motivation for using fusion frames as described below. However, this has a price as the (actual) number of measurements drastically increased to $m \cdot N$. This is clearly too many measurements in any practical scenarios, but with similar ideas, we can reach practical applications such as using multiple lower sampling rate devices in order to resolve a harder problem, typically required for much higher rate of sampling devices (see Section 1.1.1). Note also that all the results here should be put in the context of MMV problems, and not with the usual CS setup.

Casen = 2 We consider now two sets Ω_1 and Ω_2 such that $\Omega_1 \cup \Omega_2 = \{1, \dots, N\}$ and $\Omega_1 \cap \Omega_2 = \emptyset$. Without loss of generality we can assume that $\Omega_1 = \{1, \dots, N_1\}$ and $\Omega_2 = \{N_1 + 1, \dots, N_1 + N_2\}$

with $N_1 + N_2 = N$. We measure twice the unknown vector $\mathbf{x}$ as $\mathbf{y}^{(1)} = \mathbf{A}_1 \mathbf{x} + \mathbf{e}^{(1)} = \mathbf{A} P_1 \mathbf{x} + \mathbf{e}^{(1)}$ and $\mathbf{y}^{(2)} = \mathbf{A}_2 \mathbf{x} + \mathbf{e}^{(2)} = \mathbf{A} P_2 \mathbf{x} + \mathbf{e}^{(2)}$. If both N_1 and N_2 are smaller than m , then two ℓ^2 minimizations recover $\widehat{\mathbf{x}}^{(1)}$ and $\widehat{\mathbf{x}}^{(2)}$ independently and it follows that the solution $\widehat{\mathbf{x}} = \widehat{\mathbf{x}}^{(1)} + \widehat{\mathbf{x}}^{(2)}$ obeys the following error bound:

$$\|\widehat{\mathbf{x}} - \mathbf{x}\|_2^2 \leq \|\mathbf{A}_{\Omega_1}^+ \mathbf{e}^{(1)}\|_2^2 + \|\mathbf{A}_{\Omega_2}^+ \mathbf{e}^{(2)}\|_2^2. \quad (3.4)$$

The reconstruction is again perfect in the noiseless case. Moreover, as in the previous case, there is no need for a sparsity assumption on the original vector $\mathbf{x}$. The oversampling ratio is not very large as we only have a total of $2m$ measurements.

If however one at least of N_1 or N_2 (say N_1) is larger than m then we need to use other tools as we are now solving an under-determined linear system. Driven by ideas from CS, we can assume the vector to be sparse on Ω_1 . The recovery problem becomes

$$\text{Find } \widehat{\mathbf{x}}^{(1)} \text{ that minimizes } \|\mathbf{z}\|_1, \quad \text{subject to } \|\mathbf{A}_1 \mathbf{z} - \mathbf{y}^{(1)}\|_2 \leq \eta_1 \quad (3.5)$$

The ℓ^1 minimization problem is introduced as a convex relaxation of the NP-hard ℓ^0 minimization. Note that the constraints apply only on the support Ω_1 of the unknown vector. Hence the usual sparsity requirements encountered in the CS literature need not to apply to the whole vector. Unfortunately, the sparsity assumption being applied independently on Ω_1 and/or Ω_2 restricts ourselves to non-uniform recovery guarantees only, at least when considering the full set of sparse signals. The following definition will become handy in the analysis.

Definition 2 (Partial null space property). A matrix $\mathbf{A} \in \mathbb{K}^{m \times N}$ is said to satisfy a partial robust and stable null space property of order s with respect to a subset $\Omega \subset \{1, \dots, N\}$ with $N \geq |\Omega| > m$, P the orthogonal projection onto Ω , and constants $0 < \rho < 1$, $0 < \tau$ if

$$\|(P\mathbf{v})_S\|_1 \leq \rho \|(P\mathbf{v})_{\bar{S}}\|_1 + \tau \|\mathbf{A}\mathbf{v}\|_2 \quad (3.6)$$

holds for any vector $\mathbf{v}$ and any set $S \subset \Omega$ such that $|S| \leq s$.

Remark 3.2. The previous definition is a weakening of the usual robust null space property, where what happens in the complement of the set Ω is irrelevant. It is worth noticing that it also coincides with the usual definition when $\Omega = \{1, \dots, N\}$.

The partial robust null space property of the measurement matrix $\mathbf{A}$ on Ω_1 ensures that the recovered local vector $\mathbf{x}^{(1)}$ obeys the following error bound [25, Theorem 4.19]:

$$\|\widehat{\mathbf{x}}^{(1)} - P_1 \mathbf{x}\|_1 \leq \frac{2(1+\rho)}{1-\rho} \sigma_s(\mathbf{x}^{(1)})_1 + \frac{4\tau}{1-\rho} \|\mathbf{e}^{(1)}\|_2. \quad (3.7)$$

Eq. (3.7) is obtained by modifying the proofs of [25, Theorem 4.19, Lemma 4.15] and adapting them to the presence of the projection P_1 . A formal proof of a more general statement is given in the proof of the later Theorem 4.3. This yields the following direct consequence

Proposition 3.1. Let N and m be positive integers with $N > m$ and $\mathbf{A} \in \mathbb{K}^{m \times N}$. There exist two integers N_1 and N_2 such that, up to a permutation, $\Omega_1 = \{1, \dots, N_1\}$ and $\Omega_2 = \{N_1 + 1, \dots, N_1 + N_2\}$. Assume that $N_2 \leq m$ and $N_1 > m$. Assume that the matrix $\mathbf{A}$ satisfies the partial robust null space property of order s with respect to Ω_1 with constants $0 < \rho < 1$ and $\tau > 0$ and that $\mathbf{A}_{\Omega_2}$ has full rank. For $\mathbf{x} \in \mathbb{K}^N$, let $\widehat{\mathbf{x}} := \widehat{\mathbf{x}}^{(1)} + \widehat{\mathbf{x}}^{(2)}$ with $\widehat{\mathbf{x}}^{(1)}$ solution to Problem (3.5) and $\widehat{\mathbf{x}}^{(2)}$ solution to the overdetermined ℓ^2 minimization problem on Ω_2 . Then the solution obeys:

$$\|\mathbf{x} - \widehat{\mathbf{x}}\|_2 \leq \|\mathbf{A}_{\Omega_2}^+ \mathbf{e}^{(2)}\|_2 + \frac{2(1+\rho)}{1-\rho} \sigma_s(\mathbf{x}^{(1)})_1 + \frac{4\tau}{1-\rho} \|\mathbf{e}^{(1)}\|_2. \quad (3.8)$$

Moreover, the total number of measurements amounts to $2m$ for the recovery of an $N_2 + s$ sparse vector.

Recovering the vector $\mathbf{x}^{(2)}$ does not create any problem, as long as the pseudo-inverse has a reasonable ℓ^2 norm, i.e. as soon as the rank of the projection is reasonably small (see discussion below and Theorem 3.1 in particular). The recovery of the vector $\mathbf{x}^{(1)} \in \mathbb{K}^N$ is ensured provided the number of subgaussian measurements m scales as [25, Corollary 9.34]

$$m \gtrsim 2s \ln(en/s) (1 + \rho^{-1})^2. \quad (3.9)$$

Proposition 3.1 slightly reformulated yields the following result:

Corollary 3.1. Let $\mathbf{A} := \mathbf{A}P_1 + \mathbf{A}P_2 \in \mathbb{K}^{m \times N}$ with $\Omega_1 \cap \Omega_2 = \emptyset$ and $\Omega_1 \cup \Omega_2 = \{1, \dots, N\}$ and $N_1 = |\Omega_1|$, $N_2 = |\Omega_2|$. If $\mathbf{A}$ is a subgaussian matrix and $\mathbf{A}_{\Omega_2}$ has full rank, then provided that

$$m \gtrsim 2s \ln(en/s) (1 + \rho^{-1})^2,$$

any vector $\mathbf{x} \in \Theta := \Sigma_s^{N_1} + \mathbb{K}^{N_2} \subset \Sigma_{s+N_2}^N$ can be recovered with the bound (3.8).

Here, $\Sigma_s^{N_1}$ denotes the set of s sparse vectors in $\mathbb{K}^{N_1}$. The previous Corollary provides a uniform recovery result, with a proviso that we restrict the model for the vector $\mathbf{x}$.

3.1.2. Controlled deterministic projections

We look here at a scenario where the sets Ω_i (and hence the projections P_i) are completely and deterministically controlled. We consider an integer $n > 0$ and subsets $\Omega_i \subset \{1, \dots, N\}$ with $|\Omega_i| = N_i$, $\cup_{i=1}^n \Omega_i = \{1, \dots, N\}$ and $N_i \leq m$, and assume again that $\mathbf{A}$ has full rank. In this case, we can always enforce the disjointness of the support $\Omega_i \cap \Omega_j = \emptyset$, for any $i \neq j$. This yields the trivial recovery of the input signal from its local information vectors:

$$\widehat{\mathbf{x}} := \sum_{i=1}^n \widehat{\mathbf{x}}^{(i)}. \quad (3.10)$$

Here, all the local vectors are recovered via generalized Moore-Penrose pseudo inverses of the sub-matrices $\mathbf{A}_{\Omega_i}$ composed of only the columns supported on Ω_i :

$$\widehat{\mathbf{x}}^{(i)} = \mathbf{A}_{\Omega_i}^+ \mathbf{y}^{(i)} = \mathbf{x}^{(i)} + \mathbf{A}_{\Omega_i}^+ \mathbf{e}^{(i)}. \quad (3.11)$$

The error estimate follows directly:

$$\|\mathbf{x} - \widehat{\mathbf{x}}\|_2^2 \leq \sum_{i=1}^n \|\mathbf{A}_{\Omega_i}^+ \mathbf{e}^{(i)}\|_2^2 \quad (3.12)$$

Note that it holds $\|\mathbf{A}^+ \mathbf{e}\|_2 \leq \|\mathbf{A}^+\|_{2 \rightarrow 2} \|\mathbf{e}\|_2$ and that $\|\mathbf{A}^+\|_{2 \rightarrow 2} = 1/\min_{1 \leq i \leq r}(\sigma_i)$, with r the rank of $\mathbf{A}$ and $\{\sigma_i\}_{i=1}^r$ its singular values. As a consequence, when dealing with “nice” matrices, the bound (3.12) is reasonable. In a CS setup, normalized sensing matrices generated at random from a Gaussian distribution have the property of being *well-behaved*, as suggested by [25, Theorem 9.26]:

Theorem 3.1. Let $\tilde{\mathbf{A}}$ be an $m \times s$ Gaussian matrix with $m > s$ and $\mathbf{A} := \frac{1}{\sqrt{m}} \tilde{\mathbf{A}}$ be its variance-normalized counterpart. Then for $t > 0$,

$$\mathbb{P}(\sigma_{\max}(\mathbf{A}) \geq 1 + \sqrt{s/m} + t) \leq e^{-\frac{mt^2}{2}}$$

$$\mathbb{P}(\sigma_{\min}(\mathbf{A}) \leq 1 - \sqrt{s/m} - t) \leq e^{-\frac{mt^2}{2}},$$

where $\sigma_{\max}(\Theta)$ and $\sigma_{\min}(\Theta)$ are the maximum and the minimum singular values of Θ , respectively.

For small projection ranks, and $\mathbf{A}$ as in Theorem 3.1, it holds $s \leq r < m$, and therefore $\mathbb{P}(\|\mathbf{A}_{\Omega_i}^+\|_{2 \rightarrow 2} \geq \frac{1}{1 - \sqrt{\frac{s}{m}} - t}) \leq e^{-mt^2/2}$ which justifies that the bound (3.12) is small.

3.1.3. Rank-controlled projections

This scenario differs from the previous one by the fact that we may control only the ranks N_i 's of the projections but let the support of projections be random. This example is motivated by the SAR applications where the rank of the projections is controlled by the sensing device itself. In this case, for uniform recovery of any vector $\mathbf{x}$ we need to ensure that, with high probability, the whole support $\{1, \dots, N\}$ is covered by the random projections. We assume that all the projections have the same rank r for simplicity of calculations, but similar ideas apply if variations in the rank of the projections were needed. We pick uniformly at random n sets Ω_i of size r in $\{1, \dots, N\}$. It holds

$$\begin{aligned} \mathbb{P}[\exists i \in \{1, \dots, N\} : i \notin \Omega := \cup_{j=1}^n \Omega_j] &= N\mathbb{P}[1 \notin \Omega] \\ &= N(\mathbb{P}[1 \notin \Omega_1])^n = N\left(\frac{N-r}{N}\right)^n. \end{aligned} \quad (3.13)$$

This gives that $\mathbb{P}[\exists i \in \{1, \dots, N\} : i \notin \Omega := \cup_{j=1}^n \Omega_j] \leq \varepsilon$ whenever

$$n \geq \frac{\log(N/\varepsilon)}{\log(N/(N-r))}. \quad (3.14)$$

This direct consequence follows.

Proposition 3.2. Let $\mathbf{A} \in \mathbb{K}^{m \times N}$ with $m < N$, $r \leq m$, $\varepsilon > 0$, and n be such that Eq. (3.14) holds. In addition, assume that every submatrix with r columns extracted from $\mathbf{A}$ has full rank. Let $\Omega_1, \Omega_2, \dots, \Omega_n$ be n subsets in $\{1, \dots, N\}$ of size r chosen independently and uniformly at random. Any vector $\mathbf{x}$ recovered from the fusion of local recoveries as $\widehat{\mathbf{x}} := S^{-1} \sum_{i=1}^n \widehat{\mathbf{x}}^{(i)}$ from the local measurements $\mathbf{y}^{(i)} = \mathbf{A}P_i \mathbf{x} + \mathbf{e}^{(i)}$, $1 \leq i \leq n$ satisfies, with probability at least $1 - \varepsilon$

$$\|\mathbf{x} - \widehat{\mathbf{x}}\|_2^2 \leq \frac{1}{C} \sum_{i=1}^n \|\mathbf{A}_{\Omega_i}^+ \mathbf{e}^{(i)}\|_2^2 \quad (3.15)$$

where $S(\mathbf{x}) := \sum_{i=1}^n P_i \mathbf{x}$ and $\widehat{\mathbf{x}}^{(i)} := \mathbf{A}_{\Omega_i}^+ \mathbf{y}^{(i)}$ and C denotes the lower frame bound of S .

The lower frame bound is defined for general fusion frame operators as in Definition 1. The error bound above is applicable in case of any fusion/filtering scenario. In our particular set-up, we calculate exactly this bound.

Lemma 1. Given n projections P_i of rank r , the lower frame bound of the fusion frame operator $S := \sum P_i$ is given by

$$C := \min_{1 \leq k \leq N} \mathcal{M}(k) =: \mathcal{M}, \quad (3.16)$$

where $\mathcal{M}(k)$ is the multiplicity of index k . Formally, let $\Gamma_k := \{j : 1 \leq j \leq n : k \in \Omega_j\}$, then $\mathcal{M}(k) := |\Gamma_k|$.

Proof. That $C \leq \mathcal{M}$ is clear. Indeed, let $\mathbf{x} = \mathbf{e}_{k_0}$ where k_0 is one of the indices achieving the minimum and $\mathbf{e}_{k_0}$ denotes the canonical vector. Then it holds $S(\mathbf{e}_{k_0}) = \sum_{j \in \Gamma_{k_0}} \mathbf{e}_{k_0}$ and it follows that $\|S(\mathbf{x})\|_2^2 \leq \mathcal{M}^2$.

On the other hand, for any $\mathbf{x} \in \mathbb{K}^N$, the fusion frame operator can be rewritten as $S(\mathbf{x}) = \sum_{i=1}^n \mathcal{M}(i)x_i$. Consequently it holds that for any $\mathbf{x} \in \mathbb{K}^N$, $\|S(\mathbf{x})\|_2^2 \geq \mathcal{M}^2 \sum_{i=1}^N |x_i|^2$ and hence $C^2 \geq \mathcal{M}^2$. $\square$

As a consequence, it is trivial to see that for any $1 \leq k \leq N$, $\mathcal{M}(k)$ is a non-decreasing function of the number of projections n . Hence, the greater the number of projections (the redundancy, in fusion frame terms) the greater the bound C and therefore the smaller the error in the recovery, according to Eq. (3.15). This however can only hold true, as long as the noise per measurement vectors remains small.

In the case of subsets selected independently uniformly at random, we can have a precise statement. The lower frame bound is defined as the minimum number of occurrences of any index

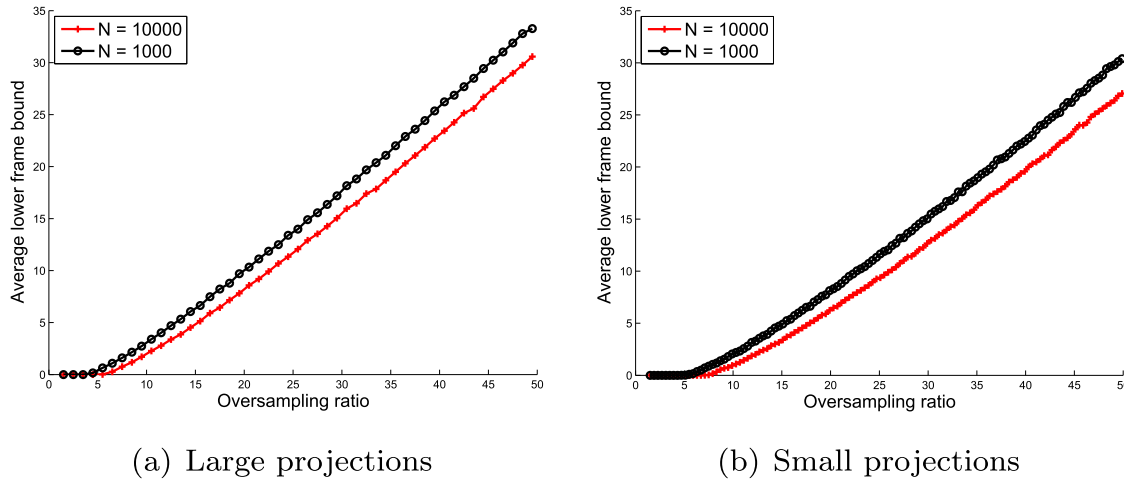


Fig. 3. Linear growth of the expectation of the lower frame bound for various use cases. The averages are calculated from 300 random experiments. The left graphs show the average lower frame bound when dealing with rather large projections (with rank $r = N/2$) while the right graphs are in the case of much smaller projections ($r = N/5$). The oversampling ratio is calculated as a linear function of the number of projections and is independent of the ambient dimension N ; $\zeta = m/N$.

$k \in \{1, \dots, N\}$. Let us consider $\mathcal{M}(k)$ and Γ_k as in the previous lemma. Let $k \in \{1, \dots, N\}$ be any index and Ω_j denote a draw of a random set. It holds $\mathbb{P}[k \in \Omega_j] = r/N$. The subsets being independent of each other, $\mathcal{M}(k)$ is a binomial random variable with probability of success $P = r/N$ for each of the n trials. It follows that $\mathbb{P}[\mathcal{M}(k) = l] = \binom{n}{l} P^l (1-P)^{n-l}$ for all $1 \leq k \leq N$ and $1 \leq l \leq n$.

Putting everything together, we get that $C := \min_{k \in \{1, \dots, N\}} \mathcal{M}(k)$ is a random variable such that

$$\mathbb{P}[C \geq l] = \mathbb{P}[\forall 1 \leq k \leq N, \mathcal{M}(k) \geq l] \quad (3.17)$$

$$\begin{aligned} &= (\mathbb{P}[\mathcal{M}(k) \geq l])^N = \left(1 - \sum_{j=1}^{l-1} \mathbb{P}[\mathcal{M}(k) = j]\right)^N \\ &= (1 - F(l-1, n, P))^N. \end{aligned} \quad (3.18)$$

These expressions resemble the calculations involved in Eqs. (3.13) and (3.14), however with a much more complicated probability distribution. To avoid unnecessarily tedious calculations that would infer the readability of the paper, we choose to leave the following result as a conjecture.

Conjecture 1. Let $\Omega_1, \Omega_2, \dots, \Omega_n$ be n subsets of r elements taken uniformly at random in $\{1, \dots, N\}$. Let $\Gamma_k := \{j: 1 \leq j \leq n: k \in \Omega_j\}$, and $\mathcal{M}(k) := |\Gamma_k|$. Then $C := \min_{1 \leq k \leq N} \mathcal{M}(k) =: \mathcal{M}$ grows at least linearly with n , i.e., there exists constants $c > 0$ and $b \geq 0$, independent of n , such that

$$\mathbb{E}[C] \geq cn - b. \quad (3.19)$$

A direct consequence of this is that one can always find a multiplicative constant $c' \leq c$ such that for a number of projections $n \geq b/(c - c')$, $\mathbb{E}[C] \geq c'n$ (the case $c = c'$ can only be true for $b = 0$, in which case, there is no need for this remark). Though the proof is not given, the result is backed up with numerical results shown in Fig. 3.³ The graphs always show the expected linear growth, independently of the parameters of the problem.

Assuming the conjecture to be true, Eq. (3.15) then simplifies to

$$\mathbb{E} \|\hat{\mathbf{x}} - \mathbf{x}\|_2^2 \leq \frac{1}{c'n} \sum_{i=1}^n \|\mathbf{A}_{\Omega_i}^+ \mathbf{e}^{(i)}\|_2^2 \leq \frac{\delta \nu}{c'}, \quad (3.20)$$

³ It is also backed up with particular cases, when the ranks are exactly half the dimension, see <http://math.stackexchange.com/questions/1135253/mean-value-of-minimum-of-binomial-variables>

where the expectation is taken over the draw of the projections, for a certain constant $c' > 0$, where we let $\delta = \max_i \|\mathbf{A}_{\Omega_i}^+\|_{2 \rightarrow 2}$ and $\nu := \max_i \|\mathbf{e}^{(i)}\|_2^2$. It shows that in expectation, the error should not grow as the number of measurements increases. Moreover, numerical results illustrated in Fig. 5 suggest that the expected reconstruction error decreases. An intuition for this behavior is provided in a simple example in Eqs. (3.24) and (3.25). From a compressed sensing perspective, it has been shown that the least favorable case appears as the case when the vectors are repeated again and again. This translates in our scenario to the case where the increase of the measurements consist solely in repeating the same projections (which would not produce a fusion frame). Considering the example of a SAR imaging process developed in Section 1.1.2, this would correspond to a plane flying over a given region multiple times, with the spatial filtering being exactly the same.

Remark 3.3. It is important to note here that there is no sparsity assumption. It is possible to recover **any** vector $\mathbf{x}$ with high probability, as long as the number of measurement vector scales reasonably with the dimension of the input space.

3.1.4. Constrained number of measurements and ranks of projections

In certain scenarios, the physical measurement devices limit our freedom in choosing the ranks and size of the measurements. We hence deal here with fixed ranks N_i 's for the projections P_i . Assume, without loss of generality, that the first $0 \leq k \leq n$ projections have ranks lower than the number of measurements m . For these subspaces, the usual ℓ^2 minimization procedure yields perfect (or optimal in the noisy case) recovery of the local information $\mathbf{x}^{(i)}$, for $1 \leq i \leq k$. For the remaining $n - k$ subspaces, the pseudo-inverse is not sufficient, and we use tools from CS. Here again we use the partial robust null space property from Definition 2 on every subspace Ω_j , for $k + 1 \leq j \leq n$. Combining results from Propositions 3.2 and 3.1 yields the following corollary as a consequence:

Corollary 3.2. Let $m < N$ and $A \in \mathbb{K}^{m \times N}$. Assume that we are given a set of n sets $\{\Omega_i\}_{i=1}^n$ in $\{1, \dots, N\}$ of respective sizes $\{N_i\}_{i=1}^n$ with $N_1 \leq N_2 \leq \dots \leq N_n$. Assume in addition that $\cup_i \Omega_i = \{1, \dots, N\}$. Moreover, there exists a unique $0 \leq k \leq n$ such that $N_k \leq m$ and $N_{k+1} > m$ (with the convention that $N_0 = 0$ and $N_{n+1} > m$). Assume that the submatrices $\mathbf{A}_{\Omega_i}$, for $1 \leq i \leq k$, have full rank. If in addition the measurement matrix satisfies a partial robust null space property of order s with respect to every subset Ω_i , $i \geq k + 1$, then, the approximation

$\widehat{\mathbf{x}}$ defined as $\widehat{\mathbf{x}} := S^{-1} \sum_{i=1}^n \widehat{\mathbf{x}}^{(i)}$ with $\widehat{\mathbf{x}}^{(i)} := \mathbf{A}_{\Omega_i}^+ \mathbf{y}^{(i)}$, for $1 \leq i \leq k$ and $\widehat{\mathbf{x}}^{(i)}$ solutions to the ℓ^2 constrained ℓ^1 minimization problems BPDN, for $k+1 \leq i \leq n$ fulfills the following estimate:

$$\|\mathbf{x} - \widehat{\mathbf{x}}\|_2 \leq \frac{1}{\mathcal{M}} \left(\sum_{i=1}^k \|\mathbf{A}_{\Omega_i}^+ \mathbf{e}^{(i)}\|_2 + \frac{2(1+\rho)}{1-\rho} \sum_{i=k+1}^n \sigma_s(\mathbf{x}^{(i)})_1 + \frac{4\tau}{1-\rho} \sum_{i=k+1}^n \|\mathbf{e}^{(i)}\|_2 \right) \quad (3.21)$$

For simplicity we have assumed that the NSP is valid on every subspaces for the same set of parameters. To avoid notational encumbrance we do not write results where the ρ , τ , and s may depend on the subspace considered.

Remark 3.4. This is a direct application of the previous results. It is obtained by recovering every local pieces of information independently from one another. It is possible to improve these estimates by a sequential procedure where we first estimate the x_j for $j \in \Omega_i$, for some $1 \leq i \leq k$, and then using this reliable estimate to improve the accuracy of the ℓ^1 minimization program.

A final remark considers the case where the rank of the projection is also random. We can think of the sets Ω_i , for $1 \leq i \leq n$ as Binomial random variables with probability of success p . In this case, the rank of the projection P_i is controlled by the expectation of this random variable. An analysis similar to the previous one can be carried over to ensure recovery of any vector with high probability.

3.2. Numerical examples

This Section describes numerical recovery results on the case described above. We first show some particular examples of recovery, when dealing with dense signals and show that we can break the traditional sparsity limit by adding a few sensors. The next example shows the behavior of the quality of the reconstruction when slowly adding sensors. Finally, the last section validates our Conjecture and show that the noise tends to decrease while adding projections, and that the overall quality of recovery scales linearly with the noise level. In another contribution, we have also verified that we can recover a very dense Fourier spectrum, by using our approach with Fourier measurement matrices, see [1].

3.2.1. Examples of recovery

The examples of reconstructions depicted on Fig. 4 were created by generating s -sparse Gaussian vectors at random with $s = 200$ for Fig. 4(a) and 4(b), and $s = 500$ for Fig. 4(c). For each case, the 250×600 matrix was generated at random with entries i.i.d. from a Gaussian distribution. In the first example, the rank is set

to $r = 300 > m$ and hence, ℓ^1 minimization (traditional basis pursuit) is used on every subspaces. This yields a total of 13 projections generated at random. For the two last examples, the ranks of the projections are set to $r = 200 < m$. In this case, only a classical ℓ^2 inversion is needed on every subspace. Here, since the ranks are smaller, the number of projections has to be increased to 22 in order to ensure that the whole set $\{1, \dots, N\}$ is covered with high probability. By looking carefully at the last example, one can see that an index with non-zero magnitude has not been selected (around index 520). These cases, however, are rare. In every figure, the red '+' crosses represent the true signal, the blue circles represent the reconstructed signal from our fusion approach, and the green 'x' correspond to the reconstruction with traditional basis pursuit. While the very last set-up (very high number of non-zero components) is clearly not suitable for usual compressed sensing, the advantage of projections and fusions can be seen even in the first two (where the number of non-zeros remains relatively small). Another aspect to look at is that when dealing with small ranks, the solutions to the ℓ^2 minimization problems are computed efficiently. As a consequence, even when the number of projections increases, the calculation of the recovered $\widehat{\mathbf{x}}$ is still orders of magnitude faster.

3.2.2. Recovery and number of projections

Given any scenario introduced above, it is expected from Conjecture 1 that increasing the number of projections (of a given rank) will increase the lower frame bound (3.16) at least linearly. This linear increase of the lower frame bound compensates for the potential increase in the reconstruction error. One could however hope for better results according to the numerical evidence illustrated in Fig. 5, where the average case error seems to be reduced as the number of projections increases. Given a set of n projections generating a fusion frame, the least favorable case, studied in [22] for the case of a single filtering operation, when adding an extra n projections, is when the exact same Ω_i are repeated. The exact behavior of the reconstruction error with respect to the number of projections is still under research.

For a total of 10 generations of random Gaussian matrices of size 300×1000 , and 10 random Gaussian 200 sparse vectors (note: that is a setting where usual compressed sensing has no recovery guarantees), we let the number of random projections of rank $r = 500$ go from 60% up to 5 times the minimum given by Eq. (3.14) (17 in our setting, for a success of covering the whole input set of $\varepsilon = 99\%$). As can be seen from the figure, the recovery is unlikely to be correct as long as the number of projections remains small. The trend is generally towards the perfect recovery - though this can only be guarantee with high accuracy, which explains the little spikes. Moreover, while the maximum might be somewhat bigger, it appears that 1) it remains within reasonable bounds (depending on the application) and 2) the average of the error among

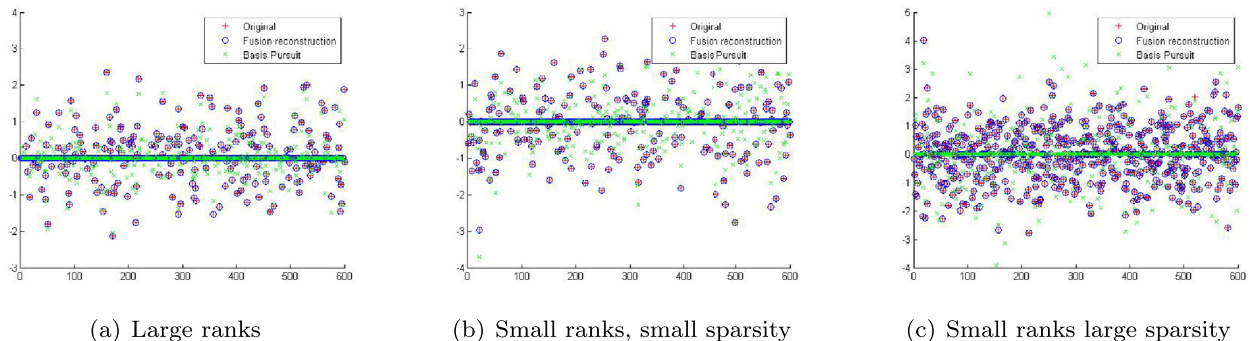


Fig. 4. Examples of fusion reconstruction.

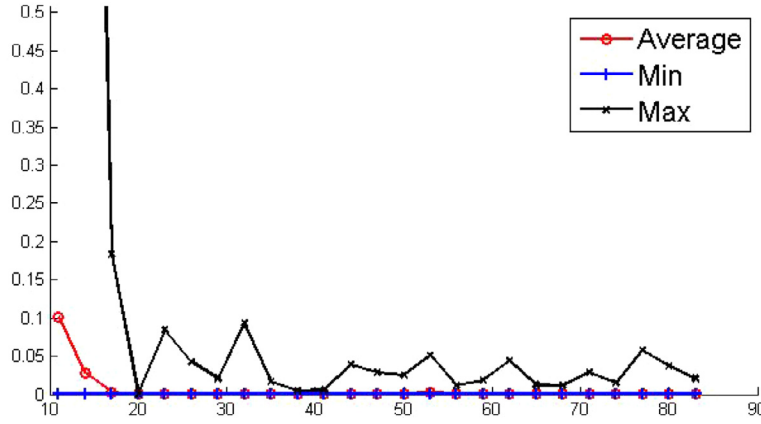
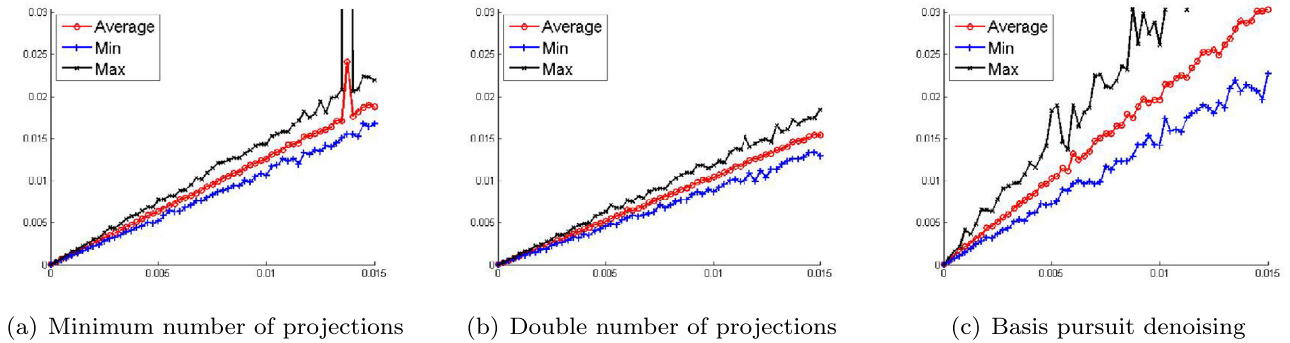
Fig. 5. ℓ^2 error of recovery for various number of projections.

Fig. 6. Behavior facing noise.

the 100 tests per number of projections is very small, suggesting that only a few of the recoveries are off. A way to understand this is to consider the case, where we have two projections that are slightly overlapping. Given the global set $\Omega = \{1, \dots, N\}$, assume we split it into Ω_1 and Ω_2 with some overlap, i.e. $\Omega = \Omega_1 \cup \Omega_2$ and $\Omega_{12} := \Omega_1 \cap \Omega_2 \neq \emptyset$. We independently solve

$$\widehat{\mathbf{x}}^{(i)} := \operatorname{argmin} \|\mathbf{x}\|_1 \quad (3.22)$$

$$\text{s.t. } \|\mathbf{A}_i \mathbf{x} - \mathbf{y}^{(i)}\|_2 \leq \eta_i \quad (3.23)$$

for $1 \leq i \leq 2$. For now we consider only the noiseless case, i.e. $\varepsilon_1 = \varepsilon_2 = 0$ which is equivalent to solving two basis pursuit optimization problems. We can decompose the solutions recovered as $\widehat{\mathbf{x}}^{(i)} = \widehat{\mathbf{x}}^{(i)}|_{\Omega_i \setminus \Omega_{12}} + \widehat{\mathbf{x}}^{(i)}|_{\Omega_{12}}$. If we assume, for simplicity, that the first component is recovered exactly, we have that $\widehat{\mathbf{x}}^{(1)} = \mathbf{x}^{(1)}$. If however the second component is not recovered accurately, we can write $\widehat{\mathbf{x}}^{(2)} = \mathbf{x}^{(2)} + \mathbf{e}^{(2)}$ (for further generalizations, we can always write $\widehat{\mathbf{x}}^{(i)} = \mathbf{x}^{(i)} + \mathbf{e}^{(i)}$, for $1 \leq i \leq n$ with $\mathbf{e}^{(i)} = 0$ in case of successful recovery on the set Ω_i). In this very simple example, we have $S(\mathbf{x}) = \sum_{i=1}^n \widehat{\mathbf{x}}^{(i)} = \mathbf{x}^{(1)} + \mathbf{x}^{(2)}$. Similarly, the inverse fusion frame operator can be seen as the point wise empirical average of the evidences (i.e. the local recoveries):

$$S^{-1}(\mathbf{x}) := \mathbf{x}^{(1)}|_{\Omega_1 \setminus \Omega_{12}} + \mathbf{x}^{(2)}|_{\Omega_2 \setminus \Omega_{12}} + \frac{\mathbf{x}^{(1)}|_{\Omega_{12}} + \mathbf{x}^{(2)}|_{\Omega_{12}}}{2}. \quad (3.24)$$

Note that the 2 in the denominator denotes the number of times that the indices in Ω_{12} are picked in the creation of the (random) projections. In other words, increasing the number of projections increases potentially the denominator under a given index

and hence reduces the error.

$$\widehat{\mathbf{x}} - \mathbf{x} = S^{-1}(\widehat{\mathbf{x}}^{(1)} + \widehat{\mathbf{x}}^{(2)} - \mathbf{x}^{(1)} - \mathbf{x}^{(2)}) = \mathbf{e}^{(2)}|_{\Omega_2 \setminus \Omega_{12}} + \frac{\mathbf{e}^{(2)}|_{\Omega_{12}}}{2} \quad (3.25)$$

As a consequence, the error (on the set Ω_{12}) is bounded by the maximum of the error from every recovery problem divided, as it corresponds to the average error on the domain.

3.2.3. Robustness to noise

In the set of experiments illustrated in Fig. 6, we have compared the robustness of our algorithm to traditional basis pursuit denoising when dealing with noise. We have set the parameters in a regime where BPDN works for fair comparison. For a 500 dimensional ambient space and 300 dimensional measurement space, we generated 5 random 80-sparse Gaussian vectors for each of the 10 randomly generated Gaussian matrices. For every single test, some additive Gaussian noise with a given ℓ^2 norm θ has been added. This means that for the case of the distributed approach, n independent noise components of norm θ have been added in total; one for each channel. The figures depict the evolution of the error of the ℓ^2 norm when the energy of the noise component increases. The black line represents the maximum of the error, the blue one the minimum, and the red one the average over every samples. The left graph shows the result when the number of projections is exactly set as in Eq. (3.14) while the second one doubles this number. The third figure shows the result when using the usual Basis Pursuit Denoising. It is important to notice that the higher peak appearing on the first figure is due to an index with non zero component from the original vector $\mathbf{x}$ not being selected at all during the random projections. This however does

not contradict the high probability of recovery. The improvement in the noise behavior from the first figure to the second shows how the fusion frame operator tends to average out the local errors to yield a better estimate (as described in the previous section). Finally, all of the algorithms scale linearly with the norm of the noise per measurements (as suggested by Eq. (3.15)) and even if the total noise is increased as the number of projections is increased, the recovery tends to be improved by considering more projections.

4. Compressed sensing theory with fusion frames

Our results so far ensure that robust and stable recovery of dense signals is possible, by smartly combining local information. In this section we show that the ideas developed in the previous Section can also be considered as an extension of the traditional CS. In particular, it gives solid mathematical foundations to our work. It is important to note that similar ideas have been developed in parallel in [18]. There, the authors introduced a model similar to (2.9), albeit asking that the fusion frame be tight with $C = D = 1$, but reconstruct the whole signal globally, without the use of the fusion process. Other authors [10] looked at CS with structured acquisition and structured sparsity. In simple words, they prove that adapting the sampling matrices to some prior knowledge of the sparsity pattern allowed for larger applicability of the CS framework.

In this section we describe the recovery of a signal $\mathbf{x}$ by means of CS in the local subspaces and fusion processing. The local pieces of information are computed as solutions to the problems

$$\begin{aligned} \min_{\mathbf{z} \in \mathbb{K}^N} \quad & \|\mathbf{z}\|_1 \\ \text{s.t.} \quad & \|\mathbf{A}_i \mathbf{z} - \mathbf{y}^{(i)}\|_2 \leq \eta_i, \quad \mathcal{P}_{1,\eta} \end{aligned}$$

In the noiseless case, the problem is solved by the basis pursuit

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{K}^N} \quad & \|\mathbf{z}\|_1 \\ \text{s.t.} \quad & \mathbf{A}_i \mathbf{z} = \mathbf{y}^{(i)}, \quad \mathcal{P}_{1,0} \end{aligned}$$

Lemma 2. Let $\widehat{\mathbf{z}}^{(i)}$ be a solution to the noisy $\mathcal{P}_{1,\eta}$ or noiseless $\mathcal{P}_{1,0}$ basis pursuit problem. Then $\widehat{\mathbf{z}}^{(i)} \in W_i$.

Proof. Let $\widehat{\mathbf{z}}^{(i)}$ be a solution and let $\tilde{\mathbf{z}} = P_i \widehat{\mathbf{z}}^{(i)}$. Then $\|\tilde{\mathbf{z}}\|_1 \leq \|\widehat{\mathbf{z}}^{(i)}\|_1 + \|(I - P_i)(\widehat{\mathbf{z}}^{(i)})\|_1$. From the optimality of $\widehat{\mathbf{z}}^{(i)}$ and the admissibility of $\tilde{\mathbf{z}}$, it follows that $\|\tilde{\mathbf{z}}\|_1 = \|\widehat{\mathbf{z}}^{(i)}\|_1$ and $\widehat{\mathbf{z}}^{(i)} = P_i(\widehat{\mathbf{z}}^{(i)})$. $\square$

4.1. Signal models and recovery conditions

4.1.1. Extension of the sparsity model

The traditional sparsity model is not appropriate in this setting. As an example, let us consider that all the s nonzero components of a vector $\mathbf{x}$ fall within a certain subspace (say W_1), there is, a priori, no hope to improve the recovery performance compared to a single sensor/subspace problem. Indeed, in this case, the recovery is ensured (locally) by CS methods if the number of observations m scales as

$$m \asymp s \log(N/s).$$

Since we are dealing here only with an identical sensing model, this yields a total number of observations m_T scaling as

$$m_T \asymp ns \log(N/s).$$

This may be acceptable if we consider only very few subspaces but may explode in certain cases. Therefore the model of distributed sparsity is more appropriate.

Definition 3. A signal $\mathbf{x} \in \mathbb{K}^N$ is said to be $\mathbf{s} = (s_1, \dots, s_n)$ -distributed sparse with respect to a fusion frame $\mathcal{W} = (W_i, P_i)_{i=1}^n$, if $\|P_i(\mathbf{x})\|_0 \leq s_i$, for every $1 \leq i \leq n$. $\mathbf{s}$ is called the *sparsity pattern* of $\mathbf{x}$ with respect to $\mathcal{W}$.

We denote by $\Sigma_{\mathbf{s}}^{(\mathcal{W})}$ the set of all $\mathbf{s}$ -distributed sparse vectors with respect to the family of subspaces $(W_i)_i$. We let $s = \|\mathbf{s}\|_1$ denote the global sparsity of the vector, with respect to $\mathcal{W}$. In the case that the sparsity of the signal is uniformly distributed among the subspaces ($s_i = s/n$), the usual CS recovery guarantees ensure us that

$$m \asymp s_i \log(N/s_i)$$

observations per subspace are required for a stable and robust recovery of the signal. This accounts for a total number of measurements scaling as $m_T \asymp ns_i \log(N/s_i) = s \log(N/s_i)$. In other words, we are able to recover similar sparsities as in the classical CS framework, while using only *cheap* sensors. But in opposition to the classical theory most of the computations can be easily carried in a distributed setting, where only pieces of the information are available. Only the fusion process requires all the local information to compute the final estimation of a signal.

Locally, it only requires solving some very small CS system, which can be done faster than solving the original one. This is also the findings found in parallel in [18], where it is concluded that the number of measurements per sensor decreases linearly with the number of sensor. We describe a similar problem, while looking at it from a different perspective. In particular, we try to find the sparsity patterns that may be recovered for a given sensor design. The motivation for this problem comes from the applications in SAR imaging where the sensor is given and the same everywhere, and where we may not have any control on the number of observations per subspace. As it will become useful later, we also need to introduce the local best approximations.

Definition 4. Let $\mathcal{W} = (W_i, P_i)_{i=1}^n$ be a fusion frame, and let $\mathbf{x} \in \mathbb{K}^N$. For $p > 0$ and a sparsity pattern $\mathbf{s} = (s_1, \dots, s_n)$ with $s_i \in \mathbb{N}$ for all $1 \leq i \leq n$, the ℓ^p errors of best $\mathbf{s}$ -term approximations are defined as the vector

$$\sigma_{\mathbf{s}}^{\mathcal{W}}(\mathbf{x})_p := (\sigma_{s_1}(P_1 \mathbf{x}), \sigma_{s_2}(P_2 \mathbf{x}), \dots, \sigma_{s_n}(P_n \mathbf{x}))^T$$

4.1.2. Partial properties

The *null space property* (NSP) has been used throughout the past decade in the CS literature as a necessary and sufficient condition for the sparse recovery problem via ℓ^0 -min. A matrix $\mathbf{A}$ is said to satisfy the (robust) null space property with parameters $\rho \in (0, 1)$ and $\tau > 0$ relative to a set $S \subset \{1, \dots, N\}$ if

$$\|\mathbf{v}_S\|_1 \leq \rho \|\mathbf{v}_{S^c}\|_1 + \tau \|\mathbf{A} \mathbf{v}\|_2, \quad \text{for all } \mathbf{v} \in \mathbb{K}^N.$$

More generally, we say that the matrix $\mathbf{A}$ satisfies the NSP of order s if it satisfies the NSP relative to all sets S such that $|S| \leq s$. We extend here this idea to the context of distributed sparsity with respect to fusion frames, as already mentioned in Definition 2. Here we talk about a sparsity pattern and ask that the NSP property be valid for all local subspaces up to a certain (local) sparsity level.

Definition 5 (Robust and stable partial null space property (RP-NSP)). Let n be an integer and $\mathcal{W} = (W_i, P_i)_{i=1}^n$ be a fusion frame for $\mathbb{K}^N$. Let $\mathbf{s} = (s_1, \dots, s_n)$ be a sequence of non negative numbers representing the sparsity pattern with respect to $\mathcal{W}$. For a number $q \geq 1$, a sensing matrix $\mathbf{A} \in \mathbb{K}^{m \times N}$ is said to satisfy the ℓ^q -RP-NSP with pattern $\mathbf{s}$ with respect to $\mathcal{W}$ and with constants $\rho_1, \dots, \rho_n \in (0, 1)$ and $\tau_1, \dots, \tau_n > 0$ if

$$\|(P_i \mathbf{v})_{S_i}\|_q \leq \frac{\rho_i}{s_i^{1-1/q}} \|(P_i \mathbf{v})_{S_i^c}\|_1 + \tau_i \|\mathbf{A} \mathbf{v}\|_2,$$

$$\text{for all } \mathbf{v} \in \mathbb{K}^N, 1 \leq i \leq n, S_i \subset W_i, \text{ and } |S_i| \leq s_i.$$

This definition is reminiscent of the work on sparse recovery with partially known support [4]. The difference here being that there is no need to enforce a condition on the vector $\mathbf{v}$ to lie in the range of the other subspaces. In a sense, this is taken care of by the fusion process and the fact that we have multiple measurement vectors.

Remark 4.1. Note that we could simplify the definition by asking that the parameters be uniform and independent of the local subspace. Namely, introducing $\tau := \max_{1 \leq i \leq n} \tau_i$ and $\rho := \max_{1 \leq i \leq n} \rho_i$, the above definition becomes

$$\left\| (P_{W_i} \mathbf{v})_{S_i} \right\|_q \leq \frac{\rho}{s^{1-1/q}} \left\| (P_{W_i} \mathbf{v})_{\bar{S}_i} \right\|_1 + \tau \|\mathbf{A} \mathbf{v}\|_2,$$

for all $\mathbf{v} \in \mathbb{K}^N$, $1 \leq i \leq n$, $S_i \subset W_i$, and $|S_i| \leq s_i$.

A stronger, but easier to verify, condition often used as a sufficient recovery condition is the by-now well known *Restricted Isometry Property* (RIP). Informally speaking a matrix is said to satisfy the $\text{RIP}(s, \delta)$ if it behaves almost like an isometry (up to a constant δ) on every s -sparse vector $\mathbf{v} \in \Sigma_s$. Formally speaking, $\mathbf{A} \in \mathbb{K}^{m \times N}$ satisfies $\text{RIP}(s, \delta)$, for some $s \geq 2$ and $\delta \in (0, 1)$ if

$$(1 - \delta) \|\mathbf{v}\|_2^2 \leq \|\mathbf{A} \mathbf{v}\|_2^2 \leq (1 + \delta) \|\mathbf{v}\|_2^2, \quad \text{for every } \mathbf{v} \in \Sigma_s. \quad (4.1)$$

The lowest δ satisfying the inequalities is called the *restricted isometry constant*. Once again, we want to derive similar properties on our sensing matrix for the distributed sparse signal model.

Definition 6 (Partial-RIP (P-RIP)). Let $\mathcal{W} = (W_i, P_i)_{i=1}^n$ be a fusion frame, and let $\mathbf{A} \in \mathbb{K}^{m \times N}$. Assume that $\mathbf{A}$ satisfies the $\text{RIP}(s_i, \delta_i)$ on W_i , with $\delta_i \in (0, 1)$, $i \in I = \{1, \dots, n\}$. Then, we say that $\mathbf{A} \in \mathbb{K}^{m \times N}$ satisfies the partial RIP with respect to $\mathcal{W}$, with bounds $\delta_1, \dots, \delta_n$ and sparsity pattern $\mathbf{s} = (s_1, \dots, s_n)$.

In other words, $\mathbf{A}$ satisfies a P-RIP conditions, if it satisfies RIP-like conditions on every subset of vectors in $\text{range}(P_i)$.

Remark 4.2. This definition is consistent with the definition of the classical RIP in the sense that the case $n = 1$ (only one projection, one subspace) recovers the usual RIP.

The P-RIP can be written in a form similar to the traditional RIP, Eq. (4.1).

Proposition 4.1. Let $\mathcal{W} = (W_i, P_i)_{i \in I}$ be a fusion frame (with frame bounds $0 < C \leq D < \infty$). Let $\mathbf{A} \in \mathbb{K}^{m \times N}$ satisfy the P-RIP with respect to $\mathcal{W}$, with bounds $\delta_1, \dots, \delta_n$ and sparsity pattern $\mathbf{s} = (s_1, \dots, s_n)$, and let $C_0 = C \min_i \{1 - \delta_i\}$, $D_0 = D \max_i \{1 + \delta_i\}$. Then, for any $\mathbf{v} \in \mathbb{K}^N$,

$$C_0 \|\mathbf{v}\|_2^2 \leq \sum_i \|\mathbf{A} \mathbf{v}_i\|_2^2 \leq D_0 \|\mathbf{v}\|_2^2.$$

Proof. Using the fusion frame inequality, and inequalities (4.1) for all $i \in I$, we obtain

$$\begin{aligned} C \min_i \{1 - \delta_i\} \|\mathbf{v}\|_2^2 &\leq \min_i \{1 - \delta_i\} \sum_i \|\mathbf{v}_i\|_2^2 \\ &\leq \sum_i (1 - \delta_i) \|\mathbf{v}_i\|_2^2 \leq \sum_i \|\mathbf{A} \mathbf{v}_i\|_2^2 \\ &\leq \sum_i (1 + \delta_i) \|\mathbf{v}_i\|_2^2 \leq \max_i \{1 + \delta_i\} \sum_i \|\mathbf{v}_i\|_2^2 \\ &\leq D \max_i \{1 + \delta_i\} \|\mathbf{v}\|_2^2. \end{aligned}$$

□

Theorem 4.1. Let $\varepsilon > 0$. Let $\mathcal{W} = (W_i, P_i)_{i=1}^n$ be a fusion frame for $\mathbb{K}^N$, $N \geq 1$. Let $\mathbf{A} \in \mathbb{K}^{m \times N}$ be a subgaussian matrix with parameters β, k . Then, there exists a constant $C = C_{\beta, k}$ such that the P-RIP constants

of $\frac{1}{\sqrt{m}} \mathbf{A}$ satisfy $\delta_{s_i} \leq \delta_i$, for $1 \leq i \leq n$ with probability at least $1 - \varepsilon$, provided

$$m \geq C \min(\delta)^{-2} (\max(s_i) \ln(eN / \max(s_i)) + \ln(2\varepsilon n))$$

Before we prove this result, we recall a standard RIP result for subgaussian matrices (Theorem 9.2 in [25]):

Theorem 4.2. Let $\mathbf{A}$ be an $m \times N$ subgaussian random matrix. Then there exists a constant $C > 0$ (depending only on subgaussian parameters β, k) such that the RIP constant of $\frac{1}{\sqrt{m}} \mathbf{A}$ satisfies $\delta_s \leq \delta$ with probability at least $1 - \varepsilon$, if

$$m \geq C \delta^{-2} (s \ln(eN/s) + \ln(2\varepsilon^{-1})).$$

Proof. (of Theorem 4.1)

For some constants $\delta_i \in (0, 1)$, let E be the event “ $\frac{1}{\sqrt{m}} \mathbf{A}$ does not satisfy P-RIP with respect to $\mathcal{W}$ with constants $\delta_1, \dots, \delta_n$ ”. Applying a union bound it follows that

$$\mathbb{P}(E) = \mathbb{P}(\exists i \in \{1, \dots, n\} : \delta_{s_i} > \delta_i) \leq \sum_{i=1}^n \mathbb{P}(\delta_{s_i} > \delta_i) = \sum_{i=1}^n \varepsilon_i$$

For some ε_i such that $\varepsilon_1 + \dots + \varepsilon_n = \varepsilon \in (0, 1)$, since the sensing matrix is the same for every sensors, there exists a unique $C > 0$ (depending on β, k) such that $\frac{1}{\sqrt{m}} \mathbf{A}$ satisfies the RIP locally within the subset W_i and for a sparsity s_i with constant $\delta_{s_i} \leq \delta_i$, provided that $m \geq \max_{1 \leq i \leq n} m_i$, with

$$m_i \geq C \delta_i^{-2} (s_i \ln(eN/s_i) + \ln(2\varepsilon_i^{-1})).$$

Additionally, since the function $s \rightarrow s \ln(eN/s)$ is monotonically increasing on $(0, N)$ and $s \ll N$, $\max_{1 \leq i \leq n} m_i \geq C \max_{1 \leq i \leq n} \delta_i^{-2} (\max_{1 \leq i \leq n} s_i \ln(eN / \max_{1 \leq i \leq n} s_i) + \ln(2\varepsilon_i^{-1}))$. Considering $\varepsilon_1 = \dots = \varepsilon_n = \varepsilon/n$ concludes the proof. □

Note: all Gaussian and Bernoulli random matrices are subgaussian random matrices, so Theorem 4.1 holds true for Gaussian and Bernoulli random matrices.

4.2. Recovery in general fusion frames settings

With the tools introduced above, we show that any signals with sparsity pattern $\mathbf{s}$ can be recovered in a stable and robust manner via the fusion frame approach described in the previous sections.

4.2.1. RP-NSP based results

Our first recovery guarantee results generalizes Thm. 3.9 from [1] and is based on the robust partial NSP, introduced in Definition 5.

Theorem 4.3. Let $\mathbf{A} \in \mathbb{K}^{m \times N}$ and $\mathcal{W} = (W_i, P_i)_{i=1}^n$ a fusion frame with frame bounds $0 < C \leq D < \infty$ and frame operator S . Let $(\mathbf{y}^{(i)})_{i=1}^n$ be the linear measurements $\mathbf{y}^{(i)} = \mathbf{A} P_i \mathbf{x} + \mathbf{e}^{(i)}$, $1 \leq i \leq n$ for some noise vectors $\mathbf{e}^{(i)}$ such that $\|\mathbf{e}^{(i)}\|_2 \leq \eta_i$. Denote by $\widehat{\mathbf{x}}^{(i)}$ the solution to the local Basis Pursuit problems $\mathcal{P}_{1, \eta}$ and let $\widehat{\mathbf{x}} = S^{-1} \sum_i \widehat{\mathbf{x}}^{(i)}$. If the matrix $\mathbf{A}$ satisfies the ℓ^1 -RP-NSP with sparsity pattern $\mathbf{s} = (s_1, \dots, s_n)$ with constants $0 < \rho_1, \dots, \rho_n < 1$ and $\tau_1, \dots, \tau_n > 0$ with respect to $\mathcal{W}$, then the estimation $\widehat{\mathbf{x}}$ approximates $\mathbf{x}$ in the following sense:

$$\|\widehat{\mathbf{x}} - \mathbf{x}\|_2 \leq \frac{2}{C} (\langle \vec{\rho}, \sigma_s^{\mathcal{W}}(\mathbf{x})_1 \rangle + \langle \vec{\tau}, \vec{\eta} \rangle), \quad (4.2)$$

where $\vec{\rho} = \left(\frac{1+\rho_i}{1-\rho_i} \right)_{i=1}^n$, $\vec{\tau} = \left(\frac{2\tau_i}{1-\rho_i} \right)_{i=1}^n$, and $\vec{\eta} = (\eta_i)_{i=1}^n$.

Proof. The solution is given by the fusion process $\widehat{\mathbf{x}} = S^{-1} \left(\sum_{i=1}^n \widehat{\mathbf{x}}^{(i)} \right)$ with $\widehat{\mathbf{x}}^{(i)}$ the solutions to the local problems $\mathcal{P}_{1, \eta}$.

It holds

$$\begin{aligned}\|\mathbf{x} - \widehat{\mathbf{x}}\|_2 &= \left\| S^{-1} \left(\sum_{i=1}^n P_i \mathbf{x} - \sum_{i=1}^n \widehat{\mathbf{x}}^{(i)} \right) \right\|_2 \\ &\leq C^{-1} \sum_{i=1}^n \|P_i \mathbf{x} - \widehat{\mathbf{x}}^{(i)}\|_2 \leq C^{-1} \sum_{i=1}^n \|P_i \mathbf{x} - \widehat{\mathbf{x}}^{(i)}\|_1.\end{aligned}$$

For a particular $i \in \{1, \dots, n\}$, we estimate the error on the subspace W_i in the ℓ^1 sense. We follow the proof techniques from [25, Section 4.3] with the adequate changes. With $\mathbf{v} := P_i \mathbf{x} - \widehat{\mathbf{x}}^{(i)}$ and $S_i \subset W_i$ the set of best s_i components of $\mathbf{x}$ supported on W_i , the ℓ^1 -RP-NSP yields

$$\|(P_i \mathbf{v})_{S_i}\|_1 \leq \rho_i \|(P_i \mathbf{v})_{\overline{S_i}}\|_1 + \tau_i \|\mathbf{A} \mathbf{v}\|_2.$$

Combining with [25, Lemma 4.15] stating

$$\|(P_i \mathbf{v})_{\overline{S_i}}\|_1 \leq \|\widehat{P_i \mathbf{x}^{(i)}}\|_1 - \|P_i \mathbf{x}\|_1 + \|(P_i \mathbf{v})_{S_i}\|_1 + 2\|(P_i \mathbf{x})_{\overline{S_i}}\|_1.$$

we arrive at

$$(1 - \rho_i) \|(P_i \mathbf{v})_{\overline{S_i}}\|_1 \leq \|\widehat{P_i \mathbf{x}^{(i)}}\|_1 - \|P_i \mathbf{x}\|_1 + 2\|(P_i \mathbf{x})_{\overline{S_i}}\|_1 + \tau_i \|\mathbf{A} \mathbf{v}\|_2.$$

Applying once again the ℓ^1 -RP-NSP, it holds

$$\begin{aligned}\|P_i \mathbf{v}\|_1 &= \|(P_i \mathbf{v})_{S_i}\|_1 + \|(P_i \mathbf{v})_{\overline{S_i}}\|_1 \leq \rho_i \|(P_i \mathbf{v})_{\overline{S_i}}\|_1 \\ &\quad + \tau_i \|\mathbf{A} \mathbf{v}\|_2 + \|(P_i \mathbf{v})_{\overline{S_i}}\|_1 \\ &\leq (1 + \rho_i) \|(P_i \mathbf{v})_{\overline{S_i}}\|_1 + \tau_i \|\mathbf{A} \mathbf{v}\|_2 \\ &\leq \frac{1 + \rho_i}{1 - \rho_i} \left(\|\widehat{P_i \mathbf{x}^{(i)}}\|_1 - \|P_i \mathbf{x}\|_1 + 2\|(P_i \mathbf{x})_{\overline{S_i}}\|_1 \right) \\ &\quad + \frac{4\tau_i}{1 - \rho_i} \|\mathbf{A} \mathbf{v}\|_2.\end{aligned}$$

We now remember Lemma 2 and notice that $\widehat{P_i \mathbf{x}^{(i)}} = \widehat{\mathbf{x}}^{(i)}$, $\widehat{\mathbf{x}}^{(i)}$ being the optimal solution to $\mathcal{P}_{1,\eta}$, it is clear that $\|\widehat{\mathbf{x}}\|_1 \leq \|P_i \mathbf{x}\|_1$ from what we can conclude that

$$\|P_i \mathbf{x} - \widehat{\mathbf{x}}^{(i)}\|_1 = \|P_i \mathbf{v}\|_1 \leq 2 \frac{1 + \rho_i}{1 - \rho_i} \sigma_s^{\mathcal{W}}(\mathbf{x})_{1,i} + \frac{4\tau_i}{1 - \rho_i} \|\mathbf{A} \mathbf{v}\|_2.$$

Summing up the contributions for all i in $\{1, \dots, n\}$ and applying the inverse frame operator finishes the proof. $\square$

Similarly, assuming ℓ^q -RP-NSP, one can adapt the proof techniques from [25, Theorems 4.22, 4.25] to the local problems. This yields the following result

Theorem 4.4. Let $\mathbf{A} \in \mathbb{K}^{m \times N}$ and $\mathcal{W} = (W_i, P_i)_{i=1}^n$ a fusion frame with frame bounds $0 < C \leq D < \infty$ and frame operator S . Let $(\mathbf{y}^{(i)})_{i=1}^n$ be the linear measurements $\mathbf{y}^{(i)} = \mathbf{A} P_i \mathbf{x} + \mathbf{e}^{(i)}$, $1 \leq i \leq n$ for some noise vectors $\mathbf{e}^{(i)}$ such that $\|\mathbf{e}^{(i)}\|_2 \leq \eta_i$. Denote by $\widehat{\mathbf{x}}^{(i)}$ the solution to the local Basis Pursuit problems $\mathcal{P}_{1,\eta}$ and let $\widehat{\mathbf{x}} = S^{-1} \sum_i \widehat{\mathbf{x}}^{(i)}$. If the matrix $\mathbf{A}$ satisfies the ℓ^2 -RP-NSP with sparsity pattern $\mathbf{s} = (s_1, \dots, s_n)$ with constants $0 < \rho_1, \dots, \rho_n < 1$ and $\tau_1, \dots, \tau_n > 0$ with respect to $\mathcal{W}$, then the estimation $\widehat{\mathbf{x}}$ approximates $\mathbf{x}$ in the following sense:

$$\|\widehat{\mathbf{x}} - \mathbf{x}\|_p \leq \frac{1}{C} \left(\frac{\langle \tilde{\rho}, \sigma_s^{\mathcal{W}}(\mathbf{x}) \rangle}{s^{1-1/p}} + \frac{\langle \tilde{\tau}, \tilde{\eta} \rangle}{s^{1/2-1/p}} \right), \quad 1 \leq p \leq 2, \quad (4.3)$$

where $\tilde{\rho} = \left(\frac{2(1+\rho_i)^2}{1-\rho_i} \right)_{i=1}^n$, $\tilde{\tau} = \left(\frac{3-\rho_1}{1-\rho_1} \tau_i \right)_{i=1}^n$, and $\tilde{\eta} = (\eta_i)_{i=1}^n$.

4.2.2. P-RIP based recovery

One can show that the P-RIP is sufficient for stable and robust recovery by combining Theorem 4.3 with the following result, showing the existence of random matrices satisfying the RP-NSP, from an RIP argument.

Theorem 4.5. Let $\mathbf{A} \in \mathbb{K}^{m \times N}$ be a matrix satisfying the P-RIP(2s, δ), with $\mathbf{s} = (s_1, \dots, s_n)$ and $\delta = (\delta_1, \dots, \delta_n)$ and $\delta_i < 4/\sqrt{41}$, for all

$1 \leq i \leq n$. Then, $\mathbf{A}$ satisfies the ℓ^2 -RP-NSP with constants $(\rho_i, \tau_i)_{i=1}^n$ where

$$\begin{aligned}\rho_i &:= \frac{\delta_i}{\sqrt{1 - \delta_i^2} - \delta_i/4} < 1 \\ \tau_i &:= \frac{\sqrt{1 + \delta_i}}{\sqrt{1 - \delta_i^2} - \delta_i/4}.\end{aligned} \quad (4.4)$$

Proof. The proof of this results consists in simply applying [25, Theorem 6.13] to every subspaces independently. $\square$

From this result and Theorem 4.4, it follows.

Theorem 4.6. Let $\mathcal{W} = (W_i, P_i)_{i=1}^n$ be a fusion frame for $\mathbb{K}^N$ with frame operator S and frame bounds $0 < C \leq D < \infty$. Let $\mathbf{A} \in \mathbb{K}^{m \times N}$ be a matrix satisfying the P-RIP(2s, δ) where $\mathbf{s} = (s_1, \dots, s_n)$ and $\delta = (\delta_1, \dots, \delta_n)$ with $\delta_i < 4/\sqrt{41}$, for all $1 \leq i \leq n$. Then any distributed-sparse vector $\mathbf{x} \in \Sigma_s^{(\mathcal{W})}$ can be recovered by solving n BPDN problems.

Assuming the noise in each BPDN problem is controlled by $\|\mathbf{e}^{(i)}\|_2 \leq \eta_i$, $1 \leq i \leq n$, and set $\text{vec} \eta = (\eta_i)_{i=1}^n$. Let $\widehat{\mathbf{x}} = S^{-1} \sum \widehat{\mathbf{x}}^{(i)}$. Then

$$\|\widehat{\mathbf{x}} - \mathbf{x}\|_2 \leq \frac{1}{C} \sum_{i=1}^n \alpha_i \frac{\sigma_s^{\mathcal{W}}(\mathbf{x})_{1,i}}{\sqrt{s_i}} + \beta_i \eta_i$$

where α_i and β_i depend only on the RIP constants δ_i .

5. Local sparsity in general dictionaries and frames

The redundancy inherent to frame structures (and their generalization to frames of subspaces) makes them appealing to signal analysis task. So far, we have used the redundancy of the fusion frame process in order to increase the global sparsity of the original vector $\mathbf{x}$ as well as increase the robustness to noise. We investigate now the use of local dictionaries in order to use the redundancy within subspaces, using the local frames for the representation of the partial information. A common scenario in applications is when $f \in \mathcal{H}$ has a sparse frame representation $f = D\mathbf{x}$, i.e. $\mathbf{x}$ is sparse, and the multiple measurements are given by

$$\mathbf{y}^{(i)} = \mathbf{A} P_i f + \mathbf{e}^{(i)} = \mathbf{A} P_i D \mathbf{x} + \mathbf{e}^{(i)}, \quad 1 \leq i \leq n.$$

Here P_i can be any projection onto a subspace of $\mathcal{H}$. In practical applications such as in SAR radar imaging, P_i can just be a projection or spatial filter onto $W_i \equiv \text{span}\{\mathbf{d}_k\}_{k \in \Omega_i}$, where $\mathbf{d}_k$ is the k^{th} column of D . Such an operation can potentially reduce the number of nonzero entries of $\mathbf{x}$ in the i^{th} observation, when the respective column vectors $\mathbf{d}_k$ are in $\ker P_i$. In particular, let us denote by $\Gamma_i \equiv \{k \mid k \notin \Omega_i, \mathbf{d}_k \in W_i\}$, and $\Lambda_i \equiv \{l \mid \mathbf{d}_l \in \ker P_i\}$.

$$\begin{aligned}P_i f &= P_i D \mathbf{x} = P_i \left(\{\mathbf{d}_k\}_{k \in \Omega_i}, \{\mathbf{d}_j\}_{j \in \Gamma_i}, \{\mathbf{d}_l\}_{l \in \Lambda_i} \right) \mathbf{x} \\ &= \left(\{\mathbf{d}_k\}_{k \in \Omega_i}, \{\mathbf{y}_j = P_i \mathbf{d}_j\}_{j \in \Gamma_i}, \{0's\}_{l \in \Lambda_i} \right) \begin{pmatrix} \mathbf{x}_{\Omega_i} \\ \mathbf{x}_{\Gamma_i} \\ \mathbf{x}_{\Lambda_i} \end{pmatrix} \\ &= \left(\{\mathbf{d}_k\}_{k \in \Omega_i}, \{\mathbf{y}_l = P_i \mathbf{d}_l\}_{l \in \Gamma_i} \right) \begin{pmatrix} \mathbf{x}_{\Omega_i} \\ \mathbf{x}_{\Gamma_i} \end{pmatrix} \\ &= D_i \mathbf{x}^{(i)},\end{aligned}$$

where $D_i \equiv \left(\{\mathbf{d}_k\}_{k \in \Omega_i}, \{\mathbf{y}_k = P_i \mathbf{d}_k\}_{k \in \Gamma_i} \right)$, and

$$\mathbf{x}^{(i)} \equiv \begin{pmatrix} \mathbf{x}_{\Omega_i} \\ \mathbf{x}_{\Gamma_i} \end{pmatrix}.$$

As a result, the i^{th} measurement becomes

$$\mathbf{y}^{(i)} = \mathbf{A} D_i \mathbf{x}^{(i)} + \mathbf{e}^{(i)}, \quad 1 \leq i \leq n,$$

or

$$\mathbf{y}^{(i)} = \mathbf{A} f^{(i)} + \mathbf{e}^{(i)}, \quad f^{(i)} = D_i \mathbf{x}^{(i)}, \quad 1 \leq i \leq n.$$

Note that the first version suggests the use of ℓ^1 synthesis methods, while the second one looks at ℓ^1 analysis tools. ℓ^1 synthesis corresponds to the usual sparse recovery, via a dictionary D :

$$\min_z \|z\|_0, \quad \text{subject to } \|ADz - y\|_2 \leq \eta.$$

The solution $\hat{f}$ is later computed as $\hat{f} = D\hat{x}$. In the ℓ^1 analysis approach, we do not care for a particular (sparse) representation of f . We just ask for this representation to have high fidelity with the data:

$$\min_g \|D^*g\|_0, \quad \text{subject to } \|Ag - y\|_2 \leq \eta \quad (5.1)$$

Here, D^* denotes the canonical dual frame. Both approaches are further detailed in the next sections. We comment that if the choice of P_i is allowed, one strategy is again to use random projections by randomly selecting the index set Ω_i to set the subspaces $W_i = \text{span}\{\mathbf{d}_j\}_{j \in \Omega_i}$.

5.1. Recovery via general ℓ^1 -analysis method

As introduced above, we try to recover f that has a sparse representation by solving Problem (5.1). While the problem is written in terms of the canonical dual frame, there is no obligation in using this particular dual frame. One may instead optimize the dual frame considered and use the sparsity-inducing dual frame [31], computed as part of the optimization problem:

$$\widehat{f^{(i)}} = \arg\min_{g, D\tilde{D}_i^*g} \|\tilde{D}_i^*g\|_1 \quad \text{s.t. } \|Ag - y^{(i)}\|_2 \leq \eta_i, \quad 1 \leq i \leq n. \quad (5.2)$$

The sparsity-inducing frame $\tilde{D}_i$ can be uniform across all i but not necessarily. The following result is known to hold for any dual frame [31].

Theorem 5.1. Let D be a general frame of $\mathbb{R}^N$ with frame bounds $0 < A \leq B < \infty$. Let $\tilde{D}$ be an alternative dual frame of D with frame bounds $0 < \tilde{A} \leq \tilde{B} < \infty$, and let $\rho = s/b$. Suppose that the matrix A satisfies the following D-RIP condition

$$\left(1 - \sqrt{\rho B \tilde{B}}\right)^2 \cdot \delta_{s+a} + \rho B \tilde{B} \cdot \delta_b < 1 - 2\sqrt{\rho B \tilde{B}} \quad (5.3)$$

for some positive integers a and b satisfying $0 < b - a \leq 3a$. Let $\hat{f}$ be the solution to the typical ℓ^1 -analysis problem

$$\hat{f} = \arg\min_g \|\tilde{D}^*g\|_1 \quad \text{s.t. } \|Ag - y\|_2 \leq \eta.$$

Then

$$\|\hat{f} - f\|_2 \leq \alpha\eta + \beta \frac{\|\tilde{D}^*f - (\tilde{D}^*f)_{s_i}\|_1}{\sqrt{s}}, \quad (5.4)$$

where α and β are some constants and $(\tilde{D}^*f)_{s_i}$ denotes the vector consisting of the s largest entries in magnitude of $\tilde{D}^*f$.

In particular, the bound (5.4) applied to the local pieces of information $\widehat{f^{(i)}}$ analyzed with the local frames $\tilde{D}_i^*$, for $1 \leq i \leq n$, obtained as solution to Problem (5.2) yields the following error bound for the reconstruct signal $\hat{f} = S^{-1} \sum_{i=1}^n \widehat{f^{(i)}}$:

$$\|\hat{f} - f\|_2 \leq \|S^{-1}\|_{2 \rightarrow 2} \sum_{i=1}^n \|\widehat{f^{(i)}} - f^{(i)}\|_2 \leq \alpha\eta + \beta \left(\sum_{i=1}^n \frac{\|\tilde{D}_i^*f^{(i)} - (\tilde{D}_i^*f^{(i)})_{s_i}\|_1}{\sqrt{s_i}} \right),$$

where $\alpha \equiv \sup_i \alpha_i \|S^{-1}\|_{2 \rightarrow 2}$, $\eta \equiv \sum_i \eta_i$, and $\beta \equiv \sup_i \beta_i \|S^{-1}\|_{2 \rightarrow 2}$. This bound is reminiscent of the traditional bounds in CS where the (local) error decays as $\sigma_s(\mathbf{x})_1/\sqrt{s}$ except that the contributions of each subspace are added together and normalized by the norm of the inverse frame operator.

5.2. Recovery via ℓ^1 -synthesis method

The gap between recovery via ℓ^1 synthesis and ℓ^1 analysis has long been studied. Interestingly, it can be shown [30] that, when using the sparsity-inducing frames described above (5.2), both approaches are equivalent. Denote by $\tilde{D}_{i,o}$ the resultant optimal dual frame and suppose that A satisfies a D-RIP property (5.3). Then it follows from [30]

$$\|\widehat{f^{(i)}} - f^{(i)}\|_2 \leq \alpha_i \eta_i + \beta_i \frac{\|\tilde{D}_{i,o}^*f^{(i)} - (\tilde{D}_{i,o}^*f^{(i)})_{s_i}\|_1}{\sqrt{s_i}}$$

for some positive constants α_i and β_i . Considering the fusion of the local information $\hat{f} = S^{-1} \left(\sum_{i=1}^n \widehat{f^{(i)}} \right)$, the following result holds true:

Proposition 5.1. Let S be the invertible fusion frame operator. Suppose A satisfies condition (5.3). Then the fused solution $\hat{f}$ has an error bound given by

$$\|\hat{f} - f\|_2 \leq \alpha\eta + \beta \left(\sum_{i=1}^n \frac{\|\tilde{D}_{i,o}^*f^{(i)} - (\tilde{D}_{i,o}^*f^{(i)})_{s_i}\|_1}{\sqrt{s_i}} \right),$$

where $\alpha \equiv \sup_i \alpha_i \|S^{-1}\|_{2 \rightarrow 2}$, $\eta \equiv \sum_i \eta_i$, and $\beta \equiv \sup_i \beta_i \|S^{-1}\|_{2 \rightarrow 2}$.

Proof. Write $f = S^{-1}Sf = S^{-1} \sum_{i=1}^n P_i f = S^{-1} \sum_{i=1}^n f^{(i)}$. Direct computation shows

$$\begin{aligned} \|\hat{f} - f\|_2 &= \|S^{-1} \sum_{i=1}^n (\widehat{f^{(i)}} - f^{(i)})\|_2 \\ &\leq \|S^{-1}\|_{2 \rightarrow 2} \sum_{i=1}^n \|\widehat{f^{(i)}} - f^{(i)}\|_2 \\ &\leq \|S^{-1}\|_{2 \rightarrow 2} \sum_{i=1}^n \left(\alpha_i \eta_i + \beta_i \frac{\|\tilde{D}_{i,o}^*f^{(i)} - (\tilde{D}_{i,o}^*f^{(i)})_{s_i}\|_1}{\sqrt{s_i}} \right) \\ &\leq \sup_i \alpha_i \|S^{-1}\|_{2 \rightarrow 2} \sum_{i=1}^n \eta_i + \\ &\quad \sup_i \beta_i \|S^{-1}\|_{2 \rightarrow 2} \left(\sum_{i=1}^n \frac{\|\tilde{D}_{i,o}^*f^{(i)} - (\tilde{D}_{i,o}^*f^{(i)})_{s_i}\|_1}{\sqrt{s_i}} \right). \end{aligned}$$

The result follows directly by setting $\alpha \equiv \sup_i \alpha_i \|S^{-1}\|_{2 \rightarrow 2}$, $\eta \equiv \sum_i \eta_i$, and $\beta \equiv \sup_i \beta_i \|S^{-1}\|_{2 \rightarrow 2}$. $\square$

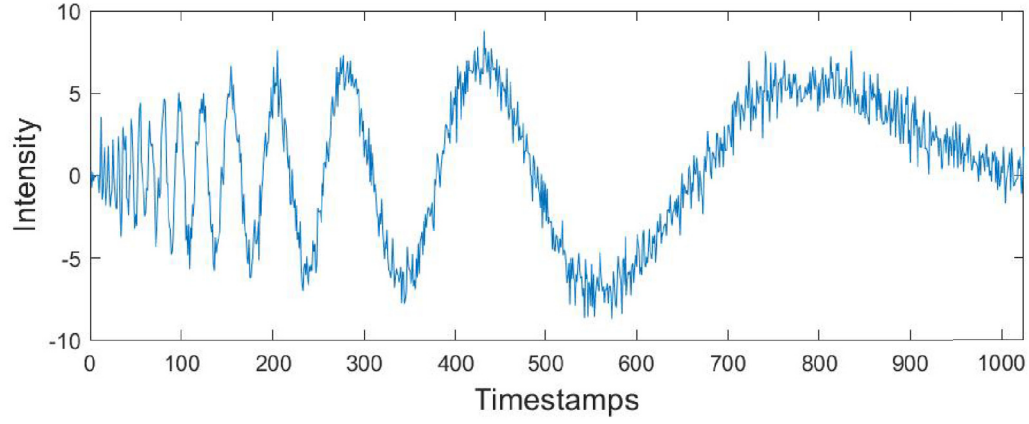
We comment that this result is not surprising due to the equivalence between the two problems described in this section when dealing with the sparsity-inducing dual frames.

6. Examples

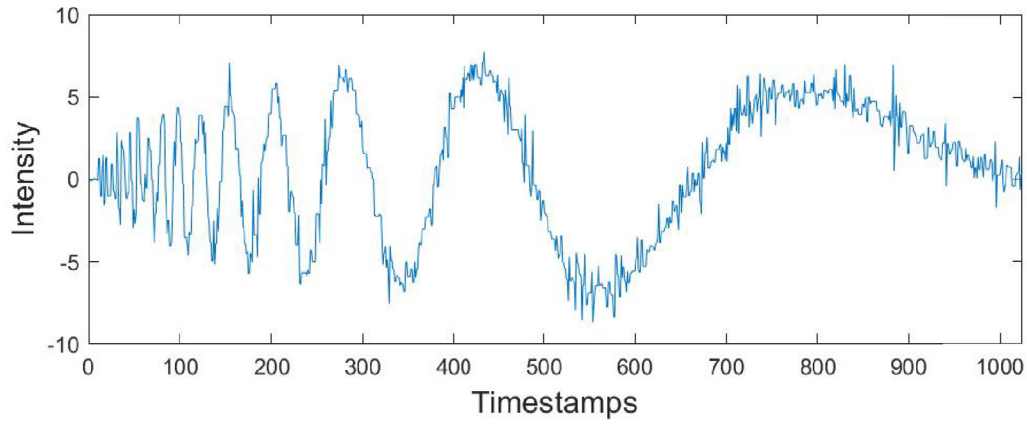
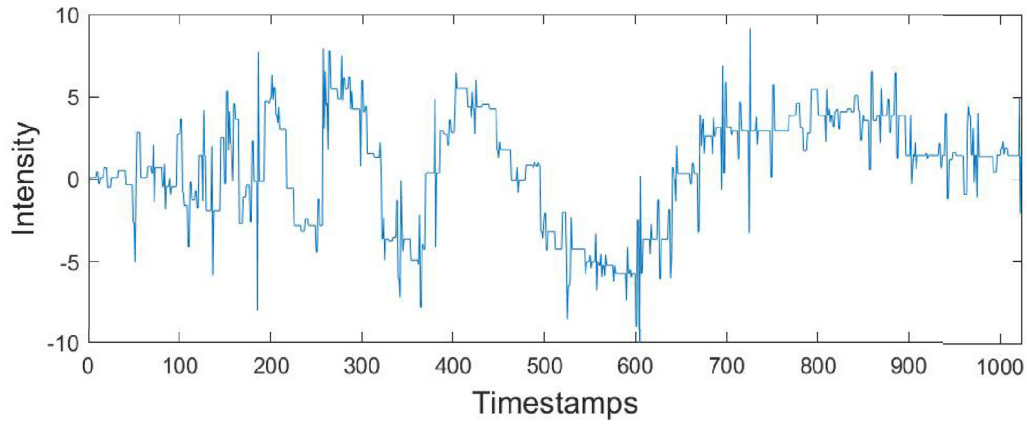
We provide in this Section some examples of applications of our approach where it yields, without any fine tuning of the parameters, comparable – if not better – results with common methods (which are precisely targeted for the given problems).

6.1. Wavelet frames and recovery of Doppler signals

This section is intended as an illustrative example and proof-of-concept of the tools developed so far. In particular, we want to show that using a fairly *poor* quality device, we are capable to recover a signal of fairly high complexity. For the sake of reproducible research, all the experiments presented in this section can be obtained and reproduced from the named authors' Github



(a) Original noisy Doppler signal

(b) Recovery from a distributed sensing approach – L^2 error: 37.18(c) Reconstruction with an ℓ^1 analysis method – L^2 error: 82.72**Fig. 7.** A noisy Doppler signal (7(a)) and its reconstructions via a Haar wavelet frame approach 7(b) and a direct ℓ^1 analysis on the Haar system 7(c).

page⁴ In this experiment, we try to recover a noisy Doppler signal (see Fig. 7(a)) using sensors with very few measurements.

To produce the Figures, we have considered a 1024 dimensional noisy Doppler signal (obtained directly via Matlab's Wavelet tool-

box). We compare a traditional ℓ^1 analysis approach, where the sparsifying basis is chosen as the Haar system. Similarly, we consider the Haar wavelet systems as subspaces for our fusion frame approach. Given that a signal $s(t)$ can be decomposed as

$$s(t) = a_l(t) + \sum_{j \geq l} d_j(t)$$

⁴ See <https://github.com/jlbouchot/FFCS> for all the self implemented files. These files require, as described in the README file, to have the access to a Haar matrix function and to have CVX [27,28] installed.

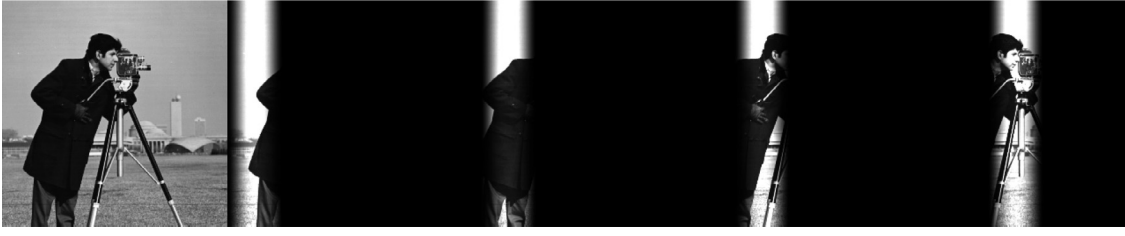


Fig. 8. Original cameraman image with the first four scene illuminations considered.

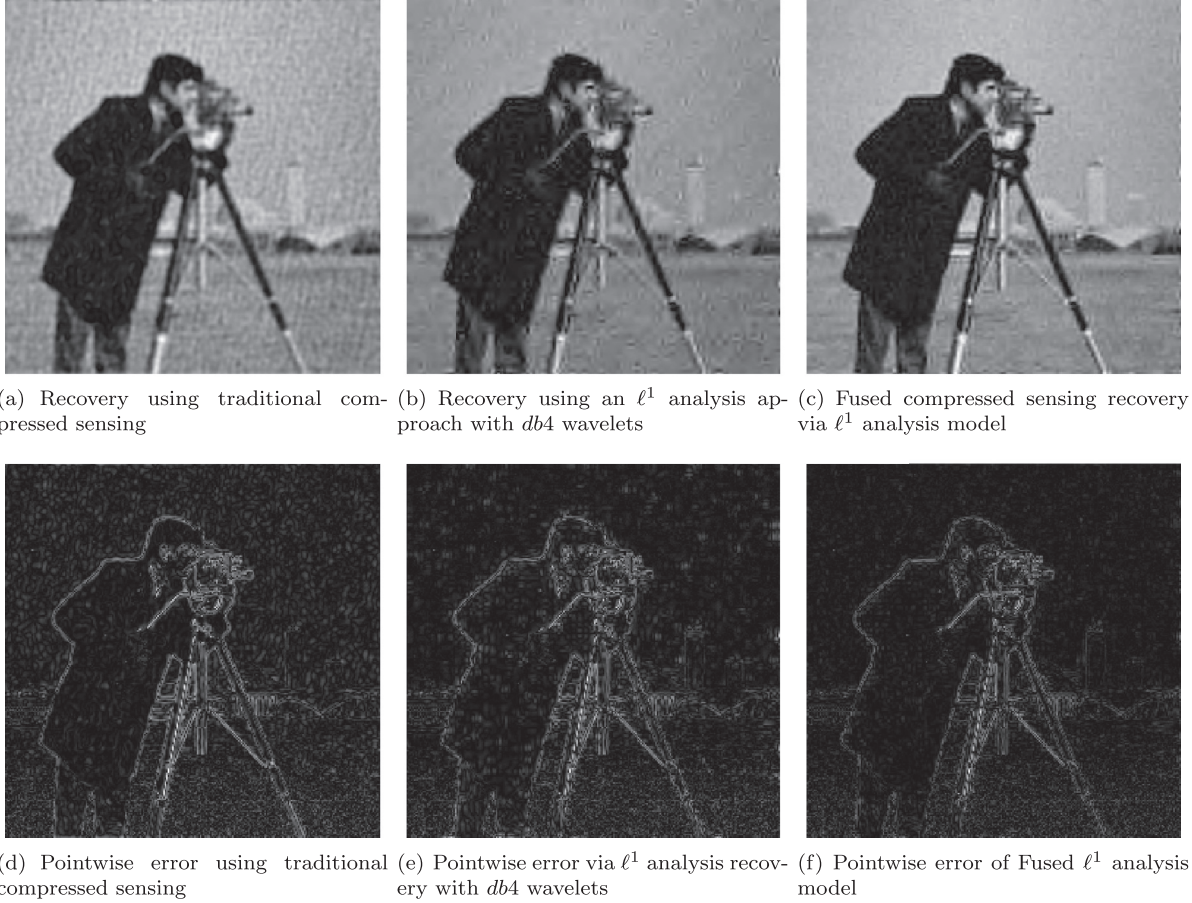


Fig. 9. Recovery results from single and multi-sensor set ups in natural images. Our approach allows to recover fine details whilst preserving the main structures of the subject.

where a_l corresponds to the approximation coefficients at level l and d_j corresponds to the details at level j . We therefore consider our projections P_j to be onto the subspaces of details W_j at each level (using the usual definition from multiresolution analysis), and a remaining low frequency approximation part. This gives us (in our setting) a grand total of $n = 9$ projections, 8 of which ($P_1, \dots, P_8$) correspond to detail coefficients, and the last one (P_0) corresponds to the low-frequency approximation. This system being orthogonal, it gives us the easy reconstruction formula

$$\hat{\mathbf{x}} = \sum_{i=0}^8 \widehat{\mathbf{x}}^{(i)}.$$

All the local information are recovered with an ℓ^1 analysis procedure, where the sparsifying basis are precisely chosen to be the Haar wavelets at the appropriate levels. Finally, the sparsity is set uniformly to 25 on each subspace and a random Gaussian matrix with $m = 174$ rows is created and some normal noise with variance

0.05 is added to the measurement vectors, all drawn independently from one another.

Fig. 7 (b) shows the reconstructed signal using our ℓ^1 -analysis fusion frame approach described in the previous section, while Fig. 7(c) is the reconstructed signal obtained from a traditional ℓ^1 -analysis approach. Both of them have been reconstructed using measurements obtained from the exact same matrix. It is important to note however, that we have not tried to optimize our parameters in this example. In particular, bearing in mind that the subspaces of higher-frequency details have higher dimensions (in fact, $\text{rank}(P_j) = \mathcal{O}(2^j)$, for $1 \leq j \leq 8$), a better choice would be to set the local sparsity to be higher in these subspaces to improve our results. This level dependent-sampling is nothing new in the community and our results corroborate those from other approaches [2]. The Figures clearly show the reconstruction capabilities of our approach even with a sub-optimal setting and not making use of any kind of redundancy. Note that mostly the high frequency components of the signal are missing, which can be overcome by a better setting of the sparsity per subspace.

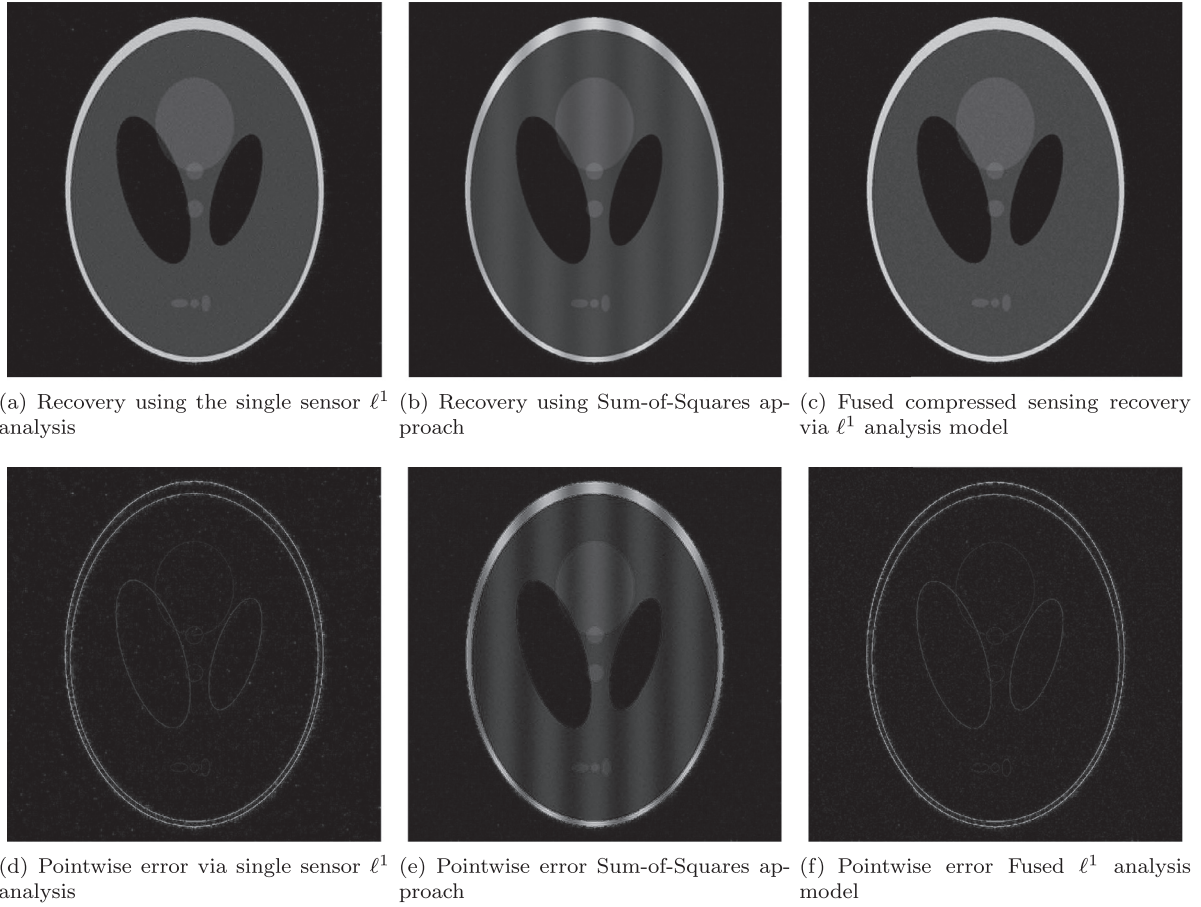


Fig. 10. Recovery of the phantom MR Image of size 1024×1024 from highly undersampled and noisy measurements. The sparse minimization is done on Daubechies 4 wavelet coefficients. Our approach allows 10(c) allows better recovery than other comparable methods 10(b).

6.2. Natural image processing

This section and the next one empirically studies the applicability of our proposed approach⁵. They detail some basic yet illustrative examples of the use of our fused sensing framework in imaging sciences. They prove both the easiness to implement all sorts of applications and the generality of the suggested approach. Further examples can be found in [Appendix A](#).

The first imaging application is concerned with natural images. To this end, we consider the traditional *cameraman* image depicted on the left of [Fig. 8](#). We simulate a moving field of view in the x direction, obtained by generating $n = 8$ traveling sine waves, as shown in [Fig. 8](#). This is motivated for its use in multi-channel MR Imaging [32,34], for which an example is given in the next section.

For this experiment, we compare the fused compressed sensing method described above with the traditional ℓ^1 analysis approach and the usual ℓ^1 minimization – albeit knowing that the traditional compressed sensing is not suited for this task. The minimization is computed with a self-implemented primal-dual algorithm which we use for all the following experiments. We exploit the fused sensing method based on the ℓ^1 analysis model presented in [Section 5](#) using a Daubechies-4 wavelet decomposition as sparsifying dictionary. The measurements are obtained by subsampling the Fourier domain at random according to a Gaussian distribution. We consider a subsampling ratio of 4.84%, which means we are taking 3174 samples from a 256×256 dimensional image. All the measurements are assumed to contain some randomly gen-

Table 1

Accuracy of the various recovery methods of the natural scene measured with the Structural SIMilarity index, Peak Signal to Noise Ratio, and pointwise ℓ^2 error.

	Compressed sensing	ℓ^1 analysis	Fused ℓ^1 analysis
SSIM	0.5473	0.5967	0.6838
PSNR	23.076	22.893	24.342
ℓ^2 error	17.965	18.348	15.530

erated additive Gaussian noise with variance 0.05. [Fig. 9](#) compares the results obtained from our approach against the single sensor approaches in the ℓ^1 analysis and traditional compressed sensing.

As can be noted, even in this drastic undersampling situation, our method is capable of handling accurate reconstructions. It can be seen that the details on the man are kept while making the noise less obvious. It is particularly visible in certain small details such as the hands of the cameraman, or the white parts of the feet of the tripod. The background also appears to be better defined in the fused image than in the classical and ℓ^1 analysis methods, which is also seen in [Table 1](#).

6.3. MR Image reconstruction

The second imaging application is concerned with medical imaging. Following the ideas from [32,34], we simulate a multi-channel MRI sensing set-up, first using the same sine waves as in the previous experiment (results displayed in [Fig 10](#)), and then using spherical beam pattern (see in [Fig. 11](#) for the results). We

⁵ All experiments can be obtained from <https://github.com/jlbouchot/FFCS>

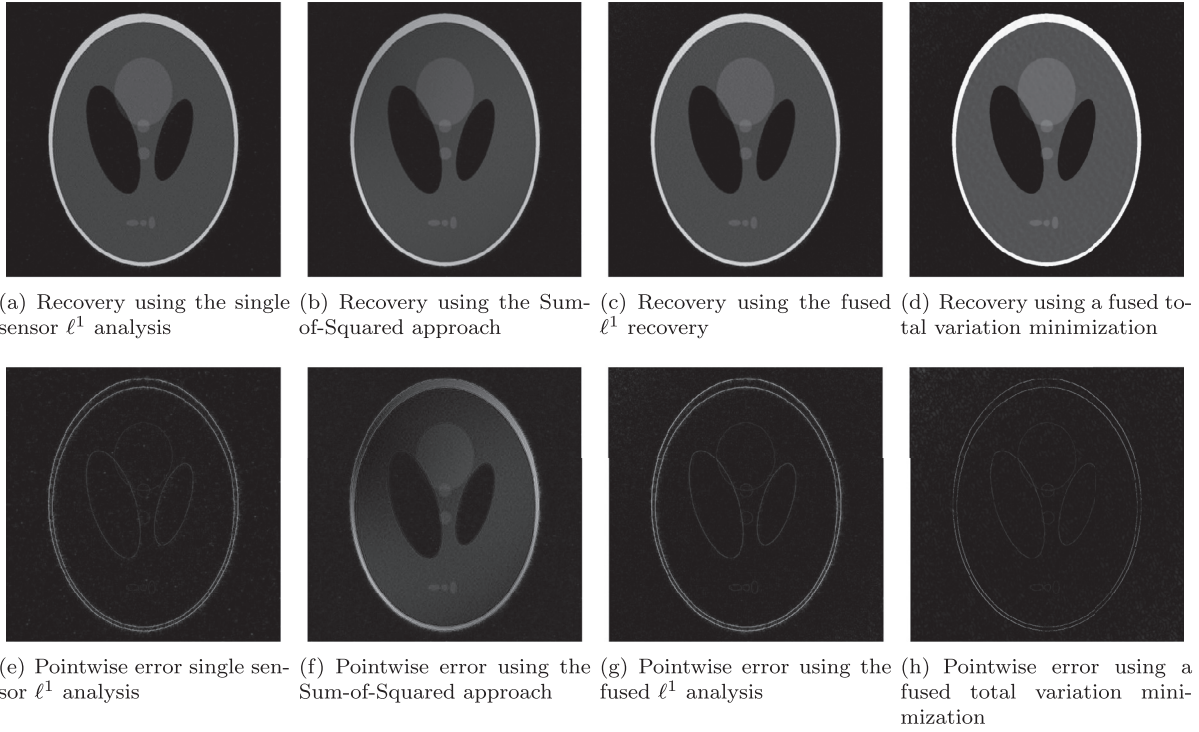


Fig. 11. Recovery of the phantom MR Image of size 1024×1024 from highly undersampled and noisy measurements (additive Gaussian noise, variance 0.05). The sparse minimization is done on Daubechies 4 wavelet coefficients and on the gradient for the rightmost image.

Table 2

Accuracy of the various recovery methods of the phantom image measured with the Structural SIMilarity index, Peak Signal to Noise Ratio, and pointwise ℓ^2 error.

	ℓ^1 analysis	Sum of Square fusion	Fused ℓ^1 analysis
SSIM	0.8022	0.4381	0.7672
PSNR	27.917	14.893	28.484
ℓ^2 error	41.159	184.36	38.558

generate the classical Shep-Logan phantom, and set its dimension to 1024×1024 .

The first experiment, depicted in Fig. 10 shows the results obtained by our method when using a Daubechies 4 sparsifying dictionary. Our results are compared to the single sensor ℓ^1 analysis, and the results obtained by the Sum-of-Squares recovery [32,34].

The recovery is obtained from noisy measurements, in which some additive Gaussian noise with variance 0.05 has been added. The measurements are obtained by subsampling 2.45% (25667 samples) of the Fourier transform. We see here that the fused compressed sensing is better capable of handling a multi-channel problem with unusual illumination (compared to the Sum-of-Squares method). Moreover, more details are preserved, when compared to the single sensor ℓ^1 analysis method see Table 2.

Some noise still appears in the image, but can easily be thresholded further if needed. One important aspect of MR Images that hasn't been considered in this research, is the fact that they are

sparse in gradient. One usually prefers to minimize the Total Variation instead of the ℓ^1 norm or another frame using ℓ^1 analysis.

Although we have not explicitly written the theory here, the recovered images in Fig. 11 show the results when using a TV minimization instead of the ℓ^1 analysis.

The results were obtained from a spherical beam illuminating the input image. The measurements are obtained by sampling 2.45% (25664 samples) of the Fourier coefficients at random iid from a Gaussian distribution. Again, some Gaussian noise with variance 0.05 is added to every measurements.

As presented in these examples, our method is capable of handling highly complex signals in potentially many dimensions yet keeping a very low number of samples taken. We can also empirically verify that the presented approach enjoys more robustness to noise and variations in scene illuminations than other known methods, numerics can be seen in Table 3.

7. Conclusion

This paper introduces a novel way to look at signal sensing and reconstruction by utilizing fusion frames. The suggested method is very general by nature and can be applied in many situations, as illustrated in the previous section. Our research suggests that it is possible to recover high complexity from many low dimensional pieces of information. This splitting of information can be either enforced by design of the sensors, or by the physical nature of the problem being considered.

Table 3

Accuracy of the various recovery methods of the phantom image measured with the Structural SIMilarity index, Peak Signal to Noise Ratio, and pointwise ℓ^2 error.

	ℓ^1 analysis	Sum of Square fusion	Fused ℓ^1 analysis	Fused total variation
SSIM	0.7972	0.4518	0.7872	0.8225
PSNR	27.907	18.719	27.909	32.606
ℓ^2 error	41.207	118.67	41.196	23.988

The present article proves that the number of measurements required locally is inversely proportional to the number of sensors, thereby allowing to use many cheap sensors. Empirically, we verified that our approach allowed to handle noise even in the case of strong undersampling of the scenes.

This work can be further developed by integrating various sensors instead of a single one, repeated n times. Further real-world applications, for instance in SAR systems, can also be implemented but the details are left for future research.

Declaration of Competing Interest

We declare no conflict of interests with any other universities or corporations.

Acknowledgment

R. A. is partially supported by the BSU Aspire Research Grant “Frame Theory and Modern Sampling Strategies”. J.-L. B. thanks the support of the [European Research Council](#) through the grant [StG 258926](#) and the organizers as well as the Hausdorff Center for Mathematics for his participation in the Hausdorff Trimester Program on Mathematics of Signal Processing. S. L. is partially supported by US NSF grants USA ([DMS-1313490](#), [DMS-1010058](#)).

Appendix A. Further numerical results

A1. Comparison with Douglas-Rachford optimization and variable density sampling (DR-VDS)

In this set of tests, we numerically assess how our approach behaves in the presence of a variable density sampling scheme which contains a low-frequency deterministic part and different random sampling patterns. This approach is motivated by results in [10,15], in which the sampling pattern is also highly structured. In particular, we look at basic variable density sampling scheme in which low frequencies are deterministically selected and the high-frequency components are sampled at random, according to either a radial pattern (see [Figs. A.12,A.14](#)) or a spiraling pattern (see [Figs. A.13,A.15](#)).

For this test, we have reused the software already available from the authors of [15]⁶. The samples were obtained by sub-sampling 5% of the Fourier coefficients and adding normal noise with 0 mean and 0.05 variance. The accuracies can be found in [Table A.4](#) for the results depicted in [Figs. A.12 and A.13](#) and in [Table A.5](#) for results depicted in [Figs. A.14 and A.15](#). As it is seen from the pointwise error figures, our fusion process is less subject to small local perturbations. Moreover, it is clear from the tables that fusing with local recovery obtained by total variation minimization is the most suitable approach.

This behavior is true for both images, which are real-world images of brain scans. It is worth keeping in mind that SSIM is a structural similarity which tries to emulate the human visual perception while the other metrics are pure machinery. As can be seen both from the figures and the tables, the proposed method performs at least as well as previous approaches, and may behave better when considering total variation minimization.

A2. Single recovery from distributed measurements

We investigate here use of a single recovery procedure from multiple measurements. In this settings, and following the ideas

Table A.4

Recovery results for the sagittal brain MRI. The results are obtained by sampling 5% of the k -space.

Sampling		DR-VDS	Fused ℓ^1	Fused TV
Radial	SSIM	0.6099	0.4366	0.7691
	PSNR	26.772	26.071	29.193
	ℓ^2 error	3.0294	3.0971	2.4104
Spiral	SSIM	0.6311	0.6830	0.8034
	PSNR	27.331	28.462	30.336
	ℓ^2 error	2.8132	2.3014	1.7364

Table A.5

Recovery results for the axial slice brain MRI. The results are obtained by sampling 5% of the k -space.

Sampling		DR-VDS	Fused ℓ^1	Fused TV
Radial	SSIM	0.6336	0.4609	0.6847
	PSNR	30.917	26.807	30.101
	ℓ^2 error	3.1783	3.1700	2.1869
Spiral	SSIM	0.6645	0.6150	0.7174
	PSNR	34.695	29.283	31.057
	ℓ^2 error	2.8709	2.4314	1.6922

Table A.6

Recovery results for the camera man image.

Noise level		DSL1	CS	L1 Analysis	Fused L1 analysis
0.020	SSIM	0.5247	0.5693	0.6432	0.6776
	PSNR	23.028	23.289	23.371	25.428
	ℓ^2 error	18.064	17.530	17.365	13.703
0.025	SSIM	0.5279	0.5711	0.6354	0.6478
	PSNR	23.000	23.240	23.273	25.064
	ℓ^2 error	18.122	17.629	17.562	14.290
0.030	SSIM	0.5200	0.5625	0.6325	0.6231
	PSNR	22.912	23.127	23.074	24.980
	ℓ^2 error	18.308	17.860	17.970	14.429

developed in [18] and described in the previous sections, we are trying to recover the unknown vector $\mathbf{x}$ from the measurements $\mathbf{y}^{(i)}$ by solving a single sparse minimization problem.

[Fig. A.16](#) shows the results obtained by various methods, namely, a single ℓ^1 minimization computed on the multi-channel measurements (similar to the approach of [18]) depicted on the first row, and traditional compressed sensing in the second row, an ℓ^1 analysis model using Daubechies 4 wavelets in the third row, and our fused framework where the local recovery are done by minimizing the coefficients on a Daubechies 4 wavelet basis.

The fusion of the local recovery shows the best of the results, which is also confirmed by the values for SSIM, PSNR, and pointwise ℓ^2 error in [Table A.6](#).

A3. Robustness to erasure

One advantage claimed for this framework is the robustness of the approach to local errors. This is believed to be a consequence of the redundancy inherent to frames. We demonstrate this point by considering the following setup. Assume given the camera man image as in the previous experiments. We emulate faulty measures by considering salt and pepper noise in the image prior to taking the Fourier measurements and filtering operations.

The graphs in [Fig. A.17](#) show the evolution of the fidelity of the recovered images, as the density of faulty pixels increases. It is evident from these figures that the fusion process (red or black curves) favors robustness in the presence of erasure. Even in a regime where up to 30% of the pixels cannot be considered reliable, our fusion process is still able to achieve high fidelity with the original images. The results shown here were also obtained with some additive Gaussian noise post measurements.

⁶ At the time of writing, the code is available at <http://chauffertn.free.fr/codes.html>.

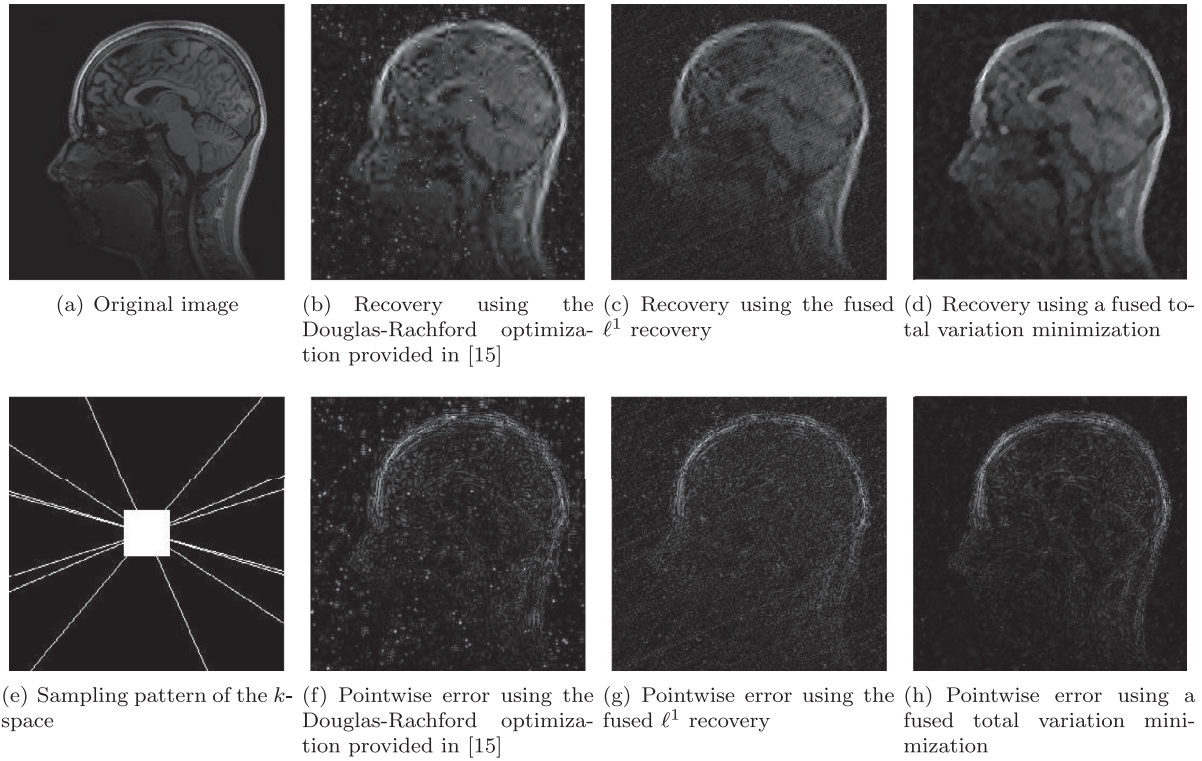


Fig. A.12. Recovery of a sagittal brain MRI from 1/20 undersampled measurements taken as radial lines in the Fourier space and a deterministic low-frequency area. The sparse minimization is done on symmet wavelet coefficients and on the gradient for the rightmost image.

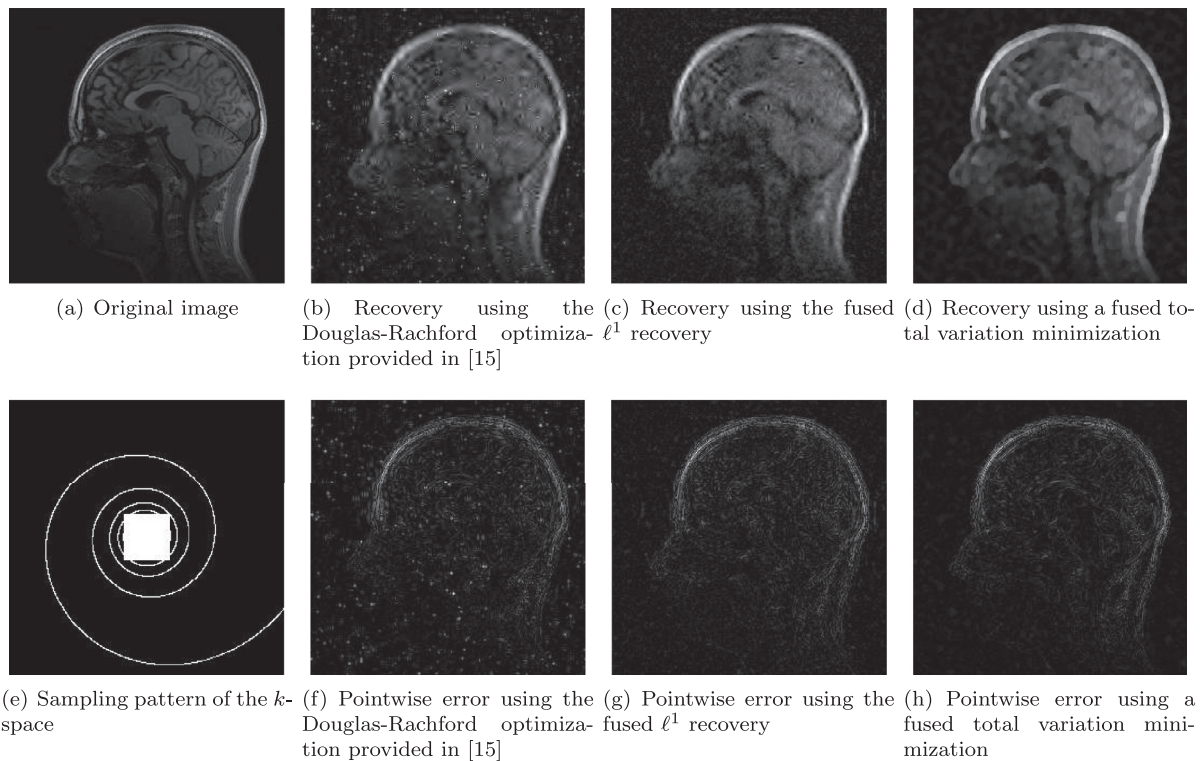


Fig. A.13. Recovery of a sagittal brain MRI from 1/20 undersampled measurements taken as a spirale in the Fourier space and a determinist low-frequency area. The sparse minimization is done on symmet wavelet coefficients and on the gradient for the rightmost image.

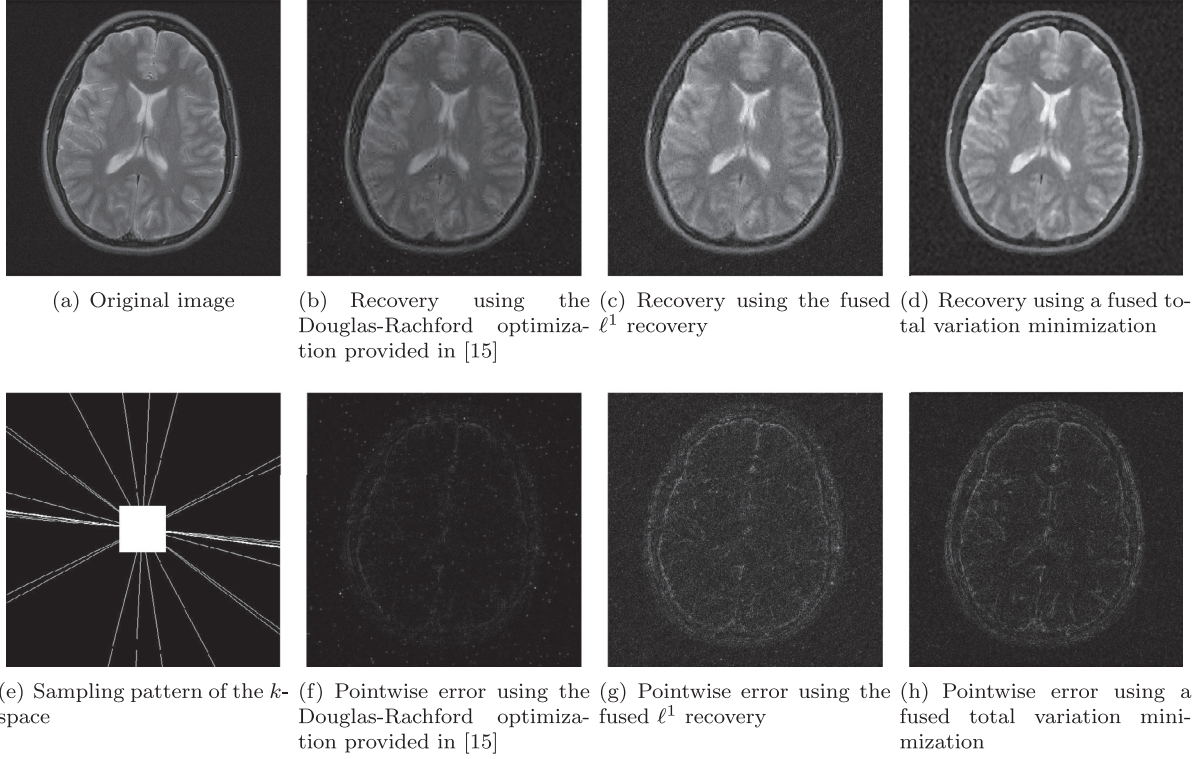


Fig. A.14. Recovery of an axial brain MRI from 1/20 undersampled measurements taken as radial lines in the Fourier space and a determinist low-frequency area. The sparse minimization is done on symmlet wavelet coefficients and on the gradient for the rightmost image.

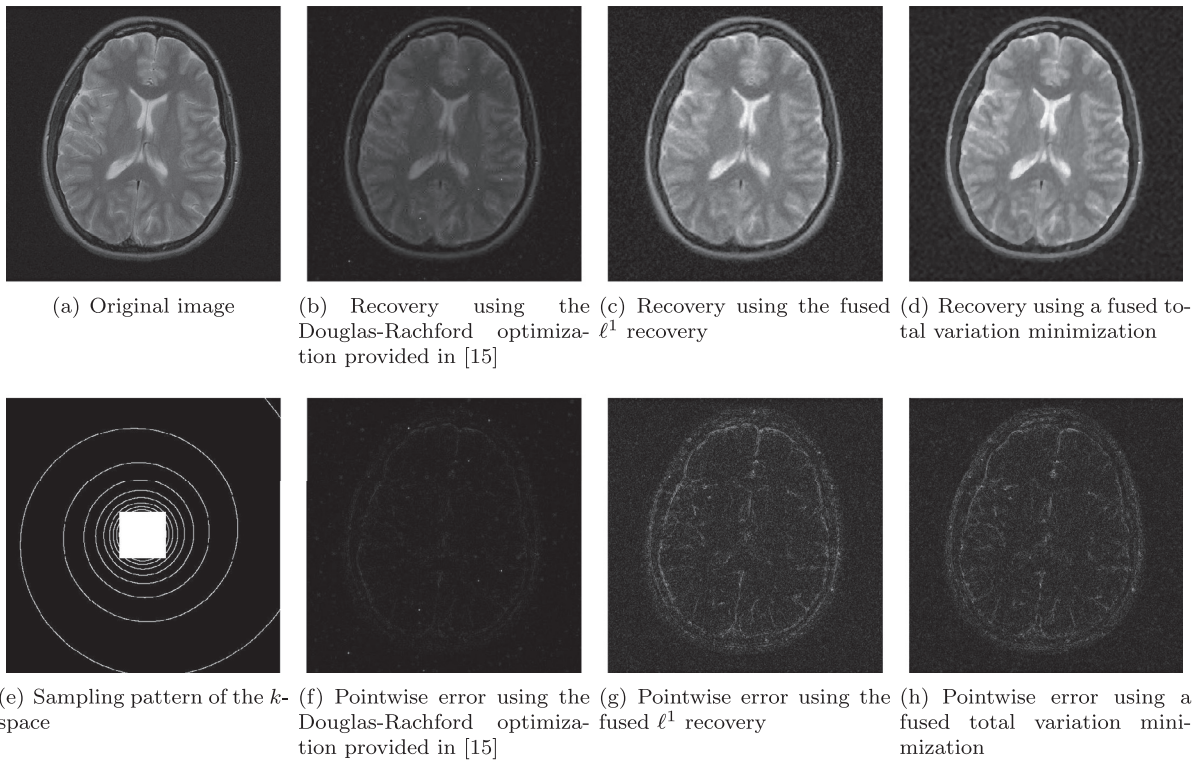


Fig. A.15. Recovery of an axial brain MRI from 1/20 undersampled measurements taken as radial lines in the Fourier space and a determinist low-frequency area. The sparse minimization is done on symmlet wavelet coefficients and on the gradient for the rightmost image.



Fig. A.16. Recovery of the camera man image when using a centralized recovery vs. local recoveries. The additive noise in the measurements is normal with variance 0.02 (left), 0.025 (central), and 0.03 (right). The first row depicts the recovery from a single, centralized, sparse approximation. The second row displays the traditional compressed sensing recovery. The third row is the ℓ^1 analysis model while the bottom row corresponds to the proposed fused ℓ^1 framework.

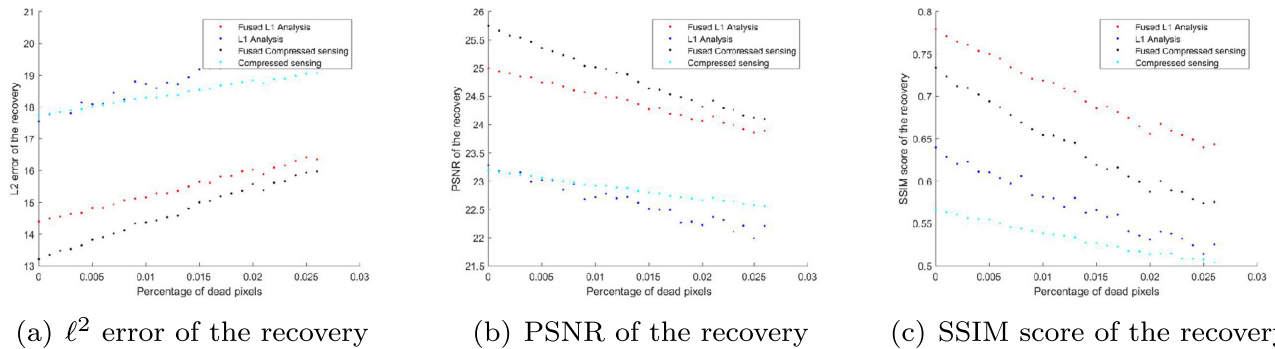


Fig. A.17. Recovery in presence of erasures in the input image.

References

- [1] R. Aceska, J.-L. Bouchot, S. Li, Fusion frames and distributed sparsity, in: Contemporary Mathematics, vol. 706, "Frames and Harmonic Analysis", 2017, pp. 47–62.
- [2] B. Adcock, A.C. Hansen, C. Poon, B. Roman, Breaking the coherence barrier: a new theory for compressed sensing, In Forum of Mathematics, Sigma, vol. 5, Cambridge University Press, 2017.
- [3] U. Ayaz, S. Dirksen, H. Rauhut, Uniform recovery of fusion frame structured sparse signals, Appl. Comput. Harmon. Anal. 41 (2) (2016) 341–361.
- [4] A.S. Bandeira, K. Scheinberg, L.N. Vicente, On partial sparse recovery, 2013, arXiv:1304.2809.
- [5] R. Baraniuk, M. Davenport, R. DeVore, M. Wakin, A simple proof of the restricted isometry property for random matrices, Construct. Approx. 28 (3) (2008) 253–263.
- [6] T. Blumensath, M.E. Davies, Iterative hard thresholding for compressed sensing, Appl. Comput. Harmon. Anal. 27 (3) (2009) 265–274.
- [7] J.-L. Bouchot, A generalized class of hard thresholding algorithms for sparse signal recovery, in: Approximation Theory XIV: San Antonio 2013, Springer, 2014, pp. 45–63.
- [8] J.-L. Bouchot, S. Foucart, P. Hitzchenko, Hard thresholding pursuit algorithms: number of iterations, Appl. Comput. Harmon. Anal. 41 (2) (2016) 412–435.
- [9] P. Boufounos, G. Kutyniok, H. Rauhut, Sparse recovery from combined fusion frame measurements, Inf. Theory, IEEE Trans. on 57 (6) (2011) 3864–3876.
- [10] C. Boyer, J. Bigot, P. Weiss, Compressed sensing with structured sparsity and structured acquisition, Appl. Comput. Harmon. Anal. 46 (2) (2019) 312–350.
- [11] J. Cahill, P. Casazza, S. Li, Non-orthogonal fusion frames and the sparsity of fusion frame operators, J. Fourier Anal. Appl. 18 (2) (2012) 287–308.
- [12] E.J. Candès, The restricted isometry property and its implications for compressed sensing, C.R. Math. 346 (9) (2008) 589–592.
- [13] P.G. Casazza, G. Kutyniok, Frames of subspaces, Contemp. Math. 345 (2004) 87–113.
- [14] P.G. Casazza, G. Kutyniok, S. Li, Fusion frames and distributed processing, Appl. Comput. Harmon. Anal. 25 (2008) 114–132.
- [15] N. Chauffert, P. Ciuciu, J. Kahn, P. Weiss, Variable density sampling with continuous trajectories, SIAM J. Imag. Sci. 7 (4) (2014) 1962–1992.
- [16] Z. Chen, J.J. Dongarra, Condition numbers of Gaussian random matrices, SIAM J. Matrix Anal. Appl. 27 (3) (2005) 603–620.
- [17] O. Christensen, An introduction to frames and Riesz bases, Springer Science & Business Media, 2003.
- [18] I.Y. Chun, B. Adcock, Compressed sensing and parallel acquisition, IEEE Trans. Inf. Theory PP (99) (2017) 1.
- [19] I.G. Cumming, F.H.c. Wong, Digital Processing of Synthetic Aperture Radar Data, Artech House, 2005.
- [20] J. Curlander, R. McDonough, Synthetic Aperture Radar, Wiley Interscience, New York, 1991.
- [21] M.F. Duarte, S. Sarvotham, D. Baron, M.B. Wakin, R.G. Baraniuk, Distributed compressed sensing of jointly sparse signals, Asilomar Conf. Signals, Sys., Comput (2005) 1537–1541.
- [22] Y.C. Eldar, H. Rauhut, Average case analysis of multichannel sparse recovery using convex relaxation, IEEE Trans. Inf. Theory 56 (1) (2010) 505–519.

- [23] M. Fornasier, H. Rauhut, Compressive Sensing, in: Handbook of Mathematical Methods in Imaging, 2011, pp. 187–228.
- [24] S. Foucart, Hard thresholding pursuit: an algorithm for compressive sensing, *SIAM J. Numer. Anal.* 49 (6) (2011) 2543–2563.
- [25] S. Foucart, H. Rauhut, A mathematical introduction to compressive sensing, Springer, 2013.
- [26] G. Franceschetti, R. Lanari, Synthetic Aperture Radar Processing, CRC Press, 1999.
- [27] M. Grant, S. Boyd, CVX: Matlab software for disciplined convex programming, version 2.1, 2014, <http://cvxr.com/cvx>.
- [28] M. Grant, S. Boyd, Graph implementations for nonsmooth convex programs, in: Recent Advances in Learning and Control, Lecture Notes in Control and Information Sciences, 2008, pp. 95–110.
- [29] R. Gribonval, H. Rauhut, K. Schnass, P. Vandergheynst, Atoms of all channels, unite! Average case analysis of multi-channel sparse recovery using greedy algorithms, *J. Fourier Anal. Appl.* 14 (5–6) (2008) 655–687.
- [30] Y. Liu, S. Li, T. Mi, H. Lei, W. Yu, Performance analysis of ℓ^1 -synthesis with coherent frames, in: Information Theory Proceedings (ISIT), 2012, IEEE International Symposium on, 2012, pp. 2042–2046. IEEE.
- [31] Y. Liu, T. Mi, S. Li, Compressed sensing with general frames via optimal-dual-based-analysis, *Inf. Theory IEEE Trans.* 58 (7) (2012) 4201–4214.
- [32] J. Ma, Generalized sampling reconstruction from fourier measurements using compactly supported shearlets, *Appl. Comput. Harmon. Anal.* 1;42 (2) (2017) 294–318.
- [33] D. Needell, J.A. Tropp, Cosamp: iterative signal recovery from incomplete and inaccurate samples, *Appl. Comput. Harmon. Anal.* 26 (3) (2009) 301–321.
- [34] K.P. Pruessmann, M. Weiger, M.B. Scheidegger, P. Boesiger, Sense: sensitivity encoding for fast MRI, *Magnetic Resonance Med.* 42 (5) (1999).
- [35] H. Rauhut, Compressive sensing and structured random matrices, *Theor. Found. Numer. Method. Sparse Recov.* 9 (2010) 1–92.
- [36] H. Rauhut, K. Schnass, P. Vandergheynst, Compressed sensing and redundant dictionaries, *Inf. Theory IEEE Trans.* 54 (5) (2008) 2210–2219.
- [37] M. Rudelson, R. Vershynin, Smallest singular value of a random rectangular matrix, *Commun. Pure Appl. Math.* 62 (12) (2009) 1707–1739.
- [38] C. Zeng, S. Li, Sparse measurements and fused sar imaging, 2015, In preparation.
- [39] T. Zhang, Sparse recovery with orthogonal matching pursuit under rip, *Inf. Theory IEEE Trans.* 57 (9) (2011) 6215–6221.



Hard thresholding pursuit algorithms: Number of iterations[☆]



Jean-Luc Bouchot, Simon Foucart^{*}, Pawel Hitczenko

ARTICLE INFO

Article history:

Received 13 February 2015

Received in revised form 27

November 2015

Accepted 1 March 2016

Available online 7 March 2016

Communicated by Spec. Issue Guest Editor

MSC:

65F10

65J20

15A29

94A12

Keywords:

Compressive sensing

Uniform sparse recovery

Nonuniform sparse recovery

Random measurements

Iterative algorithms

Hard thresholding

ABSTRACT

The Hard Thresholding Pursuit algorithm for sparse recovery is revisited using a new theoretical analysis. The main result states that all sparse vectors can be exactly recovered from compressive linear measurements in a number of iterations at most proportional to the sparsity level as soon as the measurement matrix obeys a certain restricted isometry condition. The recovery is also robust to measurement error. The same conclusions are derived for a variation of Hard Thresholding Pursuit, called Graded Hard Thresholding Pursuit, which is a natural companion to Orthogonal Matching Pursuit and runs without a prior estimation of the sparsity level. In addition, for two extreme cases of the vector shape, it is shown that, with high probability on the draw of random measurements, a fixed sparse vector is robustly recovered in a number of iterations precisely equal to the sparsity level. These theoretical findings are experimentally validated, too.

© 2016 Elsevier Inc. All rights reserved.

1. Introduction

This paper deals with the standard compressive sensing problem, i.e., the reconstruction of vectors $\mathbf{x} \in \mathbb{C}^N$ from an incomplete set of $m \ll N$ linear measurements organized in the form $\mathbf{y} = \mathbf{Ax} \in \mathbb{C}^m$ for some matrix $\mathbf{A} \in \mathbb{C}^{m \times N}$. It is now well known that if $\mathbf{x}$ is s -sparse (i.e., has only s nonzero entries) and if $\mathbf{A}$ is a random matrix whose number m of rows scales like s times some logarithmic factors, then the reconstruction of $\mathbf{x}$ is achievable via a variety of methods. The ℓ_1 -minimization is probably the most popular one, but simple iterative algorithms do provide alternative methods. We consider here the hard thresholding pursuit (HTP) algorithm [6] as well as a novel variation and we focus on the number of iterations needed for the reconstruction. This reconstruction is addressed in two settings:

[☆] S.F. and J.-L.B. partially supported by NSF (DMS-1120622), P.H. by Simons Foundation (grant 208766).

^{*} Corresponding author.

E-mail address: simon.foucart@centraliens.net (S. Foucart).

- An idealized situation, where the vectors $\mathbf{x} \in \mathbb{C}^N$ are exactly sparse and where the measurements $\mathbf{y} \in \mathbb{C}^m$ are exactly equal to $\mathbf{Ax}$. In this case, the exact reconstruction of $\mathbf{x} \in \mathbb{C}^N$ is targeted.
- A realistic situation, where the vectors $\mathbf{x} \in \mathbb{C}^N$ are not exactly sparse and where the measurements $\mathbf{y} \in \mathbb{C}^m$ contain errors $\mathbf{e} \in \mathbb{C}^m$, i.e., $\mathbf{y} = \mathbf{Ax} + \mathbf{e}$. In this case, only an approximate reconstruction of $\mathbf{x} \in \mathbb{C}^N$ is targeted. Precisely, the reconstruction error should be controlled by the sparsity defect and by the measurement error. The sparsity defect can be incorporated in the measurement error if $\mathbf{y} = \mathbf{Ax} + \mathbf{e}$ is rewritten as $\mathbf{y} = \mathbf{Ax}_S + \mathbf{e}'$ where S is an index set of s largest absolute entries of $\mathbf{x}$ and $\mathbf{e}' := \mathbf{Ax}_{\bar{S}} + \mathbf{e}$.

We shall mainly state and prove our results in the realistic situation. They specialize to the idealized situation simply by setting $\mathbf{e}' = \mathbf{0}$. In fact, setting $\mathbf{e}' = \mathbf{0}$ inside the proofs would simplify them considerably.

Let us now recall that (HTP) consists in constructing a sequence $(\mathbf{x}^n)$ of s -sparse vectors, starting with an initial s -sparse $\mathbf{x}^0 \in \mathbb{C}^N$ — we take $\mathbf{x}^0 = \mathbf{0}$ — and iterating the scheme¹

$$S^n := \text{index set of } s \text{ largest absolute entries of } \mathbf{x}^{n-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{Ax}^{n-1}), \quad (\text{HTP}_1)$$

$$\mathbf{x}^n := \operatorname{argmin}\{\|\mathbf{y} - \mathbf{Az}\|_2, \operatorname{supp}(\mathbf{z}) \subseteq S^n\}, \quad (\text{HTP}_2)$$

until a stopping criterion is met. It had been shown [10] that, in the idealized situation, exact reconstruction of every s -sparse $\mathbf{x} \in \mathbb{C}^N$ is achieved in s iterations of (HTP) with $\mathbf{y} = \mathbf{Ax}$ provided the coherence of the matrix $\mathbf{A} \in \mathbb{C}^{m \times N}$ satisfies $\mu < 1/(3s)$ (note that this condition can be fulfilled when $m \asymp s^2$). Exact and approximate reconstructions were treated in [6], where it was in particular shown that every s -sparse $\mathbf{x} \in \mathbb{C}^N$ is the limit of the sequence $(\mathbf{x}^n)$ produced by (HTP) with $\mathbf{y} = \mathbf{Ax}$ provided the 3sth restricted isometry constant of the measurement matrix $\mathbf{A} \in \mathbb{C}^{m \times N}$ obeys $\delta_{3s} < 1/\sqrt{3} \approx 0.577$ (note that this condition can be fulfilled when $m \asymp s \ln(N/s)$). As a reminder, the k th restricted isometry constant δ_k of $\mathbf{A}$ is defined as the smallest constant $\delta \geq 0$ such that

$$(1 - \delta)\|\mathbf{z}\|_2^2 \leq \|\mathbf{Az}\|_2^2 \leq (1 + \delta)\|\mathbf{z}\|_2^2 \quad \text{for all } k\text{-sparse } \mathbf{z} \in \mathbb{C}^N.$$

In fact, it was shown in [6] that the convergence is achieved in a finite number $\bar{n}$ of iterations. In the idealized situation, it can be estimated as

$$\bar{n} \leq \left\lceil \frac{\ln(\sqrt{2/3}\|\mathbf{x}\|_2/\xi)}{\ln(1/\rho_{3s})} \right\rceil, \quad \rho_{3s} := \sqrt{\frac{2\delta_{3s}^2}{1 - \delta_{3s}^2}} < 1, \quad \xi := \min_{j \in \operatorname{supp}(\mathbf{x})} |x_j|. \quad (1)$$

This paper establishes that the number of iterations can be estimated independently of the shape of $\mathbf{x}$: under a restricted isometry condition, it is at most proportional to the sparsity s , see Theorem 5 and a robust version in Theorem 6. This is reminiscent of the work of T. Zhang [15] on orthogonal matching pursuit (OMP), see also [7, Theorem 6.25] where it is proved that $\bar{n} \leq 12s$ provided that $\delta_{13s} < 1/6$.

However, (HTP) presents a significant drawback in that a prior estimation of the sparsity s is required to run the algorithm, while (OMP) does not (although stopping (OMP) at iteration $12s$ does require this estimation). We therefore consider a variation of (HTP) avoiding the prior estimation of s . We call it graded hard thresholding pursuit (GHTP) algorithm, because the index set has a size that increases with the iteration. Precisely, starting with $\mathbf{x}^0 = \mathbf{0}$, a sequence $(\mathbf{x}^n)$ of n -sparse vectors is constructed according to

¹ Exact arithmetic is assumed and, among several candidates for the index set of largest absolute entries, the smallest one in lexicographic order is always chosen.

$$S^n := \text{index set of } n \text{ largest absolute entries of } \mathbf{x}^{n-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n-1}), \quad (\text{GHTP}_1)$$

$$\mathbf{x}^n := \operatorname{argmin}\{\|\mathbf{y} - \mathbf{A}\mathbf{z}\|_2, \operatorname{supp}(\mathbf{z}) \subseteq S^n\}, \quad (\text{GHTP}_2)$$

until a stopping criterion is met. For (GHTP), too, it is established that a restricted isometry condition implies that the number of iterations needed for robust reconstruction is at most proportional to the sparsity s , see [Theorem 8](#).

One expects the number of (GHTP) iterations needed for s -sparse recovery to be exactly equal to s , though. Such a statement is indeed proved, but in the nonuniform setting rather than in the uniform setting (i.e., when successful recovery is sought with high probability on the draw of random measurements for a fixed $\mathbf{x} \in \mathbb{C}^N$ rather than for all $\mathbf{x} \in \mathbb{C}^N$ simultaneously) and in two extreme cases on the shape of the s -sparse vector to be recovered. The first case deals with vectors that are ‘flat’ on their support: robust nonuniform recovery is guaranteed with $m \asymp s \ln(N)$ Gaussian measurements, see [Proposition 9](#). The second case deals with vectors that are decaying exponentially fast on their support: robust nonuniform recovery is guaranteed with $m \asymp \max\{s, \ln(N)\}$ Gaussian measurements or with $m \asymp s \ln(N)$ Fourier measurements, see [Proposition 10](#). This is an improvement (attenuated by the extra shape condition) on uniform recovery results based on the restricted isometry property that require $m \asymp s \ln(N/s)$ Gaussian measurements or (currently) $m \asymp s \ln^4(N)$ Fourier measurements.

To conclude, the (HTP) and (GHTP) algorithms are tested numerically and benchmarked against the (OMP) algorithm. Their performance is roughly similar in terms of success rate, with different behaviors mostly explained by the shape of the vectors to be recovered. In terms of number of iterations, however, (HTP) does perform best, owing to the fact that an a priori knowledge of the sparsity s is used to run the algorithm (which also provides a convenient stopping criterion). It also seems that (GHTP) always requires less iterations than (OMP). The empirical observations that ‘flat’ vectors are least favorable, coupled with the theoretical findings of [Sections 3 and 5](#), support the beliefs that uniform recovery via (HTP) is achievable in much less iterations than the upper bound cs of [Theorem 5](#) and that nonuniform recovery via (GHTP) is achievable in exactly s iterations whatever the shape of the vector. Finally, the numerical part ends with an experimental study of the influence of the measurement error.

Throughout the paper, we make use of the following notation:

- $\mathbf{x}^* \in \mathbb{R}_+^N$ is the nonincreasing rearrangement of a vector $\mathbf{x} \in \mathbb{C}^N$, i.e.,

$$x_1^* \geq x_2^* \geq \dots \geq x_N^* \geq 0$$

and there exists a permutation π of $\{1, \dots, N\}$ such that $x_j^* = |x_{\pi(j)}|$ for all $j \in \{1, \dots, N\}$;

- S is the support or an index set of s largest absolute entries of a vector $\mathbf{x} \in \mathbb{C}^N$;
- $\mathbf{x}_T$ is the restriction of a vector $\mathbf{x} \in \mathbb{C}^N$ to an index set T — depending on the context, it is interpreted either as a vector in $\mathbb{C}^T$ or as a vector in $\mathbb{C}^N$;
- $\bar{T}$ is the complementary set of an index set T , i.e., $\bar{T} = \{1, \dots, N\} \setminus T$;
- $T \Delta T'$ is the symmetric difference of sets T and T' , i.e., $T \Delta T' = (T \setminus T') \cup (T' \setminus T)$;
- ρ_s, τ_s are quantities associated to the s th restricted isometry constant $\delta_s < 1$ via

$$\rho_s = \sqrt{\frac{2\delta_s^2}{1 - \delta_s^2}}, \quad \tau_s = \sqrt{\frac{2}{1 - \delta_s}} + \frac{\sqrt{1 + \delta_s}}{1 - \delta_s}. \quad (2)$$

2. Preliminary results

This section collects the key facts needed in the subsequent analyses of (HTP) and (GHTP). It also includes a discussion on appropriate stopping criteria.

2.1. Extensions of previous arguments

The arguments used in [6] to prove the convergence of the sequence $(\mathbf{x}^n)$ produced by (HTP) in the idealized situation, say, relied on the exponential decay of $\|\mathbf{x} - \mathbf{x}^n\|_2$, precisely

$$\|\mathbf{x} - \mathbf{x}^n\|_2 \leq \rho_{3s} \|\mathbf{x} - \mathbf{x}^{n-1}\|_2, \quad n \geq 1,$$

where $\rho_{3s} < 1$ provided $\delta_{3s} < 1/\sqrt{3}$. This followed from two inequalities derived as results of (HTP₁) and (HTP₂), namely

$$\|\mathbf{x}_{\overline{S}^n}\|_2 \leq \sqrt{2\delta_{3s}^2} \|\mathbf{x} - \mathbf{x}^{n-1}\|_2, \quad (3)$$

$$\|\mathbf{x} - \mathbf{x}^n\|_2 \leq \sqrt{\frac{1}{1 - \delta_{2s}^2}} \|\mathbf{x}_{\overline{S}^n}\|_2. \quad (4)$$

For the analyses of (HTP) and (GHTP), similar arguments are needed in the more general form below.

1. Consequence of (HTP₁)–(GHTP₁).

If $\mathbf{x} \in \mathbb{C}^N$ is s -sparse, if $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{e}$ for some $\mathbf{e} \in \mathbb{C}^m$, if $\mathbf{x}' \in \mathbb{C}^N$ is s' -sparse, and if T is an index set of $t \geq s$ largest absolute entries of $\mathbf{x}' + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}')$, then

$$\|\mathbf{x}_T\|_2 \leq \sqrt{2\delta_{s+s'+t}^2} \|\mathbf{x} - \mathbf{x}'\|_2 + \sqrt{2} \|(\mathbf{A}^*\mathbf{e})_{S\Delta T}\|_2. \quad (5)$$

Indeed, with $S := \text{supp}(\mathbf{x})$, we first notice that

$$\|(\mathbf{x}' + \mathbf{A}^*\mathbf{A}(\mathbf{x} - \mathbf{x}') + \mathbf{A}^*\mathbf{e})_T\|_2^2 \geq \|(\mathbf{x}' + \mathbf{A}^*\mathbf{A}(\mathbf{x} - \mathbf{x}') + \mathbf{A}^*\mathbf{e})_S\|_2^2.$$

Eliminating the contribution on $S \cap T$ gives

$$\|(\mathbf{x}' + \mathbf{A}^*\mathbf{A}(\mathbf{x} - \mathbf{x}') + \mathbf{A}^*\mathbf{e})_{T \setminus S}\|_2 \geq \|(\mathbf{x}' + \mathbf{A}^*\mathbf{A}(\mathbf{x} - \mathbf{x}') + \mathbf{A}^*\mathbf{e})_{S \setminus T}\|_2.$$

The left-hand side satisfies

$$\|(\mathbf{x}' + \mathbf{A}^*\mathbf{A}(\mathbf{x} - \mathbf{x}') + \mathbf{A}^*\mathbf{e})_{T \setminus S}\|_2 = \|((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}') + \mathbf{A}^*\mathbf{e})_{T \setminus S}\|_2,$$

while the right-hand side satisfies

$$\|(\mathbf{x}' + \mathbf{A}^*\mathbf{A}(\mathbf{x} - \mathbf{x}') + \mathbf{A}^*\mathbf{e})_{S \setminus T}\|_2 \geq \|\mathbf{x}_{S \setminus T}\|_2 - \|((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}') + \mathbf{A}^*\mathbf{e})_{S \setminus T}\|_2.$$

Therefore, we obtain

$$\begin{aligned} \|\mathbf{x}_{S \setminus T}\|_2 &\leq \|((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}') + \mathbf{A}^*\mathbf{e})_{T \setminus S}\|_2 + \|((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}') + \mathbf{A}^*\mathbf{e})_{S \setminus T}\|_2 \\ &\leq \sqrt{2} \|((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}') + \mathbf{A}^*\mathbf{e})_{T \Delta S}\|_2 \\ &\leq \sqrt{2} \|((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}'))_{T \Delta S}\|_2 + \sqrt{2} \|(\mathbf{A}^*\mathbf{e})_{T \Delta S}\|_2 \\ &\leq \sqrt{2} \delta_{s+s'+t} \|\mathbf{x} - \mathbf{x}'\|_2 + \sqrt{2} \|(\mathbf{A}^*\mathbf{e})_{T \Delta S}\|_2, \end{aligned} \quad (6)$$

where the last step used (3.8) of [6]. The resulting inequality is the one announced in (5).

2. Consequence of (HTP₂)–(GHTP₂).

If $\mathbf{x} \in \mathbb{C}^N$ is s -sparse, if $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{e}$ for some $\mathbf{e} \in \mathbb{C}^m$, if T is an index set of size t , and if $\mathbf{x}'$ is a minimizer of $\|\mathbf{y} - \mathbf{A}\mathbf{z}\|_2$ subject to $\text{supp}(\mathbf{z}) \subseteq T$, then

$$\|\mathbf{x} - \mathbf{x}'\|_2 \leq \sqrt{\frac{1}{1 - \delta_{s+t}^2}} \|\mathbf{x}_{\bar{T}}\|_2 + \frac{1}{1 - \delta_{s+t}} \|(\mathbf{A}^* \mathbf{e})_T\|_2. \quad (7)$$

Indeed, the vector $\mathbf{x}'$ is characterized by the orthogonality condition $(\mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}'))_T = \mathbf{0}$ (in other words, by the normal equations), therefore

$$\begin{aligned} \|(\mathbf{x} - \mathbf{x}')_T\|_2^2 &= \langle (\mathbf{x} - \mathbf{x}')_T, (\mathbf{x} - \mathbf{x}')_T \rangle = \langle (\mathbf{x} - \mathbf{x}' - \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}'))_T, (\mathbf{x} - \mathbf{x}')_T \rangle \\ &\leq |\langle (\mathbf{I} - \mathbf{A}^* \mathbf{A})(\mathbf{x} - \mathbf{x}')_T, (\mathbf{x} - \mathbf{x}')_T \rangle| + |\langle (\mathbf{A}^* \mathbf{e})_T, (\mathbf{x} - \mathbf{x}')_T \rangle| \\ &\leq \|(\mathbf{I} - \mathbf{A}^* \mathbf{A})(\mathbf{x} - \mathbf{x}')_T\|_2 \|(\mathbf{x} - \mathbf{x}')_T\|_2 + \|(\mathbf{A}^* \mathbf{e})_T\|_2 \|(\mathbf{x} - \mathbf{x}')_T\|_2 \\ &\leq \delta_{s+t} \|\mathbf{x} - \mathbf{x}'\|_2 \|(\mathbf{x} - \mathbf{x}')_T\|_2 + \|(\mathbf{A}^* \mathbf{e})_T\|_2 \|(\mathbf{x} - \mathbf{x}')_T\|_2, \end{aligned} \quad (8)$$

where the last step used (3.8) of [6]. After dividing through by $\|(\mathbf{x} - \mathbf{x}')_T\|_2$, we obtain

$$\|(\mathbf{x} - \mathbf{x}')_T\|_2 \leq \delta_{s+t} \|\mathbf{x} - \mathbf{x}'\|_2 + \|(\mathbf{A}^* \mathbf{e})_T\|_2.$$

It then follows that

$$\|\mathbf{x} - \mathbf{x}'\|_2^2 = \|(\mathbf{x} - \mathbf{x}')_T\|_2^2 + \|(\mathbf{x} - \mathbf{x}')_{\bar{T}}\|_2^2 \leq (\delta_{s+t} \|\mathbf{x} - \mathbf{x}'\|_2 + \|(\mathbf{A}^* \mathbf{e})_T\|_2)^2 + \|\mathbf{x}_{\bar{T}}\|_2^2,$$

in other words $P(\|\mathbf{x} - \mathbf{x}'\|_2) \leq 0$, where the quadratic polynomial P is defined by

$$P(z) = (1 - \delta_{s+t}^2)z^2 - 2\delta_{s+t} \|(\mathbf{A}^* \mathbf{e})_T\|_2 z - \|(\mathbf{A}^* \mathbf{e})_T\|_2^2 - \|\mathbf{x}_{\bar{T}}\|_2^2.$$

This implies that $\|\mathbf{x} - \mathbf{x}'\|_2$ is smaller than the largest root of P , i.e.,

$$\begin{aligned} \|\mathbf{x} - \mathbf{x}'\|_2 &\leq \frac{\delta_{s+t} \|(\mathbf{A}^* \mathbf{e})_T\|_2 + \sqrt{\|(\mathbf{A}^* \mathbf{e})_T\|_2^2 + (1 - \delta_{s+t}^2) \|\mathbf{x}_{\bar{T}}\|_2^2}}{1 - \delta_{s+t}^2} \\ &\leq \frac{(1 + \delta_{s+t}) \|(\mathbf{A}^* \mathbf{e})_T\|_2 + \sqrt{(1 - \delta_{s+t}^2) \|\mathbf{x}_{\bar{T}}\|_2^2}}{1 - \delta_{s+t}^2}. \end{aligned}$$

This is the inequality announced in (7).

Remark 1. The full restricted isometry property is not needed to derive (5) and (7). The restricted isometry constants only appear in (6) and (8), where $\|(\mathbf{I} - \mathbf{A}^* \mathbf{A})(\mathbf{x} - \mathbf{x}')_U\|_2$ is bounded for $U = T \Delta S$ and $U = T$. This is easier to fulfill than the restricted isometry property if $\text{supp}(\mathbf{x})$, $\text{supp}(\mathbf{x}')$, and T are all included in a fixed index set, as will be the case in Section 5.

From inequalities (5) and (7), we deduce the decay of the sequences $(\|\mathbf{x} - \mathbf{x}^n\|_2)_{n \geq 0}$ for (HTP) and $(\|\mathbf{x} - \mathbf{x}^n\|_2)_{n \geq s}$ for (GHTP).

Corollary 2. Let $\mathbf{x} \in \mathbb{C}^N$ be s -sparse and let $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{e}$ for some $\mathbf{e} \in \mathbb{C}^m$. If $(\mathbf{x}^n)$ is the sequence produced by (HTP), then

$$\|\mathbf{x} - \mathbf{x}^n\|_2 \leq \rho_{3s} \|\mathbf{x} - \mathbf{x}^{n-1}\|_2 + \tau_{2s} \|\mathbf{e}\|_2 \quad \text{for any } n \geq 1. \quad (9)$$

If $(\mathbf{x}^n)$ is the sequence produced by (GHTP), then

$$\|\mathbf{x} - \mathbf{x}^n\|_2 \leq \rho_{s+2n-1} \|\mathbf{x} - \mathbf{x}^{n-1}\|_2 + \tau_{s+n} \|\mathbf{e}\|_2 \quad \text{for any } n \geq s. \quad (10)$$

Proof. We consider (HTP) first. Applying (5) to $\mathbf{x}' = \mathbf{x}^{n-1}$ and $T = S^n$ gives

$$\|\mathbf{x}_{\overline{S^n}}\|_2 \leq \sqrt{2\delta_{3s}^2} \|\mathbf{x} - \mathbf{x}^{n-1}\|_2 + \sqrt{2} \|(\mathbf{A}^* \mathbf{e})_{S \Delta S^n}\|_2$$

and applying (7) to $T = S^n$ and $\mathbf{x}' = \mathbf{x}^n$ gives

$$\|\mathbf{x} - \mathbf{x}^n\|_2 \leq \sqrt{\frac{1}{1 - \delta_{2s}^2}} \|\mathbf{x}_{\overline{S^n}}\|_2 + \frac{1}{1 - \delta_{2s}} \|(\mathbf{A}^* \mathbf{e})_{S^n}\|_2.$$

Combining these two inequalities, while using $\delta_{2s} \leq \delta_{3s}$, yields

$$\|\mathbf{x} - \mathbf{x}^n\|_2 \leq \sqrt{\frac{2\delta_{3s}^2}{1 - \delta_{3s}^2}} \|\mathbf{x} - \mathbf{x}^{n-1}\|_2 + \left(\sqrt{\frac{2}{1 - \delta_{2s}^2}} + \frac{1}{1 - \delta_{2s}} \right) \|(\mathbf{A}^* \mathbf{e})_{S \cup S^n}\|_2.$$

In view of $\|(\mathbf{A}^* \mathbf{e})_{S \cup S^n}\|_2 \leq \sqrt{1 + \delta_{2s}} \|\mathbf{e}\|_2$ (see e.g. (3.19) in [6]), we recognize (9) with the values of ρ_{3s} and τ_{2s} given in (2).

Let us now consider (GHTP). Applying (5) to $\mathbf{x}' = \mathbf{x}^{n-1}$ and $T = S^n$ for $n \geq s$ gives

$$\|\mathbf{x}_{\overline{S^n}}\|_2 \leq \sqrt{2\delta_{s+2n-1}^2} \|\mathbf{x} - \mathbf{x}^{n-1}\|_2 + \sqrt{2} \|(\mathbf{A}^* \mathbf{e})_{S \Delta S^n}\|_2$$

and applying (7) to $T = S^n$ and $\mathbf{x}' = \mathbf{x}^n$ gives

$$\|\mathbf{x} - \mathbf{x}^n\|_2 \leq \sqrt{\frac{1}{1 - \delta_{s+n}^2}} \|\mathbf{x}_{\overline{S^n}}\|_2 + \frac{1}{1 - \delta_{s+n}} \|(\mathbf{A}^* \mathbf{e})_{S^n}\|_2.$$

Combining these two inequalities, while using $\delta_{s+n} \leq \delta_{s+2n-1}$, yields

$$\|\mathbf{x} - \mathbf{x}^n\|_2 \leq \sqrt{\frac{2\delta_{s+2n-1}^2}{1 - \delta_{s+2n-1}^2}} \|\mathbf{x} - \mathbf{x}^{n-1}\|_2 + \left(\sqrt{\frac{2}{1 - \delta_{s+n}^2}} + \frac{1}{1 - \delta_{s+n}} \right) \|(\mathbf{A}^* \mathbf{e})_{S \cup S^n}\|_2.$$

In view of $\|(\mathbf{A}^* \mathbf{e})_{S \cup S^n}\|_2 \leq \sqrt{1 + \delta_{s+n}} \|\mathbf{e}\|_2$, we recognize (10) with the values of ρ_{s+2n-1} and τ_{s+n} given in (2). $\square$

2.2. Additional arguments

While the above argument probed only the closeness of $\mathbf{x}^n$ to $\mathbf{x}$, the following lemmas examine the indices of nonzero entries of $\mathbf{x}$ that are correctly captured in the support sets produced by (HTP) and (GHTP). These lemmas are the main addition to the earlier proof techniques. They show how many iterations are necessary to increase the number of correct indices by a specified amount. The first lemma concerns (HTP).

Lemma 3. Let $\mathbf{x} \in \mathbb{C}^N$ be s -sparse and let (S^n) be the sequence of index sets produced by (HTP) with $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{e}$ for some $\mathbf{e} \in \mathbb{C}^m$. For integers $n, p \geq 0$, suppose that S^n contains the indices of p largest

absolute entries of $\mathbf{x}$. Then, for integers $k, q \geq 1$, S^{n+k} contains the indices of $p+q$ largest absolute entries of $\mathbf{x}$, provided

$$x_{p+q}^* > \rho_{3s}^k \|\mathbf{x}_{\{p+1, \dots, s\}}^*\|_2 + \kappa_{3s} \|\mathbf{e}\|_2, \quad (11)$$

where the constant κ_{3s} depends only on δ_{3s} . For instance, $\delta_{3s} < 1/3$ yields $\kappa_{3s} \leq 5.72$.

Proof. Let π be the permutation of $\{1, 2, \dots, N\}$ for which $|x_{\pi(j)}| = x_j^*$ for all $j \in \{1, 2, \dots, N\}$. Our hypothesis is $\pi(\{1, \dots, p\}) \subseteq S^n$ and our aim is to prove that $\pi(\{1, \dots, p+q\}) \subseteq S^{n+k}$. In other words, we aim at proving that the $|(\mathbf{x}^{n+k-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n+k-1}))_{\pi(j)}|$ for $j \in \{1, \dots, p+q\}$ are among the s largest values of $|(\mathbf{x}^{n+k-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n+k-1}))_i|$ for $i \in \{1, \dots, N\}$. With $\text{supp}(\mathbf{x}) \subseteq S$, $\text{card}(S) = s$, it is enough to show that

$$\min_{j \in \{1, \dots, p+q\}} |(\mathbf{x}^{n+k-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n+k-1}))_{\pi(j)}| > \max_{\ell \in \overline{S}} |(\mathbf{x}^{n+k-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n+k-1}))_\ell|. \quad (12)$$

We notice that, for $j \in \{1, \dots, p+q\}$ and $\ell \in \overline{S}$,

$$\begin{aligned} |(\mathbf{x}^{n+k-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n+k-1}))_{\pi(j)}| &\geq |x_{\pi(j)}| - |(-\mathbf{x} + \mathbf{x}^{n+k-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n+k-1}))_{\pi(j)}| \\ &\geq x_{p+q}^* - |((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n+k-1}) + \mathbf{A}^*\mathbf{e})_{\pi(j)}|, \\ |(\mathbf{x}^{n+k-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n+k-1}))_\ell| &= |(-\mathbf{x} + \mathbf{x}^{n+k-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n+k-1}))_\ell| \\ &= |((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n+k-1}) + \mathbf{A}^*\mathbf{e})_\ell|. \end{aligned}$$

Therefore, (12) will be shown as soon as it is proved that, for all $j \in \{1, \dots, p+q\}$ and $\ell \in \overline{S}$,

$$x_{p+q}^* > |((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n+k-1}) + \mathbf{A}^*\mathbf{e})_{\pi(j)}| + |((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n+k-1}) + \mathbf{A}^*\mathbf{e})_\ell|. \quad (13)$$

The right-hand side can be bounded by

$$\begin{aligned} &\sqrt{2} \|((\mathbf{I} - \mathbf{A}^*\mathbf{A})(\mathbf{x} - \mathbf{x}^{n+k-1}) + \mathbf{A}^*\mathbf{e})_{\{\pi(j), \ell\}}\|_2 \\ &\leq \sqrt{2} \|((\mathbf{I} - \mathbf{A}^*\mathbf{A})(\mathbf{x} - \mathbf{x}^{n+k-1}))_{\{\pi(j), \ell\}}\|_2 + \sqrt{2} \|(\mathbf{A}^*\mathbf{e})_{\{\pi(j), \ell\}}\|_2 \\ &\leq \sqrt{2} \delta_{2s+2} \|\mathbf{x} - \mathbf{x}^{n+k-1}\|_2 + \sqrt{2} \sqrt{1 + \delta_2} \|\mathbf{e}\|_2 \\ &\leq \sqrt{2} \delta_{3s} \left(\rho_{3s}^{k-1} \|\mathbf{x} - \mathbf{x}^n\|_2 + \frac{\tau_{2s}}{1 - \rho_{3s}} \|\mathbf{e}\|_2 \right) + \sqrt{2(1 + \delta_{2s})} \|\mathbf{e}\|_2, \end{aligned}$$

where (9) was used $k-1$ times in the last step. Using (7) and the assumption that $\pi(\{1, \dots, p\}) \subseteq S^n$, we derive

$$\begin{aligned} &\sqrt{2} \|((\mathbf{I} - \mathbf{A}^*\mathbf{A})(\mathbf{x} - \mathbf{x}^{n+k-1}) + \mathbf{A}^*\mathbf{e})_{\{\pi(j), \ell\}}\|_2 \\ &\leq \sqrt{2} \delta_{3s} \rho_{3s}^{k-1} \left(\frac{1}{\sqrt{1 - \delta_{2s}^2}} \|\mathbf{x}_{\overline{S}^n}\|_2 + \frac{1}{1 - \delta_{2s}} \|(\mathbf{A}^*\mathbf{e})_{S^n}\|_2 \right) + \left(\frac{\sqrt{2} \delta_{3s} \tau_{2s}}{1 - \rho_{3s}} + \sqrt{2(1 + \delta_{2s})} \right) \|\mathbf{e}\|_2 \\ &\leq \rho_{3s}^k \|\mathbf{x}_{\pi(\{1, \dots, p\})}\|_2 + \left(\frac{\sqrt{2} \delta_{3s} \rho_{3s}^{k-1} \sqrt{1 + \delta_s}}{1 - \delta_{2s}} + \frac{\sqrt{2} \delta_{3s} \tau_{2s}}{1 - \rho_{3s}} + \sqrt{2(1 + \delta_{2s})} \right) \|\mathbf{e}\|_2 \\ &\leq \rho_{3s}^k \|\mathbf{x}_{\{p+1, \dots, s\}}^*\|_2 + \kappa_{3s} \|\mathbf{e}\|_2, \end{aligned}$$

where the constant κ_{3s} is determined by

$$\kappa_{3s} = \frac{\sqrt{2(1+\delta_{3s})}}{1-\delta_{3s}} + \frac{\sqrt{2}\delta_{3s}\tau_{3s}}{1-\rho_{3s}}.$$

Therefore, (13) is proved as soon as condition (11) holds. The proof is completed by noticing that $\delta_{3s} \leq 1/3$ yields $\rho_{3s} \leq 1/2$, $\tau_{3s} \leq 2\sqrt{3}$, and finally $\kappa_{3s} \leq 14/\sqrt{6}$. $\square$

For (GHTP), an analog of Lemma 3 is needed in the form below.

Lemma 4. *Let $\mathbf{x} \in \mathbb{C}^N$ be s -sparse and let (S^n) be the sequence of index sets produced by (GHTP) with $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{e}$ for some $\mathbf{e} \in \mathbb{C}^m$. For integers $n \geq s$ and $p \geq 0$, suppose that S^n contains the indices of p largest absolute entries of $\mathbf{x}$. Then, for integers $k, q \geq 1$, S^{n+k} contains the indices of $p+q$ largest absolute entries of $\mathbf{x}$, provided*

$$x_{p+q}^* > \rho_{s+2n+2k}^k \|\mathbf{x}_{\{p+1, \dots, s\}}^*\|_2 + \kappa_{s+2n+2k} \|\mathbf{e}\|_2, \quad (14)$$

where the constant $\kappa_{s+2n+2k}$ depends only on $\delta_{s+2n+2k}$.

Proof. Given the permutation π of $\{1, 2, \dots, N\}$ for which $|x_{\pi(j)}| = x_j^*$ for all $j \in \{1, \dots, N\}$, our hypothesis is $\pi(\{1, \dots, p\}) \subseteq S^n$ and our aim is to prove that $\pi(\{1, \dots, p+q\}) \subseteq S^{n+k}$. For this purpose, we aim at proving that the $|(\mathbf{x}^{n+k-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n+k-1}))_{\pi(j)}|$ for $j \in \{1, \dots, p+q\}$ are among the $n+k$ largest values of $|(\mathbf{x}^{n+k-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n+k-1}))_i|$, $i \in \{1, \dots, N\}$. Since $n+k \geq s$, it is enough to show that they are among the s largest values of $|(\mathbf{x}^{n+k-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n+k-1}))_i|$, $i \in \{1, \dots, N\}$. The rest of the proof duplicates the proof of Lemma 3 modulo the change imposed by (10) for the indices of δ and κ . $\square$

2.3. Stopping the (HTP) and (GHTP) algorithms

The stopping criterion, which is an integral part of the algorithm, has not been addressed yet. We take a few moments to discuss this issue here. For (HTP), the natural stopping criterion² is $S^n = S^{n-1}$, since the algorithm always produce the same vector afterwards. Moreover, under the condition $\delta_{3s} < 1/\sqrt{3}$,

1. if $S^n = S^{n-1}$ occurs, then robustness is guaranteed, since $\mathbf{x}^n = \mathbf{x}^{n-1}$ substituted in (9) yields $\|\mathbf{x} - \mathbf{x}^{n-1}\|_2 \leq \rho_{3s} \|\mathbf{x} - \mathbf{x}^{n-1}\|_2 + \tau_{2s} \|\mathbf{e}\|_2$, i.e.,

$$\|\mathbf{x} - \mathbf{x}^{n-1}\|_2 \leq \frac{\tau_{2s}}{1 - \rho_{3s}} \|\mathbf{e}\|_2;$$

2. and $S^n = S^{n-1}$ does occur provided $\|\mathbf{e}\|_2 < 4[\tau_{2s}/(1 - \rho_{3s})] x_s^*$. Indeed, for $\|\mathbf{e}\|_2 > 0^3$ and for n large enough to have $\rho_{3s}^{n-1} \|\mathbf{x}\|_2 \leq [\tau_{2s}/(1 - \rho_{3s})] \|\mathbf{e}\|_2$, the inequality (9) applied $n-1$ times implies that $\|\mathbf{x} - \mathbf{x}^{n-1}\|_2 \leq \rho_{3s}^{n-1} \|\mathbf{x}\|_2 + [\tau_{2s}/(1 - \rho_{3s})] \|\mathbf{e}\|_2 \leq 2[\tau_{2s}/(1 - \rho_{3s})] \|\mathbf{e}\|_2$. In turn, for $j \in \text{supp}(\mathbf{x})$ and $\ell \notin \text{supp}(\mathbf{x})$, the inequalities

$$\begin{aligned} |x_j^{n-1}| &\geq |x_j| - |(\mathbf{x} - \mathbf{x}^{n-1})_j| \geq x_s^* - 2[\tau_{2s}/(1 - \rho_{3s})] \|\mathbf{e}\|_2 > 2[\tau_{2s}/(1 - \rho_{3s})] \|\mathbf{e}\|_2, \\ |x_\ell^{n-1}| &= |(\mathbf{x} - \mathbf{x}^{n-1})_\ell| \leq 2[\tau_{2s}/(1 - \rho_{3s})] \|\mathbf{e}\|_2 \end{aligned}$$

show that $S^{n-1} = \text{supp}(\mathbf{x})$. The same reasoning gives $S^n = \text{supp}(\mathbf{x})$, hence $S^n = S^{n-1}$.

² Again, exact arithmetic is assumed and the lexicographic rule to break ties applies.

³ In the case $\mathbf{e} = \mathbf{0}$, we have $\mathbf{x}^{n-1} = \mathbf{x}$ for some $n \geq 1$ (see (1)), hence $\mathbf{x}^n = \mathbf{x}$, and in particular $S^{n-1} = S^n$.

For (GHTP), a stopping criterion not requiring any estimation of s or $\|\mathbf{e}\|_2$ seems to be lost. It is worth recalling that iterative algorithms such as CoSaMP [12], IHT [2], or HTP [6] typically involve an estimation of s and that optimization methods such as quadratically constrained ℓ_1 -minimization typically involve an estimation of $\|\mathbf{e}\|_2$ — unless the measurement matrix satisfies the ℓ_1 -quotient property, see [14]. Here, we settle for a stopping criterion requiring an estimation of either one of s or $\|\mathbf{e}\|_2$, precisely (GHTP) is stopped either when $n = 4s$ or when $\|\mathbf{y} - \mathbf{Ax}^n\|_2 \leq d'\|\mathbf{e}\|_2$ for some constant $d' \geq 2.83$, say. Indeed, under the condition $\delta_{9s} < 1/3$, robustness is guaranteed from Theorem 8 by the fact that $\|\mathbf{x} - \mathbf{x}^{\bar{n}}\|_2 \leq d\|\mathbf{e}\|_2$ for some $\bar{n} \leq 4s$ and some $d \leq 2.45$. Thus, in the alternative $n = 4s$,

$$\|\mathbf{x} - \mathbf{x}^n\|_2 \leq \rho_{9s}^{4s-\bar{n}} \|\mathbf{x} - \mathbf{x}^{\bar{n}}\|_2 + \frac{\tau_{9s}}{1 - \rho_{9s}} \|\mathbf{e}\|_2 \leq \left(d + \frac{\tau_{9s}}{1 - \rho_{9s}}\right) \|\mathbf{e}\|_2 \leq d'' \|\mathbf{e}\|_2$$

and in the alternative $\|\mathbf{y} - \mathbf{Ax}^n\|_2 \leq d'\|\mathbf{e}\|_2$ (which is guaranteed to occur for some $n \leq \bar{n}$ by virtue of $\|\mathbf{y} - \mathbf{Ax}^{\bar{n}}\|_2 \leq \sqrt{1 + \delta_{9s}} \|\mathbf{x} - \mathbf{x}^{\bar{n}}\|_2 \leq \sqrt{1 + \delta_{9s}} d \|\mathbf{e}\|_2$),

$$\|\mathbf{x} - \mathbf{x}^n\|_2 \leq \frac{1}{\sqrt{1 - \delta_{9s}}} \|\mathbf{A}(\mathbf{x} - \mathbf{x}^n)\|_2 \leq \frac{1}{\sqrt{1 - \delta_{9s}}} (\|\mathbf{y} - \mathbf{Ax}^n\|_2 + \|\mathbf{e}\|_2) \leq \frac{d' + 1}{\sqrt{1 - \delta_{9s}}} \|\mathbf{e}\|_2 \leq d'' \|\mathbf{e}\|_2,$$

where $d'' := \max\{d + \tau_{9s}/(1 - \rho_{9s}), (d' + 1)/\sqrt{1 - \delta_{9s}}\} \leq 9.38$. We point out that, according to the result of [15], a similar stopping criterion involving either one of s or $\|\mathbf{e}\|_2$ can be used for (OMP). This makes the resemblance between (GHTP) and (OMP) even more compelling. Their empirical performances will be compared in Section 6.

3. Uniform recovery via hard thresholding pursuit

This section is dedicated to the main results about (HTP), namely that sparse recovery is achieved in a number of iterations at most proportional to the sparsity level, independently of the shape of the vector to be recovered. The results are uniform in the sense that the same measurement matrix allows for the recovery of all s -sparse vectors simultaneously. For clarity of exposition, we first state and prove a result in the idealized situation — only the realistic situation is considered thereafter.

Theorem 5. *If the restricted isometry constant of the matrix $\mathbf{A} \in \mathbb{C}^{m \times N}$ obeys*

$$\delta_{3s} \leq \frac{1}{\sqrt{5}},$$

then every s -sparse vector $\mathbf{x} \in \mathbb{C}^N$ is recovered from the measurement vector $\mathbf{y} = \mathbf{Ax} \in \mathbb{C}^m$ via a number $\bar{n}$ of iterations of (HTP) satisfying

$$\bar{n} \leq cs.$$

The constant $c \leq 3$ depends only on δ_{3s} . For instance, $\delta_{3s} \leq 1/3$ yields $c \leq 2$.

Proof. Let π be the permutation of $\{1, 2, \dots, N\}$ for which $|x_{\pi(j)}| = x_j^*$ for all $j \in \{1, 2, \dots, N\}$. Our goal is to prove that $\text{supp}(\mathbf{x}) \subseteq S^{\bar{n}}$ for $\bar{n} \leq 3s$, hence (HTP₂) ensures that $\mathbf{Ax}^{\bar{n}} = \mathbf{Ax}$ and $\mathbf{x}^{\bar{n}} = \mathbf{x}$ follows from $\delta_{3s} < 1$. For this purpose, we shall consider a partition $Q_1 \cup Q_2 \cup \dots \cup Q_r$, $r \leq s$, of $\text{supp}(\mathbf{x}) = \pi(\{1, \dots, s\})$. Lemma 3 enables to argue repeatedly that, if $Q_1 \cup \dots \cup Q_{i-1}$ has been correctly captured, then a suitable number k_i of additional (HTP) iterations captures $Q_1 \cup \dots \cup Q_{i-1} \cup Q_i$. The remaining step is just the estimation of $k_1 + \dots + k_r$. Let us set $q_0 = 0$ and let us define the sets $Q_1, \dots, Q_r$ inductively by

$$Q_i = \pi(\{q_{i-1} + 1, \dots, q_i\}), \quad q_i := \text{maximum index} \geq q_{i-1} + 1 \text{ such that } x_{q_i}^* > \frac{1}{\sqrt{2}} x_{q_{i-1}+1}^*. \quad (15)$$

We notice that $x_{q_i+1}^* \leq x_{q_{i-1}+1}^*/\sqrt{2}$ for all $i \in \{1, \dots, r-1\}$. With the artificial introduction of $Q_0 = \emptyset$ and $k_0 = 0$, we now prove by induction on $i \in \{0, \dots, r\}$ that

$$Q_0 \cup Q_1 \cup \dots \cup Q_i \subseteq S^{k_0+k_1+\dots+k_i}, \quad (16)$$

where $k_1, \dots, k_r$ are defined, with $\rho := \rho_{3s} < 1$, as

$$k_i := \left\lceil \frac{\ln(2(\text{card}(Q_i) + \text{card}(Q_{i+1})/2 + \dots + \text{card}(Q_r)/2^{r-i}))}{\ln(1/\rho^2)} \right\rceil. \quad (17)$$

For $i = 0$, the result holds trivially. Next, if (16) holds for $i-1$, $i \in \{1, \dots, r\}$, Lemma 3 (with $\mathbf{e} = \mathbf{0}$) guarantees that (16) holds for i , provided

$$(x_{q_i}^*)^2 > \rho^{2k_i} (\|\mathbf{x}_{Q_i}\|_2^2 + \|\mathbf{x}_{Q_{i+1}}\|_2^2 + \dots + \|\mathbf{x}_{Q_r}\|_2^2). \quad (18)$$

In view of $(x_{q_i}^*)^2 > (x_{q_{i-1}+1}^*)^2/2$ and of

$$\begin{aligned} & \|\mathbf{x}_{Q_i}\|_2^2 + \|\mathbf{x}_{Q_{i+1}}\|_2^2 + \dots + \|\mathbf{x}_{Q_r}\|_2^2 \\ & \leq (x_{q_{i-1}+1}^*)^2 \text{card}(Q_i) + (x_{q_i+1}^*)^2 \text{card}(Q_{i+1}) + \dots + (x_{q_{r-1}+1}^*)^2 \text{card}(Q_r) \\ & \leq (x_{q_{i-1}+1}^*)^2 \left(\text{card}(Q_i) + \frac{1}{2} \text{card}(Q_{i+1}) + \dots + \frac{1}{2^{r-i}} \text{card}(Q_r) \right), \end{aligned}$$

we verify that condition (18) is fulfilled thanks to the definition (17) of k_i . This concludes the inductive proof. We derive that the support $\pi(\{1, \dots, s\})$ of $\mathbf{x}$ is recovered in a number of iterations at most

$$\begin{aligned} \bar{n} &= \sum_{i=1}^r k_i \leq \sum_{i=1}^r \left(1 + \frac{\ln(2(\text{card}(Q_i) + \text{card}(Q_{i+1})/2 + \dots + \text{card}(Q_r)/2^{r-i}))}{\ln(1/\rho^2)} \right) \\ &= r + \frac{r}{\ln(1/\rho^2)} \sum_{i=1}^r \frac{1}{r} \ln(2(\text{card}(Q_i) + \text{card}(Q_{i+1})/2 + \dots + \text{card}(Q_r)/2^{r-i})). \end{aligned}$$

Using the concavity of the logarithm, we obtain

$$\begin{aligned} \ln(1/\rho^2) \frac{\bar{n} - r}{r} &\leq \ln \left(\sum_{i=1}^r \frac{2}{r} (\text{card}(Q_i) + \text{card}(Q_{i+1})/2 + \dots + \text{card}(Q_r)/2^{r-i}) \right) \\ &= \ln \left(\frac{2}{r} (\text{card}(Q_1) + (1 + 1/2)\text{card}(Q_2) + \dots + (1 + 1/2 + \dots + 1/2^{r-1})\text{card}(Q_r)) \right) \\ &\leq \ln \left(\frac{4}{r} (\text{card}(Q_1) + \text{card}(Q_2) + \dots + \text{card}(Q_r)) \right) = \ln \left(\frac{4s}{r} \right). \end{aligned}$$

Since $\ln(4x)/x \leq \ln(4)$ for $x \geq 1$, we deduce that

$$\ln(1/\rho^2) \frac{\bar{n} - r}{s} \leq \frac{\ln(4s/r)}{s/r} \leq \ln(4),$$

and we subsequently obtain

$$\bar{n} \leq r + \frac{\ln(4)}{\ln(1/\rho^2)} s \leq \left(1 + \frac{\ln(4)}{\ln(1/\rho^2)}\right) s = \frac{\ln(4/\rho^2)}{\ln(1/\rho^2)} s.$$

This is the desired result with $c = \ln(2(1 - \delta_{3s}^2)/\delta_{3s}^2)/\ln((1 - \delta_{3s}^2)/(2\delta_{3s}^2))$. We observe that $\delta_{3s} \leq 1/\sqrt{5}$ yields $c \leq \ln(8)/\ln(2) = 3$ and that $\delta_{3s} \leq 1/3$ yields $c \leq \ln(16)/\ln(4) = 2$. The proof is complete. $\square$

The approach above does not allow for the reduction of the number of iterations below $c s$. Indeed, if the s -sparse vector $\mathbf{x} \in \mathbb{C}^N$ has nonzero entries $x_j = \mu^j$ for $\mu \in (0, 1/2)$, then each set Q_i consists of one element, so there are s of these sets, and $\bar{n} = \sum_{i=1}^s k_i \geq s$. However, such a vector whose nonincreasing rearrangement decays exponentially fast is highly compressible — applying [Theorem 6](#) turns out to be more appropriate (see [Remark 7](#)). For other types of sparse vectors, the number of iterations can be significantly lowered. We give two examples below. In each of them, the partition involves a single set Q_1 . This corresponds to invoking [\(1\)](#).

Sparse vectors whose nonincreasing rearrangement decays slowly. For s -sparse vectors $\mathbf{x} \in \mathbb{C}^N$ satisfying $x_j^* = 1/j^\alpha$ for all $j \in \{1, \dots, s\}$ and for some $\alpha \geq 0$ (‘flat’ vectors are included with the choice $\alpha = 0$), the number of iterations is at most proportional to $\ln(s)$. Indeed, we have

$$\frac{\|\mathbf{x}\|_2^2}{(x_s^*)^2} = \frac{1 + \dots + 1/s^{2\alpha}}{1/s^{2\alpha}} \leq \frac{s}{1/s^{2\alpha}} = s^{2\alpha+1},$$

and it follows from [\(1\)](#) that

$$\bar{n} \leq \left\lceil \frac{\ln(\sqrt{2/3} s^{\alpha+1/2})}{\ln(1/\rho_{3s})} \right\rceil \leq c_{\delta_{3s}, \alpha} \ln(s).$$

Gaussian vectors. In numerical experiments, the nonzero entries of an s -sparse vector $\mathbf{g} \in \mathbb{C}^N$ are often taken as independent standard Gaussian random variables g_j , $j \in S$. In this case, with probability at least $1 - \varepsilon$, the vector $\mathbf{g}$ is recovered from $\mathbf{y} = \mathbf{A}\mathbf{g}$ with a number of (HTP) iterations at most proportional to $\ln(s/\varepsilon)$. Indeed, we establish below that

$$\mathbb{P}\left(g_s^* < \sqrt{\frac{\pi}{8}} \frac{\varepsilon}{s}\right) \leq \frac{\varepsilon}{2}, \quad (19)$$

$$\mathbb{P}\left(\|\mathbf{g}\|_2^2 > \frac{3\pi s}{2} \frac{\varepsilon}{s}\right) \leq \frac{\varepsilon}{2}. \quad (20)$$

Thus, with failure probability at most ε , we have $g_s^* \geq \sqrt{\pi/8} \varepsilon/s$ and $\|\mathbf{g}\|_2 \leq \sqrt{3\pi/2} \sqrt{s/\varepsilon}$. It follows from [\(1\)](#) that the number of iterations necessary to recover $\mathbf{g}$ satisfies

$$\bar{n} \leq \left\lceil \frac{\ln((2s/\varepsilon)^{3/2})}{\ln(1/\rho_{3s})} \right\rceil = \left\lceil \frac{3 \ln(2s/\varepsilon)}{2 \ln(1/\rho_{3s})} \right\rceil \leq c_{\delta_{3s}} \ln(s/\varepsilon).$$

To obtain [\(19\)](#), with g denoting a standard Gaussian random variable and with $t := \sqrt{\pi/8} \varepsilon/s$, we write

$$\mathbb{P}(g_s^* < t) = \mathbb{P}(|g_j| < t \text{ for some } j \in S) \leq s \mathbb{P}(|g| < t) = s \int_{-t}^t \frac{\exp(-u^2/2)}{\sqrt{2\pi}} du \leq s \sqrt{\frac{2}{\pi}} t = \varepsilon/2.$$

To obtain [\(20\)](#), with $u \in (0, 1/4)$ to be chosen later and with $t := (3\pi/2)s/\varepsilon$, we write

$$\mathbb{P}(\|\mathbf{g}\|_2^2 > t) = \mathbb{P}\left(\sum_{j \in S} g_j^2 > t\right) \leq \frac{\mathbb{E}\left(\exp\left(u \sum_{j \in S} g_j^2\right)\right)}{\exp(ut)} = \frac{\prod_{j \in S} \mathbb{E}(\exp(ug_j^2))}{\exp(ut)} = \frac{((1-2u)^{-1/2})^s}{\exp(ut)}.$$

Using $(1-2u)^{-1/2} = (1+2u/(1-2u))^{1/2} \leq (1+4u)^{1/2} \leq \exp(2u)$ for $u \in (0, 1/4)$, we derive

$$\mathbb{P}(\|\mathbf{g}\|_2^2 > t) \leq \exp(2us - ut) = \exp\left(-us\left(\frac{3\pi}{2\varepsilon} - 2\right)\right) = \frac{\varepsilon}{2},$$

where we have chosen $us = \ln(2/\varepsilon)/(3\pi/(2\varepsilon) - 2)$, so that $u \leq us < 1/4$ when $\varepsilon < 1/2$.

We now state the extension of [Theorem 5](#) to the realistic situation. We need to assume that the measurement error is not too large compared with the smallest nonzero absolute entry of the sparse vector — such a condition (quite common in the literature, see e.g. [\[11, Theorem 3.2\]](#) or [\[9, Theorem 4.1\]](#)) is further discussed in Subsection 6.3.

Theorem 6. *If the restricted isometry constant of the matrix $\mathbf{A} \in \mathbb{C}^{m \times N}$ obeys*

$$\delta_{3s} \leq \frac{1}{3},$$

then the sequence $(\mathbf{x}^n)$ produced by (HTP) with $\mathbf{y} = \mathbf{Ax} + \mathbf{e} \in \mathbb{C}^m$ for some s -sparse $\mathbf{x} \in \mathbb{C}^N$ and some $\mathbf{e} \in \mathbb{C}^m$ with $\|\mathbf{e}\|_2 \leq \gamma x_s^$ satisfies*

$$\|\mathbf{x} - \mathbf{x}^{\bar{n}}\|_2 \leq d\|\mathbf{e}\|_2, \quad \bar{n} \leq cs.$$

The constants $c \leq 3$, $d \leq 2.45$, and $\gamma \geq 0.079$ depend only on δ_{3s} .

Proof. Let π be the permutation of $\{1, 2, \dots, N\}$ for which $|x_{\pi(j)}| = x_j^*$ for all $j \in \{1, 2, \dots, N\}$. As before, we partition $\text{supp}(\mathbf{x}) = \pi(\{1, \dots, s\})$ as $Q_1 \cup Q_2 \cup \dots \cup Q_r$, $r \leq s$, where the Q_i are defined as in [\(15\)](#). With $Q_0 = \emptyset$ and $k_0 = 0$, we prove by induction on $i \in \{0, \dots, r\}$ that $Q_0 \cup Q_1 \cup \dots \cup Q_i \subseteq S^{k_0 + k_1 + \dots + k_i}$, where $k_1, \dots, k_r$ are defined, with $\rho := \rho_{3s} < 1$, by

$$k_i := \left\lceil \frac{\ln(16(\text{card}(Q_i) + \text{card}(Q_{i+1})/2 + \dots + \text{card}(Q_r)/2^{r-i}))}{\ln(1/\rho^2)} \right\rceil. \quad (21)$$

The induction hypothesis holds trivially for $i = 0$. Next, if it holds for $i - 1$, $i \in \{1, \dots, r\}$, then [Lemma 3](#) guarantees that it holds for i , provided

$$x_{q_i}^* > \rho^{k_i} \sqrt{\|\mathbf{x}_{Q_i}\|_2^2 + \|\mathbf{x}_{Q_{i+1}}\|_2^2 + \dots + \|\mathbf{x}_{Q_r}\|_2^2 + \kappa_{3s}\|\mathbf{e}\|_2}. \quad (22)$$

With the choice $\gamma := (2\sqrt{2} - 1)/(4\kappa_{3s})$, we have $\kappa_{3s}\|\mathbf{e}\|_2 \leq \kappa_{3s}\gamma x_s^* \leq (1/\sqrt{2} - 1/4)x_{q_{i-1}+1}^*$. Moreover, in view of $x_{q_i}^* > x_{q_{i-1}+1}^*/\sqrt{2}$ and of

$$\sqrt{\|\mathbf{x}_{Q_i}\|_2^2 + \|\mathbf{x}_{Q_{i+1}}\|_2^2 + \dots + \|\mathbf{x}_{Q_r}\|_2^2} \leq x_{q_{i-1}+1}^* \sqrt{\text{card}(Q_i) + \frac{1}{2}\text{card}(Q_{i+1}) + \dots + \frac{1}{2^{r-i}}\text{card}(Q_r)},$$

we verify that [\(22\)](#) is fulfilled thanks to the definition [\(21\)](#) of k_i . This concludes the induction. In particular, the support $\pi(\{1, \dots, s\})$ of $\mathbf{x}$ is included in $S^{\bar{n}}$, where $\bar{n}$ can be estimated (using similar arguments as before) by

$$\bar{n} = \sum_{i=1}^r k_i \leq c s, \quad c := \frac{\ln(16/\rho^2)}{\ln(1/\rho^2)}.$$

It follows from (HTP₂) that $\|\mathbf{y} - \mathbf{A}\mathbf{x}^{\bar{n}}\|_2 \leq \|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2 = \|\mathbf{e}\|_2$. In turn, we derive that

$$\|\mathbf{x} - \mathbf{x}^{\bar{n}}\|_2 \leq \frac{1}{\sqrt{1 - \delta_{\bar{n}}}} \|\mathbf{A}(\mathbf{x} - \mathbf{x}^{\bar{n}})\|_2 \leq \frac{1}{\sqrt{1 - \delta_{cs}}} (\|\mathbf{y} - \mathbf{A}\mathbf{x}^{\bar{n}}\|_2 + \|\mathbf{e}\|_2) \leq \frac{2}{\sqrt{1 - \delta_{cs}}} \|\mathbf{e}\|_2 =: d\|\mathbf{e}\|_2.$$

We note that $\delta_{3s} \leq 1/3$ yields $c \leq 3$, then $d \leq \sqrt{6}$, and finally $\gamma \geq (4 - \sqrt{2})\sqrt{3}/56$. $\square$

Remark 7. When the vector $\mathbf{x}$ is not exactly s -sparse, we denote by S an index set of its s largest absolute entries. Writing $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{e}$ as $\mathbf{y} = \mathbf{A}\mathbf{x}_S + \mathbf{e}'$ with $\mathbf{e}' := \mathbf{A}\mathbf{x}_{\bar{S}} + \mathbf{e}$, Theorem 6 guarantees that, under the restricted isometry condition $\delta_{3s} \leq 1/3$,

$$\|\mathbf{x} - \mathbf{x}^{\bar{n}}\|_2 \leq \|\mathbf{x}_{\bar{S}}\|_2 + \|\mathbf{x}_S - \mathbf{x}^{\bar{n}}\|_2 \leq \|\mathbf{x}_{\bar{S}}\|_2 + d\|\mathbf{e}'\|_2, \quad \bar{n} \leq cs,$$

provided $\|\mathbf{e}'\|_2 \leq \gamma x_s^*$. Let us now assume that $\mathbf{e} = \mathbf{0}$ for simplicity. With $S_1, S_2, \dots$ denoting index sets of s largest absolute entries of $\mathbf{x}_{\bar{S}}$, of next s largest absolute entries of $\mathbf{x}_{\bar{S}}$, etc., we notice that

$$\|\mathbf{e}'\|_2 = \|\mathbf{A}\mathbf{x}_{\bar{S}}\|_2 \leq \sum_{k \geq 1} \|\mathbf{A}\mathbf{x}_{S_k}\|_2 \leq \sqrt{1 + \delta_s} \sum_{k \geq 1} \|\mathbf{x}_{S_k}\|_2.$$

In particular, vectors whose nonincreasing rearrangement decays as $x_j^* = \mu^j$ for $\mu \in (0, 1/2)$ satisfy $\|\mathbf{A}\mathbf{x}_{\bar{S}}\|_2 \leq c_{\delta_s} \mu^{s+1}$, so that $\|\mathbf{e}'\|_2 \leq \gamma x_s^*$ holds for small enough μ . Taking $\|\mathbf{x}_{\bar{S}}\|_2 \leq c' \mu^{s+1}$ into account, too, the reconstruction bound becomes $\|\mathbf{x} - \mathbf{x}^{\bar{n}}\|_2 \leq c''_{\delta_s} \mu^{s+1}$. As a matter of fact, the same bound is valid if s is replaced by a smaller value t (even $t = 1$), so the reconstruction error becomes quite satisfactory after a relatively small number ct of iterations.

4. Uniform recovery via graded hard thresholding pursuit

This section contains the main result about (GHTP) in the uniform setting, namely the fact that the reconstruction of all s -sparse vectors is achieved under a restricted isometry condition in a number of iterations at most proportional to s . This is comparable to the main result about (HTP), but we recall the added benefits that (GHTP) does not need an estimation of the sparsity to run and that the stopping criterion involves an estimation of either one of the sparsity or the measurement error — the only other such algorithm we are aware of is (OMP). In passing, we note that (in both uniform and nonuniform settings) the number of (GHTP) iterations for s -sparse recovery cannot be less than s , since $t < s$ iterations only produce t -sparse vectors. We prove the result directly in the realistic situation and we invite the reader to weaken the restricted isometry condition in the idealized situation.

Theorem 8. *If the restricted isometry constant of the matrix $\mathbf{A} \in \mathbb{C}^{m \times N}$ obeys*

$$\delta_{9s} \leq \frac{1}{3},$$

then the sequence $(\mathbf{x}^n)$ produced by (GHTP) with $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{e} \in \mathbb{C}^m$ for some s -sparse $\mathbf{x} \in \mathbb{C}^N$ and some $\mathbf{e} \in \mathbb{C}^m$ with $\|\mathbf{e}\|_2 \leq \gamma x_s^$ satisfies*

$$\|\mathbf{x} - \mathbf{x}^{\bar{n}}\|_2 \leq d\|\mathbf{e}\|_2, \quad \bar{n} \leq cs.$$

The constants $c \leq 4$, $d \leq 2.45$, and $\gamma \geq 0.079$ depend only on δ_{9s} .

Proof. Let π be the permutation of $\{1, 2, \dots, N\}$ for which $|x_{\pi(j)}| = x_j^*$ for all $j \in \{1, 2, \dots, N\}$. Our goal is to show that $\text{supp}(\mathbf{x}) = \pi(\{1, \dots, s\}) \subseteq S^{\bar{n}}$ for some $\bar{n} \leq 4s$. The arguments are very similar to the ones used in the proof of [Theorem 5](#), except for the notable difference that the first s iterations are ignored. We still partition $\pi(\{1, \dots, s\})$ as $Q_1 \cup Q_2 \cup \dots \cup Q_r$, $r \leq s$, where the Q_i are defined as in [\(15\)](#), and we prove by induction on $i \in \{0, \dots, r\}$ that

$$Q_0 \cup Q_1 \cup \dots \cup Q_i \subseteq S^{s+k_0+k_1+\dots+k_i}, \quad (23)$$

where $Q_0 = \emptyset$, $k_0 = 0$, and $k_1, \dots, k_r$ are defined, with $\rho := \rho_{9s} \leq 1/2$, by

$$k_i := \left\lceil \frac{\ln(16(\text{card}(Q_i) + \text{card}(Q_{i+1})/2 + \dots + \text{card}(Q_r)/2^{r-i}))}{\ln(1/\rho^2)} \right\rceil. \quad (24)$$

Based on arguments already used, we observe that

$$\sum_{i=1}^r k_i \leq \frac{\ln(4/\rho^2)}{\ln(1/\rho^2)} s \leq 3s.$$

We now note that [\(23\)](#) holds trivially for $i = 0$. Next, if [\(23\)](#) holds for $i - 1$, $i \in \{1, \dots, r\}$, then [Lemma 4](#) guarantees that [\(23\)](#) holds for i , provided

$$x_{q_i}^* > \rho^{k_i} \sqrt{\|\mathbf{x}_{Q_i}\|_2^2 + \|\mathbf{x}_{Q_{i+1}}\|_2^2 + \dots + \|\mathbf{x}_{Q_r}\|_2^2 + \kappa_{9s} \|\mathbf{e}\|_2}, \quad (25)$$

since $\rho_{s+2(s+k_1+\dots+k_{i-1})+2k_i} \leq \rho_{9s}$. With $\gamma := (2\sqrt{2} - 1)/(4\kappa_{9s}) \geq (4 - \sqrt{2})\sqrt{3}/56$, we verify that [\(25\)](#) is fulfilled thanks to the definition [\(24\)](#) of k_i . This concludes the induction. In particular, the support $\pi(\{1, \dots, s\})$ of $\mathbf{x}$ is included in $S^{\bar{n}}$ with $\bar{n} = s + k_1 + \dots + k_r \leq 4s$. We derive that $\|\mathbf{y} - \mathbf{Ax}^{\bar{n}}\|_2 \leq \|\mathbf{y} - \mathbf{Ax}\|_2 = \|\mathbf{e}\|_2$ from [\(HTP₂\)](#), and then

$$\|\mathbf{x} - \mathbf{x}^{\bar{n}}\|_2 \leq \frac{1}{\sqrt{1 - \delta_{\bar{n}}}} \|\mathbf{A}(\mathbf{x} - \mathbf{x}^{\bar{n}})\|_2 \leq \frac{1}{\sqrt{1 - \delta_{4s}}} (\|\mathbf{y} - \mathbf{Ax}^{\bar{n}}\|_2 + \|\mathbf{e}\|_2) \leq \frac{2}{\sqrt{1 - \delta_{9s}}} \|\mathbf{e}\|_2,$$

which is the desired result with $d = 2/\sqrt{1 - \delta_{9s}} \leq \sqrt{6}$. $\square$

5. Nonuniform recovery via graded hard thresholding pursuit

In this section, we demonstrate that s -sparse recovery via exactly s iterations of (GHTP) can be expected in the nonuniform setting, i.e., given an s -sparse vector $\mathbf{x} \in \mathbb{C}^N$, a random measurement matrix allows for the recovery of this specific $\mathbf{x}$ with high probability. Precisely, we obtain results in two extreme cases: when the nonincreasing rearrangement of the sparse vector does not decay much and when it decays exponentially fast. [Proposition 9](#) covers the first case, suspected to be the least favorable one — see Subsection 6.2. The result and its proof have similarities with the work of [\[13\]](#), which was extended to incorporate measurement error in [\[9\]](#). The result is stated directly in this realistic situation, and it is worth pointing out its validity for all measurement errors $\mathbf{e} \in \mathbb{C}^m$ simultaneously.

Proposition 9. *Let $\lambda \geq 1$ and let $\mathbf{x} \in \mathbb{C}^N$ be an s -sparse vector such that $x_1^* \leq \lambda x_s^*$. If $\mathbf{A} \in \mathbb{R}^{m \times N}$ is a (normalized) Gaussian random matrix with*

$$m \geq Cs \ln(N),$$

then it occurs with probability at least $1 - 2N^{-c}$ that, for all $\mathbf{e} \in \mathbb{C}^m$ with $\|\mathbf{e}\|_2 \leq \gamma x_s^*$, the sequences (S^n) and $(\mathbf{x}^n)$ produced by (GHTP) with $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{e}$ satisfy, at iteration s ,

$$S^s = \text{supp}(\mathbf{x}) \quad \text{and} \quad \|\mathbf{x} - \mathbf{x}^s\|_2 \leq d \|\mathbf{e}\|_2.$$

The constants γ and d depend only on λ , while the constant C depends on γ and c .

Proof. The result is in fact valid if (normalized) Gaussian random matrices are replaced by matrices with independent subgaussian entries with mean zero and variance $1/m$. One part of the argument relies on the fact that, for a fixed index set S of size s ,

$$\mathbb{P}(\|\mathbf{A}_S^* \mathbf{A}_S - \mathbf{I}\|_{2 \rightarrow 2} > \delta) \leq 2 \exp(-c' \delta^2 m) \quad (26)$$

provided $m \geq C's/\delta^2$, with c', C' depending only on the subgaussian distribution (see e.g. [7, Theorem 9.9], which leads to the improvement [7, Theorem 9.11] of [1, Theorem 5.2] in terms of dependence of constants on δ). Another part of the argument relies on the fact that, for a vector $\mathbf{v} \in \mathbb{C}^N$ and an index $\ell \in \{1, \dots, N\}$, with $\mathbf{a}_\ell$ denoting the ℓ th column of the matrix $\mathbf{A}$,

$$\mathbb{P}(|\langle \mathbf{a}_\ell, \mathbf{v} \rangle| > t \|\mathbf{v}\|_2) \leq 4 \exp(-c'' t^2 m) \quad (27)$$

for a constant c'' depending only on the subgaussian distribution (see e.g. [7, Theorem 7.27] in the case $\mathbf{v} \in \mathbb{R}^N$). With $S := \text{supp}(\mathbf{x})$, let us now define, for $n \in \{1, \dots, s\}$, two random variables χ_n and ζ_n as

$$\begin{aligned} \chi_n &:= [(\mathbf{x}^{n-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n-1}))_S]_n^*, \\ \zeta_n &:= [(\mathbf{x}^{n-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n-1}))_{\bar{S}}]_1^*, \end{aligned}$$

i.e., χ_n is the n th largest value of $|(\mathbf{x}^{n-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n-1}))_j|$ when j runs through S and ζ_n is the largest value of $|(\mathbf{x}^{n-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n-1}))_\ell|$ when ℓ runs through $\bar{S}$. We are going to prove that, with high probability, $S^n \subseteq S$ for all $n \in \{1, \dots, s\}$, which follows from $\chi_n > \zeta_n$ for all $n \in \{1, \dots, s\}$. The failure probability of this event is

$$\begin{aligned} P &:= \mathbb{P}(\exists n \in \{1, \dots, s\} : \zeta_n \geq \chi_n \text{ and } (\chi_{n-1} > \zeta_{n-1}, \dots, \chi_1 > \zeta_1)) \\ &\leq \mathbb{P}\left(\|\mathbf{A}_{S \cup \{\ell\}}^* \mathbf{A}_{S \cup \{\ell\}} - \mathbf{I}\|_{2 \rightarrow 2} > \delta \text{ for some } \ell \in \bar{S}\right) \end{aligned} \quad (28)$$

$$+ \sum_{n=1}^s \mathbb{P}\left(\zeta_n \geq \chi_n, (\chi_{n-1} > \zeta_{n-1}, \dots, \chi_1 > \zeta_1), (\|\mathbf{A}_{S \cup \{\ell\}}^* \mathbf{A}_{S \cup \{\ell\}} - \mathbf{I}\|_{2 \rightarrow 2} \leq \delta \text{ for all } \ell \in \bar{S})\right), \quad (29)$$

where the constant $\delta = \delta_\lambda$ is to be chosen later. According to (26), the term in (28) is bounded by $2(N-s) \exp(-c' \delta^2 m)$ since a proper choice of C (depending on λ through δ) guarantees that $m \geq C'(s+1)/\delta^2$. Let us turn to the term in (29) and let us use the shorthand $\mathbb{P}'(E)$ to denote the probability of an event E intersected with the events $(\chi_{n-1} > \zeta_{n-1}, \dots, \chi_1 > \zeta_1)$ and $(\|\mathbf{A}_{S \cup \{\ell\}}^* \mathbf{A}_{S \cup \{\ell\}} - \mathbf{I}\|_{2 \rightarrow 2} \leq \delta \text{ for all } \ell \in \bar{S})$, which are assumed to hold below. On the one hand, with $T^{s-n+1} \subseteq S$ representing an index set corresponding to $s-n+1$ smallest values of $|(\mathbf{x}^{n-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n-1}))_j|$ when j runs through S , we notice that

$$\begin{aligned} \chi_n &\geq \frac{1}{\sqrt{s-n+1}} \|(\mathbf{x}^{n-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n-1}))_{T^{s-n+1}}\|_2 \\ &\geq \frac{1}{\sqrt{s-n+1}} \left(\|\mathbf{x}_{T^{s-n+1}}\|_2 - \|((\mathbf{A}^* \mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n-1}))_{T^{s-n+1}}\|_2 - \|(\mathbf{A}^* \mathbf{e})_{T^{s-n+1}}\|_2 \right). \end{aligned}$$

Then, since the condition $\chi_{n-1} > \zeta_{n-1}$ yields $S^{n-1} \subseteq S$, we obtain

$$\|(\mathbf{A}^* \mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n-1})\|_{T^{s-n+1}} \leq \|(\mathbf{A}_S^* \mathbf{A}_S - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n-1})\|_2 \leq \delta \|\mathbf{x} - \mathbf{x}^{n-1}\|_2.$$

We also use the inequality $\|(\mathbf{A}^* \mathbf{e})_{T^{s-n+1}}\|_2 \leq \sqrt{1+\delta} \|\mathbf{e}\|_2$ to derive

$$\chi_n \geq \frac{1}{\sqrt{s-n+1}} \left(\|\mathbf{x}_{T^{s-n+1}}\|_2 - \delta \|\mathbf{x} - \mathbf{x}^{n-1}\|_2 - \sqrt{1+\delta} \|\mathbf{e}\|_2 \right).$$

On the other hand, since $S^{n-1} \subseteq S$, we have

$$\begin{aligned} \zeta_n &= \max_{\ell \in \bar{S}} |(\mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n-1}))_\ell| \leq \max_{\ell \in \bar{S}} |(\mathbf{A}^* \mathbf{A}(\mathbf{x} - \mathbf{x}^{n-1}))_\ell| + \max_{\ell \in \bar{S}} |(\mathbf{A}^* \mathbf{e})_\ell| \\ &\leq \max_{\ell \in \bar{S}} |\langle \mathbf{a}_\ell, \mathbf{A}(\mathbf{x} - \mathbf{x}^{n-1}) \rangle| + \sqrt{1+\delta} \|\mathbf{e}\|_2. \end{aligned}$$

It follows that

$$\begin{aligned} \mathbb{P}'(\zeta_n \geq \chi_n) &\leq \mathbb{P}' \left(\max_{\ell \in \bar{S}} |\langle \mathbf{a}_\ell, \mathbf{A}(\mathbf{x} - \mathbf{x}^{n-1}) \rangle| \geq \frac{1}{\sqrt{s-n+1}} \left(\|\mathbf{x}_{T^{s-n+1}}\|_2 - \delta \|\mathbf{x} - \mathbf{x}^{n-1}\|_2 \right) - 2\sqrt{1+\delta} \|\mathbf{e}\|_2 \right) \\ &\leq \mathbb{P}' \left(\max_{\ell \in \bar{S}} |\langle \mathbf{a}_\ell, \mathbf{A}(\mathbf{x} - \mathbf{x}^{n-1}) \rangle| \geq \frac{\delta}{\sqrt{s}} \|\mathbf{x} - \mathbf{x}^{n-1}\|_2 \right), \end{aligned}$$

where the last step will be justified by the inequality

$$\frac{1}{\sqrt{s-n+1}} \|\mathbf{x}_{T^{s-n+1}}\|_2 - 2\sqrt{1+\delta} \|\mathbf{e}\|_2 \geq \frac{2\delta}{\sqrt{s-n+1}} \|\mathbf{x} - \mathbf{x}^{n-1}\|_2.$$

In order to fulfill this inequality, we remark that $\|\mathbf{x}_{T^{s-n+1}}\|_2 \geq \sqrt{s-n+1} x_s^*$, that $\|\mathbf{e}\|_2 \leq \gamma x_s^*$, and, thanks to the extension of (7) mentioned in Remark 1, that

$$\begin{aligned} \|\mathbf{x} - \mathbf{x}^{n-1}\|_2 &\leq \frac{1}{\sqrt{1-\delta^2}} \|\mathbf{x}_{\bar{S}^{n-1}}\|_2 + \frac{1}{1-\delta} \|(\mathbf{A}^* \mathbf{e})_{S^{n-1}}\|_2 \leq \frac{\sqrt{s-n+1}}{\sqrt{1-\delta^2}} x_1^* + \frac{\sqrt{1+\delta}}{1-\delta} \|\mathbf{e}\|_2 \\ &\leq \sqrt{s-n+1} \left(\frac{\lambda}{\sqrt{1-\delta^2}} x_s^* + \frac{\sqrt{1+\delta} \gamma}{1-\delta} x_s^* \right), \end{aligned}$$

so that it is enough to choose δ then γ (depending on λ) small enough to have

$$1 - 2\sqrt{1+\delta} \gamma \geq 2\delta \left(\frac{\lambda}{\sqrt{1-\delta^2}} + \frac{\sqrt{1+\delta} \gamma}{1-\delta} \right).$$

Taking $\|\mathbf{A}(\mathbf{x} - \mathbf{x}^{n-1})\|_2 \leq \sqrt{1+\delta} \|\mathbf{x} - \mathbf{x}^{n-1}\|_2$ into account, we obtain

$$\mathbb{P}'(\zeta_n \geq \chi_n) \leq \mathbb{P}' \left(\max_{\ell \in \bar{S}} |\langle \mathbf{a}_\ell, \mathbf{A}(\mathbf{x} - \mathbf{x}^{n-1}) \rangle| \geq \frac{\delta}{\sqrt{1+\delta} \sqrt{s}} \|\mathbf{A}(\mathbf{x} - \mathbf{x}^{n-1})\|_2 \right). \quad (30)$$

Let us now remark that the condition $\chi_1 = [(\mathbf{A}^* \mathbf{A}_S \mathbf{x} + \mathbf{A}^* \mathbf{e})_S]_1^* > \zeta_1 = [(\mathbf{A}^* \mathbf{A}_S \mathbf{x} + \mathbf{A}^* \mathbf{e})_{\bar{S}}]_1^*$ implies that the random set S^1 (corresponding to the largest value of $\mathbf{A}_S^* \mathbf{A}_S \mathbf{x} + \mathbf{A}_S^* \mathbf{e}$) depends only on the random submatrix $\mathbf{A}_S$, so that the random vector $\mathbf{x}^1 = \mathbf{A}_{S^1}^\dagger \mathbf{y}$ depends only on $\mathbf{A}_S$, too; then, since $\mathbf{x}^1$ is supported on $S^1 \subseteq S$, the condition $\chi_2 = [(\mathbf{x}^1 + \mathbf{A}^* \mathbf{A}_S(\mathbf{x} - \mathbf{x}^1) + \mathbf{A}^* \mathbf{e})_S]_2^* > \zeta_2 = [(\mathbf{x}^1 + \mathbf{A}^* \mathbf{A}_S(\mathbf{x} - \mathbf{x}^1) + \mathbf{A}^* \mathbf{e})_{\bar{S}}]_1^*$

implies that the random set S^2 (corresponding to the two largest values of $\mathbf{x}_S^1 + \mathbf{A}_S^* \mathbf{A}_S (\mathbf{x} - \mathbf{x}^1) + \mathbf{A}_S^* \mathbf{e}$) depends only on the random submatrix $\mathbf{A}_S$, so that the random vector $\mathbf{x}^2 = \mathbf{A}_{S^2}^\dagger \mathbf{y}$ depends only on $\mathbf{A}_S$, too; and so on until we infer that the random vector $\mathbf{x}^{n-1}$, supported on $S^{n-1} \subseteq S$, depends only on $\mathbf{A}_S$, and in turn so does $\mathbf{A}(\mathbf{x} - \mathbf{x}^{n-1})$. Exploiting the independence of $\mathbf{A}(\mathbf{x} - \mathbf{x}^{n-1})$ and of the columns $\mathbf{a}_\ell$ for $\ell \in \bar{S}$, the estimate (27) can be applied in (30) to yield

$$\begin{aligned} \mathbb{P}'(\zeta_n \geq \chi_n) &\leq \sum_{\ell \in \bar{S}} \mathbb{P}\left(|\langle \mathbf{a}_\ell, \mathbf{A}(\mathbf{x} - \mathbf{x}^{n-1}) \rangle| \geq \frac{\delta}{\sqrt{1+\delta}\sqrt{s}} \|\mathbf{A}(\mathbf{x} - \mathbf{x}^{n-1})\|_2\right) \\ &\leq 4(N-s) \exp\left(-\frac{c''\delta^2 m}{(1+\delta)s}\right). \end{aligned}$$

Altogether, the failure probability P satisfies

$$\begin{aligned} P &\leq 2(N-s) \exp(-c'\delta^2 m) + 4s(N-s) \exp\left(-\frac{c''\delta^2 m}{(1+\delta)s}\right) \\ &\leq 2(N-1) \exp(-c'\delta^2 m) + N^2 \exp\left(-\frac{c''\delta^2 m}{(1+\delta)s}\right). \end{aligned}$$

In particular, we have $\chi_s > \zeta_s$, and in turn $S^s = S$, with failure probability at most P . Then, since $\|\mathbf{A}_S^* \mathbf{A}_S - \mathbf{I}\|_{2 \rightarrow 2} \leq \delta$ with failure probability given in (26), we also have

$$\|\mathbf{x} - \mathbf{x}^s\|_2 \leq \frac{1}{\sqrt{1-\delta}} \|\mathbf{A}(\mathbf{x} - \mathbf{x}^s)\|_2 \leq \frac{1}{\sqrt{1-\delta}} (\|\mathbf{y} - \mathbf{A}\mathbf{x}^s\|_2 + \|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2) \leq \frac{2}{\sqrt{1-\delta}} \|\mathbf{e}\|_2,$$

which is the desired result with $d = 2/\sqrt{1-\delta}$. The resulting failure probability is bounded by

$$2N \exp(-c'\delta^2 m) + N^2 \exp\left(-\frac{c''\delta^2 m}{(1+\delta)s}\right) \leq 2N^2 \exp\left(-\frac{c'''\delta^2 m}{s}\right) \leq 2N^{-c},$$

where the last inequality holds with a proper choice of the constant C in $m \geq Cs \ln(N)$. $\square$

The previous result is limited by its nonapplicability to random partial Fourier matrices. For such matrices, the uniform results of the two previous sections — or any recovery result based on the restricted isometry property — do apply, but the restricted isometry property is only known to be fulfilled with a number of measurements satisfying $m \geq cs \ln^4(N)$. This threshold can be lowered in the nonuniform setting by reducing the power of the logarithm. This is typically achieved via ℓ_1 -minimization (see [4, Theorem 1.3], [3, Theorem 1.1], or [8, V"3]), for which the successful recovery of a vector does not depend on its shape but only on its sign pattern. We establish below a comparable result for (GHTP): fewer measurements guarantee the high probability of nonuniform s -sparse recovery via exactly s iterations of (GHTP). There is an additional proviso on the vector shape, namely its nonincreasing rearrangement should decay exponentially fast. This is admittedly a strong assumption, but possible improvements in the proof strategy (perhaps incorporating ideas from Proposition 9) suggest that the result could be extended beyond this class of sparse vectors for Fourier measurements. Incidentally, for this class of vectors, it is striking that the number of Gaussian measurements can be reduced even below the classical reduction (see [5, Theorems 1.2 and 1.3]) of the constant c in $m \approx cs \ln(N/s)$ when passing from the uniform to the nonuniform setting.

Proposition 10. *Let $\mu \in (0, 1)$ and let $\mathbf{x} \in \mathbb{C}^N$ be an s -sparse vector such that $x_{j+1}^* \leq \mu x_j^*$ for all $j \in \{1, \dots, s-1\}$. If $\mathbf{A} \in \mathbb{R}^{m \times N}$ is a (normalized) Gaussian random matrix with*

$$m \geq \max\{Cs, C' \ln(N)\},$$

or if $\mathbf{A} \in \mathbb{C}^{m \times N}$ is a (normalized) random partial Fourier matrix with

$$m \geq C'' s \ln(N),$$

then it occurs with probability at least $1 - 2N^{-c}$ that, for all $\mathbf{e} \in \mathbb{C}^m$ with $\|\mathbf{e}\|_2 \leq \gamma x_s^*$, the sequences (S^n) and $(\mathbf{x}^n)$ produced by (GHTP) with $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{e}$ satisfy, at iteration s ,

$$S^s = \text{supp}(\mathbf{x}) \quad \text{and} \quad \|\mathbf{x} - \mathbf{x}^s\|_2 \leq d \|\mathbf{e}\|_2.$$

The constants C , γ , and d depend only on μ , while the constants C' and C'' depend on μ and c .

Proof. The result would also be valid if (normalized) Gaussian random matrices were replaced by (normalized) subgaussian random matrices and if (normalized) random partial Fourier matrices were replaced by (normalized) random sampling matrices associated to bounded orthonormal systems. We recall (see [7, chapter 12] for details) that a bounded orthonormal system is a system $(\phi_1, \dots, \phi_N)$ of functions, defined on a set $\mathcal{D}$ endowed with a probability measure ν , that satisfy, for some constant $K \geq 1$,

$$\sup_{t \in \mathcal{D}} |\phi_j(t)| \leq K \quad \text{and} \quad \int_{\mathcal{D}} \phi_k(t) \overline{\phi_\ell(t)} d\nu(t) = \delta_{k,\ell} \quad (31)$$

for all $j, k, \ell \in \{1, \dots, N\}$. The normalized random sampling matrix $\mathbf{A} \in \mathbb{C}^{m \times N}$ associated to $(\phi_1, \dots, \phi_N)$ has entries

$$A_{k,\ell} = \frac{1}{\sqrt{m}} \phi_\ell(t_k), \quad k \in \{1, \dots, m\}, \quad \ell \in \{1, \dots, N\},$$

where the sampling points $t_1, \dots, t_m \in \mathcal{D}$ are selected independently at random according to the probability measure ν . For instance, the random partial Fourier matrix corresponds to the choices $\mathcal{D} = \{1, \dots, N\}$, $\nu(T) = \text{card}(T)/N$ for all $T \subseteq \{1, \dots, N\}$, and $\phi_k(t) = \exp(i2\pi kt)$, in which case $K = 1$. The argument here relies on the fact (see [7, Theorem 12.12]) that, for a fixed index set S of size s ,

$$\mathbb{P}(\|\mathbf{A}_S^* \mathbf{A}_S - \mathbf{I}\|_{2 \rightarrow 2} > \delta) \leq 2s \exp\left(-\frac{3\delta^2 m}{8K^2 s}\right). \quad (32)$$

Let π be the permutation of $\{1, 2, \dots, N\}$ for which $|x_{\pi(j)}| = x_j^*$ for all $j \in \{1, 2, \dots, N\}$ and let Π_n denote the set $\pi(\{1, 2, \dots, n\})$ for each $n \in \{1, \dots, N\}$. Let $\delta = \delta_\mu > 0$ be chosen small enough to have $2\delta/\sqrt{1-\delta^2} \leq (1-\mu)\sqrt{1-\mu^2}/3$ — for reason that will become apparent later. Invoking (26) and (32) for the sets $\Pi_s \cup \{k\}$, $k \notin \Pi_s$, ensures that

$$\|\mathbf{A}_{\Pi_s \cup \{k\}}^* \mathbf{A}_{\Pi_s \cup \{k\}} - \mathbf{I}\|_{2 \rightarrow 2} \leq \delta. \quad (33)$$

In the Gaussian case, provided $m \geq C_\mu s$, this holds with failure probability at most

$$2(N-s) \exp(-c_\mu m) \leq 2N \exp(-c_\mu C'_{\mu,c} \ln(N)) \leq 2N^{-c}$$

where $C'_{\mu,c}$ is chosen large enough. In the Fourier case, this holds with failure probability at most

$$2(N-s)s \exp\left(-\frac{c_\mu m}{s}\right) \leq 2N^2 \exp(-c_\mu C''_{\mu,c} \ln(N)) \leq 2N^{-c}$$

where $C''_{\mu,c}$ is chosen large enough. We are going to prove by induction that $S^n = \Pi_n$ for all $n \in \{0, 1, \dots, s\}$. The result holds trivially for $n = 0$. To increment the induction from $n - 1$ to n , $n \in \{1, \dots, s\}$, we need to show that

$$\min_{j=1,\dots,n} |(\mathbf{x}^{n-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n-1}))_{\pi(j)}| > \max_{\ell=n+1,\dots,N} |(\mathbf{x}^{n-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n-1}))_{\pi(\ell)}|.$$

We observe that for $j \in \{1, \dots, n\}$ and $\ell \in \{n+1, \dots, N\}$,

$$\begin{aligned} |(\mathbf{x}^{n-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n-1}))_{\pi(j)}| &\geq |x_{\pi(j)}| - |((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n-1}) + \mathbf{A}^*\mathbf{e})_{\pi(j)}| \\ &\geq x_n^* - |((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n-1}))_{\pi(j)}| - |(\mathbf{A}^*\mathbf{e})_{\pi(j)}|, \\ |(\mathbf{x}^{n-1} + \mathbf{A}^*(\mathbf{y} - \mathbf{A}\mathbf{x}^{n-1}))_{\pi(\ell)}| &\leq |x_{\pi(\ell)}| + |((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n-1}) + \mathbf{A}^*\mathbf{e})_{\pi(\ell)}| \\ &\leq x_{n+1}^* + |((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n-1}))_{\pi(\ell)}| + |(\mathbf{A}^*\mathbf{e})_{\pi(\ell)}|. \end{aligned}$$

Hence, it is enough to prove that, for all $j \in \{1, \dots, n\}$ and $\ell \in \{n+1, \dots, N\}$,

$$x_n^* > x_{n+1}^* + |((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n-1}))_{\pi(j)}| + |(\mathbf{A}^*\mathbf{e})_{\pi(j)}| + |((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n-1}))_{\pi(\ell)}| + |(\mathbf{A}^*\mathbf{e})_{\pi(\ell)}|. \quad (34)$$

For any $k \in \{1, \dots, N\}$, using the induction hypothesis and (33), we obtain

$$|((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n-1}))_k| \leq \|(\mathbf{A}^*_{\Pi_s \cup \{k\}} \mathbf{A}_{\Pi_s \cup \{k\}} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n-1})\|_2 \leq \delta \|\mathbf{x} - \mathbf{x}^{n-1}\|_2. \quad (35)$$

Moreover, in view of the extension of (7) mentioned in Remark 1, we have

$$\|\mathbf{x} - \mathbf{x}^{n-1}\|_2 \leq \frac{1}{\sqrt{1-\delta^2}} \|\mathbf{x}_{\Pi_{n-1}}\|_2 + \frac{1}{1-\delta} \|(\mathbf{A}^*\mathbf{e})_{\Pi_{n-1}}\|_2. \quad (36)$$

Combining (35) and (36) gives, for any $k \in \{1, \dots, N\}$,

$$\begin{aligned} |((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n-1}))_k| + |(\mathbf{A}^*\mathbf{e})_k| &\leq \frac{\delta}{\sqrt{1-\delta^2}} \|\mathbf{x}_{\Pi_{n-1}}\|_2 + \frac{\delta}{1-\delta} \|(\mathbf{A}^*\mathbf{e})_{\Pi_{n-1}}\|_2 + |(\mathbf{A}^*\mathbf{e})_k| \\ &\leq \frac{\delta}{\sqrt{1-\delta^2}\sqrt{1-\mu^2}} x_n^* + \frac{1}{1-\delta} \|(\mathbf{A}^*\mathbf{e})_{\Pi_s \cup \{k\}}\|_2, \end{aligned}$$

where we have used $\|\mathbf{x}_{\Pi_{n-1}}\|_2^2 \leq (x_n^*)^2(1 + \mu^2 + \dots + (\mu^2)^{s-n}) \leq (x_n^*)^2/(1 - \mu^2)$ in the last step. Using also the estimate imposed by (33) for the maximal singular value of $\mathbf{A}_{\Pi_s \cup \{k\}}$ in

$$\|(\mathbf{A}^*\mathbf{e})_{\Pi_s \cup \{k\}}\|_2 \leq \sqrt{1+\delta} \|\mathbf{e}\|_2 \leq \sqrt{1+\delta} \gamma x_s^* \leq \sqrt{1+\delta} \gamma x_n^*,$$

we arrive at

$$|((\mathbf{A}^*\mathbf{A} - \mathbf{I})(\mathbf{x} - \mathbf{x}^{n-1}))_k| + |(\mathbf{A}^*\mathbf{e})_k| \leq \left(\frac{\delta}{\sqrt{1-\delta^2}\sqrt{1-\mu^2}} + \frac{\sqrt{1+\delta}\gamma}{1-\delta} \right) x_n^*.$$

Taking the fact that $x_{n+1}^* \leq \mu x_n^*$ into account, we observe that (34) is fulfilled as soon as

$$1 > \mu + \frac{2\delta}{\sqrt{1-\delta^2}\sqrt{1-\mu^2}} + \frac{2\sqrt{1+\delta}\gamma}{1-\delta}.$$

The latter holds with the choice of δ made at the beginning and then with an appropriate choice of γ (depending only on μ). At this point, we have justified inductively that $S^n = \Pi_n$ for all $n \in \{1, \dots, s\}$,

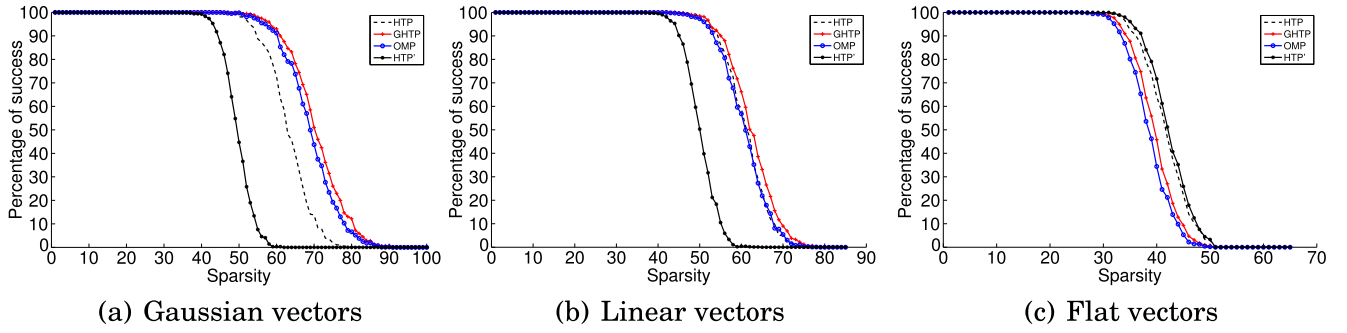


Fig. 1. Frequency of successful recoveries for (HTP), (GHTP), (OMP), and (HTP') using Gaussian measurements.

hence in particular that $S^s = \Pi_s = \text{supp}(\mathbf{x})$. To conclude the proof, we remark that (GHTP₂) yields $\|\mathbf{y} - \mathbf{A}\mathbf{x}^s\|_2 \leq \|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2 = \|\mathbf{e}\|_2$ and we also use the estimate imposed by (33) for the minimal singular value of $\mathbf{A}_{\Pi_s}$ to derive

$$\|\mathbf{x} - \mathbf{x}^s\|_2 \leq \frac{1}{\sqrt{1-\delta}} \|\mathbf{A}(\mathbf{x} - \mathbf{x}^s)\|_2 \leq \frac{1}{\sqrt{1-\delta}} (\|\mathbf{y} - \mathbf{A}\mathbf{x}^s\|_2 + \|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2) \leq \frac{2}{\sqrt{1-\delta}} \|\mathbf{e}\|_2.$$

The constant $d := 2/\sqrt{1-\delta}$ depends only on μ . $\square$

6. Numerical experiments

This section provides some empirical validation of our theoretical results. We focus on the performances of (HTP), (GHTP), and its natural companion (OMP). All the tests were carried out for Gaussian random matrices $\mathbf{A} \in \mathbb{R}^{m \times N}$ with $m = 200$ and $N = 1000$. We generated 100 such random matrices. To assess the influence of the decay in the nonincreasing rearrangement of s -sparse vectors $\mathbf{x} \in \mathbb{R}^N$ to be recovered, we tested ‘flat’ vectors with $x_j^* = 1$ for $j \in \{1, \dots, s\}$, ‘linear’ vectors with $x_j^* = (s + 1 - j)/s$ for $j \in \{1, \dots, s\}$, and Gaussian vectors whose s nonzero entries are independent standard normal random variables. We generated 10 such vectors per matrix. All these experiments can be reproduced by downloading the associated MATLAB files from the authors’ webpages. These files contain supplementary numerical tests, e.g. one involving Fourier submatrices to illustrate Proposition 10.

6.1. Frequency of successful recoveries

We present here a comparison between (HTP), (GHTP), and (OMP) in terms of successful recovery — this complements the experiments of [6] where (HTP) was compared to basis pursuit, compressive sampling matching pursuit, iterative hard thresholding, and normalized iterative hard thresholding. We have also included a comparison with a version of (HTP), denoted (HTP'), where the number of largest absolute entries kept at each iteration is $2s$ instead of s . The natural stopping criterion $S^n = S^{n-1}$ was used for (HTP) and (HTP'), and we used the criterion⁴ $[\text{supp}(\mathbf{x}) \subseteq S^n \text{ or } \|\mathbf{x} - \mathbf{x}^n\|_2 / \|\mathbf{x}\|_2 < 10^{-4}]$ for (GHTP) and (OMP).

Fig. 1 suggests that no algorithm consistently outperforms the other ones, although (GHTP) is always marginally better than (OMP). Moreover, (HTP') underperforms in two cases, possibly because theoretical requirements such as restricted isometries properties are harder to fulfill when $2s$ replaces s . The shape of the vector to be recovered has a strong influence on the success rate, in contrast with basis pursuit whose success depend only on the sign pattern.

⁴ The true sparse solution is available in this experiment, unlike in a practical scenario.

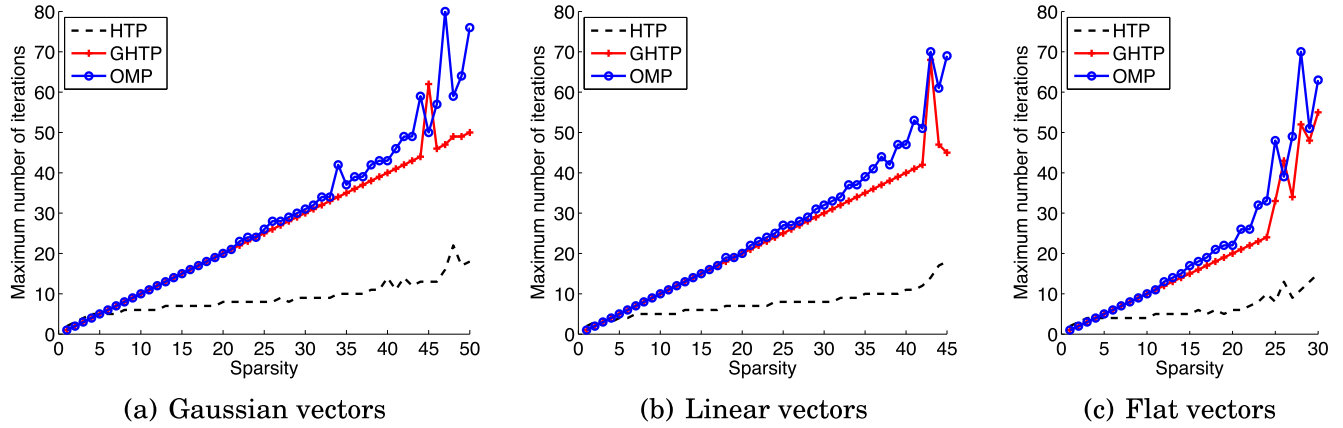


Fig. 2. Number of iterations for (HTP), (GHTP), and (OMP) using Gaussian measurements.

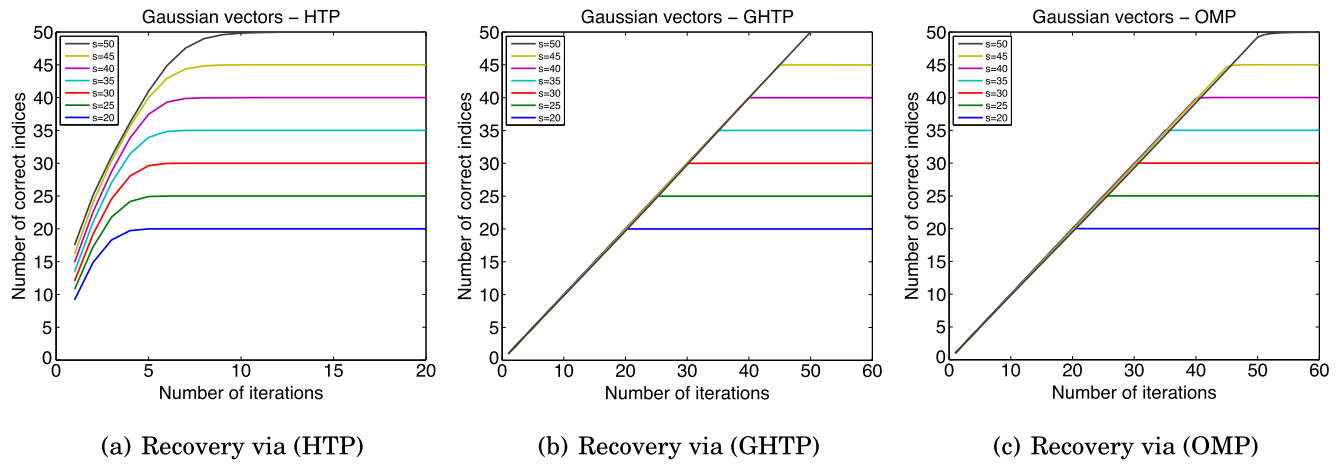


Fig. 3. Number of support indices of Gaussian vectors correctly captured by the active set as a function of the iteration count for (HTP), (GHTP), and (OMP) using Gaussian measurements.

6.2. Number of iterations and indices correctly captured

We now focus on the information provided by our experiments in terms of number of iterations needed for the recovery of a sparse vector and in terms of number of indices from the true support correctly captured in the active sets as a function of the iteration count. First, Fig. 2 displays the maximum over all our trials for the number $\bar{n}(s)$ of iterations needed for s -sparse recovery as a function of s in the sparsity regions where recovery is certain (see Fig. 1). With this indicator, (HTP) is the best algorithm — thanks to the fact that a prior knowledge of s is exploited. For this algorithm, no notable difference seems to result from the vector shape (but remember that it did influence the success rate), so the upper bound $\bar{n}(s) \leq cs$ from Theorem 5 is likely to be an overestimation to be replaced by the bound $\bar{n}(s) \leq c \ln(s)$, which is known to be valid for ‘flat’ vectors. As for (GHTP) and (OMP), we observe that the vector shape influences the regions (predicted by Propositions 9 and 10) of s -values for which $\bar{n}(s) = s$ and that (GHTP) generally requires fewer iterations than (OMP), probably thanks to the fact that incorrectly selected indices can be rejected at the next (GHTP) iteration (however, this complicates speed-ups based on QR -updates).

Next, Figs. 3, 4, and 5 display the number $i(n)$ of indices from the true support that are correctly captured in the active set S^n as a function of the iteration count n . We point out that these figures do not reflect the worst case anymore, but instead they present an average over all successful recoveries. Once again, (HTP) is the superior algorithm because several correct indices can be captured at once. For (GHTP) and (OMP), the ideal situation corresponds to the selection of correct indices only, i.e., $i(n) = n$ for all $n \leq s$. This seems to occur for small sparsities, with larger regions of validity for vectors whose nonincreasing rearrangements

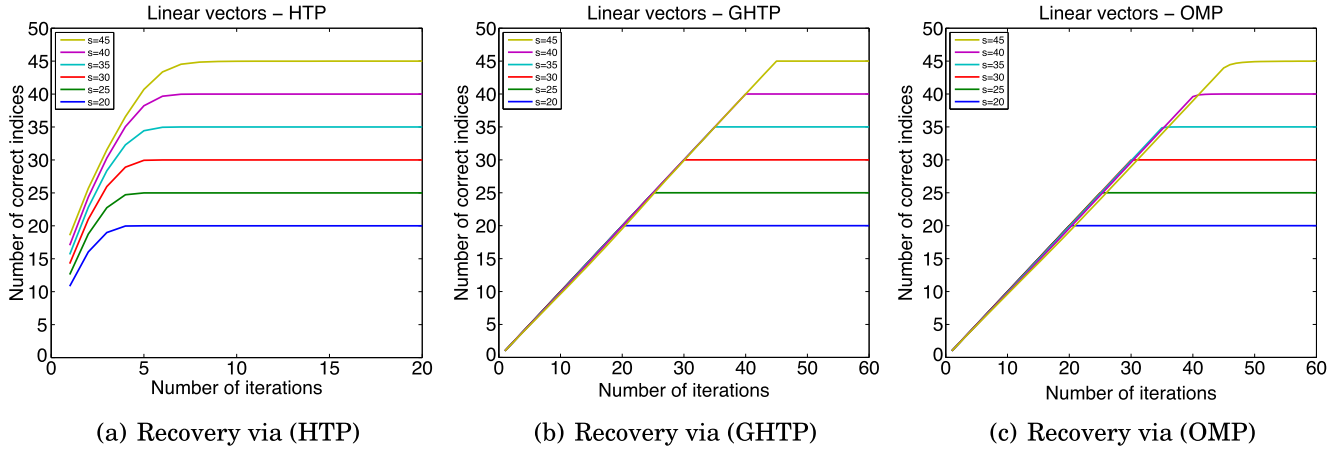


Fig. 4. Number of support indices of linear vectors correctly captured by the active set as a function of the iteration count for (HTP), (GHTP), and (OMP) using Gaussian measurements.

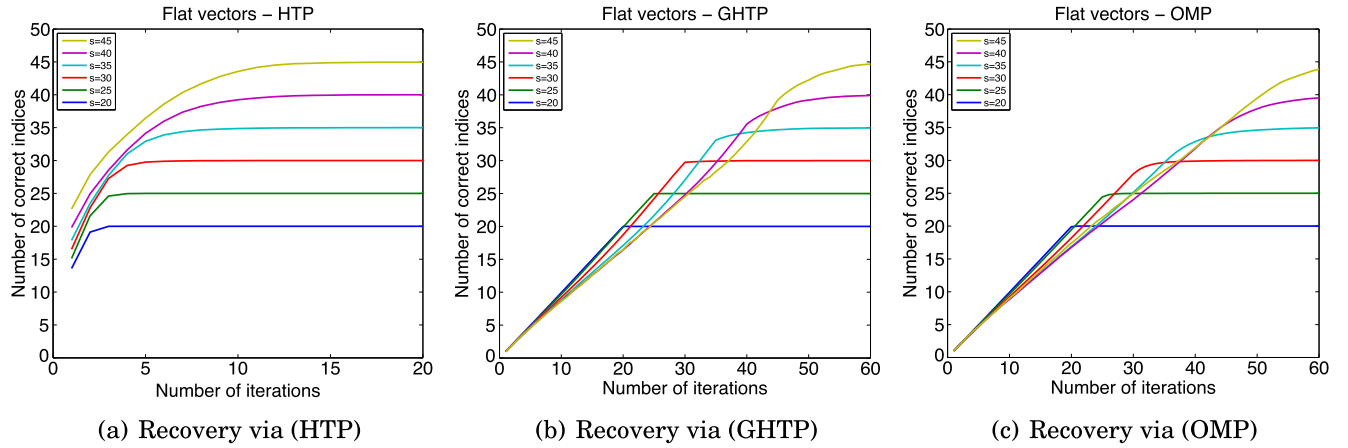


Fig. 5. Number of support indices of flat vectors correctly captured by the active set as a function of the iteration count for (HTP), (GHTP), and (OMP) using Gaussian measurements.

decay quickly. This is an encouraging fact since $i(n) = n$ has been justified for all $n \leq s$ in the proof of Proposition 9 for the least favorable case of ‘flat’ vectors. For these vectors, Fig. 5 also reveals that, even in the region where s -sparse recovery via (GHTP) and (OMP) is certain, the recovery is sometimes achieved in more than s iterations.

Finally, Fig. 6 shows the effect of overestimating the sparsity level as $2s$ instead of s in each hard thresholding pursuit iteration. We notice a slight improvement for small sparsities, as more correct indices can be captured at each iteration, but a degradation for larger sparsities, again possibly due to harder theoretical requirements.

6.3. Robustness to measurement error

Our final experiment highlights the effect of the measurement error $\mathbf{e} \in \mathbb{C}^m$ on the recovery via (HTP), (GHTP), and (OMP) of sparse vectors $\mathbf{x} \in \mathbb{C}^N$ acquired through $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{e} \in \mathbb{C}^m$. For a sparsity set to 25, with other parameters still fixed at $m = 200$ and $N = 1000$, the test was carried out on ‘linear’ and ‘flat’ s -sparse vectors. For an error level η varying from 0 to 5 by increment of 0.01, we generated 100 Gaussian matrices $\mathbf{A}$, then 10 instances of a random support S and a Gaussian noise vector $\mathbf{e}$ normalized by $\|\mathbf{e}\|_2 = \eta x_s^*$, and we run the three algorithms on the resulting 1000 measurement vectors per error level. After the algorithms exited, we averaged the number of indices from the true support correctly caught, as well as the reconstruction error measured in ℓ_2 -norm. Under restricted isometry conditions, the proofs of

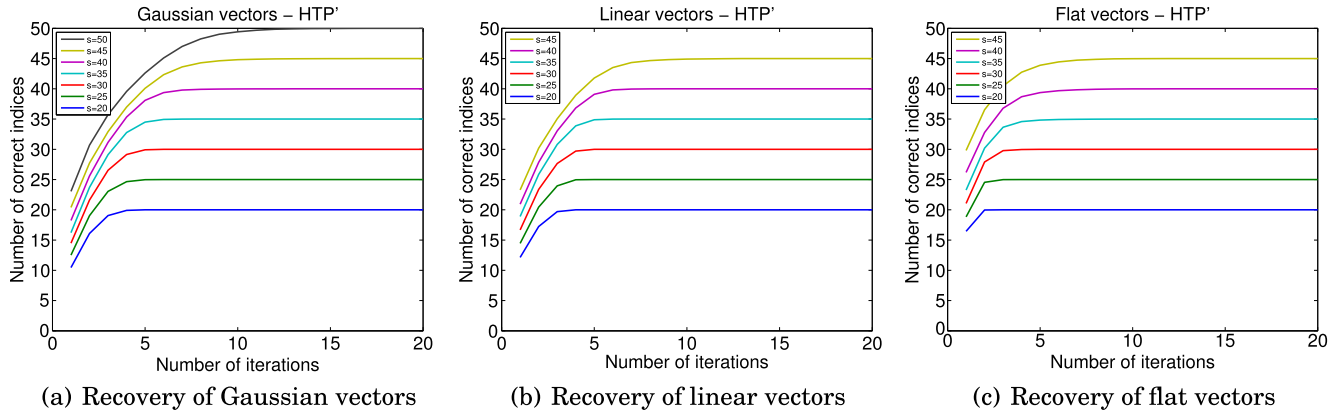


Fig. 6. Number of support indices correctly captured by the active set as a function of the iteration count for (HTP') using Gaussian measurements.

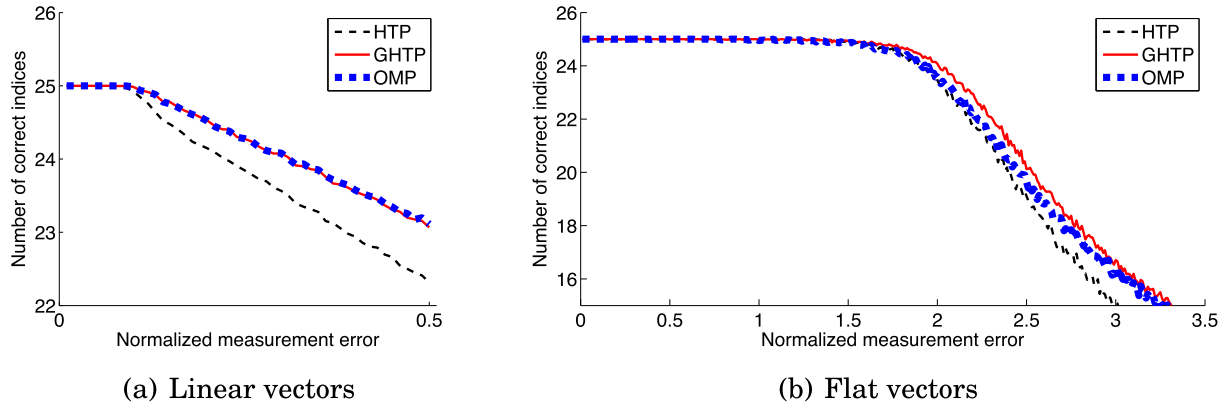


Fig. 7. Number of indices correctly captured as a function of the measurement error.

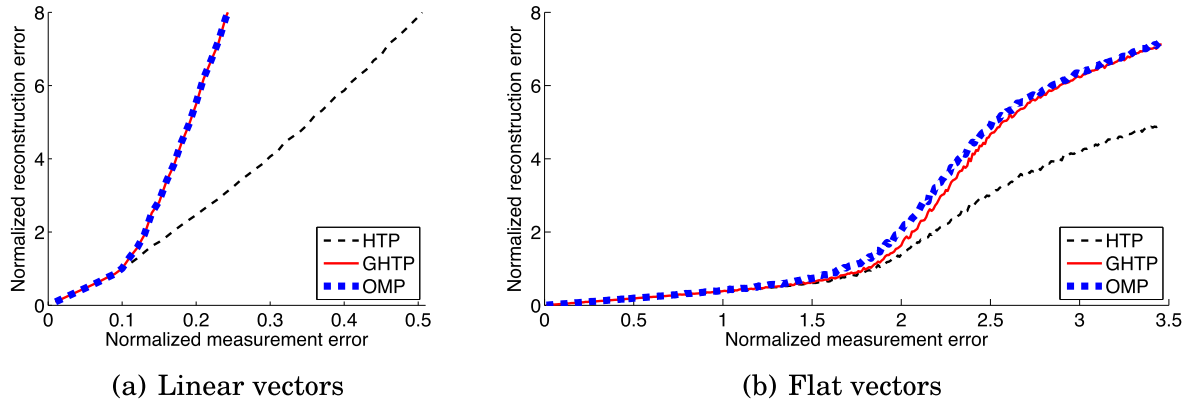


Fig. 8. Reconstruction error as a function of the measurement error.

Theorems 6 and 8 guarantee that all indices are correctly caught provided η is below some threshold η_* . This phenomenon is apparent in [Fig. 7](#) (abscissa: $\eta = \|\mathbf{e}\|_2/x_s^*$, ordinate: $\text{card}(S \cap S^\sharp)$, where $S^\sharp$ is the support of the output $\mathbf{x}^\sharp$ of the algorithm). The figure also shows that the threshold is approximately the same for the three algorithms and that it is higher for ‘flat’ vectors. For ‘linear’ vectors, the value $\eta_* \approx 0.1$ does not differ too much from the value $\gamma \approx 0.079$ obtained in [Theorems 6 and 8](#). Once the threshold is passed, we also notice that (HTP) catches fewer correct indices than (GHTP) and (OMP), probably due to the fact that it produces index sets of smaller size. In terms of reconstruction error, [Fig. 8](#) (abscissa: $\eta = \|\mathbf{e}\|_2/x_s^*$, ordinate: $\|\mathbf{x} - \mathbf{x}^\sharp\|_2/x_s^*$) displays transitions around the same values η_* as [Fig. 7](#). Although the ratio $\|\mathbf{x} - \mathbf{x}^\sharp\|_2/\|\mathbf{e}\|_2$ of reconstruction error by measurement error increases after the threshold, it does

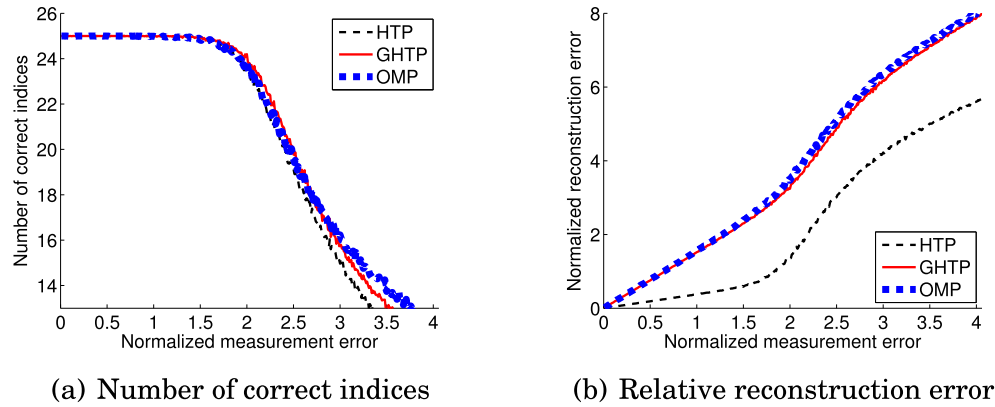


Fig. 9. Recovery of flat vectors with realistic stopping criteria.

not rule out a robustness estimate of the type $\|\mathbf{x} - \mathbf{x}^\sharp\|_2 \leq d \|\mathbf{e}\|_2$ valid for all $\mathbf{e} \in \mathbb{C}^m$ with an absolute constant d (this is established in [6] for (HTP) under a restricted isometry constant), since the prospective estimate is intuitively easier to satisfy when $\|\mathbf{e}\|_2$ becomes larger. We finally notice that (HTP) delivers lower reconstruction errors than (GHTP) and (OMP), probably due to the smaller index set involved in the final orthogonal projection.

We close the numerical section by revisiting the recovery of ‘flat’ vectors when the realistic stopping criteria are used, namely $n = 4s$ for (GHTP) as suggested by Theorem 8, $n = 12s$ for (OMP) as detailed in [7], and $S^n = S^{n-1}$ for (HTP) as before. We still consider measurements corrupted by Gaussian error and record the number of indices correctly captured together with the (relative) reconstruction error. Although we chose to show only the results for the least favorable case of ‘flat’ vectors, other tests (linear and Gaussian vectors) have been performed in the MATLAB files accompanying this article. Comparing Fig. 9 with Figs. 7 and 8 confirms that a realistic stopping criterion does not invalidate the observations made earlier when the ground truth was known.

References

- [1] R. Baraniuk, M. Davenport, R. DeVore, M. Wakin, A simple proof of the restricted isometry property for random matrices, *Constr. Approx.* 28 (3) (2008) 253–263.
- [2] T. Blumensath, M.E. Davies, Iterative hard thresholding for compressed sensing, *Appl. Comput. Harmon. Anal.* 27 (3) (2009) 265–274.
- [3] E.J. Candès, Y. Plan, A probabilistic and RIPless theory of compressed sensing, *IEEE Trans. Inform. Theory* 57 (11) (2011) 7235–7254.
- [4] E.J. Candès, J. Romberg, T. Tao, Robust uncertainty principles: exact signal reconstruction from highly incomplete frequency information, *IEEE Trans. Inform. Theory* 52 (2) (2006) 489–509.
- [5] D. Donoho, J. Tanner, Counting faces of randomly projected polytopes when the projection radically lowers dimension, *J. Amer. Math. Soc.* 22 (1) (2009) 1–53.
- [6] S. Foucart, Hard thresholding pursuit: an algorithm for compressive sensing, *SIAM J. Numer. Anal.* 49 (6) (2011) 2543–2563.
- [7] S. Foucart, H. Rauhut, *A Mathematical Introduction to Compressive Sensing*, Birkhäuser, 2013.
- [8] J.-P. Kahane, Variantes sur un théorème de Candès, Romberg et Tao, *Ann. Inst. Fourier* 63 (6) (2013) 2081–2096.
- [9] J. Lin, S. Li, Nonuniform support recovery from noisy random measurements by orthogonal matching pursuit, *J. Approx. Theory* 165 (2013) 20–40.
- [10] A. Maleki, Coherence analysis of iterative thresholding algorithms, in: *Proc. of 47th Annual Allerton Conference on Communication, Control, and Computing*, 2009, pp. 236–243.
- [11] D. Needell, Noisy signal recovery via iterative reweighted L1-minimization, in: *Proc. Asilomar Conference on Signals, Systems, and Computers*, Pacific Grove, CA, Nov. 2009.
- [12] D. Needell, J.A. Tropp, CoSaMP: iterative signal recovery from incomplete and inaccurate samples, *Appl. Comput. Harmon. Anal.* 26 (3) (2009) 301–321.
- [13] J.A. Tropp, A. Gilbert, Signal recovery from random measurements via orthogonal matching pursuit, *IEEE Trans. Inform. Theory* 53 (2007) 4655–4666.
- [14] P. Wojtaszczyk, Stability and instance optimality for Gaussian measurements in compressed sensing, *Found. Comput. Math.* 10 (1) (2010) 1–13.
- [15] T. Zhang, Sparse recovery with orthogonal matching pursuit under RIP, *IEEE Trans. Inform. Theory* 57 (9) (2011) 6215–6221.

Compressed Sensing Petrov-Galerkin Approximations for Parametric PDEs

Jean-Luc Bouchot, Benjamin Bykowski, and Holger Rauhut

Chair for Mathematics C (Analysis)

RWTH Aachen University

Email: {bouchot,rauhut}@mathc.rwth-aachen.de
benjamin.bykowski@rwth-aachen.de

Christoph Schwab

Seminar of Applied Mathematics

ETH Zurich

Email: christoph.schwab@sam.math.ethz.ch

Abstract—We consider the computation of parametric solution families of high-dimensional stochastic and parametric PDEs. We review theoretical results on sparsity of polynomial chaos expansions of parametric solutions, and on compressed sensing based collocation methods for their efficient numerical computation.

With high probability, these randomized approximations realize best N -term approximation rates afforded by solution sparsity and are free from the curse of dimensionality, both in terms of accuracy and number of samples evaluations (i.e. PDE solves).

Through various examples we illustrate the performance of Compressed Sensing Petrov-Galerkin (CSPG) approximations of parametric PDEs, for the computation of (functionals of) solutions of integral and differential operators on high-dimensional parameter spaces. The CSPG approximations reduce the number of PDE solves, as compared to Monte-Carlo methods, while being likewise nonintrusive, and being “embarrassingly parallel”, unlike dimension-adaptive collocation or Galerkin methods.

I. INTRODUCTION

A. Problem statement

We consider the problem of computing an (accurate) approximation of a parametric family u defined on a bounded, “physical” domain $D \subset \mathbb{R}^d$ and depending on possibly countably many parameters $y = (y_j)_{j \geq 1} \in U = [-1, 1]^{\mathbb{N}}$ implicitly through the parametric operator equation

$$A(y)u(y) = f. \quad (1)$$

Equation (1) is assumed to be well-posed in a reflexive, separable Banach space V , uniformly for all parameter sequences y . From a Bochner space perspective, with y distributed according to a certain distribution μ on U , we are looking for strongly μ -measurable maps $u \in \mathcal{L}(U, V; \mu)$. For simplicity, we assume here as in [1], [2] that $A(y)$ in (1) can be expressed as a linear combination of bounded linear operators $(A_j)_{j \geq 0} : V \rightarrow W'$, where W is a suitable second (separable and reflexive) Banach space, such that $A(y) = A_0 + \sum_{j \geq 1} y_j A_j$ in $\mathcal{L}(V, W')$. We hasten to add that the principal conclusions of the present work apply to larger classes of countably-parametric operator equations (1) with sparse solution families. Given a parametric family $f(y) \in W'$, we wish to approximate numerically the corresponding parametric solution family $\{u(y) : y \in U\}$. To simplify the exposition, we restrict ourselves in this note to nonparametric data f ; all

results which follow apply verbatim in the general case. We consider the more practical case of approximating functionals of the parametric solution: given $\mathcal{G} \in \mathcal{L}(V, \mathbb{R})$, approximate numerically the map $F : U \rightarrow \mathbb{R}$, $F(y) := \mathcal{G}(u(y))$.

B. Tools and ideas

The present CSPG approach is motivated by recent sparsity results for the parametric solution maps in [3], [4] where sparsity results for coefficient sequences of *generalized polynomial chaos* (GPC) expansions of $u(y)$ were established. The term GPC expansion refers to $u(y) = \sum_{j \in \mathcal{F}} x_j \varphi_j(y)$ for some basis $(\varphi_j)_{j \in \mathcal{F}}$ of $L^2_\mu(U)$, indexed on the countable family $\mathcal{F}$. Due to the boundedness and linearity of $\mathcal{G}$, we can similarly write $F(y) = \sum_j g_j \varphi_j$. The coefficient sequence $(g_j)_{j \geq 0}$ being compressible (i.e., there exists $0 < p < 1$, such that $\|g\|_p < \infty$), the computational challenge is to recover only the most important ones using only as few as possible solves of (1), while ensuring a given accuracy. The sparsity of the GPC coefficient sequence together with Stechkin inequality ensures that $\sigma_s(g)_q \leq s^{1/q-1/p} \|g\|_p$, $0 < p < q$, where $\sigma_s(g)_q$ denotes the best s -term approximation of g in the q norm. In other words, the coefficient sequence g can be compressed with high accuracy and hence (see Theorem 1 in Section II-B) the function F can be well approximated. To overcome the curse of dimensionality incurred by CS based methods (see [2, Eq.(1.16)]), we propose an extension based on $\ell_{p,\omega}$ spaces [5].

The rest of the paper is organized as follows. Section II reviews the background on weighted compressed sensing, Petrov-Galerkin approximations, and their combined use for high-dimensional PDEs. Section III introduces various model problems to numerically validate the theoretical results. We provide numerical experiments which show that the CSPG method can be easily implemented for various linear functionals $\mathcal{G}$, even with a large number of parameters. For the parametric model diffusion problems, we compare the convergence of the expectation with that of Monte-Carlo methods.

II. REVIEW ON CSPG APPROXIMATIONS

A. Generalized Polynomial Chaos (GPC) expansions and tensorized Chebyshev polynomials

GPC expansion of the parametric solution family $\{u(y) : y \in U\}$ of Eq. (1) are orthogonal expansions of $u(y)$ in terms

of tensorized Chebyshev polynomials $(T_\nu)_{\nu \in \mathcal{F}}$, with $\mathcal{F} := \{\nu \in \mathbb{N}_0^N : |\text{supp}(\nu)| < \infty\}$:

$$u(y) = \sum_{\nu \in \mathcal{F}} d_\nu T_\nu(y). \quad (2)$$

Note that here the coefficients d_ν are functions in V and that the expansion is in terms of the parameter sequences $y \in U$ and not in the spatial coordinates. The tensorized Chebyshev polynomials are defined as $T_\nu(y) = \prod_{j=1}^\infty T_{\nu_j}(y_j)$, $y \in U$, with $T_j(t) = \sqrt{2} \cos(j \arccos(t))$ and $T_0(t) \equiv 1$.

Defining σ the probability measure on $[-1; 1]$ as $d\sigma(t) := \frac{dt}{\pi\sqrt{1-t^2}}$, Chebyshev polynomials form an orthonormal system in the sense that $\int_{-1}^1 T_k(t)T_l(t)d\sigma(t) = \delta_{k,l}$ for $k, l \in \mathbb{N}_0$. Similarly, with the product measure $d\eta(y) := \bigotimes_{j \geq 1} d\sigma(y_j)$, the tensorized Chebyshev polynomials are orthonormal in the sense that $\int_{y \in U} T_\mu(y)T_\nu(y)d\eta(y) = \delta_{\mu,\nu}$, for $\mu, \nu \in \mathcal{F}$.

An important result proven in [6] ensures the ℓ_p summability, for some $0 < p \leq 1$, of the Chebyshev polynomial chaos expansion (2) of the solution of the model affine-parametric diffusion equation

$$-\nabla \cdot (a \nabla u) = f, \quad (3)$$

where a admits the affine-parametric expansion

$$a(y; x) = a_0(x) + \sum_{j \geq 1} y_j a_j(x). \quad (4)$$

Specifically, $\|(\|d_\nu\|_V)_{\nu \in \mathcal{F}}\|_p^p = \sum_{\nu \in \mathcal{F}} \|d_\nu\|_V^p < \infty$ under the condition that the sequence of infinity norms of the a_j is itself ℓ_p summable: $\|(\|a_j\|_\infty)_{j \geq 1}\|_p < \infty$.

These results were extended to the weighted ℓ_p spaces (see Section II-B) for the more general parametric operator problem, Eq.(1) with linear parameters, in [2]: $(\|d_\nu\|_V)_{\nu \in \mathcal{F}} \in \ell_{\omega,p}(\mathcal{F})$ under certain assumptions, and with Eq. (6) defining the weighted ℓ_p spaces.

B. Weighted compressed sensing

The last decade has seen the emergence of compressed sensing [7] as a method to solve underdetermined linear systems $Ax = y$ under sparsity constraints $\|x\|_0 := |\text{supp}(x)| \leq s$. Some work has focused on finding conditions on A such that

$$x^\# = \text{argmin} \|x\|_1, \quad \text{s.t. } \|Ax - y\|_2 \leq \eta \quad (5)$$

yields an exact and unique solution. One known condition is that the matrix A should fulfill a *Restricted Isometry Property* (RIP) of order s and constant δ for reasonable values of s and δ . A is said to have $\text{RIP}(s, \delta)$ if, for any s -sparse vector x , $\|Ax\|_2^2 - \|x\|_2^2 \leq \delta \|x\|_2^2$. In other words, for small δ , the matrix A behaves almost like an isometry on sparse vectors.

In parallel, the community also focused on finding matrices obeying the RIPs. While the problem of finding deterministic matrices with interesting RIPs is still open, it has been proven that random matrices are good candidates [8].

More recently, results from compressed sensing have been generalized to handle weighted sparsity [5]. In a similar manner, given a weight sequence $(\omega_j)_{j \in \Lambda}$, $\omega_j \geq 1$, one can derive

weighted sparsity of a vector x as $\|x\|_{\omega,0} := \sum_{j \in \text{supp}(x)} \omega_j^2$. Similarly, weighted ℓ_p norms are defined, for $0 < p \leq 2$ as $\|x\|_{\omega,p} = \left(\sum_{j \in \Lambda} \omega_j^{2-p} |x_j|^p \right)^{1/p}$ and the associated spaces are defined as

$$\ell_{p,\omega}(\Lambda) := \{x \in \mathbb{R}^\Lambda : \|x\|_{p,\omega} < \infty\}. \quad (6)$$

Weighted ℓ_p can be further extended to functions $f = \sum_{j \geq 1} x_j \varphi_j$ expanded in certain basis elements $(\varphi_j)_{j \geq 1}$ as $\|f\|_{\omega,p} := \|x\|_{\omega,p}$. This in particular is interesting for function interpolation from very few samples:

Theorem 1 (Complete statement in [5]). *For $(\varphi_j)_{j \in \Lambda}$ a countable orthonormal system and a sequence of weights $\omega_j \geq \|\varphi_j\|_\infty$, there exists a finite subset Λ_0 with $N = |\Lambda_0|$ and $s > 2\|\omega\|_\infty^2$ such that drawing $m \geq c_0 s \log^3(s) \log(N)$ samples at random ensures that, with high probability,*

$$\|f - f^\#\|_\infty \leq \|f - f^\#\|_{\omega,1} \leq c_1 \sigma_s(f)_{\omega,1}, \quad (7)$$

$$\|f - f^\#\|_2 \leq d_1 \sigma_s(f)_{\omega,1} / \sqrt{s} \quad (8)$$

where $f := \sum_{j \in \Lambda} x_j \varphi_j$ is the function to recover, and $f^\#(y) := \sum_{j \in \Lambda_0} x_j^\# \varphi_j(y)$ is the solution obtained via weighted ℓ_1 minimization.

In particular, the previous theorem offers error bounds in both L_2 and L_∞ norms.

C. Petrov-Galerkin Approximations

Petrov-Galerkin (PG) approximations of a function $u \in V$ defined on D are based on one-parameter families of nested and finite-dimensional subspaces $\{V^h\}_h$ and $\{W^h\}_h$ which are dense in V and W , respectively (with the discretization parameter h denoting e.g. meshwidth of triangulations in Finite Element methods, or the reciprocal of the spectral order in spectral methods). Given $y \in U$, the PG projection of $u(y)$ onto V^h , $u^h(y) := G_h(u(y))$, is defined as the unique solution of the finite-dimensional, parametric variational problem: Find $u^h \in V^h$ such that

$$(A(y)u^h(y))(w^h) = f(w^h), \quad \forall w^h \in W^h. \quad (9)$$

Note that for $A(y)$ boundedly invertible and under some (uniform w.r. to $y \in U$ and w.r. to h) inf-sup conditions (see, e.g. [1, Prop. 4]) the PG projections G_h in (9) are well-defined and quasioptimal *uniformly w.r. to $y \in U$* : there exists a constant $C > 0$ such that for all $y \in U$ and $h > 0$ sufficiently small it holds

$$\|u(y) - u^h(y)\|_V \leq C \inf_{v^h \in V^h} \|u(y) - v^h\|_V. \quad (10)$$

D. Review of the CSPG algorithm

We review the CSPG algorithm of [2]. It relies on the evaluation of m solutions $b^{(l)} := \mathcal{G}(u(y^{(l)}))$ of Eq. (1) given m randomly chosen parameter instances $y^{(l)} = (y_j^{(l)})_{j \geq 1}$, $1 \leq l \leq m$. These real-valued solutions $b^{(l)}$ are then used in a weighted ℓ_1 minimization program in order to determine the most significant coefficients in expansion (2), see Algorithm 1 (see [2] for more details)

Data: Weights $(v_j)_{j \geq 1}$, an accuracy ε and sparsity s parameters, a multiindex set $\mathcal{J}_0^s$ with $N = |\mathcal{J}_0^s| < \infty$, a compressibility parameter $0 < p < 1$, and a number of samples m .

Result: $(g_\nu^\#)_{\nu \in \mathcal{J}_0^s}$, a CS-based approximation of the coefficients $(g_\nu)_{\nu \in \mathcal{J}_0^s}$.

for $l = 1, \dots, m$ **do**
 | Draw $y^{(l)}$ at random;
 | Numerically estimate $b^{(l)}$;
end

$$\begin{aligned} (\Phi)_{l,\nu} &\leftarrow T_\nu(y^{(l)}), \quad \nu \in \mathcal{J}_0^s, 1 \leq l \leq m \\ \omega_\nu &\leftarrow 2^{\|\nu\|_0/2} \prod_{j:\nu_j \neq 0} v_j^{\nu_j}, \quad \nu \in \mathcal{J}_0^s \end{aligned} \quad (11)$$

Compute $g^\#$ as the solution of

$$\min \|g\|_{\omega,p}, \quad \text{s.t. } \|\Phi g - b\|_2 \leq 2\sqrt{m}\varepsilon \quad (12)$$

Algorithm 1: Pseudo-code for the CSPG Algorithm

Before we justify the use of this algorithm, it is worth mentioning the roles of the different parameters: ε is an accuracy parameter that will have an impact on the discretization parameter h and on the number of samples required. The importance and impact of the choice of the weight sequence $(v_j)_{j \geq 1}$ is still not clear and is left for future research. The index set $\mathcal{J}_0^s \subset \mathcal{F}$ (see definition below) acts as an estimation of the optimal set containing the best s -(weighted) sparse approximation of F .

Theorem 2 ([2]). *Let $u(y)$ be the solution to Eq. (1) such that A_0 is boundedly invertible and $\sum_{j \geq 1} \beta_{0,j} \leq \kappa \in (0, 1)$ where $\beta_{0,j} := \|A_0^{-1} A_j\|_{\mathcal{L}(V,V)}$, and the scales of smoothness spaces have a parametric regularity property (see [2, Assumption 2.3]). If the sequence $(v_j)_{j \geq 1}$ is such that $\sum_j \beta_{0,j} v_j^{(2-p)/p} \leq \kappa_{v,p} \in (0, 1)$ and $(\beta_{0,j})_j \in \ell_{v,p}$ then $g \in \ell_{\omega,p}$ with ω as in Eq. (11). Choose an accuracy $\varepsilon > 0$ and sparsity s such that*

$$\sqrt{5} 4^{1-1/p} s^{1/2-1/p} \|g\|_{\omega,p} \leq \varepsilon \leq C_2 s^{1/2-1/p} \|g\|_{\omega,p}, \quad (13)$$

with $C_2 > \sqrt{5} 4^{1-1/p}$ independent of s and define $\mathcal{J}_0^s := \{\nu \in \mathcal{F} : 2^{\|\nu\|_0} \prod_{j:\nu_j \neq 0} v_j^{2\nu_j} \leq s/2\}$ with $N := |\mathcal{J}_0^s|$. Pick $m \asymp Cs \log^3(s) \log(N)$ (C a universal constant) samples i.i.d at random w.r.t. the Chebyshev distribution and define $\tilde{F}(y) := \sum_{\nu \in \mathcal{J}_0^s} g_\nu^\#(y)$ with $g^\#$ solution of Eq. (12). Then, there exists a (universal) constant C' such that, with probability at least $1 - 2N^{-\log^3(s)}$, it holds:

$$\|F - \tilde{F}\|_2 \leq C' \|g\|_{\omega,p} s^{1/2-1/p} \leq C_g'' \left(\frac{\log^3(m) \log(N)}{m} \right)^{1/p-1} \quad (14)$$

$$\|F - \tilde{F}\|_\infty \leq C' \|g\|_{\omega,p} s^{1-1/p} \leq C_g'' \left(\frac{\log^3(m) \log(N)}{m} \right)^{1/p-1} \quad (15)$$

where C_g'' depends on C' and $\|g\|_{\omega,p}$.

Remark 1. It is worth mentioning that many different parameters are to be tuned and their interdependencies are still not clear. The choice of the accuracy ε is constrained by the chosen weighted sparsity s and the norm of the GPC coefficient vector $\|g\|_{\omega,p}$. Finding estimates to this unknown is not a trivial task and depends on a certain compressibility parameter p . Some analysis is needed for every particular case in order to estimate this norm. Finally, the last parameters to look at are the weight sequence ω and the original set $\mathcal{J}_0^s$. This set only needs to contain a subset Λ_N of optimal coefficients of the GPC expansion. The ones introduced here may not be optimal and finding other good candidates is left for further contributions.

III. NUMERICAL EXAMPLES

For the empirical validation of the theories introduced in the previous sections, we consider several model problems in 1-dimensional physical domains. We are mostly interested in the high-dimensionality of the parameter space U . The physical domain of the operator equation (1) renders the PG discretization more complicated but does not affect the CS access to the parameter space U .

In the numerical examples we consider model affine-parametric diffusion equations (3), (4). As stated in the introduction, we consider $y = (y_j)_{j \geq 1} \in U$ and $x \in D = [0, 1]$. For the practical numerical results, we use trigonometric polynomials as basis functions a_j in (4), for a constant α :

$$a_{2j-1}(x) = \frac{\cos(j\pi x)}{j^\alpha}, \quad a_{2j}(x) = \frac{\sin(j\pi x)}{j^\alpha}. \quad (16)$$

We used the nominal field $a_0 = 0.1 + 2\zeta(\alpha, 1)$ where $\zeta(\alpha, 1) := \sum_{n \geq 1} \frac{1}{n^\alpha}$. The remaining parameters are set as:

- $f \equiv 10$,
- For $j \geq 1$, $\tilde{v}_j = \gamma j^\tau$, where $\gamma = 1.015$ and $\tau = \log(\lfloor \sqrt{s}/2\gamma \rfloor) / \log(d)$, then $v_{2j-1} = v_{2j} = \tilde{v}_j$, $j \geq 1$ (see below for some remarks about d),¹
- $\alpha = 2$, $\varepsilon = 0.5 \cdot 10^{-9}$,
- $m = 2s \log(N)$ with $\mathcal{J}_0^s$ denoting the initial set of important coefficients of the GPC, and $N := |\mathcal{J}_0^s|$. s varies from 30 to 500, see Table I for examples.²

Note that no values are assigned for the compressibility parameter p . While it is of prime importance in the theoretical analysis and for the understanding of the algorithm, it has no effect on the implementation. For information purposes, we have also given the bounds on the error in Table I for a compressibility parameter $p = 1/2$ (i.e. $\text{err}^{1/p-1/2}$ and $\text{err}^{1/p-1}$, resp. for the L^2 and L^∞ norms, where $\text{err} = \frac{\log^3(m) \log(N)}{m}$). Be aware that these estimations do not include the constant $\mathcal{Q}_g''$ in front in Eqs. (14) and (15). Another remark regarding the implementation is that the number of variables has been truncated to $d = 20$ parameters (i.e. k stops at 10 in the expansion of the diffusion coefficient).

¹doubling the v_j ensures that the sin and cos functions with the same frequencies are given the same weights.

²This number of samples is different than the one suggested in the theorem. This is motivated by results on nonuniform recovery from random matrices in compressed sensing - see [7, Ch.9.2].

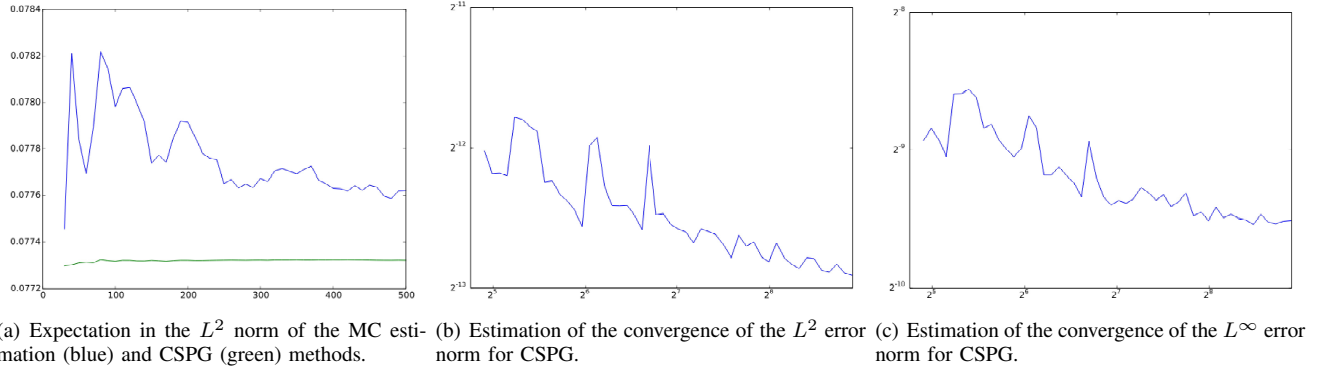


Fig. 1. Numerical results of the average of the solution of the operator equation (1) (see text for details).

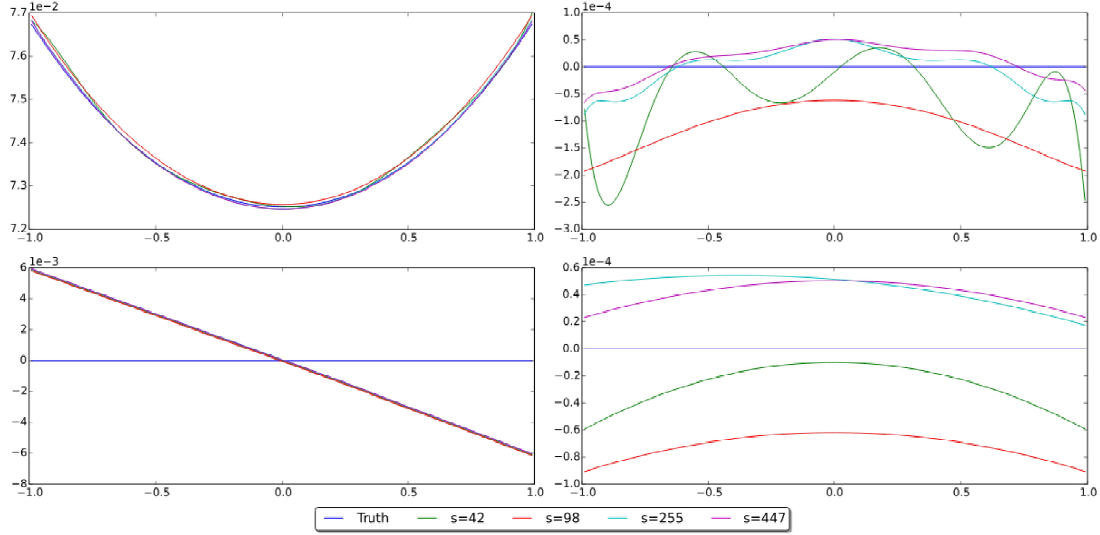


Fig. 2. Convergence pointwise for various sparsity levels.

 TABLE I
 DEPENDENCY OF THE NUMBER OF SAMPLES ON THE SPARSITY AND BOUNDS OF THE L^2 AND L^∞ ERROR BOUNDS FOR $p = 1/2$.

s	30	100	200	300	400	500
N	1554	5712	10006	12690	15601	17043
m	441	1731	3685	5670	7725	9744
L^2	1.9396	1.4393	1.1765	1.0373	0.9469	0.8802
L^∞	3.7619	2.0717	1.3843	1.0759	0.8967	0.7747

While we would ideally like to compare the approximations $\tilde{F}$ with the true functions F in the L^2 and L^∞ norms, this is unrealistic. We provide a lower bound on the L^∞ norm based on independent draws of test samples $z^{(l)}$, $1 \leq l \leq m_{\text{test}} := 10000$. The error in the L^2 norm can be accurately estimated (through Bernstein inequality) by sums of differences.

On the other hand, a comparison with usual Monte Carlo methods can be done only when comparing the first coefficient g_0 of the expansion (2), the expectation of the function, and the Monte-Carlo estimate. Indeed, for $F(y) := \sum_{\nu \in \mathcal{F}} g_\nu T_\nu(y)$, for T_ν the tensorized Chebyshev polynomials, it holds $\mathbb{E}[F] = g_0$. Monte-Carlo estimation is done as the empirical mean

$E_m[F] := \frac{1}{m} \sum_{l=1}^m F(y^{(l)})$. It is known (see [9, Lemma 4.1]) that for a number of samples m drawn at random it holds

$$\|E_m[F] - \mathbb{E}[F]\|_{L^2(U, \eta)} \leq m^{-1/2} \|F\|_{L^2(U, \eta)} \quad (17)$$

A. Average

For this first use case, we consider the functional $I : V \rightarrow \mathbb{R}$ defined as $I(u(y)) := \int_{x \in D} u(y; x) dx$, and $D = [0, 1]$. Fig. 1(a) illustrates the convergence of both the Monte-Carlo and the CSPG method. The x -axis corresponds to the sparsity parameter s , or, seen differently, to the number of samples used (see Table I for the correspondance).

The CSPG method shows a much better convergence of the expectation. The other interesting point to note and that has not been studied so far, is the stability of the first coefficient. While the number of samples required to ensure small L^2 and L^∞ errors is important (as illustrated in Figs. 1(b) and 1(c) respectively) it seems that the reliable estimation of the first coefficient is not drastically affected by strong undersampling.

Figs. 1(b) and 1(c) respectively illustrate the estimation of the empirical L^2 and the L^∞ errors. These values have to be compared with the last two rows of Table I.

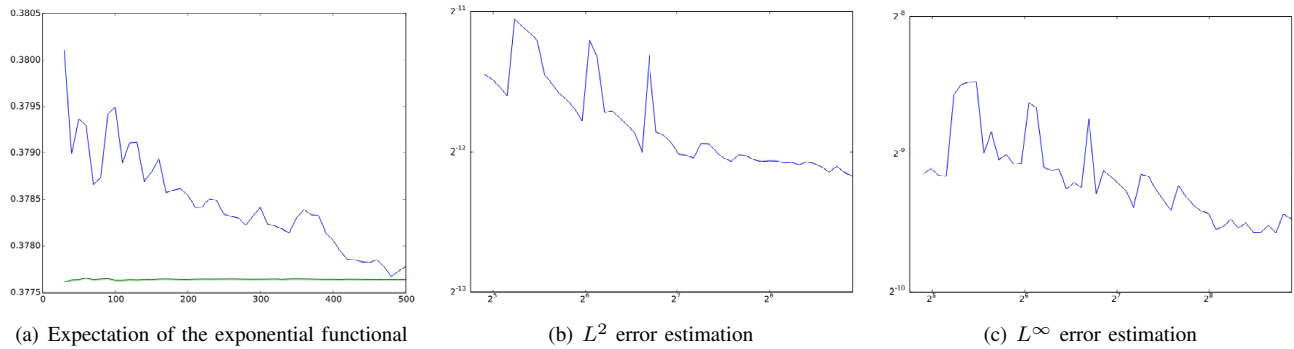


Fig. 3. Numerical results of the exponential weighted integral.

We can also look at the pointwise convergence of the approximation in U with respect to the sparsity (or number of samples). Fig. 2 shows the values of $t \mapsto \tilde{F}(te_k)$, where $e_k = (\delta_{kj})_j$, for $-1 \leq t \leq 1$. The upper left corner shows the computed solution of $t \mapsto \tilde{F}(te_1)$ for various sparsity levels. The remaining three quadrants correspond to the (pointwise) error for $k = 1, 2, 3$. The CSPG approximated solutions converge, as s increases, to the true solution.

B. Exponential convolution kernel

We now study the accuracy of the CSPG method when we consider an exponential weighted integral centered at a point $x_0 = 0.5$: $F(y) := u(y) * K(x_0)$ where the convolution kernel is given, for a constant $\mathcal{N}$, by $K(x) := \frac{1}{\mathcal{N}} e^{-\frac{|x-x_0|}{\sigma}}$.

Again we can notice the faster convergence rate of the CSPG method. Moreover, the bounds derived in Theorem 2 are very crude and easily achieved numerically. Fig. 3 illustrates the convergence of the CSPG algorithm. In the left graph, the expectation is shown with respect to the (weighted) sparsity. The blue curve illustrates the convergence of the MC method, while the rather flat constant line implies that the CSPG method computes the first coefficient of the GPC expansion g_0 well very soon. The two following log-log plots show the decrease of the L^2 and L^∞ errors. As the sparsity increases, the results are more accurate and, seem to be more stable.

Remark 2. Though our experiments suggest otherwise, the ordering of the coefficients is not required for our CS approach.

IV. CONCLUSION

This paper shows the practical and numerical use of a compressed-sensing based Petrov-Galerkin approximation method for computing the functionals of solutions of high-dimensional parametric elliptic equations. We have empirically verified the theoretical results and compared them, when possible, to MC methods, where the convergence appears to be slower than the proposed approach. Although only model, affine-parametric problems were considered here, we emphasize that the presently proposed CSPG approach is nonintrusive, and exploits sparsity of the GPC expansion; therefore, its range of application extends to non-affine parametric operator equations, such as PDEs in uncertain domains. The presently

proposed CS based GPC recovery does not, a-priori, impose specific structure on the sets of active coefficients as required in some adaptive collocation methods, see for instance [1], [10]. It is also important to put our results in perspective with those of [11] where probabilistic error bounds were derived.

The promising results presented here require further analysis as to the quantitative effect of the different parameters, in particular regarding the weights in Eq. (11) (and associated v_j 's), are not well understood. Moreover, it will be of interest to later derive methods based on these techniques to recover the full, parametric solution of (1), rather than only a functional.

ACKNOWLEDGMENT

J.-L. B. and H. R. are funded by the European Research Council through the Starting Grant StG 258926 (SPALORA).

C. S. acknowledges support from the Swiss National Science Foundation.

REFERENCES

- [1] C. Schwab, "Qmc galerkin discretization of parametric operator equations," in *Monte Carlo and Quasi-Monte Carlo Methods 2012*. Springer, 2013, pp. 613–629.
- [2] H. Rauhut and C. Schwab, "Compressive sensing Petrov-Galerkin approximation of high-dimensional parametric operator equations," arXiv preprint arXiv:1410.4929, 2014.
- [3] A. Cohen, R. DeVore, and C. Schwab, "Analytic regularity and polynomial approximation of parametric and stochastic elliptic PDE's," *Analysis and Applications*, vol. 9, no. 01, pp. 11–47, 2011.
- [4] A. Cohen, R. DeVore, and C. Schwab, "Convergence rates of best N-term Galerkin approximations for a class of elliptic sPDEs," *Foundations of Computational Mathematics*, vol. 10, no. 6, pp. 615–646, 2010.
- [5] H. Rauhut and R. Ward, "Interpolation via weighted ℓ_1 minimization," *Applied and Computational Harmonic Analysis*, 2015, to appear.
- [6] M. Hansen and C. Schwab, "Analytic regularity and nonlinear approximation of a class of parametric semilinear elliptic PDEs," *Mathematische Nachrichten*, vol. 286, no. 8–9, pp. 832–860, 2013.
- [7] S. Foucart and H. Rauhut, *A mathematical introduction to compressive sensing*. Springer, 2013.
- [8] R. Baraniuk, M. Davenport, R. DeVore, and M. Wakin, "A simple proof of the restricted isometry property for random matrices," *Constructive Approximation*, vol. 28, no. 3, pp. 253–263, 2008.
- [9] A. Barth, C. Schwab, and N. Zollinger, "Multi-level monte carlo finite element method for elliptic pdes with stochastic coefficients," *Numerische Mathematik*, vol. 119, no. 1, pp. 123–161, 2011.
- [10] C. Schillings and C. Schwab, "Sparse, adaptive smolyak quadratures for bayesian inverse problems," *Inverse Problems*, vol. 29, no. 6, p. 065011, 2013.
- [11] A. Doostan and H. Owhadi, "A non-adapted sparse approximation of pdes with stochastic inputs," *Journal of Computational Physics*, vol. 230, no. 8, pp. 3015–3034, 2011.



Discrepancy norm: Approximation and variations



Jean-Luc Bouchot^{a,*}, Frank Bauer^b

^a Department of Mathematics, Drexel University, 3141 Chestnut Street, Philadelphia, United States

^b DZ BANK AG, Kapitalmärkte Handel, Frankfurt am Main, Germany

ARTICLE INFO

Article history:

Received 30 July 2013

Received in revised form 11 May 2014

Keywords:

Misalignment

Discrepancy norm

Correlations

Image registration

Signal alignment

p -norm approximation

ABSTRACT

This paper introduces an approach for the minimization of the discrepancy norm. The general idea is to replace the infinity norms appearing in the definition by L^p norms which are differentiable and to make use of this approximation for local optimization.

We will show that the discrepancy norm can be approximated up to any ε and the robustness of this approximation is shown. Moreover, analytical formulation of the derivative of the discrepancy correlation function is given.

In a following step we extend the results to higher dimensional data and derive the related forms for approximations and differentiations.

© 2014 Elsevier B.V. All rights reserved.

1. Introduction

This paper deals with the use of the discrepancy norm as introduced by Weyl [1] in the context of signal analysis. A great deal of research in discrepancy theory has been done in numerical integration [2], analysis of randomness [3] and uniformity [4], low-discrepancy sequences for sampling [5], computational geometry [6], and geometric approximation [7]. We invite the reader to have a look at the book of Chazelle for a rather complete review of topics in discrepancy theory [8].

Here we investigate another application of discrepancy theory, namely to signal analysis. As re-introduced by Neunzert and Welton [9] and later by Moser [10] the discrepancy norm becomes an interesting tool for pattern recognition. We revisit here some previous work [11] where an approximation of the discrepancy norm was developed for discrete time series analysis. We extend our results to continuous measurable functions and compute its analytic derivative that can be used for optimization purposes.

In a second step we generalize these results to higher-dimensional signals by considering directional discrepancies and combining them together. Artificial experiments are provided to validate our theoretical findings.

Our contribution is two-fold. We first extend previous results [11] obtained for one-dimensional discrete signals to the more general case of continuous measurable functions. We show that our approximation can be done up to any ε for any function from a measurable space with finite measure. These results are also proven in higher dimension. Second we compute the derivative of this approximation that comes handy for optimization (for instance in the context of signal alignment).

We first start by reviewing the basics regarding the discrepancy norm in Section 2: its definition and its useful properties. Of particular interest for signal alignment is the monotonicity property of its autocorrelation function (Property C3). Then Section 3 introduces the approximation through L^p -norms and its derivative for one dimensional data. We show some convergence criteria to prove the robustness of the approaches. Section 4 extends the previous results to higher dimensional

* Corresponding author.

E-mail address: jean-luc.bouchot@drexel.edu (J.-L. Bouchot).

data. We see how the problem arise in the discrepancy norm's definition when dealing with higher dimensions. Numerical experiments are given along the theoretical results.

2. Discrepancy norm

2.1. Introduction and first definitions

Discrepancy measurements have been studied for a very long time and date back to Hermann Weyl's theory [1]. Here we are more interested in its application in terms of similarity measure [12].

As seen in [13,14], the discrepancy norm introduced here is particularly suited for image registration. Other approaches have been studied recently for pattern recognition [9] and vision purposes [15]. It has been reintroduced for general measurable functions by Moser [10].

Definition 1 (*Discrepancy Norm*). Let $(\mathbb{R}, \Sigma, \mu)$ be a measure space with μ a finite measure. The discrepancy norm of a function $f \in L(\mathbb{R}, \mu)$ is defined as

$$\|f\|_D = \sup_{[a,b] \subset \mathbb{R}} \left| \int_a^b f d\mu \right|.$$

2.2. Properties

It has been proven that the discrepancy norm shows particularly interesting properties for analyzing misalignments in signals [12] compared to other traditional measures.

Definition 2 (*Misalignment Functions*). Given a distance measure

$$d : L(\mathbb{R}, \mu) \times L(\mathbb{R}, \mu) \rightarrow \mathbb{R}^+$$

one can define a misalignment measure of a signal as follows:

$$\Delta_d[f](t) := d(f, f_t)$$

where f_t is a translation of f defined as $\forall x \in \mathbb{R}, f_t(x) = f(x - t)$.

In [10] three criteria have been introduced to evaluate the ability of a metric to act as a misalignment measure:

C1 Positive-definiteness:

$$\Delta_d[f](t) = 0 \Leftrightarrow t = 0$$

C2 Continuity:

$$\Delta_d[f](t) \rightarrow 0, \quad \text{for } t \rightarrow 0$$

C3 Monotonicity:

$$\forall 1 \leq \lambda, \quad \forall t \in \mathbb{R}, \quad \Delta_d[f](t) \leq \Delta_d[f](\lambda t).$$

Property 1 (*Compatibility of the Discrepancy Norm*). Given any measurable function $f \in L(\mathbb{R}, \mu), f \geq 0, \mu$ almost everywhere, the discrepancy norm fulfills the three criteria described above.

Note that this result is stated on the real line but it could be adapted on the torus to deal with periodic functions. With such function, criteria C1 and C3 need to be adapted. We actually have a more stronger result regarding the continuity criterion:

Property 2 (*Lipschitz Continuity*). The variations of the discrepancy misalignment function are bounded by its infinity norm:

$$\Delta_{\|\cdot\|_D}[f](t) \leq |t| \|f\|_\infty.$$

The next property defines another computation for the discrepancy norm which appears to be much faster in the applications.

Property 3 (*Linear Computation*). Let $f \in L(\mathbb{R}, \mu)$, the following holds true

$$\|f\|_D = \max_{b \in \mathbb{R}} \int_{-\infty}^b f d\mu - \min_{a \in \mathbb{R}} \int_{-\infty}^a f d\mu.$$

This last formulation allows us to compute the discrepancy of a function by means of integral images [16,17]. These integral images are, in practice, computed in a linear time with respect to the number of samples.

3. Approximation and optimization: the one dimensional case

In this section we extend the results shown in [11] in the context of discrete one dimensional signals. We extend here this approach to more general measurable functions and derive formula to assess the quality of the approximation. Finally a formula for the derivative of the discrepancy correlation function is given in the last subsection.

3.1. Approximation of discrete signals

This subsection is intended as an introduction for our next results and is based on the work of Bouchot et al. [11].

In this case, we consider the counting measure μ_c and a function of bounded support. Without loss of generality, we have $f : D := \{1, \dots, N\} \rightarrow \mathbb{R}$ and the discrepancy norm reads

$$\|f\|_D = \max_{i,j \in D} \left| \sum_{k=1}^j f_k - \sum_{k=1}^i f_k \right| = \max_{j \in D} \left\{ 0, \sum_{k=1}^j f_k \right\} - \min_{i \in D} \left\{ 0, \sum_{k=1}^i f_k \right\}.$$

Note that the 0 in the max, min operators is essential as the sums should be taken starting at $-\infty$. But the function can be extended with 0 outside the domain D . We can also equivalently write $\|f\|_D = \max_{j \in D_0} \sum_{k=0}^j f_k - \min_{i \in D_0} \sum_{k=0}^i f_k$ where we define $D_0 = D \cup \{0\}$ and $f_0 = 0$.

Before we recall the idea of the approximation and its property, we give some definitions.

Definition 3 (Generalized Mean). Let ϕ be a continuous invertible function, let $f = \{f_k\}_{k=1}^N$, we can define [18] a mean as

$$M_\phi(f) = \phi^{-1} \left(\frac{\sum_{k=1}^N \phi(f_k)}{N} \right).$$

Moreover, as we would expect from a mean, we have $\min_k f_k \leq M_\phi(f) \leq \max_k f_k$ (see [19, Chapter II] for the proof).

Now if we consider taking L^p norms as ϕ in the previous definition, this yields the following p means:

$$M_p(f) = \left(\frac{\sum_{k=1}^N |f_k|^p}{N} \right)^{1/p}.$$

In our previous work we have introduced an approximation of the discrepancy norm for discrete one dimensional signals as described in the following theorem.

Theorem 1. Let $f \in \ell^1 = L(\mathbb{Z}, \mu_c)$, we define

$$\gamma_p : \ell^1 \rightarrow \mathbb{R}^+ \\ f \mapsto \gamma_p(f) := \ln \left(\frac{M_p(\chi(f))}{M_{-p}(\chi(f))} \right)$$

where $\chi(f)(k) := \exp(\sum_{l=0}^k f_l)$ is the exponential of the cumulative function.

The following holds:

$$\gamma_p(f) \leq \|f\|_D < \gamma_p(f) + \frac{2}{p} \ln(N+1)$$

and γ_p is positive definite.

Proofs and details about this theorem can be found in [11].

Corollary 1 (Choice of p).

$$\forall \varepsilon > 0, \forall f \in \ell^1, \forall p \in \mathbb{R}, \quad p \geq p_0 := \frac{2}{\varepsilon} \ln(N+1) \Rightarrow |\gamma_p(f) - \|f\|_D| \leq \varepsilon.$$

Proof. According to the previous theorem, we have $|\gamma_p(f) - \|f\|_D| < \frac{2}{p} \ln(N+1)$ so the corollary holds true whenever $\frac{2}{p} \ln(N+1) \leq \varepsilon \Leftrightarrow p \geq p_0$ as given in the statement. $\square$

This last part allows to compute an optimal approximation in a practical way. Indeed, due to the power p coming in the approximation process, the danger of getting overflow increases with better approximations.

Conversely computing the approximation of a signal of 1000 samples at order $p = 8$ creates an error not larger than 0.86256, completely independent of the signal's magnitude.

3.2. Approximation of continuous measurable functions

Now that we have seen how practical and close the approximation can be, we want to extend this to continuous measurable functions. From now on we go back to the setting of measure space on $\mathbb{R} : (\mathbb{R}, \Sigma, \mu)$ with μ being a finite measure. In the same way, we can introduce the continuous counter-part of the generalized means

$$M_\phi(f) = \phi^{-1} \left(\frac{\int_{\mathbb{R}} \phi(f(x)) d\mu(x)}{\int_{\mathbb{R}} d\mu} \right).$$

And the following bounding property still holds $\inf_{x \in \mathbb{R}} f(x) \leq M_\phi(f) \leq \sup_{x \in \mathbb{R}} f(x)$ (where $\inf$ and $\sup$ are understood as their essential definitions: $\text{ess. sup}(f) = \inf \{C \in \mathbb{R} : \mu\{x : f(x) > \sup f\} = 0\}$). We also redefine the p -means M_p in the same way.

The proposed approximation is reintroduced in the same way (with capital letter for the continuous case). Let us first define χ as the exponential of the cumulative function, i.e.:

$$\begin{aligned} \chi &: \begin{cases} L(\mathbb{R}, \mu) \rightarrow L(\mathbb{R}, \mu) \\ f \mapsto \chi(f) : \chi(f)(x) = e^{\int_{-\infty}^x f(t) d\mu(t)} \end{cases} \\ \Gamma_p &: \begin{cases} L^1(\mathbb{R}, \mu) \rightarrow \mathbb{R}^+ \\ f \mapsto \Gamma_p(f) := \ln \left(\frac{M_p(\chi(f))}{M_{-p}(\chi(f))} \right). \end{cases} \end{aligned}$$

Note that the function f is not necessarily defined on the whole real line but on an interval or union of intervals $I \subset \mathbb{R}$. In this case we can consider its extension $\tilde{f}$ on the real line as $\tilde{f}(x) = f(x)$ if $x \in I$ and $\tilde{f} = 0$ elsewhere. While this extension is not the unique, nor the smartest, option it is sufficient for now.

The following theorem holds:

Theorem 2 (Approximation of the Discrepancy Norm for 1D Continuous Functions). Γ_p defines an approximation of the discrepancy norm in the sense that

- (i) $\forall f \in L(\mathbb{R}, \mu), \Gamma_p(f) \xrightarrow{p \rightarrow \infty} \|f\|_D$
- (ii) $\Gamma_p(f) \leq \|f\|_D$
- (iii) $\forall \varepsilon > 0, \exists p^* : \forall p \in \mathbb{R}, p \geq p^* \Rightarrow |\Gamma_p(f) - \|f\|_D| \leq \varepsilon.$

Proof. Let $f \in L(\mathbb{R}, \mu)$, we need to compute $d_p(f) := \|f\|_D - \Gamma_p(f)$. First we rewrite $\|f\|_D$ another way, so that it gets comparable to Γ_p .

$$\begin{aligned} \|f\|_D &= \sup_b \int_{-\infty}^b f d\mu - \inf_a \int_{-\infty}^a f d\mu \\ &= \sup_b \ln(\chi(f)(b)) + \sup_a \ln(\chi(-f)(a)) \\ &= \ln \left(\sup_b \chi(f)(b) \right) + \ln \left(\sup_a \chi(-f)(a) \right) = \ln(\|\chi(f)\|_\infty \cdot \|\chi(-f)\|_\infty). \end{aligned}$$

Plugging this into the expression of d_p , and noting that $\Gamma_p(f) = \ln \left(\frac{\|\chi(f)\|_p}{\mu(\mathbb{R})^{1/p}} \cdot \frac{\|\chi(-f)\|_p}{\mu(\mathbb{R})^{1/p}} \right)$ we get:

$$d_p(f) = \ln \left(\frac{\|\chi(f)\|_\infty \mu(\mathbb{R})^{1/p}}{\|\chi(f)\|_p} \cdot \frac{\|\chi(-f)\|_\infty \mu(\mathbb{R})^{1/p}}{\|\chi(-f)\|_p} \right).$$

We make use of the following lemma:

Lemma 1 (p -norm Approximation of the ∞ Norm). Let f be in $L^q \cap L^\infty$ for a certain $q \in \mathbb{R}$. It holds:

$$\|f\|_p \xrightarrow{p \rightarrow \infty} \|f\|_\infty.$$

A proof of this lemma can be found in [20, Chapter III, Theorem 14F].

If we apply the previous lemma to $\chi(f)$ and $\chi(-f)$, we get that $\frac{\|\chi(f)\|_\infty}{\|\chi(f)\|_p} \xrightarrow{p \rightarrow \infty} 1$ and $\mu(\mathbb{R})^{1/p} \xrightarrow{p \rightarrow \infty} 1$. The same holds for $\chi(-f)$ and therefore we have $d_p(f) \xrightarrow{p \rightarrow \infty} 0$ which proves the first point of the theorem.

For the second point, we make use of the following inequality:

$$\forall f \in L^p(\mathbb{R}, \mu) \cap L^q(\mathbb{R}, \mu), \quad 1 \leq p \leq q \leq \infty, \quad \|f\|_p \leq \mu(\mathbb{R})^{1/p - 1/q} \|f\|_q. \quad (1)$$

Thus applying this inequality to $\chi(f)$ and $\chi(-f)$ yields

$$\ln \left(\frac{\mu(\mathbb{R})^{(1/p)} \|\chi(f)\|_\infty}{\|\chi(f)\|_p} \right) \geq 0, \quad \text{and} \quad \ln \left(\frac{\mu(\mathbb{R})^{(1/p)} \|\chi(-f)\|_\infty}{\|\chi(-f)\|_p} \right) \geq 0.$$

And therefore, we get the second results by summing up both contributions.

To compute a minimum rank of convergence within ε , we make use of the following lemma:

Lemma 2 (Eventual Convergence of L^p Means Towards ∞ Norm). Let $\bar{\varepsilon} > 0$ and $g \in L^q \cap L^\infty$ for a certain $q \in \mathbb{R}$. Let

$$p_0 = p_0(t, \bar{\varepsilon}, g) = \frac{\ln \left(\frac{\mu(E_g(t))}{\mu(\mathbb{R})} \right)}{\ln \left(\frac{1-\bar{\varepsilon}}{1-t} \right)}, \quad \text{with } E_g(t) = \{x : |g(x)| > \|g\|_\infty(1-t)\}. \text{ It holds}$$

$$\forall p > \max\{q, p_0\}, \quad \frac{M_p(g)}{\|g\|_\infty} \geq 1 - \bar{\varepsilon}.$$

Proof. The convergence is clear due to the previous lemma and inequality (1). Now fix $\bar{\varepsilon} > 0$ and let $\|g\|_\infty > \bar{\varepsilon} \geq t = \alpha \bar{\varepsilon} > 0$ with $0 < \alpha < 1$ and consider the set $E_g(t) = \{x : |g(x)| > \|g\|_\infty(1-t)\}$, it holds:

$$M_p(f) = \left(\int_{\mathbb{R}} |g|^p d\mu \right)^{1/p} \geq \left(\frac{\mu(E_g(t))}{\mu(\mathbb{R})} \right)^{1/p} (\|g\|_\infty(1-t)).$$

Based on this, we are looking for a lower-bound p_0 of p such that

$$\frac{M_p(g)}{\|g\|_\infty} \geq 1 - \bar{\varepsilon}.$$

We have:

$$M_p(g) \geq \left(\frac{\mu(E_g(t))}{\mu(\mathbb{R})} \right)^{1/p} (\|g\|_\infty(1-t)) \geq \|g\|_\infty(1-\bar{\varepsilon})$$

and hence

$$\left(\frac{\mu(E_g(t))}{\mu(\mathbb{R})} \right)^{1/p} \geq \frac{1-\bar{\varepsilon}}{1-t}, \quad \text{or} \quad p \geq \frac{\ln \left(\frac{\mu(E_g(t))}{\mu(\mathbb{R})} \right)}{\ln \left(\frac{1-\bar{\varepsilon}}{1-t} \right)}.$$

The last expression holds for any $\bar{\varepsilon} > t > 0$. For instance, for $t = \bar{\varepsilon}/2$ (or $\alpha = 1/2$)

$$\forall p \geq p_0 = \frac{\ln \left(\frac{\mu(E_g(\bar{\varepsilon}/2))}{\mu(\mathbb{R})} \right)}{\ln \left(2 \frac{1-\bar{\varepsilon}}{2-\bar{\varepsilon}} \right)}, \quad M_p(g) \geq \|g\|_\infty(1-\bar{\varepsilon}).$$

This finishes the proof of the lemma. $\square$

Now we apply this lemma to both $\chi(f)$ and $\chi(-f)$ and choosing $\bar{\varepsilon} = 1 - e^{-\varepsilon/2}$ and we therefore get two constants $p^+ = p_0(t, \bar{\varepsilon}, \chi(f))$ and $p^- = p_0(t, \bar{\varepsilon}, \chi(-f))$ and we have $\forall p \geq p^* := \max\{p^+, p^-\}$, $\Gamma_p(f) \geq \|f\|_D - \varepsilon$.

This finishes the proof of the theorem. $\square$

Remarks. This lower bound depends strongly on the function f through the set $E_f(t)$; this is due to the non-equivalence of norms in the continuous case (in the discrete case, the bounds given for the theorem were crisp and uniform).

In the proof we have made use of an upper level set $E_f(t)$ and have proven that we get an upper bound for $t = \varepsilon/2$. It would work for $t = \varepsilon/4$ too and more generally for all $0 < t < \varepsilon < 1$. We have no idea at the moment what would be the influence of this on the estimated p^* .

Finding a nice p^* is a crucial problem for practical applications. Indeed because we are taking the p^{th} power of an exponential, the risk to encounter overflow is high. Therefore we need to ensure a good approximation but keeping the power p as small as possible.

Example of lower bounds. For illustration purposes let us see this effect on some toy examples.

We consider three functions:

$$f_1 : \begin{cases} [0, 5] \rightarrow \mathbb{R} \\ x \mapsto 1 \quad \text{if } x \in [1, 2] \cap [3, 4], \quad 0 \quad \text{elsewhere} \end{cases}$$

$$f_2 : \begin{cases} [-10, 10] \rightarrow \mathbb{R} \\ x \mapsto \sin(x) \end{cases}$$

$$f_3 : \begin{cases} [-10, 10] \rightarrow \mathbb{R} \\ x \mapsto a \cdot x + \sin(x) \end{cases}$$

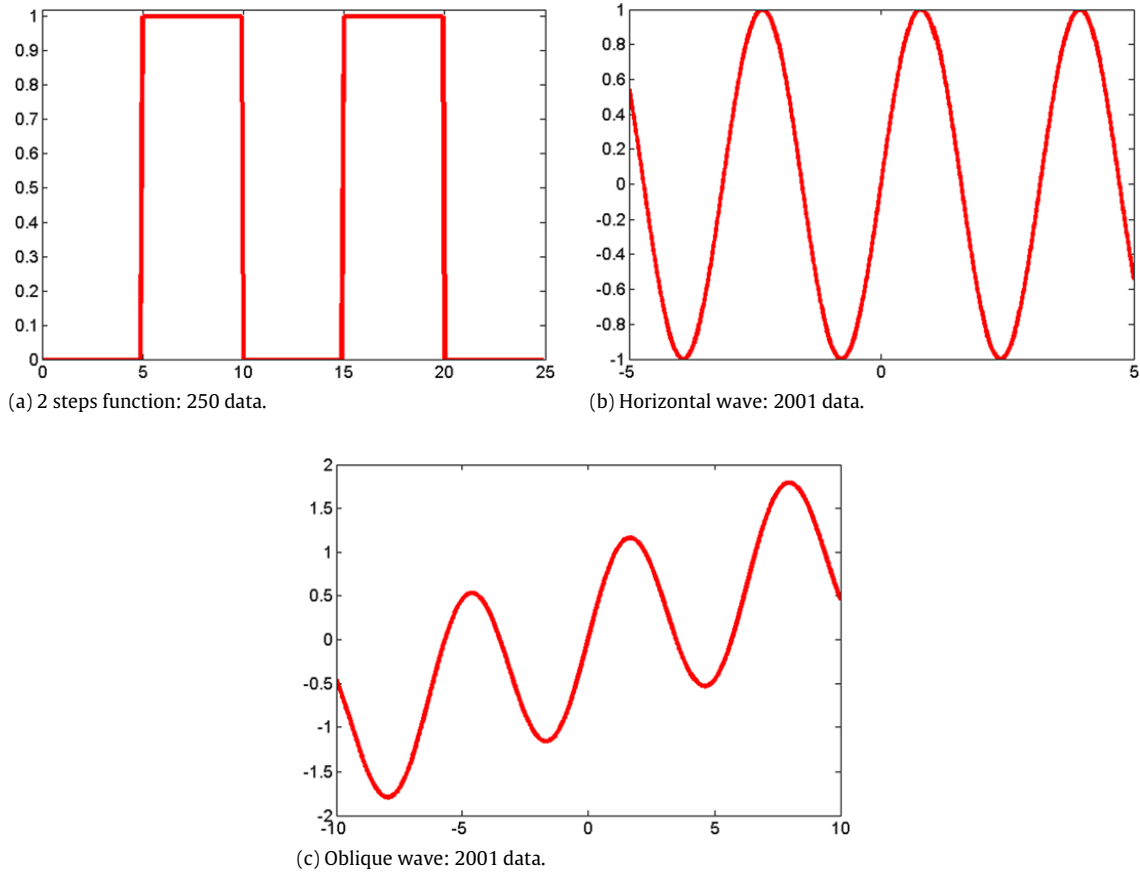


Fig. 1. Our dataset of 1 dimensional toy functions used in our paper.

Table 1

Examples of approximation and lower bound estimations on some toy functions.

Function	ε_0	α	p^*	$\Gamma_{p^*}(f)$	$\ f\ _D$	$\overline{p^*}$
2 steps	0.1	1/2	100.77	99.97	100	32
	5	0.2	2.68	98.81	100	1
Horizontal sines	0.1	1/4	257.73	99.96	100.00	101
	0.1	1/2	348.72	99.97	100.00	101
	0.1	3/4	666.56	99.98	100.00	101
	0.05	1/2	767.06	99.98	100.00	212
	2	1/2	13.45	99.39	100.00	4
Oblique sines	1	1/2	105.99	685.72	685.85	13
	2	1/4	44.64	685.55	685.85	7

where we have chosen a to be around $1/10$. In our case, we consider a measure as a combination of uniformly distributed Diracs (every 0.02 for f_1 and every 0.01 for the two others). f_1, f_2, f_3 are illustrated in Fig. 1.

We start by analyzing the convergence of the p -norms approximations towards the discrepancy norm. The first two statements of Theorem 2 tell that Γ_p gets to $\|\cdot\|_D$ from below with p increasing. Fig. 2 depicts this behavior. One sees that the values of Γ_p functions increase drastically with p close to 0 and tends to stabilize to $\|\cdot\|_D$ early.

The next point we need to have a look at is the reliability of our estimator for the value of p^* . While this value is clearly not optimal (we have left potentially a lot of the function aside when deriving the estimation), we would like it to be relatively small anyway. As this is the value we are going to use afterwards for our tests, we need it to be small enough, in order not to get into machine overflow.

We have gathered results based on our three toy functions. For all of them, we have noted the p^* defined in Theorem 2, and the smallest integer value, denoted by $\overline{p^*}$, for which the approximation up to an ε_0 holds. Moreover, as it can be seen in the proof of the theorem, one can tune the threshold used for the decomposition of the function. This threshold has also been analyzed for the horizontal sine wave function. The results are summarized in Table 1.

It comes out that we indeed get a close approximate of the discrepancy norm of a function by means of p -norms approximation. The estimation of a good p^* , however, is strongly dependent on the choice of the threshold. Fortunately first tests tend to show that a threshold in the order of 0.1 or 0.2 yields stable and reliable results.

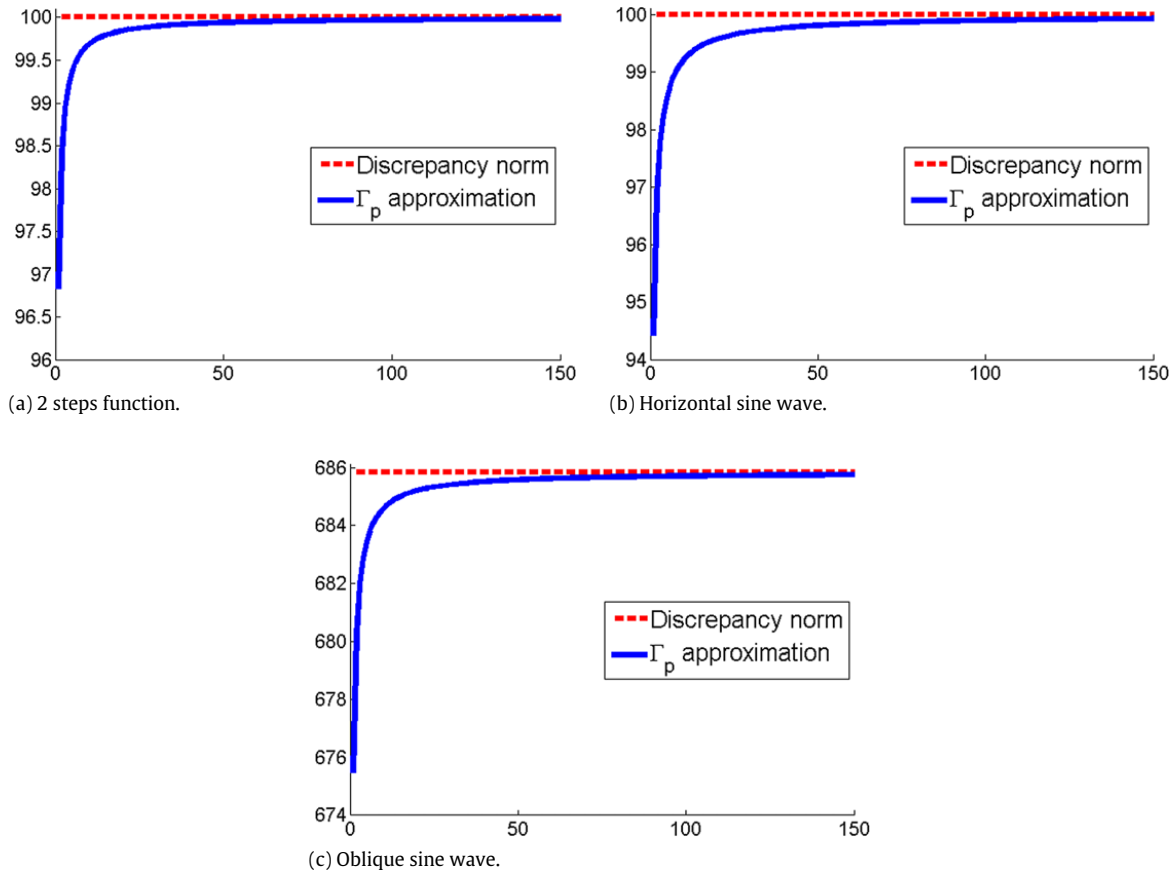


Fig. 2. Convergence of the Γ_p functions towards the discrepancy norm. The horizontal axis corresponds to changing values of p while the vertical one is the output of the Γ_p and discrepancy norm functions.

Fig. 3 shows the behavior of the estimation of p^* for different thresholding values. For the rest of the paper we use in our applications a threshold value of 0.2 which seems to lie in a rather stable area and which achieves low values of p^* .

Other properties—convolution. One advantage of this approximation lies in the possibility to solve any misalignment problem. Indeed if we consider f_t as the function f translated by an offset t , we can calculate the correlation function $h(t) = \Gamma_p(f_t - g)$ by simply computing two convolutions. As it was first shown in [11] for the discrete case, we show here that the arguments still hold in case of measurable functions.

Theorem 3. Let f and g be two measurable functions in the finite σ -algebra, h can be computed by means of two convolution operators (up to a normalization factor):

$$h = \frac{1}{p} \ln (\chi_p(f) * \chi_p(-g)) + \frac{1}{p} \ln (\chi_p(g) * \chi_p(-f))$$

where $\chi_p(x) = e^{p \int_{-\infty}^x f(y) d\mu(y)}$.

Proof. We have

$$h(t) = \Gamma_p(f_t - g) = \frac{1}{p} \ln \left(\int_{\mathbb{R}} \chi_p(f_t - g) \right) + \frac{1}{p} \ln \left(\int_{\mathbb{R}} \chi_p(g - f_t) \right) - 2 \ln (\mu(\mathbb{R})).$$

Let us consider only the first part of this equation (and the second part is done exactly the same way).

It holds:

$$\begin{aligned} \int_{\mathbb{R}} \chi_p(f_t - g)(x) d\mu(x) &= \int_{\mathbb{R}} e^{p \int_{-\infty}^{x-t} f(y) d\mu(y) - p \int_{-\infty}^x g(y) d\mu(y)} d\mu(x) \\ &= \int_{\mathbb{R}} \chi_p(f)(x-t) \cdot \chi_p(-g)(x) d\mu(x) \end{aligned}$$

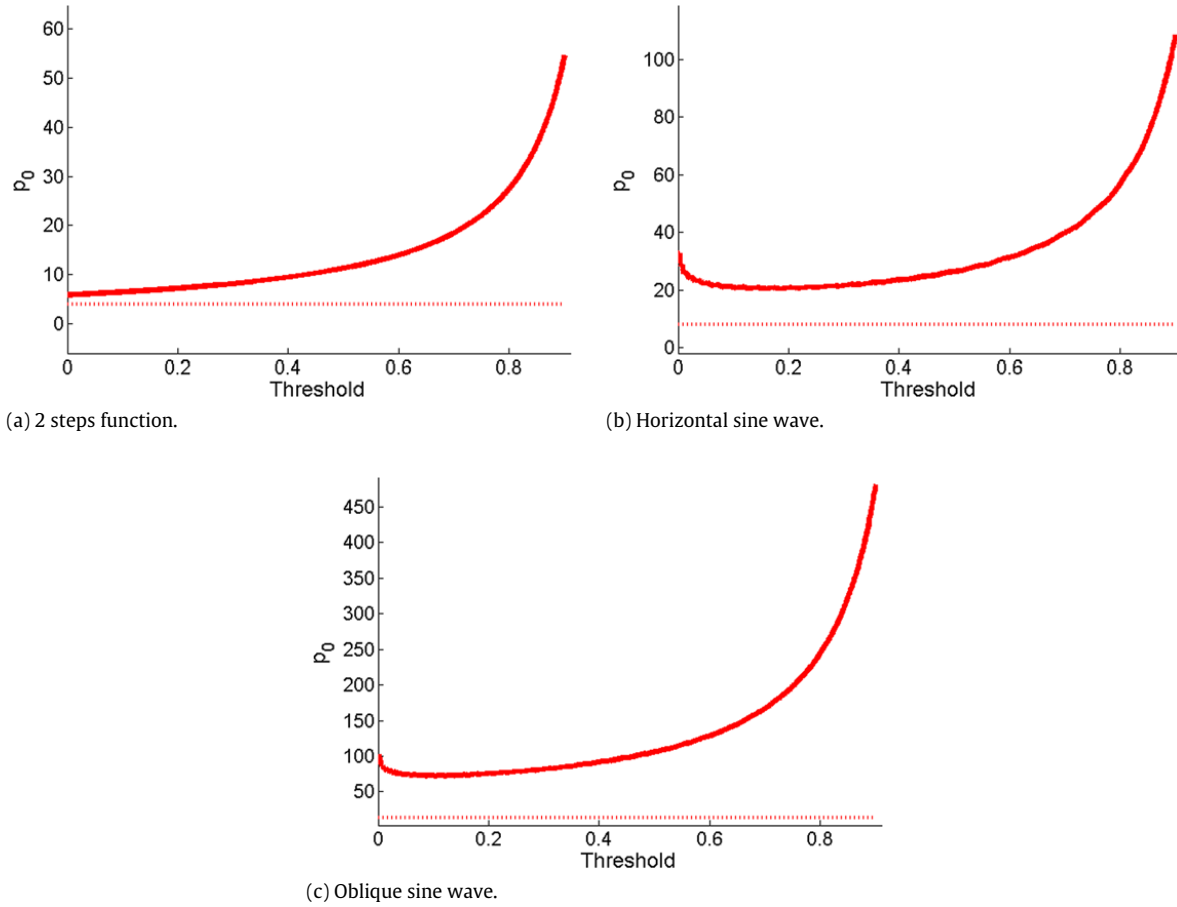


Fig. 3. Computed values of the p^* estimator. The continuous curves show the estimated p^* while the dotted straight line corresponds to the $\overline{p^*}$ as described in Table 1.

which is the definition of the correlation of two functions. The second term directly gives:

$$\int_{\mathbb{R}} \chi_p(g - f_t)(x) d\mu(x) = \int_{\mathbb{R}} \chi_p(g)(x) \cdot \chi_p(-f)(x - t) d\mu(x).$$

Hence, $h(t)$ can be computed for all t with only two convolutions. $\square$

This formula can become very handy to speed up the calculations whenever needed.

3.3. Differentiation of the discrepancy misalignment function

In this subsection we want to introduce a way of optimizing the discrepancy norm based correlation function. Given a function f and a reference g , we want to find the optimal translation parameter t^* such that $\|f_t - g\|_D$ is minimal. We consider the following optimization problem:

$$t^* = \operatorname{argmin}_{t \in \mathbb{R}} \|f_t - g\|_D.$$

We would like to optimize the objective function $J(t) = \|f_t - g\|_D$ making use of the monotonicity property of the discrepancy norm when facing misaligned functions. However, due to its definition with max and min, the discrepancy norm is not everywhere differentiable (it is however almost everywhere differentiable due to Rademacher's theorem). Moreover, as it can be seen from Fig. 4, the objective function shows some kind of plateau in some pathological cases which yield any gradient based method to fail in finding a correct solution. Therefore, we want to make use of the previous approximation to compute an approximated gradient to the objective function. As we demonstrate, we can derive a formula based on a scalar product which allows us to compute the derivative very efficiently.

Theorem 4 (Derivative of the Discrepancy Correlation). *Given a continuously differentiable function f and a function g in $L(\mathbb{R}; \mu)$ the derivative of $J(t)$ can be approximated in the following way:*

$$\frac{\partial J}{\partial t}(t) \approx \left\langle \int_{x=-\infty}^{\cdot} f'(x - t) d\mu(x), \left(\frac{\chi(-f_t + g)}{\|\chi(-f_t + g)\|_p} \right)^p - \left(\frac{\chi(f_t - g)}{\|\chi(f_t - g)\|_p} \right)^p \right\rangle. \quad (2)$$

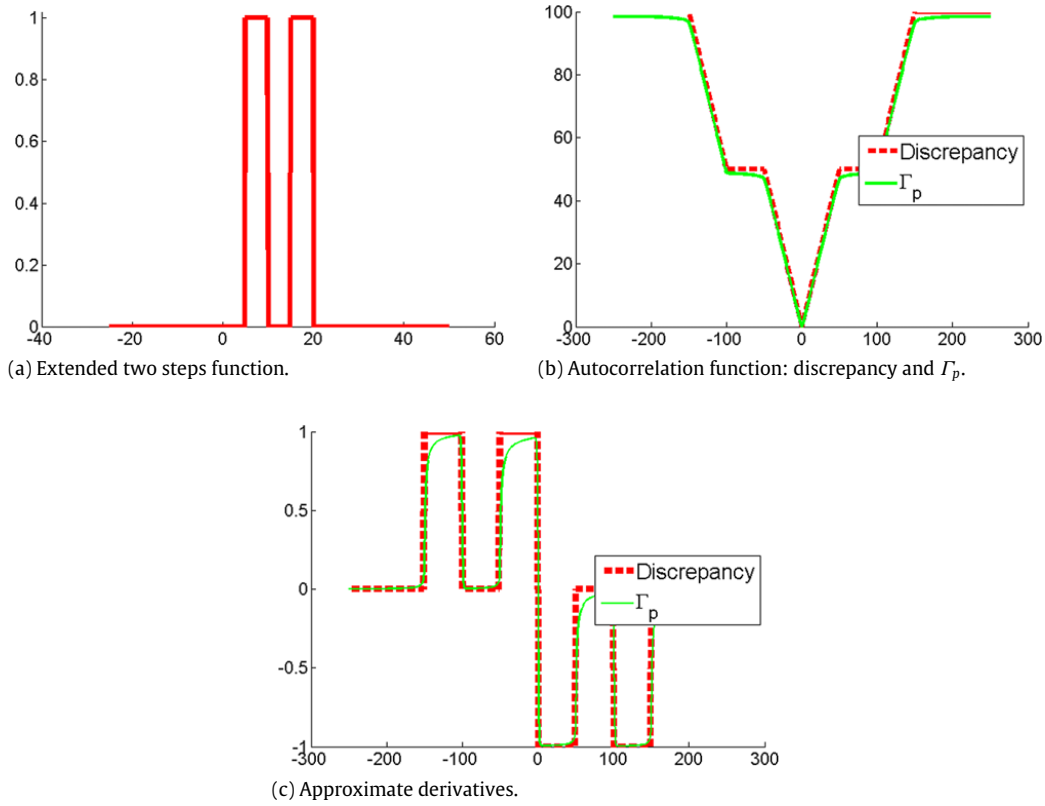


Fig. 4. This figure shows how a gradient descent based optimization algorithm might fail when trying to locally optimize the discrepancy correlation function. Indeed the correlation function (second figure) shows some plateau where the derivative (illustrated on the third figure) is 0. On the other hand, using an appropriate p -norm approximation allows to overcome this effect while keeping a really close objective function.

Proof. We need to compute the derivative of the discrepancy norm of a difference. As it has been seen in the previous section, it can be approximated by means of L^p norms and we get

$$\begin{aligned} \frac{\partial J}{\partial t}(t) &\approx \frac{\partial \Gamma_p}{\partial t}(f_t - g) = \frac{\partial}{\partial t} \left(\frac{1}{p} \ln (\| \chi(f_t - g) \|_p^p) + \frac{1}{p} \ln (\| \chi(-f_t + g) \|_p^p) + C \right) \\ &= \frac{1}{p} \frac{\partial}{\partial t} (\Gamma_p^{(+)}(f_t - g) + \Gamma_p^{(-)}(f_t - g)), \end{aligned}$$

where we have defined $\Gamma_p^{(+)}(h) := \ln (\| \chi(h) \|_p^p) := \Gamma_p^{(-)}(-h)$,

$$\begin{aligned} \frac{\partial \Gamma_p^{(\pm)}}{\partial t}(f_t - g) &= \frac{1}{\| \chi(\pm(f_t - g)) \|_p^p} \frac{\partial}{\partial t} \| \chi(\pm(f_t - g)) \|_p^p, \quad \text{with } F_p(h) := \| \chi(h) \|_p^p \\ \frac{\partial F_p}{\partial t}(\pm(f_t - g)) &= \frac{\partial}{\partial t} \int_{\mathbb{R}} |\chi(\pm(f_t - g))|^p d\mu = \frac{\partial}{\partial t} \int_{\mathbb{R}} e^{\pm p(F_t(x) - G(x))} d\mu(x) \\ &= \int_{\mathbb{R}} \frac{\partial}{\partial t} e^{\pm p(F_t(x) - G(x))} d\mu(x) \\ &= \mp \int_{\mathbb{R}} p \left(\int_{x=-\infty}^{\cdot} f'(x - t) d\mu(x) \right) \chi(\pm(f_t - g))^p d\mu \end{aligned}$$

and we finally get

$$\begin{aligned} \frac{\partial \Gamma_p}{\partial t}(f_t - g) &= \frac{-1}{\| \chi(f_t - g) \|_p^p} \int_{\mathbb{R}} \left(\int_{x=-\infty}^{\cdot} f'(x - t) d\mu(x) \right) \chi(f_t - g)^p d\mu \\ &\quad + \frac{1}{\| \chi(-f_t + g) \|_p^p} \int_{\mathbb{R}} \left(\int_{x=-\infty}^{\cdot} f'(x - t) d\mu(x) \right) \chi(-f_t + g)^p d\mu \\ \frac{\partial \Gamma_p}{\partial t}(f_t - g) &= \left\langle \int_{x=-\infty}^{\cdot} f'(x - t) d\mu(x), \left(\frac{\chi(-f_t + g)}{\| \chi(-f_t + g) \|_p} \right)^p - \left(\frac{\chi(f_t - g)}{\| \chi(f_t - g) \|_p} \right)^p \right\rangle \end{aligned}$$

and this finishes the proof. $\square$

Note that the theorem is stated for a continuous differentiable function f with no more constraints on the function g . A first remark is to notice that we could swap the constraints from f to g without changing the results, due to the commutativity of the convolution. As we stated earlier, f might be defined on a bounded domain only in which case the function f actually corresponds to the extension to the real line which might not be differentiable at the boundary of the domain. In this case, we can consider differentiating f in the sense of distribution which would add a Dirac δ distribution at the boundary for f' . This δ term added will have measure 0 for continuous measure or will be added to the total in case of a discrete measure. From a more functional analytic perspective, we could consider f to be a function of bounded variation and consider the first term in Eq. (2) to be the cumulative variation of f up to a certain point. We may equivalently rewrite:

$$\frac{\partial J}{\partial t}(t) \approx \left\langle \text{CV}[f](\cdot - t) d\mu(x), \left(\frac{\chi(-f_t + g)}{\|\chi(-f_t + g)\|_p} \right)^p - \left(\frac{\chi(f_t - g)}{\|\chi(f_t - g)\|_p} \right)^p \right\rangle$$

where $\text{CV}[f](\cdot - t)$ corresponds, in the case of a differentiable function whose derivative is Riemann-integrable, to

$$\text{CV}[f](x) = \int_{-\infty}^x f'(y) d\mu(y).$$

While formula (2) seems complicated, we see that it simplifies in the case of real discrete signals.

Corollary 2 (Derivative for Discrete Uniform Measure). Assume μ is a uniform discrete measure on a bounded domain. Then the derivative of the discrepancy autocorrelation function can be approximated by

$$\frac{\partial J}{\partial t}(t) \approx \left\langle f_t, \left(\frac{\chi(-f_t + g)}{\|\chi(-f_t + g)\|_p} \right)^p - \left(\frac{\chi(f_t - g)}{\|\chi(f_t - g)\|_p} \right)^p \right\rangle,$$

or equivalently:

$$\frac{\partial J}{\partial t}(t) \approx \sum_{i=1}^N f_t(i) \left(\left(\frac{\chi(-f_t + g)(i)}{\|\chi(-f_t + g)\|_p} \right)^p - \left(\frac{\chi(f_t - g)(i)}{\|\chi(f_t - g)\|_p} \right)^p \right),$$

where N denotes the size of the discrete finite signal (i.e. vector) we consider.

As stated earlier let us see on the toy example two steps function how the discrepancy norm can be problematic for local gradient based optimization algorithms. If we consider the autocorrelation function of the 2 steps example based on the discrepancy norm as well as on the Γ_p functions (see Fig. 4), it appears that even on really dummy easy example, the discrepancy norm does not appear to be well suited for an optimization based procedure.

However, using the above introduced approximations allows us to have an appropriate idea of what the discrepancy objective function looks like while keeping differentiable functions and avoiding plateaus which causes the derivative to vanish.

Now let us see how it behaves on some concrete examples. If we consider the horizontal wave function f and take it as a pattern we would like to align into a bigger pattern g , the discrepancy norm and its approximation(s) should ideally behave similarly when translating f along g . As toy examples, we consider two cases for g . The first one extends the f pattern only by padding 0s outside the domain of f . The second case mirrors the function f at its borders. These examples are illustrated in Fig. 5(a)–(c). The last figure illustrates the case when the frequency of the wave is twice as big.

For a given function, we have computed the discrepancies between the local window of g and our pattern f . The same has been done for Γ_p where p has been chosen according to Theorem 2 using $\varepsilon = 0.1$ and a threshold of 0.2. The values are illustrated for displacement in the range of $[-N, N]$ when using a uniform Dirac comb as measure in Fig. 5(d)–(f). It appears that the approximation overlaps well with the original discrepancy autocorrelation function. Moreover, as we are aiming at minimizing this correlation function, we can remark that the different local minima and global minimum are located at the same places.

Finally, the last row of Fig. 5 shows two approximations of the discrepancy correlation. One is concerned with the analytical derivative of the Γ_p function over the shift, and the other one is computed as a finite difference of the discrepancy correlation: $\frac{\partial}{\partial t} (\Delta_{\|\cdot\|_D}[f](t)) \approx \Delta_{\|\cdot\|_D}[f](t+1) - \Delta_{\|\cdot\|_D}[f](t)$. See that not only is the approximated derivative fast to compute (only p -norms computations and a scalar product) we also get results pretty close to what numerical differentiation would give. Finally we can see that this approximation is suitable for any gradient based local optimization methods.

4. Approximation and optimization: the higher dimensional case

In this section we consider functions $f \in L(\Omega, \mathbb{R}; \mu)$ where:

- $\Omega \subseteq \mathbb{R}^d$ is open,
- d is the dimension of the ambient space,
- μ is a finite measure.

We also consider the extended function $\tilde{f}$ that matches f on Ω and is 0 elsewhere. Both f and $\tilde{f}$ have the same discrepancy as introduced in the following sections.

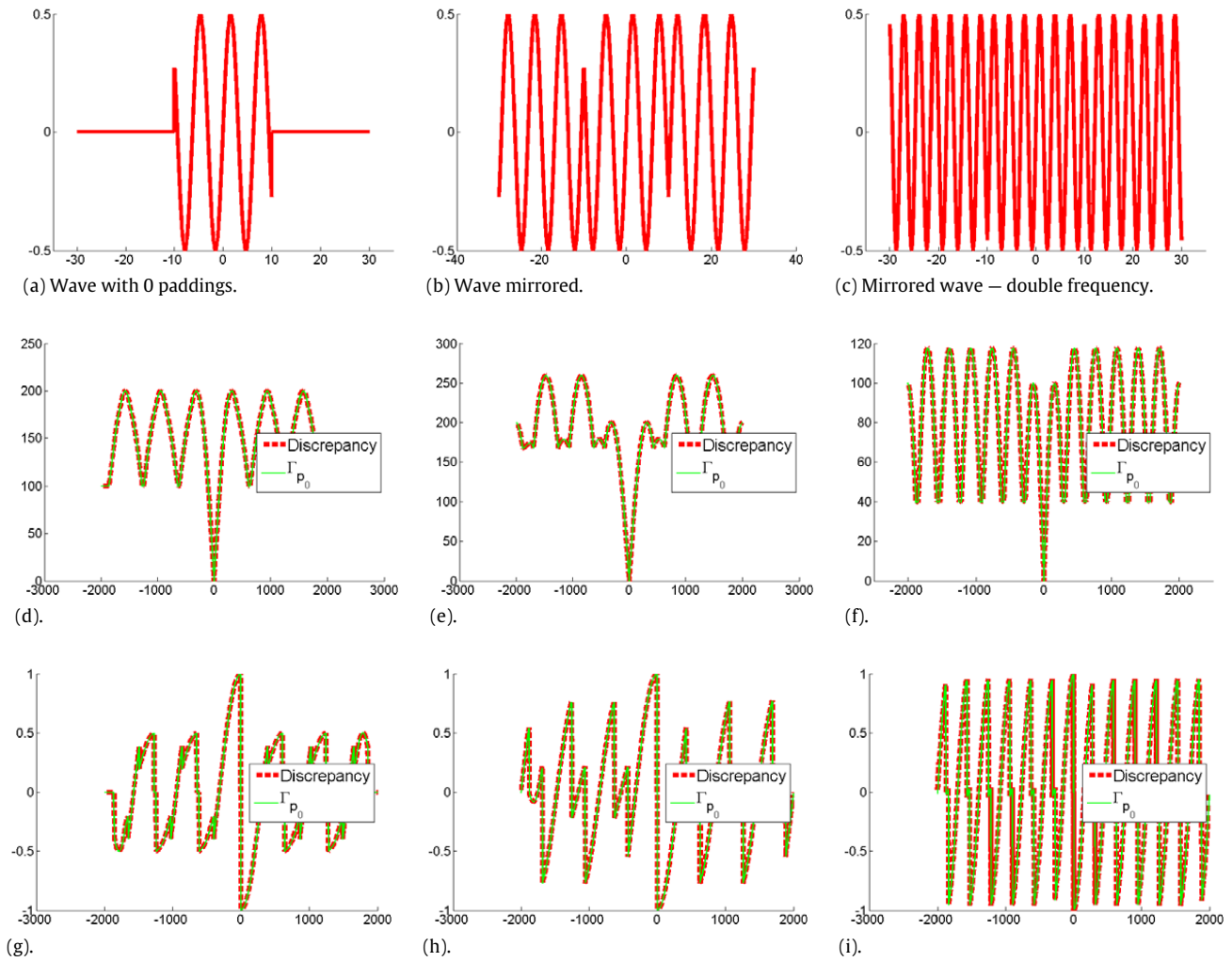


Fig. 5. Sample sine wave with their discrepancy autocorrelation functions and their approximate derivatives. The first row shows the input signal extended with 0 padding (first column) and mirroring the wave (2nd and 3rd columns). 2nd row shows the autocorrelation function based on the discrepancy norm and its approximation. Last row shows the approximated derivative together with the finite difference of the discrepancy autocorrelation.

4.1. Approximation of the discrepancy norm of a continuous multivariate signal

Now that the theory is clear for 1 dimensional functions, we would like to extend it to handle multivariate functions. According to [10], there are different ways of extending the discrepancy norm to higher dimensional functions but we consider only the following:

Definition 4 (Multivariate Discrepancy Norm). The discrepancy norm for multivariate functions can be defined by extending [Property 3](#) to multidimensional integration. Let $\iota \in \{-1, 1\}^d$ and to simplify the notations, let us define, for $s \in \mathbb{R}^d$, $[s] :=]-\infty, s_1[\times \cdots \times]-\infty, s_d[$. For a given ι , we denote by $\iota \cdot [s]$, the set composed of a product of intervals: $\iota \cdot [s] :=]-\iota_1\infty, \iota_1s_1[\times \cdots \times]-\iota_d\infty, \iota_ds_d[$, where the intervals might be flipped. Then we have:

$$\|f\|_l^{(d)} := \max_{\iota \in \{-1, 1\}^d} \left\{ \sup_{s \in \mathbb{R}^d} \int_{\iota \cdot [s]} f \, d\mu - \inf_{s \in \mathbb{R}^d} \int_{\iota \cdot [s]} f \, d\mu \right\}.$$

And this equation can be numerically implemented by means of integral images [16,17]. For simplicity, we use the following notation:

$$D^{(\iota)}(f) := \sup_{s \in \mathbb{R}^d} \int_{\iota \cdot [s]} f \, d\mu - \inf_{s \in \mathbb{R}^d} \int_{\iota \cdot [s]} f \, d\mu.$$

It defines the discrepancy of a function given a direction of integration (by direction, we mean, from which corner of the space we start the integral image.)

Therefore we can equivalently write $\|f\|_I^{(d)} = \max_{1 \leq i \leq 2^d} \{D^{(i)}(f)\}$. We also define an approximation function through p -norms:

$$\Gamma_p^{(i)}(f) := \ln(M_p(\chi^{(i)}(f)) \cdot M_p(\chi^{(i)}(-f))) = \ln\left(\frac{\|\chi^{(i)}(f)\|_p}{\mu(\mathbb{R}^d)^{1/p}} \cdot \frac{\|\chi^{(i)}(-f)\|_p}{\mu(\mathbb{R}^d)^{1/p}}\right)$$

with $\chi^{(i)}(f)(x) = e^{\int_{\mathbb{R}^d} f(s) d\mu^s}$. As in the previous sections, we have the following approximation theorem:

Theorem 5 (Approximation of the Directional Discrepancy Norm Multivariate Functions). $\Gamma_p^{(i)}$ defines an approximation of the directional discrepancy norm in the sense that

- (i) $\forall f \in L(\mathbb{R}^d; \mu), \Gamma_p^{(i)}(f) \xrightarrow{p \rightarrow \infty} D^{(i)}(f)$
- (ii) $\Gamma_p(f) \leq D^{(i)}(f)$
- (iii) $\forall \varepsilon > 0, \exists p^{(d, \varepsilon)} : \forall p \in \mathbb{R}, p \geq p^{(d, \varepsilon)} \Rightarrow |\Gamma_p^{(i)}(f) - D^{(i)}(f)| \leq \varepsilon$.

Proof. The proof is the same as for Theorem 2, for continuous univariate functions.

However, for the last point, p^* has to be changed according to the dimension, with $\bar{\varepsilon} := 1 - e^{\varepsilon/2}$

$$p^{(d, \varepsilon)} := \max \left\{ \frac{\ln \frac{\mu(E_{\chi^{(i)}(f)}(t))}{\mu(\mathbb{R}^d)}}{\ln \frac{1-\bar{\varepsilon}}{1-t}}, \frac{\ln \frac{\mu(E_{\chi^{(i)}(-f)}(t))}{\mu(\mathbb{R}^d)}}{\ln \frac{1-\bar{\varepsilon}}{1-t}} \right\}. \quad \square$$

However, the problem is not solved yet. Indeed, we are able to smoothly approximate the directional discrepancies, but we still have to combine them by taking the max over all 2^d possible directions. Fortunately, this max over a finite set can be once again approximated with a reasonable error using q -norms:

$$\|f\|_D = \max_{i \in \{-1, 1\}^d} D^{(i)}(f) \approx \|\vec{D}(f)\|_q$$

where $\vec{D} = [D^{(i^{(1)})}, \dots, D^{(i^{(2^d)})}]^T$ denotes the column vector composed of all directional discrepancies.

Finally we can combine both p - q -norms approximation:

$$\Gamma_{p,q}^{(d)}(f) := \left(\frac{\sum_{i \in \{-1, 1\}^d} \Gamma_p^{(i)}(f)^q}{2^d} \right)^{1/q} = M_q(\vec{\Gamma}_p)$$

where we have $\vec{\Gamma}_p = [\Gamma_p^{(i^{(1)})}, \dots, \Gamma_p^{(i^{(2^d)})}]^T$.

Corollary 3 (Convergence of $\Gamma_{p,q}$). Given a function $f \in L(\mathbb{R}^d, \mathbb{R}, \mu)$, the following holds:

$$\forall \varepsilon > 0, \exists \mathbf{p}^{(d)} := (p^{(d)}, q^{(d)}) : \forall \mathbf{p} \geq \mathbf{p}^{(d)}, \left| \Gamma_{p,q}^{(d)}(f) - \|f\|_I^{(d)} \right| \leq \varepsilon.$$

Note that $\mathbf{p} = (p, q) \geq \mathbf{p}^{(d)}$ reads $p \geq p^{(d)}$ and $q \geq q^{(d)}$.

Proof. Let $0 < \lambda < 1$ and $\varepsilon > 0$.

We need to estimate the quantity $\delta_{p,q} := \left| \|f\|_I^{(d)} - \Gamma_{p,q}^{(d)}(f) \right|$.

$$\delta_{p,q} = \|f\|_I^{(d)} - M_q(\vec{D}(f)) + M_q(\vec{D}(f)) - \Gamma_{p,q}^{(d)}(f) =: d_1 + d_2.$$

Now let us start with the first component

$$d_1 = \|f\|_I^{(d)} - M_q(\vec{D}(f)) = \|\vec{D}(f)\|_\infty - M_q(\vec{D}(f))$$

this last difference actually lies in a finite-dimensional vector space (2^d dimensions) and we have the following norm comparisons:

$$\|\vec{D}(f)\|_\infty \leq \|\vec{D}(f)\|_q \leq 2^{d/q} \|\vec{D}(f)\|_\infty$$

which means that for q large enough ($q \geq q_0$), we have $\left| \|\vec{D}(f)\|_q - \|\vec{D}(f)\|_\infty \right| < (1 - \lambda)\varepsilon$. Indeed

$$\|\vec{D}(f)\|_\infty \leq \|\vec{D}(f)\|_q \leq 2^{d/q} \|\vec{D}(f)\|_\infty$$

is equivalent to

$$\frac{\|\vec{D}(f)\|_\infty}{2^{d/q}} \leq M_q(\vec{D}(f)) \leq \|\vec{D}(f)\|_\infty$$

which can be reformulated as

$$\frac{\|\vec{D}(f)\|_\infty}{2^{d/q}} - \|\vec{D}(f)\|_\infty \leq M_q(\vec{D}(f)) - \|\vec{D}(f)\|_\infty \leq 0$$

whence for $-(1-\lambda)\varepsilon \leq \|\vec{D}(f)\|_\infty (2^{-d/q} - 1)$ we get the results.

$$1 - \frac{(1-\lambda)\varepsilon}{\|\vec{D}(f)\|_\infty} \leq 2^{-d/q}$$

$$\frac{d \ln(2)}{\ln\left(\frac{\|\vec{D}(f)\|_\infty}{\|\vec{D}(f)\|_\infty - (1-\lambda)\varepsilon}\right)} \leq q.$$

And we set $q_0 := d \ln 2 / \ln\left(\frac{\|f\|_p^{(d)}}{\|f\|_p^{(d)} - (1-\lambda)\varepsilon}\right)$.

Now let us compute an estimate for the second part fixing $q = q_0$, and try to get $d_2 \leq \lambda\varepsilon$ to prove the corollary:

$$d_2 = M_q(\vec{D}(f)) - \Gamma_{p,q}^{(d)}(f) = M_q(\vec{D}(f)) - M_q(\vec{\Gamma}_p(f)) = \frac{\|\vec{\Gamma}_p(f)\|_q - \|\vec{D}(f)\|_q}{2^{d/q}} \leq \lambda\varepsilon.$$

Thus applying the estimates of the triangular inequalities for the q -norm, it suffices to find p such that

$$\forall \iota \in \{-1, 1\}^d, \quad D^{(\iota)}(f) - \Gamma_p^{(\iota)}(f) \leq \lambda\varepsilon$$

which implies $\sum_{\iota \in \{-1, 1\}^d} |D^{(\iota)}(f) - \Gamma_p^{(\iota)}(f)|^p \leq 2^d \lambda^q \varepsilon^q$ and consequently $\|\vec{D}(f) - \vec{\Gamma}_p(f)\|_q \leq 2^{d/q} \lambda \varepsilon$.

Now we get that for all ι , p has to be greater than $p^{(\iota)}$ according to [Theorem 5](#). In other terms, by defining $p_0 := \max_{\iota} p^{(\iota)}$ it ensures that $d_2 \leq \lambda\varepsilon$ and therefore we have $\delta_{p,q} \leq (1-\lambda)\varepsilon + \lambda\varepsilon$ which finishes the proof. $\square$

Remark that the most important part of the approximation, and the smallest one we need to get, is the one regarding p . The one regarding q is numerically more stable. Moreover, q is just a classical 2^d dimensional vector norm which will be used for d being practically 2 (for images) or 3 (for volumes) and does not create troubles for applications. Therefore we might suggest to get λ as close to 1 as possible.

Another remark comes from the computation of q_0 where the value of the discrepancy norm of f is actually needed. In this case one can question itself whether all these calculations make sense if we anyway have to compute the discrepancy norm to get an estimator of it. However in the practical example concerned with autocorrelation, we estimate this value only once for the whole registration procedure.

Examples of lower bounds. Here we introduce three toy examples of functions defined on $\mathbb{R}^2$, which we use as test samples.

$$f_1 : \begin{cases} [1, 50] \times [1, 50] \rightarrow \mathbb{R} \\ \mathbf{x} = (x, y) \mapsto 1_{[10, 20[} + 1_{[30, 40[} \end{cases}$$

$$f_2 : \begin{cases} [1, 50] \times [1, 50] \rightarrow \mathbb{R} \\ \mathbf{x} = (x, y) \mapsto \sin(\mathbf{u}^T \mathbf{x}) \end{cases}$$

$$f_3 : \begin{cases} [1, 50] \times [1, 50] \rightarrow \mathbb{R} \\ \mathbf{x} = (x, y) \mapsto \frac{1}{\|\mathbf{x}\|_2} \sin(\|\mathbf{x}\|_2). \end{cases}$$

In the definition of f_2 the vector $\mathbf{u}$ is characterized by two parameters: an angle α and a period T , so that we can also write $f_2(x, y) = \sin\left(\frac{\cos(\alpha)x + \sin(\alpha)y}{T}\right)$.

These examples are depicted in [Fig. 6](#). It must be noted that the second and first images (or functions) are oscillating around 0 while the first one is non negative. This has a particular effect as it can be seen in the results of [Table 2](#).

[Table 2](#) gathers some results of the convergence and quality of the approximation for the different test functions with different parameters. It is divided into three groups of columns: the first one describes the different parameters used for the experiments (function, the maximal error expected ε , the percentage of ε needed for the thresholding process in the computation of $p_0 \alpha \in [0, 1]$ and the repartition of the error between p -mean and q -mean approximations $\lambda \in [0, 1]$), the results obtained for with our approximations and estimations (p_0 and q_0 computed according to [Corollary 3](#), the value of the

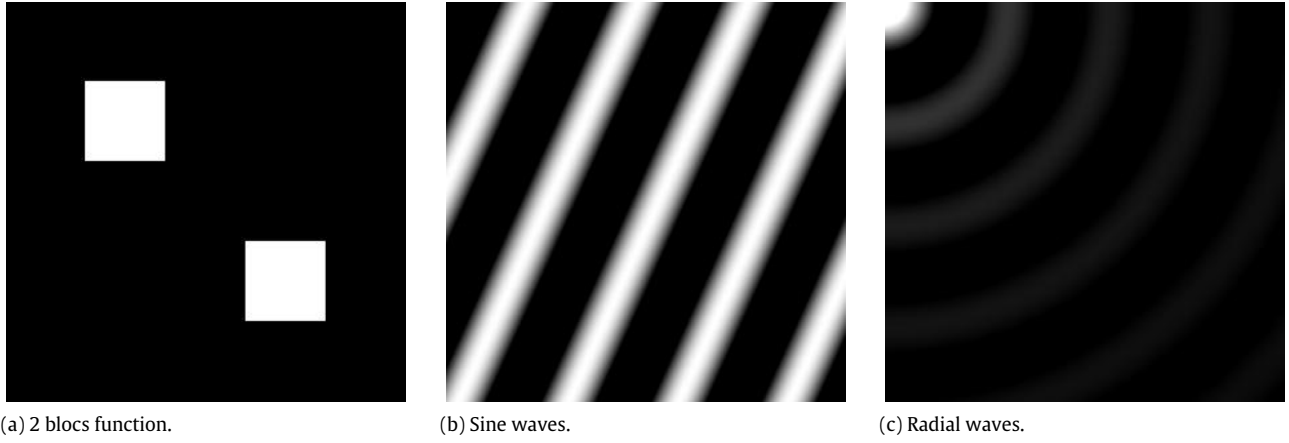


Fig. 6. Our dataset of 2 dimensional toy functions used in our paper.

Table 2

Examples of approximation and lower bound estimations on some toy functions.

Function	ε	α	λ	p_0	q_0	Γ_{p_0, q_0}	$\ \cdot\ _f^{(2)}$	$\bar{p}_0$	$\bar{q}_0$
Blocs	1	0.75	0.75	29.88	1108.3	199.87	200	6	1
	1	0.75	0.5	46.92	553.82	199.92	200	6	1
	1	0.75	0.25	98.30	368.98	199.96	200	6	1
	1	0.5	0.5	24.24	553.82	199.84	200	6	1
	10	0.5	0.5	1.71	54.76	197.73	200	1	1
Sines	1	0.25	0.5	40.52	154.09	54.97	55.83	29	155
	1	0.3	0.5	39.32	154.09	54.96	55.83	29	155
	1	0.5	0.5	53.70	154.09	55.05	55.83	29	155
	1	0.75	0.5	90.59	154.09	55.16	55.83	29	155
	2.5	0.3	0.5	13.56	61.22	53.63	55.83	11	60
Radial	0.25	0.5	0.5	246.52	179.30	16.10	16.23	125	90
	0.5	0.5	0.5	121.39	89.30	15.98	16.23	62	42
	1	0.5	0.5	58.92	44.30	15.73	16.23	31	17
	1	0.5	0.4	74.52	36.80	15.75	16.23	38	13

approximation Γ_{p_0, q_0} and the value of the discrepancy norm) and the optimal exponents $\bar{p}_0, \bar{q}_0$ achievable with the given current set of parameters. They are computed as follows:

$$\bar{q}_0 = \min \left\{ q \in \mathbb{N} : \|f\|_f^{(2)} - M_q(\vec{D}(f)) < (1 - \lambda)\varepsilon \right\}$$

$$\bar{p}_0 = \min \left\{ p \in \mathbb{N} : M_{\bar{q}_0}(\vec{D}(f)) - M_{\bar{q}_0}(\vec{D}_p(f)) < \lambda\varepsilon \right\}.$$

It is important to us that the estimated p_0 and q_0 stay close to the optimal values $\bar{p}_0$ and $\bar{q}_0$ so that the quality of the approximation is preserved, without getting into numerical troubles.

Concerning the first example of the two blocs functions, we see that our estimates are far above the optimal ones. This is due to the fact that the function is non negative. In this case, $\forall q \in \mathbb{R}, M_q(\vec{D}(f)) = \|\vec{D}(f)\|_\infty = \min(\vec{D}(f)) = \|f\|_f^{(2)}$ so that no error is done on any non negative functions during the q -means approximation.

The second function is an example of a (at least locally, due to the finiteness of the image) periodic function. In this case we notice that our estimations are close to the optimal ones (sometimes even smaller due to the discretization of the computation of $\bar{q}_0$ and $\bar{p}_0$). Why this happens to be for such periodic signals has to be studied in more details and goes beyond the scope of this research.

In the last example of spherical waves, we can see that estimated powers are within two to three times bigger than the optimal ones yielding an error about half the maximal error expected.

What come out of this table is that the choice of an adequate λ is vital to avoid numerical problems. Indeed, if not well tuned, it leads to high p or q powers and yields numerical overflows. Fig. 7 shows the impact of this λ on the estimated p_0 and q_0 . It seems that, apart from the blocs samples, a λ within the range $[0.3, 0.7]$ gives a good compromise between a high q or a high p (see Fig. 7), while keeping a good accuracy (see Fig. 8).

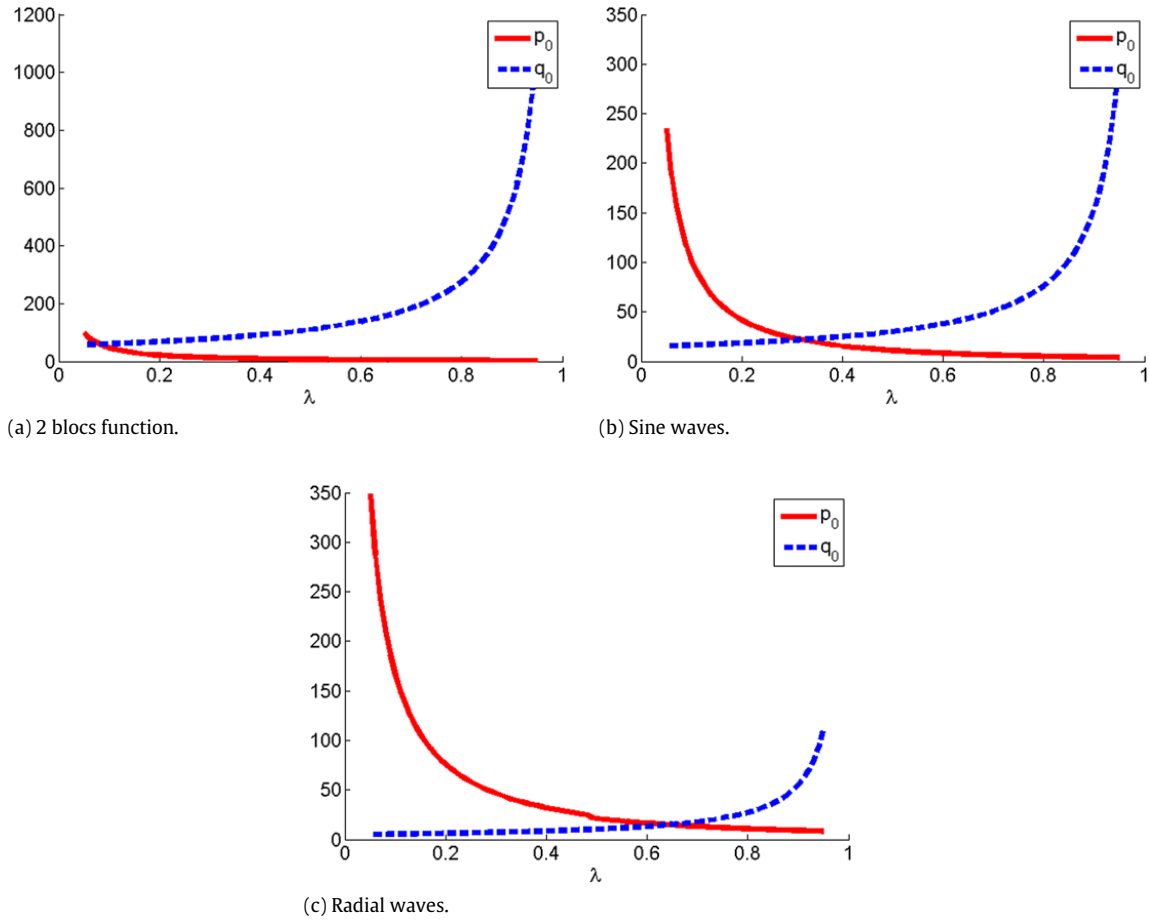


Fig. 7. Evolution of the values of p_0 and q_0 depending on the choice of λ .

4.2. Differentiation of the discrepancy correlation: the multivariate case

As in the one dimensional case, we introduce the discrepancy correlation function which we wish to minimize $J^{(d)}(\mathbf{t}) := \|f_{\mathbf{t}} - g\|_l^{(d)}$, where f and g denote two patterns we want to align.

Theorem 6 (Gradient Computation of the Discrepancy Correlation for Multivariate Differentiable Functions). *Let f be a continuously differentiable and measurable function and g be measurable. Let $J^{(d)}(\mathbf{t}) := \|f_{\mathbf{t}} - g\|_l^{(d)}$ denote the discrepancy correlation function. Then the derivatives of $J^{(d)}(\mathbf{t})$ can be approximated in the following way:*

$$\frac{\partial J}{\partial \mathbf{t}_i}(\mathbf{t}) \approx \frac{1}{2^{d/q}} \frac{\|\vec{f}_p(\mathbf{f}_{\mathbf{t}} - g)\|_q}{\|\vec{f}_p(\mathbf{f}_{\mathbf{t}} - g)\|_q^q} \sum_{i \in \{-1, 1\}^d} \Gamma_p^{(i)}(\mathbf{f}_{\mathbf{t}} - g)^{q-1} \left\langle \theta_i^{(i)}(\cdot - \mathbf{t}), \delta_{\chi^{(i)}} \right\rangle \quad (3)$$

where we use the following notations:

$$\begin{aligned} \theta_i^{(i)}(x - \mathbf{t}) &= \int_{t \in [x]} \frac{\partial f}{\partial \mathbf{x}_i}(s - t) d\mu(s) \\ \delta_{\chi^{(i)}} &= \left(\frac{\chi^{(i)}(g - f_{\mathbf{t}})}{\|\chi^{(i)}(g - f_{\mathbf{t}})\|_p} \right)^p - \left(\frac{\chi^{(i)}(f_{\mathbf{t}} - g)}{\|\chi^{(i)}(f_{\mathbf{t}} - g)\|_p} \right)^p. \end{aligned} \quad (4)$$

Note that we consider here a function f defined on $\mathbb{R}^d$. In the case that f is only defined on a domain $\Omega \subsetneq \mathbb{R}^d$ we would expect the function to be smooth at the boundary such that its extension on the whole space is continuously differentiable. We can extend these results by considering a function in a Sobolev space $W^{1,1}(\Omega)$ instead.

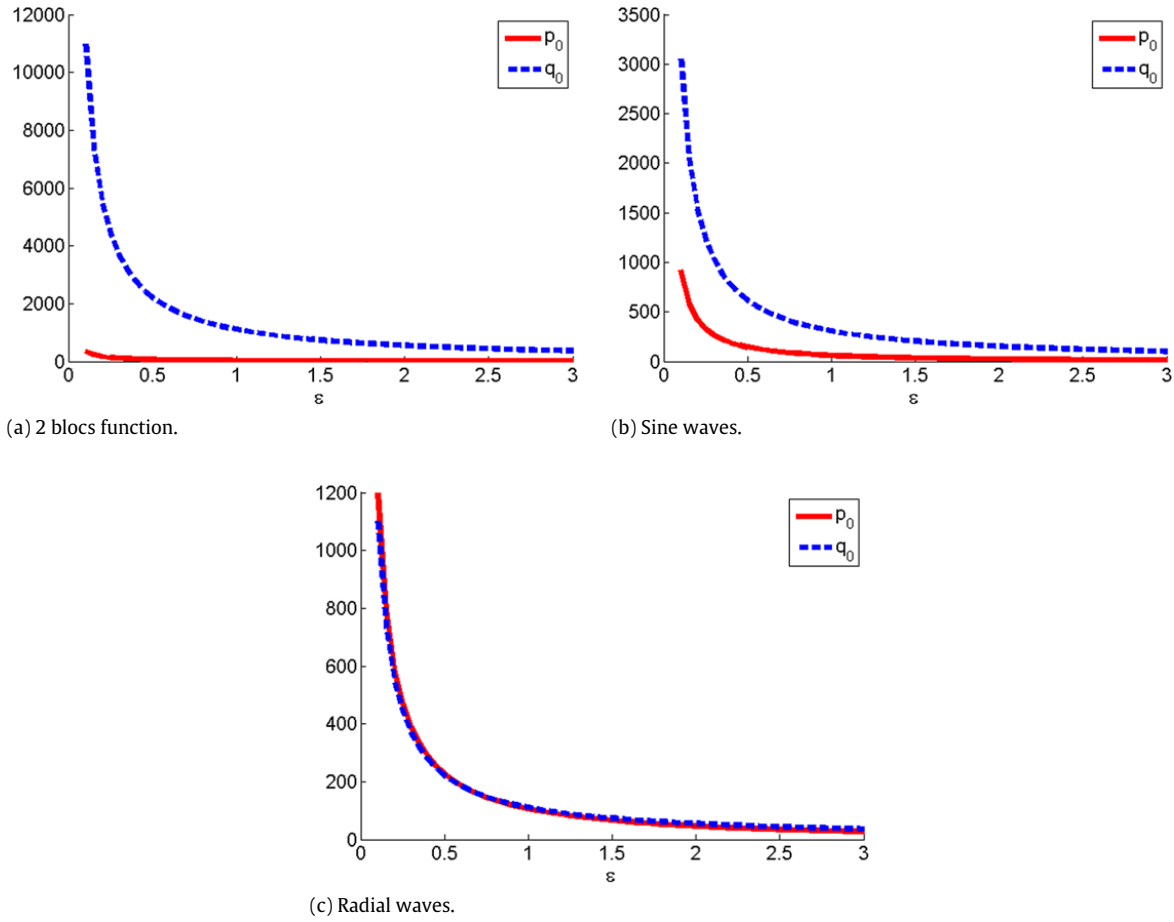


Fig. 8. Evolution of the values of p_0 and q_0 depending on the choice of ε .

Proof. Let $\Delta_{p,q}[f, g](\mathbf{t}) := \Gamma_{p,q}^{(d)}(\mathbf{f}_t - \mathbf{g}) = \frac{(\sum_i \Gamma_p^{(i)}(\mathbf{f}_t - \mathbf{g})^q)^{1/q}}{2^d}$. We approximate $J^{(d)}(\mathbf{t}) \approx \Delta_{p,q}[f, g](\mathbf{t})$.
Let $i \in \{1, \dots, d\}$, we have

$$\begin{aligned} \frac{\partial \Delta_{p,q}[f, g]}{\partial \mathbf{t}_i}(\mathbf{t}) &= \frac{1}{2^{d/q}} \frac{1}{q} \left(\sum_i \Gamma_p^{(i)}(\mathbf{f}_t - \mathbf{g})^q \right)^{1/q-1} \frac{\partial \sum_i \Gamma_p^{(i)}(\mathbf{f}_t - \mathbf{g})^q}{\partial \mathbf{t}_i} \\ &= \frac{1}{2^{d/q}} \frac{1}{q} \frac{\|\vec{\Gamma}_p(\mathbf{f}_t - \mathbf{g})\|_q}{\|\vec{\Gamma}_p(\mathbf{f}_t - \mathbf{g})\|_q^q} \frac{\partial \sum_i \Gamma_p^{(i)}(\mathbf{f}_t - \mathbf{g})^q}{\partial \mathbf{t}_i} \\ \frac{\partial \sum_i \Gamma_p^{(i)}(\mathbf{f}_t - \mathbf{g})^q}{\partial \mathbf{t}_i} &= q \sum_i \Gamma_p^{(i)}(\mathbf{f}_t - \mathbf{g})^{q-1} \frac{\partial \Gamma_p^{(i)}(\mathbf{f}_t - \mathbf{g})}{\partial \mathbf{t}_i} \\ \frac{\partial \Gamma_p^{(i)}(\mathbf{f}_t - \mathbf{g})}{\partial \mathbf{t}_i} &= \left\langle \theta_i^{(i)}(\cdot - \mathbf{t}), \delta_{\chi^{(i)}} \right\rangle. \end{aligned}$$

By combining every components together we get

$$\frac{\partial \Delta_{p,q}[f, g]}{\partial \mathbf{t}_i}(\mathbf{t}) = \frac{1}{2^{d/q}} \frac{\|\vec{\Gamma}_p(\mathbf{f}_t - \mathbf{g})\|_q}{\|\vec{\Gamma}_p(\mathbf{f}_t - \mathbf{g})\|_q^q} \sum_{i \in \{1, \dots, d\}} \Gamma_p^{(i)}(\mathbf{f}_t - \mathbf{g})^{q-1} \left\langle \theta_i^{(i)}(\cdot - \mathbf{t}), \delta_{\chi^{(i)}} \right\rangle. \quad \square$$

4.3. Discussions

Implementation. Eq. (3) seems quite complicated and not really suitable for practical use. We first look at and decompose all the different components of the formula. The first one $\frac{1}{2^{d/q}} \frac{\|\vec{\Gamma}_p(\mathbf{f}_t - \mathbf{g})\|_q}{\|\vec{\Gamma}_p(\mathbf{f}_t - \mathbf{g})\|_q^q}$ is just a multiplicative factor and can be simplified as $2^{-d} \Gamma_{p,q}(\mathbf{f}_t - \mathbf{g})^{1-q}$ which depends neither on i nor ι .

The sum on the right can be understood as a scalar product in $\mathbb{R}^{2^d}$. As ι travels all over $\{-1, 1\}^d$, each $\Gamma_p(\iota)$ takes value in $\mathbb{R}$ (and actually only positive values as already seen). So does the scalar product (in $\mathbb{R}^d$ this time) $\left\langle \theta_i^{(\iota)}(\cdot - \mathbf{t}), \delta_{\chi^{(\iota)}} \right\rangle$; so that altogether the sum can be interpreted as $\left(\overrightarrow{\Gamma_p(f_{\mathbf{t}} - g)}^{q-1}, \overrightarrow{\Theta_i(\mathbf{t})} \right)$ where $\overrightarrow{\Theta_i(\mathbf{t})} = \left[\int_{\mathbf{x} \in \mathbb{R}^d} \theta_i^{(\iota)}(\mathbf{x} - \mathbf{t}) \delta_{\chi^{(\iota)}}(\mathbf{x}) \mu(\mathbf{x}) \right]_{\iota \in \{-1, 1\}^d}$. The power of vector $\overrightarrow{\Gamma_p}$ is to be understood componentwise.

In many practical cases, the μ measure will be slowly varying (if not completely uniform on a compact support) and discrete. In this configuration, and in particular in the case of image, $d = 2$, we have (assuming $\iota = (1, 1)$ i.e., the summation starts from the $-\infty, -\infty$ corner; the other cases are computed similarly)

$$\begin{aligned} \theta_i^{(\iota)}(\mathbf{x} - \mathbf{t}) &= \int_{\iota \cdot [x]} \frac{\partial f}{\partial \mathbf{t}_i}(s - t) d\mu(s) \\ &\approx \sum_{l=-\infty}^{x_1} \sum_{m=-\infty}^{x_2} \frac{\partial f}{\partial \mathbf{t}_i}(l - t_1, m - t_2) \mu(l, m). \end{aligned}$$

The partial derivative can be approximated by finite differences; in the continuous case, the fundamental theorem of calculus applies.

$$\begin{aligned} \theta_i^{(\iota)}(\mathbf{x} - \mathbf{t}) &= \sum_{l=-\infty}^{x_1} \sum_{m=-\infty}^{x_2} (f(l - t_1 + 1, m - t_2) - f(l - t_1, m - t_2)) \mu(l, m) \\ &\approx \sum_{m=-\infty}^{x_2} f(x_1 + 1 - t_1, m - t_2) \mu(x_1 + 1, m) \end{aligned}$$

and the last formula tells that we only need to do some kind of integral images in $n - 1$ directions (apart from the one in which we wish to differentiate). This is easy and efficient to compute when working on images.

Non-differentiability of f . **Theorem 6** requires f to be continuously differentiable. The only reason to have this is to have $\theta_i^{(\iota)}$ in Eq. (4) to be well defined. We could also consider f to be only μ -almost everywhere differentiable. In this case we have that $\frac{\partial f}{\partial \mathbf{x}_i}(s - t)$ exists almost everywhere and hence Eq. (4) is well defined. Evidence suggests that similar results exist for the function of bounded variations defined on Ω but the proof of this statement being more technical is left aside for this paper.

5. Conclusion

We have reintroduced here the use of the discrepancy norm as a tool to measure correlations and (dis-)similarities between finite-length (or finite energy) signals. We proved that this norm can be approximated as close as we want by L^p norms and that this norm can be used in gradient based optimization procedures for, for instance, alignment purposes. Moreover, calculations may be accelerated using a convolution formula that takes advantage of this new approximation.

As these results hold also in high-dimensional signals, we believe that our results could be applied to image registration, for instance using a log-polar transform [21,22], or to any phase-based alignment.

Some care should however be taken when computing this approximation as values may get very high very soon with the degree of approximation increasing.

Acknowledgments

This work was initiated while JLB was funded by the Austrian Science Fund (FWF, grant no. P21496 N23, while he was with Department of Knowledge-based Mathematical System, Johannes Kepler University, Linz, Austria). He thanks now the National Science Foundation (grant DMS-1120622) for financial support. The authors are thankful to the anonymous reviewers for their precious comments.

References

- [1] H. Weyl, Über die Gleichverteilung von Zahlen mod. Eins, *Math. Ann.* 77 (1916) 313–352.
- [2] S. Haber, Numerical evaluation of multiple integrals, *SIAM Rev.* 12 (4) (1970) 481–526.
- [3] R. Coveyou, R.D. MacPherson, Fourier analysis of uniform random number generators, *J. ACM* 14 (1967) 100–119.
- [4] J. Koksma, Ein mengentheoretischer satz über die gleichverteilung modulo eins, *Compos. Math.* 2 (1935) 250–258.
- [5] H. Niederreiter, *Random Number Generation and Quasi-Monte Carlo Methods*, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 1992.
- [6] B. Chazelle, The discrepancy method in computational geometry, in: *Handbook of Discrete and Computational Geometry*, CRC Press, 2004, pp. 983–996.
- [7] J. Matoušek, *Geometric Discrepancy: An Illustrated Guide*, Vol. 18, Springer, 1999.
- [8] B. Chazelle, *The Discrepancy Methods: Randomness and Complexity*, Cambridge University Press, 2002.

- [9] H. Neunzert, B. Wetton, Pattern recognition using measure space metrics, Tech. Rep. 28, Universität Kaiserslautern, Fachbereich Mathematik, 1987.
- [10] B. Moser, A similarity measure for image and volumetric data based on hermann weyl's discrepancy, *IEEE Trans. Pattern Anal. Mach. Intell.* 33 (11) (2011) 2321–2329. <http://dx.doi.org/10.1109/TPAMI.2009.50>.
- [11] J.-L. Bouchot, J. Himmelbauer, B. Moser, On autocorrelation based on Hermann Weyl's discrepancy norm for time series analysis, in: A. Prieto (Ed.), *The 2010 International Joint Conference on Neural Networks, IJCNN*, IEEE, 2010, pp. 3139–3145. <http://dx.doi.org/10.1109/IJCNN.2010.5596843>.
- [12] B. Moser, G. Stübl, J.-L. Bouchot, On a non-monotonicity effect of similarity measures, *Similarity-Based Pattern Recognit.* (2011) 46–60.
- [13] J.-L. Bouchot, G. Stübl, B. Moser, A template matching approach based on the discrepancy norm for defect detection on regularly textured surfaces, in: *Proc. SPIE 8000, Tenth International Conference on Quality Control by Artificial Vision, 80000K* (July 12, 2011); <http://dx.doi.org/10.1117/12.889865>.
- [14] G. Stübl, J.-L. Bouchot, P. Haslinger, B. Moser, Discrepancy norm as fitness function for defect detection on regularly textured surfaces, in: *Pattern Recognition*, Springer, 2012, pp. 428–437.
- [15] P. Bauer, U. Bodenhofer, E.P. Klement, A fuzzy algorithm for pixel classification based on the discrepancy norm, in: *Proc. of the 5th IEEE Int. Conf on Fuzzy Syst., Vol. III*, New Orleans, LA, 1996, pp. 2007–2012.
- [16] F. Crow, Summed-area tables for texture mapping, *ACM SIGGRAPH Comput. Graph.* 18 (3) (1984) 207–212.
- [17] P. Viola, M. Jones, Robust real-time object detection, *Int. J. Comput. Vis.* 57 (2) (2001) 137–154.
- [18] A. Kolmogorov, G. Castelnuovo, Sur la notion de la moyenne, G. Bardi, tip. della R. Accad. dei Lincei, 1930.
- [19] G. Hardy, J. Littlewood, G. Pólya, *Inequalities*, Cambridge Univ Pr., 1952.
- [20] L. Loomis, *An Introduction to Abstract Harmonic Analysis*, Van Nostrand Reinhold, 1953.
- [21] G. Wolberg, S. Zokai, Robust image registration using log-polar transform, in: *2000 International Conference on Image Processing, 2000. Proceedings*, vol. 1, 2000, pp. 493–496. <http://dx.doi.org/10.1109/ICIP.2000.901003>.
- [22] S. Zokai, G. Wolberg, Image registration using log-polar mappings for recovery of large-scale similarity and projective transformations, *IEEE Trans. Image Process.* 14 (10) (2005) 1422–1434. <http://dx.doi.org/10.1109/TIP.2005.854501>.

Scaled-Distance-Transforms and Monotonicity of Autocorrelations

Jean-Luc Bouchot and Frédéric Morain-Nicolier

Abstract—In this letter, we investigate the use of distance transforms in a scale space domain for the image (and signal) misalignment problem. We show that it is possible to build an autocorrelation function that is monotonic with respect to the amount of translation. This creates a new paradigm for image comparison and gives yet a new generalization of distance transforms to grey-level images. Its behavior is analyzed on a natural scene image and its robustness against noise is verified numerically.

Index Terms—Autocorrelation, distance transforms, Hausdorff measure, misalignment, scale-space.

I. INTRODUCTION

ACCORDING to some research in the neuroscience (see for instance Cadieu *et al.* [1]), the human brain manages to recognize some objects by a cascade of different detectors with increasing complexity. It is now clear that the neurons are activated in presence of certain fixed stimuli. In particular it seems that edges play an important role in the detection process. Due, however to the scale invariance nature of the human visual system, it is beforehand impossible to tell the scale of an object activating a neuron at a given moment.

On the other side, computer vision scientists have been working intensively on structural similarity measures. A major piece of work is dealing with image quality estimation and yields one of the best measures so far [2]. The authors compared images by separating their resemblances into three components. First a luminance normalization, second a contrast normalization, while the third component seems to have the most impact and is described as being a structural component of the image. More recently, some work have been done on monogenic representations and phase-based imaging for image comparison [3], [4]. It is clear since the original work of [5] that structural information is contained in phases.

Manuscript received April 02, 2014; accepted May 13, 2014. Date of publication May 16, 2014; date of current version June 25, 2014. This work was initiated while J.-L. Bouchot was with the Department of Knowledge-based Mathematical Systems, Johannes Kepler University of Linz, supported by the Austrian Science Fund (FWF) under Grant P21496 N23. He is now supported by the National Science Foundation under Agreement DMS 1120622. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Kai-Kuang Ma.

J.-L. Bouchot is with the Department of Mathematics, Drexel University, Philadelphia, PA 19104 USA (e-mail: jean-luc.bouchot@drexel.edu).

F. Morain-Nicolier is with the Centre de Recherche en Sciences et Technologies d'Information et Communications, University of Reims-Champagne-Ardenne, IUT Troyes, Troyes, France (e-mail: frederic.nicolier@univ-reims.fr).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/LSP.2014.2325407

It is also understood that edges play a great role in machine recognition. The HOG descriptor [6] gives a description of local neighborhoods only based on weighted edges. Based on this, Shrivastava *et al.* [7] recently described how considering the whole environment (i.e. the whole image) for comparison, even though one uses local descriptors, can drastically improve some results.

In this letter, we investigate a new way to look at multivariate real-valued functions by defining a distance transform in a scale space. This approach is particularly suited for the misalignment problem, e.g. for image registration and stereo vision. Unlike most of the similarities used in image comparison and pattern matching [8], our scaled distance transform yields a monotonic behavior when dealing with autocorrelations.

A. Monotonicity of Autocorrelation and Misalignment

In this note, we are particularly interested in similarity measures designed for the following misalignment problem:

$$t^* := \arg \min_{t \in \mathbb{R}^n} J(f_t, g) \quad (1)$$

where

- f and g are two multivariate signals: $f, g : \mathbb{R}^n \rightarrow \mathbb{R}$,
- f_t corresponds to the translation of the signal f by a vector t , and
- J corresponds to an objective function. It can be understood as a distance measure or a similarity measure (in which case we would want t^* to maximize this equation).

When $f = g$, we are dealing with an autocorrelation minimization. In order to use local minimization techniques, it is essential for the objective function f to show some kind of monotonicity with respect to the amount of misalignment such as $J(f_{\lambda_1 t}, f) \leq J(f_{\lambda_2 t}, f)$ for all $0 \leq \lambda_1 \leq \lambda_2$. However, as described in [8], most of the usual similarity measures or distance metrics, such as any f -divergences and Minkowski distances, do not fulfil this property. This has led researchers to increase the work on local feature selections. Unfortunately, while SIFT [9] and HOG [6] like features appear robust for natural images, they behave poorly, for instance, in interferometric imaging (such as those obtained by Optical Coherence Tomography). In such cases, purely structure based measures show stronger potential. To the best of our knowledge, only measures based on the discrepancy norm [10] and Hausdorff measures [11] show this monotonic behavior.

B. Organization and Contribution

In this letter we investigate a new similarity measure in a scale-space that fulfills the above monotonicity criterion. It is based on a combination of distance transforms at different

scales. We start in Section II by re-introducing the mandatory background. In particular basics about distance-transforms and scale space are given. Then, we introduce our novel approach in Section III. We derive our *scaled-distance-transform* (SDT) for one dimensional signals and analyze its theoretical properties. Section III-B extends our results to higher dimensional signals. Finally Sections IV and V give some numerical results and draw conclusions.

II. BACKGROUND REVIEW

A. Distance Transforms and Hausdorff Measures

In his novel work, Baddeley [12] introduced a metric based on the Hausdorff measure to compare two binary images. While his idea was first to compare the efficacy of different edge detection algorithms, it has led to many research in the area of binary pattern matching. In particular it introduced the notion of Local Distance Maps that also show a monotonic behavior with respect to the displacement.

In this section we consider two bounded open sets A and B in an ambient metric space $(\mathbb{R}^n, d)$.

Definition 1: (Distance transform to the foreground). The distance transform is defined as

$$\forall \mathbf{x} \in \mathbb{R}^n, DT_d[A](\mathbf{x}) = d(\mathbf{x}, A) = \min_{\mathbf{y} \in A} (d(\mathbf{x}, \mathbf{y})). \quad (2)$$

Remark 1: The output of the distance transform might be infinite for $\mathbf{x} \rightarrow \pm\infty$. We can restrict the domain of definition of the DT to a bounded domain Ω containing A .

Remark 2: This is generally called the distance to the foreground. A similar definition, with the distance transform to the background may be defined. We will not give any further details about this in the rest of this note. The distance transform to the background is finite as long as A is bounded.

From there, it is easy to define the distance transform of binary functions (i.e. indicator functions). As long as these functions have support on a bounded domain we can identify them with their support and consider the distance transform of their support. We will equivalently write

$$DT_d[A] = DT_d[\chi_A] \quad (3)$$

where χ_A represents the indicator function of the set A . Comparing indicator functions can also be done with the Hausdorff metric [11], [13]. It is defined as follows:

Definition 2: (Hausdorff distance). Given two bounded sets $A, B \subset (\mathbb{R}^n, d)$, the Hausdorff distance is defined as

$$H(A, B) = \max(\sup_{\mathbf{x} \in A} (d(\mathbf{x}, B)), \sup_{\mathbf{y} \in B} (d(\mathbf{y}, A))) \quad (4)$$

This Hausdorff distance can however be reformulated in a simpler manner [12]

$$\begin{aligned} H(A, B) &= \sup_{x \in \Omega} |d(x, A) - d(x, B)| \\ &= \sup_{x \in \Omega} |DT_d[A](x) - DT_d[B](x)| \end{aligned} \quad (5)$$

which is in this case implemented by running two distance transform algorithm. Some efficient and robust approximations have been developed that makes this idea tractable [14], [15], [16].

The importance of this distance is justified by the following theorem:

Theorem 1: Given a non-empty finite set $A \subset \mathbb{R}^n$ and a vector $\mathbf{t} \in \mathbb{R}^n$, it holds [11]:

$$H(A, T_{\mathbf{t}}A) = \|\mathbf{t}\|$$

where the norm depends on the distance used in the definition above and $T_{\mathbf{t}}A$ is the translation of A by a vector $\mathbf{t}$.

As a consequence, we have the monotonicity along a line of the autocorrelation function:

Corollary 1: Let $\mathbf{t} \in \mathbb{R}^n$ be a vector and $A \subset \mathbb{R}^n$ a non-empty finite set. For any two constants $0 \leq \lambda_1 \leq \lambda_2$, it holds:

$$H(A, T_{\lambda_1 \mathbf{t}}A) \leq H(A, T_{\lambda_2 \mathbf{t}}A)$$

B. Scale Space

As this area can be very detailed if we want an exhaustive description, we only review the topics needed for this letter: the Gaussian scale space. We refer the curious reader to [17] for more details.

Scale space representations have first been introduced and studied in details in [18] and then used in different computer vision algorithms [9], [6]. It allows to give a description of an image based on its spatial coordinates as well as in terms of scale. Given the well known Gaussian smoothing kernel $g(x, y; s) := \frac{1}{2\pi s} e^{-\frac{x^2+y^2}{2s}}$ depending on the continuous scale parameter $s \in \mathbb{R}^+$, one can define the scale space representation of an image I as, $L(x, y; 0) = I$ and for s as above,

$$L(x, y; s) = (g(\cdot, \cdot; s) * I)(x, y) \quad (6)$$

where $*$ denotes the product of convolution. These ideas have been successfully used in the context of feature detection such as SIFT [9] or HOG [6].

III. SCALED DISTANCE TRANSFORMS

The whole idea of our new approach is to use the scale-space representation to embed the structures of images in a more robust similarity measure that is monotonic with respect to the amount of translation. We first look at the one-dimensional case and deal with higher-dimensional signals in the following section. On the other hand, our approach can be seen as yet another generalization of the distance transform to continuous, or, at least, non-binary signals. However, unlike other generalizations [19], [20], [21], the autocorrelation function based on the Hausdorff distance in a scale space shows the monotonic behavior expected. Our generalization differs also from others by the fact that it extends the idea of edge comparison, as suggested by the original work.

A. One-Dimensional Signals

Edges are found as zero-crossing of the Laplacian of the scale-space representation: a point $\mathbf{x}$ is said to be an edge at a given scale $s \geq 0$ if the Gaussian regularized derivative reaches an extremum at $\mathbf{x}$. In other words, we want both a non-vanishing first derivative and a zero-crossing of the second one.

In the one-dimensional case, this translates to:

$$\begin{aligned} (\partial_x g_s * f)(\mathbf{x}) &\neq 0, \\ (\partial_{xx} g_s * f)(\mathbf{x}) &= 0. \end{aligned}$$

We denote by $E(f; s)$ the set of edges found in f at scale s . It is defined as

$$\begin{aligned} E(f; s) &:= \{\mathbf{x} \in \mathbb{R} : (\partial_{xx} g_s * f)(\mathbf{x}) \neq 0\} \\ &\cap \{\mathbf{x} \in \mathbb{R} : (\partial_{xxx} g_s * f)(\mathbf{x}) = 0\} \end{aligned} \quad (7)$$

This can equivalently be written as $E(f; s) = (\partial_{xx} g_s * f)^{-1}(\mathbb{R} \setminus \{0\}) \cap (\partial_{xxx} g_s * f)^{-1}(\{0\})$, where the inverse should be understood in an algebraic way as

$$\mathbf{x} \in (\partial_{xx} g_s * f)^{-1}(\{0\}) \Leftrightarrow (\partial_{xxx} g_s * f)(\mathbf{x}) = 0$$

Definition 3: Let f be a function defined on $\Omega \subset \mathbb{R}$, open and bounded, with values in $\mathbb{R}$. Its scaled-distance transform is a positive real-valued mapping from $\Omega \times \mathbb{R}^+$:

$$SDT[f] : \begin{cases} \Omega \times \mathbb{R}^+ & \rightarrow \mathbb{R}^+ \\ (\mathbf{x}, s) & \mapsto SDT[f](\mathbf{x}, s) \end{cases} \quad (8)$$

$$SDT[f](\mathbf{x}, s) = \min \{d(\mathbf{x}, \mathbf{y}), \mathbf{y} \in E(f; s)\} \quad (9)$$

B. Higher Dimensional Signals

We use a very similar approach to define the scaled-distance transform in a multivariate setting. However, here we calculate the distance transform to an edge, which can appear in any direction. We say that an edge located at $\mathbf{x}$ is oriented in the i^{th} direction, $1 \leq i \leq n$, if

$$\begin{aligned} \mathbf{x} \in E_i(f; s) &:= \{\mathbf{x} \in \mathbb{R}^n : (\partial_{x_i} g_s * f)(\mathbf{x}) \neq 0\} \\ &\cap \{\mathbf{x} \in \mathbb{R}^n : (\partial_{x_i x_i} g_s * f)(\mathbf{x}) = 0\}. \end{aligned} \quad (10)$$

The set of all edge locations at scale s is defined as

$$E(f; s) = \bigcup_i E_i(f; s). \quad (11)$$

Hence we get the following definition:

Definition 4: Let f be a function defined on $\Omega \subset \mathbb{R}^n$, open and bounded, with values in $\mathbb{R}$. Its scaled-distance transform is a positive real-valued mapping from $\Omega \times \mathbb{R}^+$:

$$SDT[f] : \begin{cases} \Omega \times \mathbb{R}^+ & \rightarrow \mathbb{R}^+ \\ (\mathbf{x}, s) & \mapsto SDT[f](\mathbf{x}, s) \end{cases} \quad (12)$$

$$SDT[f](\mathbf{x}, s) = \min \{d(\mathbf{x}, \mathbf{y}), \mathbf{y} \in E(f; s)\} \quad (13)$$

C. Comparing SDTs and Monotonicity

Given a finite measure ν on $\mathbb{R}^+$ and a distance d on $\mathbb{R}^n$, we suggest to compare two SDT representations with

$$\delta_{SDT}(f, g) = \int_{s \geq 0} \delta(SDT[f](\cdot, s), SDT[g](\cdot, s)) d\nu(s). \quad (14)$$

Proposition 1: Given a function f in $(\Omega \subset \mathbb{R}^n, d)$. Its autocorrelation function $J_H[f](\mathbf{t}) := \delta_{SDT}(f, f_{\mathbf{t}})$ where the Hausdorff distance is used as δ is monotonic with the respect to the



Fig. 1. Mansion image with the reference patch.

displacement. For a certain $\mathbf{t} \in \mathbb{R}^n$ and two constants $0 \leq \lambda_1 \leq \lambda_2$, it holds

$$J_H[f](\lambda_1 \mathbf{t}) \leq J_H[f](\lambda_2 \mathbf{t}) \quad (15)$$

Note that this property is stated for functions defined on Ω which means that the objective function J_H needs not to be defined for all $\mathbf{t}$'s. Fortunately, a simple completion of the function with 0's will solve any problems. The proof relies on the following lemma.

Lemma 1: Given a function f , the set of its edges at a given scale s is anti-invariant with respect to translations:

$$E(T_{\mathbf{t}} f; s) = T_{-\mathbf{t}} E(f; s)$$

Proof: To prove the lemma it is sufficient to show that for any $\mathbf{x} \in E(f; s)$, $T_{-\mathbf{t}} \mathbf{x} \in E(T_{\mathbf{t}} f; s)$. The other part of the inequality is obtained with $f = T_{-\mathbf{t}} g$.

Equations (10) and (11) state that $\mathbf{x} \in E(f; s)$ if there exists an $i \in \{1, \dots, n\}$ such that $(\partial_{x_i} g_s * f)(\mathbf{x}) \neq 0$ and $(\partial_{x_i x_i} g_s * f)(\mathbf{x}) = 0$. In particular we have

$$\begin{aligned} \int_{\tau \in \mathbb{R}^n} \partial_{x_i x_i} g_s(\mathbf{x} - \tau) f(\tau) d\tau &= 0, \\ \int_{\mathbf{u} \in \mathbb{R}^n} \partial_{x_i x_i} g_s(\mathbf{x} + \mathbf{t} - \mathbf{u}) f(\mathbf{u} - \mathbf{t}) d\mathbf{u} &= 0, \\ &= \int_{\mathbf{u} \in \mathbb{R}^n} \partial_{x_i x_i} g_s(T_{-\mathbf{t}} \mathbf{x} - \mathbf{u}) T_{\mathbf{t}} f(\mathbf{u}) d\mathbf{u} = 0. \end{aligned}$$

A similar result is obtained regarding the first derivative. Hence $T_{-\mathbf{t}} \mathbf{x} \in E(T_{\mathbf{t}} f; s)$ which concludes the proof. $\square$

Proof. (of the proposition): With the use of the previous lemma, the proof of Proposition 1 is rather straightforward. Indeed, for a fixed $s \in \mathbb{R}^+$, for two positive constants $0 \leq \lambda_1 \leq \lambda_2$ and a translation vector $\mathbf{t} \in \mathbb{R}^n$, we have that $H(SDT[f](\cdot, s), SDT[T_{\lambda_1 \mathbf{t}} f](\cdot, s)) \leq H(SDT[f](\cdot, s), SDT[T_{\lambda_2 \mathbf{t}} f](\cdot, s))$; due to Corollary 1 and the definition 4 of the SDT combined with the previous lemma.

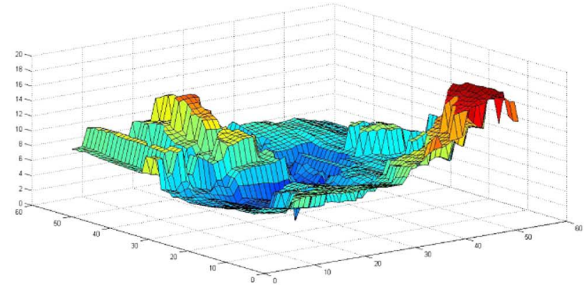
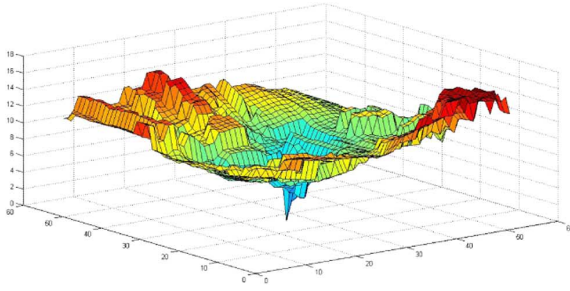


Fig. 2. Discrepancies of the translated pattern in the noise free case. x and y axis corresponds to the displacement of the window that is compared to the black box from Fig. 1.

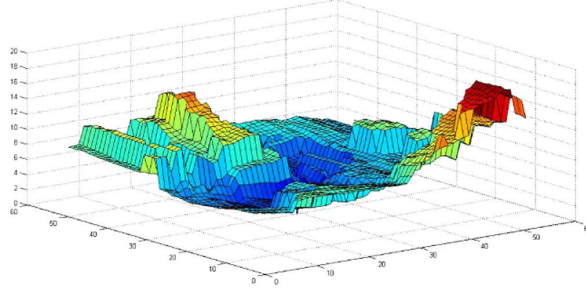
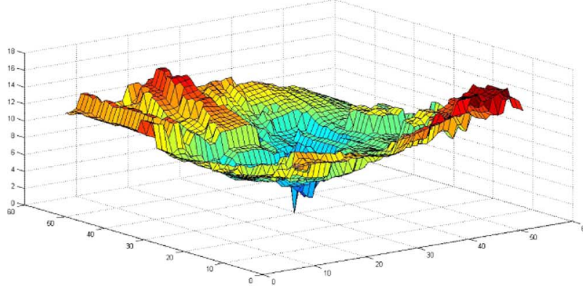


Fig. 3. Discrepancies of the translated pattern in the noisy case. x and y axis corresponds to the displacement of the window that is compared to the black box from Fig. 1.

Hence, by definition of a positive finite measure, we have that

$$\begin{aligned} & \int_{s \geq 0} H(SDT[f](\cdot; s), SDT[T_{\lambda_1 t} f](\cdot; s)) d\nu(s) \\ & \leq \int_{s \geq 0} H(SDT[f](\cdot; s), SDT[T_{\lambda_2 t} f](\cdot; s)) d\nu(s), \end{aligned}$$

which is the claimed result. $\square$

IV. NUMERICAL RESULTS

A. Monotonicity

To analyze the monotonicity property of the SDT, we compare a sliding patch around the black box from the mansion pattern, Fig. 1. This patch corresponds to a 51×51 pixel area. We compare its SDT with other patches in its neighborhood in the image with translations from -25 pixels to $+25$ pixels in all directions.¹

For the numerical experiments we need to specify how to deal with the finite measure specified in the combination of the Hausdorff distances at every level. We suggest three ideas but it is evident that prior knowledge on the geometry of the scene may help the design of better ones. We consider:

- $\nu = \chi_S$, $S \subset \mathbb{R}^+$; in this case, only a subset of scales is given importance (we do not use this approach further)
- $\nu = 1$; in this case we give the same emphasis on all scales without further considerations
- $\nu(s) = Cs$; here we give more emphasize on objects at coarser scales. It is motivated by the fact that noise and unwanted details will most likely appear in fine scales.

Moreover we apply the SDT on discrete images and therefore use the usual Gaussian pyramid as a scale space. The results of

the local discrepancy can be seen in Fig. 2. It is important to notice that the monotonicity along a line is not valid due to the fact that we consider “real” patches or a real image and thus we are not dealing with exact autocorrelation. However, it is still locally true and the surface shows few local minima.

B. Noise Robustness

It is well known that the Hausdorff distance is however rather unstable when facing noise. This is due to the $\max$ in its definition. To overcome this problem, it has been suggested to average the contribution of all the pixels via the use of a p -mean:

$$H(A, B) \approx \left(\frac{\sum_{x \in X} |d(x, A) - d(x, B)|^p}{|X|} \right)^{1/p} \quad (16)$$

And as p gets bigger this approximation gets better but more sensitive to noise. For this series of tests we have considered the black box reference window as noise free and have added Gaussian noise to the sliding window. The dissimilarity maps can be seen in Fig. 3, where we have $p = 16$.

V. CONCLUSION

We have introduced an approach that generalizes the classical distance transform to non-binary signals based on a scale space representation. Compared to other generalizations, the autocorrelation function based on our scaled distance transform shows a monotonic behavior with respect to the misalignment. This generates new ideas for signal and image registration.

The idea to use the scale space for edge detection is not new, it can be understood as the building blocks of wavelet analysis, for instance. However, the monotonicity property of the autocorrelation function described above may motivate new research in image alignments based on wavelet representations and/or multiresolution analysis.

¹Note that all the data for this note can be obtained from the authors' websites: <http://www.math.drexel.edu/jb3455/publi.html> or <http://pixel-shaker.fr/>

REFERENCES

- [1] C. Cadieu, M. Kouh, A. Pasupathy, C. E. Connor, M. Riesenhuber, and T. Poggio, "A model of V4 shape selectivity and invariance," *J. Neurophysiology*, vol. 98, no. 3, pp. 1733–1750, 2007.
- [2] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.
- [3] S. Held, M. Storath, P. Massopust, and B. Forster, "Steerable wavelet frames based on the Riesz transform," *IEEE Trans. Image Process.*, vol. 19, no. 3, pp. 653–667, 2010.
- [4] S. Bernstein, J.-L. Bouchot, M. Reinhardt, and B. Heise, "Generalized analytic signals in image processing: Comparison, theory and applications," in *Quaternion and Clifford Fourier transform and Wavelets*, ser. Trends in Mathematics. Basel, Switzerland: Springer, 2013, pp. 221–246.
- [5] A. V. Oppenheim and J. S. Lim, "The importance of phase in signals," *Proc. IEEE*, vol. 69, no. 5, pp. 529–541, 1981.
- [6] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *IEEE Computer Soc. Conf. Computer Vision and Pattern Recognition, 2005 CVPR 2005.*, 2005, pp. 886–893.
- [7] A. Shrivastava, T. Malisiewicz, A. Gupta, and A. A. Efros, "Data-driven visual similarity for cross-domain image matching," *ACM Trans. Graphics*, p. 154, 2011.
- [8] B. A. Moser, J.-L. Bouchot, and G. Stübl, "On a non-monotonicity effect of similarity measures," in *Similarity-Based Pattern Recognition*. Berlin, Germany: Springer, 2011, pp. 46–60.
- [9] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, no. 2, pp. 91–110, 2004.
- [10] B. A. Moser, "A similarity measure for image and volumetric data based on Hermann Weyl's discrepancy," *IEEE Trans. Patt. Anal. Mach. Intell.*, vol. 33, no. 11, pp. 2321–2329, 2011.
- [11] D. P. Huttenlocher, G. A. Klanderman, and W. J. Rucklidge, "Comparing images using the Hausdorff distance," *IEEE Trans. Patt. Anal. Mach. Intell.*, vol. 15, no. 9, pp. 850–863, 1993.
- [12] A. J. Baddeley, "An error metric for binary images," in *Proc. Int. Workshop on Robust Computer Vision*, 1992, pp. 59–78.
- [13] D. P. Huttenlocher and W. J. Rucklidge, "A multi-resolution technique for comparing images using the Hausdorff distance," in *1993 IEEE Computer Soc. Conf. Computer Vision and Pattern Recognition, 1993 Proc. CVPR '93*, 1993, pp. 705–706.
- [14] U. Montanari, "A method for obtaining skeletons using a quasi-Euclidean distance," *J. ACM*, vol. 15, no. 4, pp. 600–624, 1968.
- [15] G. Borgefors, "Distance transformations in digital images," *Computer Vis. Graph. Image Process.*, vol. 34, no. 3, pp. 344–371, 1986.
- [16] G. Borgefors, "On digital distance transforms in three dimensions," *Comput. Vis. Image Understand.*, vol. 64, no. 3, pp. 368–376, 1996.
- [17] T. Lindeberg, "Scale-space for discrete signals," *IEEE Trans. Patt. Anal. Mach. Intell.*, vol. 12, no. 3, pp. 234–254, 1990.
- [18] T. Lindeberg, "Scale-space theory: A basic tool for analyzing structures at different scales," *J. Appl. Statist.*, vol. 21, pp. 225–270, 1994.
- [19] D. L. Wilson, A. J. Baddeley, and R. A. Owens, "A new metric for grey-scale image comparison," *Int. J. Comput. Vis.*, vol. 24, pp. 5–17, 1997.
- [20] D. Coquin and P. Bolon, "Application of Baddeley's distance to dissimilarity measurement between gray scale images," *Patt. Recognit. Lett.*, vol. 22, pp. 1483–1502, 2001.
- [21] I. S. Molchanov and P. Terán, "Distance transforms for real-valued functions," *J. Math. Anal. Applicat.*, vol. 278, pp. 472–484, 2003.